THE SECOND PUBLICATION OF A RESEARCH PROGRAM

Grammar in Numbers

Arithmetic, Number Theory, and the Computational Grammar of Number

Abed Kadan

Founding Principal Researcher

First edition (1.0), October 2026

Grammar in Numbers: Arithmetic, Number Theory, and the Computational Grammar of Number. Abed Kadan, Founding Principal Researcher. First edition (1.0), October 2026.

Content version `cc88d59d24bd` (SHA-256 prefix of the canonical sources); source build `19ce8d9` ; generated 2026-10-18 from `book/content` by `book/tools/build.mjs` and `book/tools/pdf.mjs` . A website edition built from the same sources prints the same content version in its footer; this build does not deploy it, and no online edition is implied by this PDF.

Publication status. A research monograph. Not peer reviewed. Proofs marked “proved here” are self-checked and checked by computation on small cases; nearly all of the mathematics is classical, and it is labelled so. Machine claims name their toolchain and say whether they were measured, emulated or taken from a specification.

What this book claims, and what it does not. This book does not claim new theorems of number theory, new algorithms, new complexity results or a replacement for any established theory. It claims a framework (arithmetic as computational grammars with four separable layers and converse admissibility), a vocabulary of failure (no solution versus non-unique), executable reference semantics with tests, and reproducible measurements of real machines. Every formal claim carries its evidence category and a ledger identifier.

This edition: 42 chapters, 3 appendices, 38 definitions, 83 proved or cited results, 18 empirical and implementation results, 12 counterexamples, 2 conjectures and open problems, 121 executed code examples, 73 references.

Code, data, experiments and the research ledger: `https://github.com/akadaan310/grammarinnumbers` . Errata: `https://github.com/akadaan310/grammarinnumbers/issues` . The research program and its first publication, *Computational Grammar Theory*: `https://www.abedkadaan.com` .

Relationship to the earlier milestone: the session-1 artefact `Grammar-in-Numbers.pdf` (working edition 0.1, a concatenation of the research ledger) is preserved unchanged in the repository; the differences are recorded in `EDITION_HISTORY.md` .

Contents

PART I

What Is a Number?

Number as object, value, symbol, numeral, count, position, state, address and machine representation — and what is assumed when we say that 1 + 1 = 2.

1 **The Question: What Does $1 + 1$ Ask?** MIXED EVIDENCE

2 **Number, Numeral, Encoding, Machine** MIXED EVIDENCE

PART II

Successor, Counting, and Numerical Transition

0 → 1 → 2 → 3 → 4: the smallest grammar, its converse, induction, recursion and the cost of counting.

3 **The Smallest Grammar: $0 \to 1 \to 2 \to 3 \to 4$** MIXED EVIDENCE

4 **Recursion, Induction, and the Cost of Counting** MIXED EVIDENCE

PART III

Digits, Numerals, and Positional Representation

Numerals as programs: place value, canonical forms, zero's empty address, other digit grammars, and why $1 + 1 = 10_2$ says nothing new about the value.

5 **Positional Notation as an Operational Grammar** MIXED EVIDENCE

6 **Other Digit Grammars** MIXED EVIDENCE

7 **$1 + 1 = 10_2$: Value and Representation** MIXED EVIDENCE

PART IV

Arithmetic as Formal Operations

What an operation is: domains, closure, admissibility, operation contracts, and inverse operations as converse relations.

8 **What an Operation Is: Domains, Layers, and Contracts** MIXED EVIDENCE

9 **Inverse Operations as Converse Relations** MIXED EVIDENCE

PART V

Addition and Carry

Addition from first principles; the carry automaton, its three-element monoid, and parallel prefix adders.

10 **Addition from First Principles** MIXED EVIDENCE

11 **Carry: Automata, Monoids, and Parallel Prefix** MIXED EVIDENCE

PART VI

Subtraction and Inverses

Why $0 - 1$ has no answer in $\mathbb{N}$ and one in $\mathbb{Z}$, and the other ways systems make subtraction total.

12 **Subtraction and the Construction of the Integers** MIXED EVIDENCE

13 **Closure, Partiality, and the Price of Totality** MIXED EVIDENCE

PART VII

Multiplication

Multiplication as recursion, product and scaling; why $\times 0$ gives 0; and the algorithms from shift-and-add to $O(n \log n)$.

14 **Multiplication: Recursion, Products, and Scaling** MIXED EVIDENCE

15 **Algorithms for Multiplication** MIXED EVIDENCE

PART VIII

Division and the Boundary of Zero

The central part: division as a converse, the image/kernel theorem, 1/0 versus 0/0, every way of giving 1/0 a value and what it costs, division with remainder, and the grammar of zero.

16 **Division as the Converse of Multiplication** MIXED EVIDENCE

17 **$1/0$ and $0/0$: Two Different Failures** MIXED EVIDENCE

18 **Giving $1/0$ a Value: What Every Extension Gives Up** MIXED EVIDENCE

19 **Division with Remainder, and How Machines Divide** MIXED EVIDENCE

20 **The Grammar of Zero and One** MIXED EVIDENCE

PART IX

Binary and Digital Arithmetic

Bits are not numbers: words, masks, shifts, two's complement as truncated 2-adic integers, and arithmetic circuits.

21 **Bits, Words, and Two’s Complement** MIXED EVIDENCE

22 **Arithmetic Circuits** MIXED EVIDENCE

PART X

Machines That Compute Arithmetic

An expression traced from characters to instructions, and what real compilers, languages, instruction sets and floating-point units do — measured, emulated, or specified.

23 **From Symbol to Instruction** MIXED EVIDENCE

24 Integer Division by Zero on Real Machines MIXED EVIDENCE

25 Floating Point: Rounding, Special Data, and Flags MIXED EVIDENCE

26 Overflow, Checked Arithmetic, and Arbitrary Precision MIXED EVIDENCE

PART XI

Number Systems as Computational Domains

ℕ, ℤ, ℚ, ℝ, ℂ, ℤ/n, finite fields and p-adic integers: constructions, closure, inverses, order, representation and cost.

27 ℕ, ℤ, ℚ: Constructions, Order, and Representation MIXED EVIDENCE

28 ℝ and ℂ: Completeness, Approximation, and Branches MIXED EVIDENCE

29 ℤ/n, Finite Fields, and the 2-adic Integers MIXED EVIDENCE

PART XII

Number Theory from First Principles

Divisibility, primes and unique factorization; congruences, inverses and the Chinese remainder theorem; arithmetic functions and recurrences.

30 Divisibility, Primes, and Unique Factorization MIXED EVIDENCE

31 Congruences, Orders, and the Converse of Exponentiation MIXED EVIDENCE

32 Arithmetic Functions and the Dirichlet Ring MIXED EVIDENCE

PART XIII

Algorithms in Number Theory

Euclid and its relatives, exponentiation by squaring, sieves, primality tests and factorization — each as a transition system with an invariant and a counted cost.

33 Euclid’s Algorithm as a Transition System MIXED EVIDENCE

34 Exponentiation: The Exponent’s Numeral as a Program MIXED EVIDENCE

35 Sieves, Primality Tests, and Factoring Baselines MIXED EVIDENCE

PART XIV

The Complexity of Arithmetic

Unit cost, word RAM and bit complexity kept apart; measured exponents; and what the shape of an evaluation does and does not change.

36 Three Cost Models, Kept Apart MIXED EVIDENCE

37 Measured Complexity: What the Experiments Show and What They Cannot EMPIRICAL OBSERVATION

PART XV

Numerical Grammar and Computational Grammar Theory

Exact correspondences, useful analogies and non-equivalences between this book and Computational Grammar Theory.

38 Numerical Grammar and Computational Grammar Theory MIXED EVIDENCE

PART XVI

Programming and AI Interfaces

Operation contracts as an interface between a request and an execution system, measured against real toolchains; and the SDK.

39 Operation Contracts as an Interface Between Request and Execution MIXED EVIDENCE

40 The SDK as Executable Semantics IMPLEMENTATION RESULT

PART XVII

Findings, Limits, and Open Problems

What has been proved, reproduced, implemented, observed, conjectured and left open.

41 Findings: What Was Proved, Measured, Conjectured, Refuted, and Not Shown MIXED EVIDENCE

42 Open Problems OPEN PROBLEM

Appendices

Appendix A Notation and Conventions DEFINITION

Appendix B Reproducibility, the SDK, and the Laboratory IMPLEMENTATION RESULT

Appendix C A Guide to the Research Ledger DEFINITION

Index of definitions and results

Glossary

Bibliography

PART I

What Is a Number?

Number as object, value, symbol, numeral, count, position, state, address and machine representation — and what is assumed when we say that 1 + 1 = 2.

Chapter 1

The Question: What Does $1 + 1$ Ask?

> *Every expression sits on a network of definitions, representations, laws, algorithms and machine decisions. Familiar notation hides the network; it does not remove it.*

LEARNING OBJECTIVES

- State the central research question of this book and explain why it is not assumed to be answered.
- Descend from the expression $1 + 1$ through thirteen layers, from mathematical object to counterexample, and say what each layer contributes.
- Recognize six familiar expressions — $1 + 1$, $0/1$, $1/0$, $0/0$, $1 + 1 = 10_2$ and $2147483647 + 1$ — as six *different kinds* of question.
- Read the evidence badges and ledger identifiers that accompany every formal claim in the book.
- Know exactly what this book claims to contribute, and what it does not.

1.1 Six familiar expressions

Here are six things anyone who has been to school can read:

$$1 + 1, \qquad 0/1, \qquad 1/0, \qquad 0/0, \qquad 1 + 1 = 10_2, \qquad 2147483647 + 1.$$

They look alike. Each is a short string of digits and operator symbols, and the first two have answers everyone knows. But they are not six instances of one kind of question. The first asks for a value and gets one. The second asks for a value and gets one too, although many people hesitate. The third asks for a value that does not exist: there is no number c with $0 \cdot c = 1$. The fourth asks for a value that is not *determined*: every number c satisfies $0 \cdot c = 0$. The fifth is not a computation at all but a claim that two different words name the same value. The sixth has the answer 2147483648 in the integers, has the answer -2147483648 on most processors if the operands are 32-bit integers, and has *no meaning at all* in the C programming language, whose standard declares signed overflow undefined.

None of this is new mathematics. The point of this book is that it is not *visible* in the notation, and that making it visible is a computational task with a precise structure. We will spend seventeen parts making it visible.

1.2 The research question

The book investigates one question, formulated by Abed Kadan as the founding question of this publication:

> **Can numerical objects, arithmetic operations, number representations and numerical algorithms be understood as computational grammars whose admissible transitions, semantics, representations and costs can be formally specified and experimentally validated?**

The question has a weak reading and a strong reading. The weak reading asks whether arithmetic can be *described* in this vocabulary: whether there is a consistent way to say, of every operation, what its inputs and outputs are, when it is admissible, how its result is represented, what algorithm computes it and what it costs. The strong reading asks whether the description *does work*: whether it yields new theorems, better algorithms, sharper explanations of machine behaviour, or interfaces that make computation more reliable.

We do not assume either answer. The weak reading turns out to be well supported: every operation studied in this book has a precise description of that kind, and every failure we study is classified by the layer at which it occurs and the condition that fails. The strong reading is answered more carefully, and partly negatively. The mathematics this book uses is, with very few exceptions, classical, and it is labelled so. What the framework adds is organization, vocabulary, executable reference semantics and measurements — and, in one case we will measure (Chapter 39), a demonstrable improvement in predicting what real programs do. The final part of the book (Chapter 41) states exactly which reading has been established and to what extent.

1.3 Thirteen layers beneath $1 + 1$

What does it take for $1 + 1$ to *be* 2? The table below descends through thirteen layers. Each row is a question that someone — a mathematician, a compiler writer, a hardware designer, a person testing an answer — must have answered for the statement to mean anything.

DEFINITION 1.1 (Layered analysis of an arithmetic request) DEFINITION GIN-DEF-090

A **layered analysis** of an arithmetic expression e evaluated in a domain D answers, in order: (1) which *mathematical objects* are involved; (2) which *symbols and numerals* name them; (3) whether e is a sentence of the *formal syntax*; (4) what e *denotes* under the semantic interpretation; (5) whether every operation in e is *admissible* at its arguments; (6) which *state transition* each operation performs; (7) how values are *represented*; (8) which *algorithm* computes each operation; (9) what the *machine* executes; (10) which *invariants* make the algorithm correct; (11) what the computation *costs* under a stated model; (12) how the result is *verified* empirically; (13) where the analysis *fails*: its limits and counterexamples.

The thirteen questions are not independent, and several are classical subjects of their own (logic answers (1)–(4), algorithm analysis (8), (10) and (11), computer architecture (9)). Their value as a list is that they force each answer to be stated, rather than assumed.

LAYER	FOR $1 + 1$	WHERE IN THE BOOK
1. object	the element $S(S(0))$ of the natural numbers, reached from 0 by two successor moves	Chapter 3
2. symbol and numeral	the digit `1`, the operator symbol `+`; the result named `2`, $\texttt{10}_2$, `II`, `SS0`	Chapter 5
3. syntax	`1 + 1` is a sentence of the expression grammar: a binary node `+` with two numeral leaves	Chapter 8
4. semantics	the value of the tree is $\mathrm{add}(1, 1)$, defined by primitive recursion	Chapter 4
5. admissibility	addition is total on $\mathbb{N}$: no condition can fail	Chapter 8
6. transition	one application of the move “add 1”, that is, one successor step from state 1 to state 2	Chapter 3
7. representation	in binary the digits `1` and `1` produce sum bit `0` and carry `1`: the word `10`	Chapter 7
8. algorithm	schoolbook addition with carry, least significant digit first	Chapter 10

LAYER	FOR $1 + 1$	WHERE IN THE BOOK
9. machine	an `add` instruction on two registers; a half adder computes $\mathrm{XOR}(1,1) = 0$ and $\mathrm{AND}(1,1) = 1$	Chapter 22
10. invariant	after position i, the bits written so far plus 2^{i+1} times the carry equal the sum of the operands' low $i + 1$ bits	Chapter 10
11. cost	$\Theta(L)$ bit operations for L-bit operands; one instruction while operands fit in a word	Chapter 36
12. verification	executed: every implementation in this book agrees, and the result is checked against independent arithmetic	Appendix B
13. limits	in 32-bit hardware arithmetic the same algorithm computes $2147483647 + 1 = -2147483648$; in ISO C that expression is undefined	Chapter 26

A reader who already knows arithmetic will find nothing surprising in any single row. The surprise, if there is one, is the length of the table — and the fact that row 13 exists at all: the most familiar algorithm in mathematics has a domain outside which it computes something else.

1.4 The same request in many domains

The software that accompanies this book, the Python package `ginsdk` (Chapter 40), evaluates an arithmetic expression in a chosen domain and reports the layer at which evaluation stopped. Here is the first example of the book, executed when the book is built:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e in ["1 + 1", "0 / 1", "1 / 0", "0 / 0"]:
    o = evaluate(e, "Q")
    print(f"{e:6s} in Q:  {o.status:12s} {o.display or o.reason}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 + 1  in Q:  value        2
0 / 1  in Q:  value        0
```

```
1 / 0  in Q:  no-solution  0 · c = 0 for every c, so 0 · c = 1 would require 0 = 1
0 / 0  in Q:  non-unique   0 · c = 0 holds for every c: the equation does not determine c
```

The two failures are different, and the program says so: `1 / 0` fails because *no* rational number solves the defining equation; `0 / 0` fails because *every* rational number does. This distinction is the subject of Part VIII. It is classical — it is the difference between the image and the kernel of the map $c \mapsto 0 \cdot c$ — but it is lost whenever both cases are reported as "undefined" or "error".

Now hold the expression fixed and change the domain:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for d in ["Q", "binary64", "Python", "int32 x86-64", "int32 AArch64", "int32 RISC-V", "int32 C", "int32 Java"]:
    o = evaluate("1 / 0", d)
    print(f"{d:14s} {o.status:12s} {o.display or o.reason.split(' — ')[0].split('  [')[0]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
Q              no-solution  0 · c = 0 for every c, so 0 · c = 1 would require 0 = 1
binary64       special      Infinity
Python         exception    ZeroDivisionError: division by zero
int32 x86-64   trap         #DE (divide error)
int32 AArch64  value        0
int32 RISC-V   value        -1
int32 C        undefined    undefined behaviour
int32 Java     exception    ArithmeticException: / by zero
```

The same five characters `1 / 0` have, across these domains, *seven* distinct outcomes: no solution, a special datum that is not a number (`Infinity`), a language exception, a hardware trap, the value 0, the value -1, and no meaning at all. Two of the outcomes look like ordinary numbers. They are not solutions of anything: $0 \cdot 0 \neq 1$ and $0 \cdot (-1) \neq 1$. They are *conventions* adopted by the designers of the AArch64 and RISC-V instruction sets, for reasons we will examine in Chapter 24. In this book they are called **totalizations**: values supplied where the mathematics supplies none.

1.5 Kinds of claim, and how this book labels them

A book about the foundations of arithmetic is surrounded by millennia of mathematics. Almost every true statement in it is known. The danger is not that the book will say false things — although it may, and every claim is set out so that it can be checked — but that it will present known things as discoveries, or present a reformulation as if it were a theorem. To prevent this, every formal claim carries one of the labels below, and a stable identifier in the research ledger.

LABEL	MEANING IN THIS BOOK
Definition	a precise object; not a discovery
Axiom	an assumption of a formal system under study
Lemma, Proposition, Theorem, Corollary	a statement with a proof, given here or cited
CLASSICAL RESULT	known in the literature; cited; a proof may be included for completeness
PROVED HERE	the proof in this book was written for it and checked by computation on small cases; *this is not a claim of novelty*
REINTERPRETATION	a known fact re-expressed in this book's vocabulary; not a new result
REPRODUCED RESULT	a published result reproduced here independently by computation
IMPLEMENTATION RESULT	a property of code written for this book, on a stated test domain
EMPIRICAL OBSERVATION	a reproducible measurement, with its environment; not a proof
COUNTEREXAMPLE	an instance where a plausible claim fails
CONJECTURE	believed, not established
OPEN PROBLEM	a precisely stated question this book cannot answer

Identifiers have the form `GIN-THM-001`, `GIN-OBS-009`, `GIN-NEG-005`. Each links to its entry in the research ledger, which records its proof or data, its relation to prior work and its history. The ledger is the canonical record; the book is its exposition. Identifiers of the parent research program, Computational Grammar Theory, have the form `CGT-THM-003` and link to that separate publication.

CLAIM BOUNDARY

What the badges do not mean. “Proved here” means the proof is in this book and machine-checked on small instances by `experiments/check_theorems.py` ; it does not mean independently reviewed, and it almost never means new. No result in this book has been peer reviewed. “Empirical” results hold on the stated host, toolchain and version; the book never generalizes a measurement on one machine to “the machine”.

1.6 Relationship to Computational Grammar Theory

This book is the second publication of a research program whose first publication, *Computational Grammar Theory* (CGT), studies how computational structures — arrays, trees, graphs, ranked collections — expose and constrain their operations through grammars, and when such grammars change the cost of computing. CGT is the *earlier* book. This book is its sequel in the order of research, but in the order of ideas it is a prequel: it descends beneath CGT’s data structures to the numbers, numerals and arithmetic operations from which every address, index and cost in CGT is built.

The two publications are independent. Each has its own numbering, ledger, software and evidence; no claim here depends on a claim there. Where a concept genuinely corresponds — CGT’s grammatical *address* and the positional numeral, for instance (Chapter 5) — the correspondence is stated precisely and both sides are cited. Part XV (Chapter 38) collects the correspondences and also the places where the analogy breaks.

1.7 How to read this book

The parts build on each other, but readers with different purposes can take different paths.

- **A reader who knows ordinary arithmetic** should read Parts I–IV in order, then Part VIII (division and zero), then whatever interests them. The executable examples can be skipped; the prose does not depend on them.
- **A programmer** should read Parts I, IV, VIII, IX and X, and then Part XVI, which shows how operation contracts predict what real programs do.
- **A mathematician** will find the mathematics classical and is invited to read the proofs critically. Parts VIII, XI and XII contain the most mathematics; the open problems are in Chapter 42.
- **A computer scientist** should read Parts V, VII, IX, X and XIV, where every cost is stated in an explicit model and measured.

- **An AI researcher** should read Part IV and Part XVI, and the counterexamples throughout: they are the cases in which a fluent description of arithmetic is wrong.

Every chapter ends with exercises, most with solutions that can be expanded.

EXERCISE 1.1

For each of the six expressions of §1.1, say whether it asks for a value, makes a claim, or both; and if it asks for a value, say whether the integers supply exactly one.

▶ Solution to Exercise 1.1

EXERCISE 1.2

Write the thirteen-layer table of §1.3 for the expression $6/3$. Which layer would change if the expression were $6/4$ evaluated in $\mathbb{Z}$?

▶ Solution to Exercise 1.2

EXERCISE 1.3

On AArch64, integer division of 1 by 0 yields 0. Explain in one sentence why 0 is not "the answer" to $1/0$, and in one sentence why returning it is nevertheless a consistent design.

▶ Solution to Exercise 1.3

EXERCISE 1.4

The book's example checker executes every code block marked as runnable and refuses to build the book if the printed output differs from the output shown. Name one kind of error in this chapter that the checker *cannot* catch.

▶ Solution to Exercise 1.4

EXERCISE 1.5

Classify each statement with one of the labels of §1.5: (a) “In $\mathbb{Z}/6$ the equation $4c = 2$ has exactly two solutions.” (b) “On this host, `1.0/0.0` in Python raises ZeroDivisionError.” © “Positional notation is a compression of unary notation.” (d) “Every finite integral domain is a field.”

▶ Solution to Exercise 1.5

SUMMARY

- The book asks whether arithmetic can be specified and tested as computational grammars, and does not assume the answer.
- Thirteen layers lie beneath $1 + 1$, from mathematical object to counterexample; the book states each one.
- Six familiar expressions are six different kinds of question; `1 / 0` alone has seven distinct outcomes across common domains.
- Every claim is labelled by kind of evidence and identified in the research ledger; “proved here” never means “new”.

Chapter 2

Number, Numeral, Encoding, Machine

> *A digit is not a number, a numeral is not a number, a bit pattern is not a number, and a register is not a number. Each of them can stand for one.*

LEARNING OBJECTIVES

- Distinguish the value 3 from the digit `3`, the numerals `3`, 11_2, `III`, `SSS0`, the bit patterns that encode it, and a machine state that holds one of them.
- Define the four layers — value, numeral, encoding, machine state — and the four maps between them: denotation, encoding, decoding and execution.
- State precisely what is assumed when we say that a number exists, that two numerals denote the same number, and that an operation returns a result.
- Recognize statements that silently cross layers, and rewrite them so that they do not.

2.1 Twelve readings of “3”

Write the character `3` on a page. What has been written? The question has at least twelve answers, and they are not interchangeable.

READING	WHAT “3” IS	LAYER
mathematical object	the element $S(S(S(0)))$ of the natural numbers	value
count	the number of elements of $\{a, b, c\}$	value (via cardinality)
position	the fourth place in $0, 1, 2, 3, \ldots$ — or the third, if one counts from 1	value with an order convention
state	a node of the successor transition system	value as state

READING	WHAT “3” IS	LAYER
address	the canonical numeral of 3 in some digit grammar: `3` , 11_2 , 10_3 , `SSS`	numeral
digit	the symbol `3` of the decimal alphabet	numeral alphabet
symbol	a token in an expression such as `3 + 1`	syntax
sequence	the one-letter word `3` in the language of decimal numerals	numeral
machine representation	`0x00000003` as a 32-bit integer; `0x4008000000000000` as an IEEE binary64 number	encoding
transition result	the value of `1 + 2` , of $S(2)$, of `6 / 2`	value (output of an operation)
algebraic element	$3 \in \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$; the class $3 + n\mathbb{Z}$ in $\mathbb{Z}/n$; and $3 = 0$ in $\mathbb{Z}/3$	value in a chosen structure
computational value	what a register holds after `mov eax, 3`	machine state

Where do the readings agree? Every one of them denotes, encodes or holds the same natural number. Where do they diverge? A *digit* is not a number: `3` is a symbol, and in base 3 there is no digit `3` at all. A *numeral* is not a number: `03` and `3` are different words with the same value. A *binary representation* is not the number: 11_2 and `3` are different words. A *machine word* is not an integer: the 32-bit pattern `0xFFFFFFFF` is -1 under one reading and 4294967295 under another. A *residue class* is not an integer: $3 = 0$ in $\mathbb{Z}/3$, while $3 \neq 0$ in $\mathbb{Z}$.

Most of the time nothing goes wrong when these readings are blurred. This chapter separates them so that, in the rest of the book, we can say precisely what goes wrong when something does.

2.2 The four layers

DEFINITION 2.1 (Number (value)) DEFINITION GIN-DEF-001

A **natural number** is an element of a *successor algebra* $(\mathbb{N}, 0, S)$: a set $\mathbb{N}$ with a distinguished element 0 and an injective map $S : \mathbb{N} \to \mathbb{N}$ with $0 \notin S(\mathbb{N})$, such that every subset of $\mathbb{N}$ that contains 0 and is closed under S is all of $\mathbb{N}$. Integers, rationals, reals and complex numbers are defined from $\mathbb{N}$ by standard constructions (Part XI). A **value** is an element of one of these structures. Values are not strings and have no digits.

The definition is Dedekind's (1888) and, in axiomatic form, Peano's (1889) [Dedekind 1888; Peano 1889]. Its last clause — the induction axiom — is what makes $\mathbb{N}$ the *smallest* set reachable from 0 by successor steps. Without it, a structure could contain extra elements that are never reached by counting.

THEOREM 2.2 (Dedekind's categoricity theorem) CLASSICAL RESULT GIN-HIST-002

Any two successor algebras $(\mathbb{N}, 0, S)$ and $(\mathbb{N}', 0', S')$ are isomorphic, and the isomorphism is unique: there is exactly one bijection $h : \mathbb{N} \to \mathbb{N}'$ with $h(0) = 0'$ and $h(Sn) = S'(hn)$ for all n.

Proof.

Existence. By the recursion theorem — itself proved from the induction axiom — there is a unique map $h : \mathbb{N} \to \mathbb{N}'$ with $h(0) = 0'$ and $h \circ S = S' \circ h$; symmetrically there is a unique $g : \mathbb{N}' \to \mathbb{N}$ with $g(0') = 0$ and $g \circ S' = S \circ g$. Then $g \circ h : \mathbb{N} \to \mathbb{N}$ satisfies $(g \circ h)(0) = 0$ and $(g \circ h) \circ S = S \circ (g \circ h)$, as does the identity; by the uniqueness clause of the recursion theorem, $g \circ h = \mathrm{id}$. Likewise $h \circ g = \mathrm{id}$. *Uniqueness.* Any h' with the same two properties agrees with h on 0, and agrees on Sn whenever it agrees on n; the set where $h' = h$ therefore contains 0 and is closed under S, so by induction it is all of $\mathbb{N}$. ∎

This theorem is what entitles us to speak of *the* natural numbers. It is also our first example of a pattern that recurs throughout the book: a *structure* is characterized by its operations and their laws, and any two *representations* of it — sets, numerals, bit patterns — are related by a unique structure-preserving translation. The value 3 is what all representations of the third successor of zero have in common.

DEFINITION 2.3 (Numeral and digit) DEFINITION GIN-DEF-002

A **numeral** is a word over a finite **digit** alphabet, read in a *numeral system* that assigns it a value. `2`, $\texttt{10}_{\texttt{2}}$, `II`, `SS0` and `0010` are numerals. A digit is a letter of a numeral alphabet; a digit is a symbol, not a number, even when a one-letter numeral and its only digit are spelled alike.

DEFINITION 2.4 (Encoding) DEFINITION GIN-DEF-003

An **encoding** of a set V of values in a format F is a partial injective map $\mathrm{enc} : V \rightharpoonup \{0,1\}^w$ together with a total decoding map $\mathrm{dec} : \{0,1\}^w \to V \cup S_F$, where S_F is a set of *special data* that are not values. For IEEE 754 binary formats, S_F contains $+\infty$, $-\infty$ and NaN, and the sign of zero is extra information that is not part of the value.

DEFINITION 2.5 (Machine state) DEFINITION GIN-DEF-004

A **machine state** assigns bit strings to registers and memory, together with status information: flags, pending exceptions, the trap state. An **instruction** is a partial map on states. The **execution** of an expression is a sequence of instructions; its **outcome** is either a final state from which a value is decoded, or a trap or exception state.

The four definitions give four layers, and between them four maps:

$$\underbrace{\text{numeral}}_{\text{word}} \xrightarrow{\text{denotation}} \underbrace{\text{value}}_{\text{element}} \xrightarrow{\text{encoding (partial)}} \underbrace{\text{bit string}}_{w \text{ bits}}, \qquad \text{bit string} \xrightarrow{\text{decoding (total)}} \text{value} \cup \text{special data}, \qquad \text{state} \xrightarrow{\text{execution}} \text{state}.$$

A claim about arithmetic names its layer. "$1 + 1 = 2$" is a statement about values. "$1 + 1 = 10_2$" is the same statement about values, with the right-hand side written in a different numeral system. "`0x7FFFFFFF + 1` is `0x80000000`" is a statement about encodings under wraparound. "After `add eax, 1` the overflow flag is set" is a statement about machine states. The statements are related, but they are not the same statement, and a proof of one is not a proof of another.

2.3 Encodings are partial; decodings forget

The asymmetry between encoding and decoding in Definition 2.4 is deliberate. An encoding is *partial* because a fixed number of bits can name only finitely many values: $2^{53} + 1$ is an integer,

but it has no binary64 encoding (Chapter 25). A decoding is *total* because every bit pattern must be read as *something* — and the something need not be a value.

PROPOSITION 2.6 (Encoding and decoding are not inverse to each other)

PROVED HERE GIN-PROP-050

For every encoding, $\mathrm{dec}(\mathrm{enc}(v)) = v$ *for every* v *in the domain of* enc. *The other composite is not the identity in general:*

1. *In IEEE binary64, exactly* $2^{53} - 2$ *of the* 2^{64} *bit patterns decode to the single datum NaN, and two patterns (* `0x0000000000000000` *and* `0x8000000000000000`*) decode to the value* 0.
2. *The same 32-bit pattern decodes to different values under different encodings:* `0xFFFFFFFF` *is* -1 *in two's complement and* 4294967295 *as an unsigned integer, and it is a NaN as an IEEE binary32 pattern.*

Proof.

The first claim is the definition of a decoding as a left inverse of the encoding. (1) A binary64 pattern is a NaN exactly when its 11 exponent bits are all ones and its 52 fraction bits are not all zero: 2 signs times $2^{52} - 1$ fractions gives $2^{53} - 2$ patterns. A pattern with all exponent and fraction bits zero decodes to ± 0, and both signed zeros have the value 0. (2) Two's complement reads a w-bit pattern β as $\beta - 2^w$ when its top bit is set, so `0xFFFFFFFF` $= 2^{32} - 1$ reads as -1; unsigned reading gives $2^{32} - 1$; as binary32 the exponent field `11111111` with non-zero fraction is a NaN. ∎

The second part of the proposition is the source of a large class of programming errors: the bits are the same, the *type* — which encoding is in force — is not. A bit pattern carries no type of its own.

COUNTEREXAMPLE 2.7 (A bit pattern determines a value) COUNTEREXAMPLE GIN-NEG-018

The claim "a register holds a number" is false at the encoding layer. The pattern `0xFFFFFFFF` is -1, 4294967295, or not a number at all, depending on the encoding in force; the pattern `0x40490FDB` is the integer 1078530011 and also the binary32 approximation of π.

Here is the second pattern decoded three ways by the SDK:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import struct
from ginsdk import ieee

w = 0x40490FDB
print("unsigned :", w)
print("signed   :", w - (1 << 32) if w >> 31 else w)
x = ieee.from_bits(ieee.BINARY32, w)
print("binary32 :", x, "  exact value", x.value())
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
unsigned : 1078530011
signed   : 1078530011
binary32 : 3.1415927   exact value 13176795/4194304
```

The binary32 number is the dyadic rational $13176795/2^{22}$, which is close to π but is not π; no finite bit pattern encodes π in any of the formats of this book.

2.4 What is assumed when we say a number exists

Three familiar sentences carry hidden assumptions.

"The number 3 exists." At the value layer, this means: in a successor algebra, the term $S(S(S(0)))$ denotes an element. That is guaranteed once a successor algebra exists at all. Its existence is not provable from nothing; it is an axiom of the background mathematics (in set theory, the axiom of infinity supplies a set closed under $x \mapsto x \cup \{x\}$, from which von Neumann's $\mathbb{N}$ is cut out). In this book we take the existence of $\mathbb{N}$ for granted, as all of ordinary mathematics does, and we never need more. At the encoding layer, "3 exists" is a different and *falsifiable* claim: the format has an encoding of 3. It is true for every format in this book; the corresponding claim for $2^{53}+1$ in binary64 is false.

"The numerals `3` and `0011`$_2$ denote the same number." This is a statement about two words and a denotation map: $\mathrm{val}_{10}(3) = \mathrm{val}_2(0011)$. It is decidable — by computing both values, or by reducing both words to canonical form and comparing — and its proof is a computation, not an insight. Chapter 5 shows that for standard positional systems two words have the same value exactly when they differ by leading zeros, so equality of numerals within one base reduces to equality of words after deleting leading zeros (GIN-PROP-001). Across bases it requires conversion, whose cost is the subject of Chapter 37.

"The operation returns a result." Three things are asserted at once: the operation is *defined* at these arguments (totality), its result is *unique* (functionality), and the result is *in the domain* (closure). Each can fail independently. Subtraction in $\mathbb{N}$ fails closure at $(0, 1)$; division in $\mathbb{Q}$ fails

totality at $(1, 0)$ and functionality at $(0, 0)$ — which are the same failures, seen as a converse problem, that Part VIII will call *no solution* and *non-unique*. A machine can fail in yet another way: it may return a result that is not the mathematical one (wraparound, rounding), or return nothing and transfer control (a trap).

> REMARK
>
> The words “undefined”, “invalid” and “error” are used for all of these failures in ordinary speech, and in many programming languages. This book never uses “undefined” as a final answer, except in its technical sense in the C language standard (“undefined behaviour”), where it means that the *program* has no meaning. For every failure it says which condition fails, at which layer.

2.5 Crossing layers safely

Every computation crosses layers: a program reads numerals, encodes values, executes instructions, decodes results and prints numerals again. The crossings are where meaning can change. A safe crossing has three properties, which can be checked:

1. **The source is in the domain of the map.** The value must have an encoding in the target format (otherwise rounding or overflow intervenes); the bit string must be read with the encoding that produced it.
2. **The map commutes with the operation, or the discrepancy is stated.** Hardware addition on 32-bit two’s-complement words commutes with integer addition *modulo* 2^{32}, not in $\mathbb{Z}$; binary64 multiplication commutes with real multiplication followed by one rounding.
3. **The reverse crossing recovers the value.** Printing a binary64 number with too few digits and reading it back can change it; printing with the shortest round-tripping digit string cannot.

The software of this book makes the crossings explicit. Its evaluator never reports a bare number; it reports the layer reached, the domain, and the representation:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for d in ["Z", "int32 x86-64", "binary32"]:
    o = evaluate("16777216 + 1", d)
    print(f"{d:13s} {o.display:10s} {o.representation}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
Z             16777217   {'exact': '16777217', 'binary': '100000000000000000000001', 'hexadecimal': '1000001',
'bit length': 25}
int32 x86-64  16777217   {"two's complement": '00000001000000000000000000000001', 'hex': '0x01000001'}
binary32      16777216   {'datum': '16777216', 'sign': '0', 'exponent': '10010111', 'fraction':
'00000000000000000000000', 'exact value': '16777216'}
```

In binary32 the sum $2^{24} + 1$ is not representable: the format has 24 significant bits, and $2^{24} + 1$ needs 25. The exact sum is rounded to the nearest representable number, which — a tie between 2^{24} and $2^{24} + 2$, broken towards the even significand — is 2^{24} itself. The equation $x + 1 = x$ has a solution in binary32. Chapter 25 explains why, and which other familiar laws fail.

EXERCISE 2.1

Give the reading (row of the table in §2.1) of "3" in each sentence: (a) "Turn to page 3." (b) "The array has 3 elements." © "`x = 3` sets the register to 3." (d) "In $\mathbb{Z}/3$, $3 = 0$." (e) "`3` is not a base-2 digit."

▶ Solution to Exercise 2.1

EXERCISE 2.2

How many binary32 bit patterns are NaNs? How many encode the value 0? How many encode an integer?

▶ Solution to Exercise 2.2

EXERCISE 2.3

Show that for 32-bit two's complement, $\mathrm{dec}(\mathrm{enc}(a) \oplus \mathrm{enc}(b)) \equiv a + b \pmod{2^{32}}$, where $\oplus$ is the hardware addition of bit patterns as unsigned integers modulo 2^{32}. When is the congruence an equality?

▶ Solution to Exercise 2.3

EXERCISE 2.4

Is the sentence "the number 10^{400} exists in binary64" true? Rewrite it as one true sentence about values and one true sentence about encodings.

▶ Solution to Exercise 2.4

EXERCISE 2.5

In Python, `float(2**53 + 1) == float(2**53)` is `True`. Which crossing of §2.5 fails, and which property?

▶ Solution to Exercise 2.5

SUMMARY

- "3" has at least twelve readings; they agree on the value and diverge on everything else.
- The four layers are value, numeral, encoding and machine state; the maps between them are denotation, encoding, decoding and execution.
- Encodings are partial and injective; decodings are total and forget (Proposition 2.6); a bit pattern has no type of its own.
- "A number exists", "two numerals are equal" and "an operation returns a result" each hide assumptions that can fail independently.

PART II

Successor, Counting, and Numerical Transition

0 → 1 → 2 → 3 → 4: the smallest grammar, its converse, induction, recursion and the cost of counting.

Chapter 3

The Smallest Grammar: $0 \to 1 \to 2 \to 3 \to 4$

> *Before addition there is counting, and before counting there is a single move: go to the next one.*

LEARNING OBJECTIVES

- Read $0 \to 1 \to 2 \to 3 \to 4$ as four applications of one move in a transition system, and state the structural grammar that generates the natural numbers.
- Explain why the converse move (predecessor) is partial, and why that is the first boundary of arithmetic.
- Give the segment notation $0\{1, 2, 3\}4$ a precise syntax and semantics, decide what it adds and what it does not, and use it to reason about off-by-one errors.
- Show that fixed-width machine integers violate exactly one Peano axiom, and identify which.

3.1 The structural grammar of $\mathbb{N}$

The natural numbers are generated by a grammar with one constant and one unary operation:

$$\mathbb{N} \;\to\; 0 \;\mid\; S(\mathbb{N}). \tag{3.1}$$

Its terms are 0, $S(0)$, $S(S(0))$, $S(S(S(0)))$, and so on. By Dedekind's theorem (GIN-HIST-002) there is, up to unique isomorphism, exactly one structure in which these terms denote distinct elements and every element is denoted by one of them. We write 1 for $S(0)$, 2 for $S(S(0))$, and so on; the digits are abbreviations, not new objects.

DEFINITION 3.1 (Successor grammar) DEFINITION GIN-DEF-009

The **successor grammar** is the structural grammar (3.1). Its terms are the *successor terms* $S^n(0)$. Its unique model is the successor algebra of Definition GIN-DEF-001. A successor term is a numeral in *unary*: one symbol `S` per unit, followed by `0`.

Read as a *transition system*, the same grammar has states and moves.

DEFINITION 3.2 (Successor transition system) DEFINITION GIN-DEF-011

$T_{\mathbb{N}}$ is the labelled transition system whose states are the natural numbers, with the move S (defined at every state, $n \mapsto n+1$) and its converse P (defined at every state except 0, $n+1 \mapsto n$). The **boundary** of a set of states X is the set of $x \in X$ at which S or P leads out of X or is undefined.

In this reading the familiar progression

$$0 \xrightarrow{S} 1 \xrightarrow{S} 2 \xrightarrow{S} 3 \xrightarrow{S} 4$$

is four applications of a single move. Three properties make $T_{\mathbb{N}}$ the smallest interesting laboratory in this book.

1. **Every state is reached from** 0 **by exactly one word.** The word is S^n. The unary numeral of n is therefore at once its *name*, its *address* (the path from the root) and the *program* that computes it. In richer grammars these three roles separate; here they coincide.
2. **The converse is partial.** P is undefined at 0, because 0 is not a successor. This is the first boundary of arithmetic, and it is precisely the boundary that the subtraction $0 - 1$ meets in $\mathbb{N}$ (Chapter 12).
3. **Adding one *is* a transition.** By the primitive-recursive definition of addition (Chapter 4), $m + 1 = S(m)$ and $m + k$ is k applications of S. The expression $n + 1$ is not merely notation for "the next number"; it is one step of the grammar.

3.2 The partial converse and the first boundary

Why is P undefined at 0? Because the equation $S(c) = 0$ has no solution in $\mathbb{N}$: the axiom $0 \notin S(\mathbb{N})$ says so. And why is P a *function* where it is defined? Because S is injective: $S(c) = S(c')$ implies $c = c'$. These two properties — no solution at 0, at most one solution everywhere — are the first

instance of a pattern that organizes the whole of Part VIII. A converse operation is admissible exactly where its defining equation has one and only one solution.

STATE X	S DEFINED?	P DEFINED?	EQUATION FOR $P(X)$	SOLUTIONS
0	yes, $S(0) = 1$	no	$S(c) = 0$	none
1	yes	yes, $P(1) = 0$	$S(c) = 1$	exactly $c = 0$
$n \geq 1$	yes	yes	$S(c) = n$	exactly $c = n - 1$

There is no state at which P has two solutions. The second kind of failure — *too many* solutions — does not occur in the successor grammar; it first appears with multiplication, at $0 \cdot c = 0$.

3.3 The segment notation $a\{I\}b$

In the research mandate for this book, Abed Kadan proposed writing the progression from 0 to 4 as

$$0\{1, 2, 3\}4,$$

with the endpoints outside the braces and the intermediate states inside, and asked whether the notation could be given a precise, useful meaning. A notation should not become foundational merely because it is suggestive; it must first have a syntax, an interpretation, and a purpose. We give all three, and then decide what it adds.

DEFINITION 3.3

For integers $a < b$, the **segment** $[a, b]$ of $T_{\mathbb{N}}$ (or of its integer analogue $T_{\mathbb{Z}}$) is the transition system on $\{a, \ldots, b\}$ with S defined on $a, \ldots, b-1$ and P defined on $a+1, \ldots, b$. Its **boundary** is $\{a, b\}$ and its **interior** is $\{a+1, \ldots, b-1\}$.

The string $a\{i_1, \ldots, i_k\}b$ is **well formed** iff $a < b$ and $(i_1, \ldots, i_k) = (a+1, \ldots, b-1)$; it then **denotes** the segment $[a, b]$. Thus `0{1,2,3}4` denotes $[0, 4]$ and `3{}4` denotes the segment with empty interior; `0{1,3}4`, `0{3,2,1}4` and `4{}4` are ill formed.

Notice what the definition does: it makes the notation *redundant*. The interior is determined by the endpoints, so writing it out carries no information — *except* the information needed to check it. That redundancy is the notation's main formal property, and it is a useful one. A redundant

notation can be validated, and an invalid instance can be *diagnosed*: not merely rejected, but told which condition it violates. The SDK implements exactly this.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.peano import parse_segment

for s in ["0{1,2,3}4", "3{}4", "0{1,3}4", "0{3,2,1}4", "4{}4", "-2{-1,0,1}2", "0{1,2,3,4,5}4"]:
    p = parse_segment(s)
    print(f"{s:16s} {'well formed' if p.ok else 'ill formed '}  {p.diagnosis}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0{1,2,3}4        well formed  segment from 0 to 4: 4 successor transitions, interior of size 3
3{}4             well formed  segment from 3 to 4: 1 successor transition, interior of size 0
0{1,3}4          ill formed   interior: interior misses [2]
0{3,2,1}4        ill formed   order: interior must be listed in successor order
4{}4             ill formed   boundary: need a < b, got 4 ≥ 4
-2{-1,0,1}2      well formed  segment from -2 to 2: 4 successor transitions, interior of size 3
0{1,2,3,4,5}4    ill formed   interior: [4, 5] lie outside the open interval (0, 4)
```

3.3.1 What the notation displays

The notation separates, visually, the two kinds of state in a segment. A boundary state has *exactly one* admissible move inside the segment: at a only S (going down would leave), at b only P. An interior state has *both*. So $0\{1,2,3\}4$ is a picture of an admissibility pattern: outside the braces, one move; inside, two.

LABORATORY (WEBSITE EDITION)

Text alternative: the laboratory writes a natural number in unary, bijective base 2, binary, ternary, balanced ternary, decimal and hexadecimal. For $n = 4$: `SSSS0` , `12` , `100` , `11` , `11` , `4` , `4` . The value is the invariant; the words differ.

3.3.2 Three counts, and the fencepost

A segment has three natural sizes, and confusing them is one of the oldest errors in programming:

$$\#\text{elements of } [a,b] = b - a + 1, \qquad \#\text{transitions} = b - a, \qquad \#\text{interior} = b - a - 1.$$

For $0\{1,2,3\}4$ these are 5, 4 and 3. A fence 4 metres long with a post every metre has 5 posts and 4 gaps; a loop `for i in range(a, b)` executes $b - a$ times and never visits b. The segment notation

writes all three counts on the page: the elements are everything written, the interior is what is in braces, and the transitions are the commas plus the two braces. The off-by-one error is, in this vocabulary, a confusion of the boundary with the interior.

3.3.3 A calculus of segments

The research ledger asked whether the notation supports a *calculus* — operations on segments with laws — rather than display only (GIN-OPEN-008). It does, and the calculus turns out to be a classical one.

PROPOSITION 3.4 (Gluing segments; the boundary of a glued segment)

PROVED HERE GIN-PROP-051

Define the partial operation ***gluing****: if* $\sigma = a\{I\}b$ *and* $\tau = b\{J\}c$ *share the endpoint* b*, then* $\sigma \oplus \tau = a\{I, b, J\}c$*; otherwise* $\sigma \oplus \tau$ *is undefined. Then:*

1. $\sigma \oplus \tau$ *is well formed whenever* σ *and* τ *are, and gluing is associative wherever defined.*
2. *The boundary map* $\partial(a\{I\}b) = \{a, b\}$ *satisfies* $\partial(\sigma \oplus \tau) = \partial\sigma \triangle \partial\tau$*, the symmetric difference.*
3. *Every well-formed segment is, uniquely, a glued sequence of segments with empty interior:* $a\{a+1, \ldots, b-1\}b = a\{\}(a+1) \oplus \cdots \oplus (b-1)\{\}b$.

Proof.

(1) The interior I, b, J is $(a+1, \ldots, b-1, b, b+1, \ldots, c-1)$, which is exactly the interior required for endpoints $a < c$. Both ways of gluing $\rho \oplus \sigma \oplus \tau$ give the interior of the outer endpoints, so they coincide. (2) $\{a, b\} \triangle \{b, c\} = \{a, c\}$ because b appears in both and $a \neq c$. (3) Induction on $b - a$, using (1). ∎

REMARK

Proposition 3.4 is the one-dimensional case of a classical construction. Segments with empty interior are the *edges* of the path graph; glued sequences are paths; and the boundary rule $\partial(\sigma \oplus \tau) = \partial\sigma \triangle \partial\tau$ is the boundary operator on 1-chains with coefficients in $\mathbb{Z}/2$, where the shared endpoint cancels because $1 + 1 = 0$. The calculus therefore exists, but it is the calculus of paths and chains, known for a century. This is the verdict recorded in the ledger as GIN-REI-010: the notation is *formal* — checkable, diagnosable, composable — and adds *no new mathematics*.

CLAIM BOUNDARY

The segment notation is used in this book for display and validation only. No theorem depends on it. Proposals to read more into it — for instance, as a new number system or a new kind of object — are not supported by anything established here.

3.4 Counting

To *count* a finite set A is to establish a bijection between A and an initial segment of the successor chain. Two conventions are in common use, and they differ by exactly the fencepost of §3.3:

- counting **from** 1: a bijection $A \to \{1, \ldots, n\}$, the segment $1\{\ldots\}n$; the last number reached *is* the count n;
- counting **from** 0 (indices, offsets): a bijection $A \to \{0, \ldots, n-1\}$; the count is the first number *not* reached.

The first convention identifies the *last position* with the *size*; the second identifies *position* with *distance from the start*. Positional notation, memory addresses and most programming languages use the second, because it makes the address of the i-th element base $+ i \cdot$ size with no correction term. Mathematics uses both. Neither is more correct; they are two numerations of the same segment, one shifted by one, and mixing them is the fencepost error.

That counting produces a *well-defined* number — that a set cannot be counted to 3 one way and 4 another — is the pigeonhole principle: there is no bijection between $\{1, \ldots, m\}$ and $\{1, \ldots, n\}$ for $m \neq n$. It is proved by induction on m, using the injectivity of S. Counting is thus the first place where the axioms of the successor grammar do real work.

3.5 The successor on a machine

A processor's increment instruction implements the successor on w-bit words. It does so on a finite set, and a finite set cannot satisfy all of Dedekind's conditions. Which one fails?

PROPOSITION 3.5 (Fixed-width successor violates exactly one Peano axiom)

PROVED HERE GIN-PROP-052

Let $W = \{0, 1, \ldots, 2^w - 1\}$ and $s(x) = (x + 1) \bmod 2^w$, the unsigned increment. Then s is injective, every subset of W containing 0 and closed under s is all of W, and $0 = s(2^w - 1)$. Thus $(W, 0, s)$ satisfies injectivity and induction but violates "0 is not a successor". The transition system is a single cycle of length 2^w: the predecessor is total, and there is no boundary.

Proof.

s is a bijection of the finite set W (its inverse is $x \mapsto (x - 1) \bmod 2^w$), hence injective. Starting from 0, the iterates $0, 1, \ldots, 2^w - 1$ are all distinct and exhaust W, so any set containing 0 and closed under s contains all of W. Finally $s(2^w - 1) = 2^w \bmod 2^w = 0$. ∎

So the machine successor is not a broken version of the mathematical one; it is the successor of a *different* structure, the cyclic group $\mathbb{Z}/2^w$, in which the boundary at 0 has been removed by joining the top of the chain to the bottom. The signed reading changes the labels, not the cycle: the state after $2^{31} - 1$ is labelled -2^{31}. Every "overflow" of a fixed-width counter is a step across the seam of this cycle.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
w = 4
s = lambda x: (x + 1) % 2 ** w
cycle, x = [0], s(0)
while x != 0:
    cycle.append(x)
    x = s(x)
signed = lambda x: x - 2 ** w if x >= 2 ** (w - 1) else x
print("unsigned:", cycle, "→ back to 0")
print("signed:  ", [signed(x) for x in cycle])
print("predecessor of 0:", (0 - 1) % 2 ** w, "   successor of 7 (signed):", signed(s(7)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
unsigned: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15] → back to 0
signed:   [0, 1, 2, 3, 4, 5, 6, 7, -8, -7, -6, -5, -4, -3, -2, -1]
predecessor of 0: 15    successor of 7 (signed): -8
```

EXERCISE 3.1

In $T_{\mathbb{N}}$, how many words over $\{S, P\}$ of length 4 lead from state 0 to state 2 without ever applying an undefined move? List them.

▶ Solution to Exercise 3.1

EXERCISE 3.2

Is `-1{0}1` well formed? What does it denote, and what are its three counts? Glue it with `1{2,3}4` and give the boundary of the result.

▶ Solution to Exercise 3.2

EXERCISE 3.3

A program processes the elements with indices $3, 4, \ldots, 17$ inclusive. Write the range as a segment, give its three counts, and write the Python `range` that visits exactly these indices.

▶ Solution to Exercise 3.3

EXERCISE 3.4

Find a structure $(X, 0, s)$ that satisfies "s is injective" and "$0 \notin s(X)$" but violates induction. What does counting from 0 fail to reach?

▶ Solution to Exercise 3.4

EXERCISE 3.5

For $w = 8$, how many applications of the signed increment take 100 to -100? Express the general answer for w bits.

▶ Solution to Exercise 3.5

SUMMARY

- $\mathbb{N}$ is generated by $\mathbb{N} \to 0 \mid S(\mathbb{N})$; read as a transition system, $0 \to 1 \to 2 \to 3 \to 4$ is four applications of one move.
- The predecessor is the converse of the successor: undefined at 0 (no solution), unique elsewhere. This is the first boundary of arithmetic.
- The segment notation $a\{I\}b$ has a precise syntax and semantics; its redundancy makes it checkable and diagnosable; its gluing calculus is the classical calculus of paths and chains. It adds no new mathematics.
- Fixed-width machine integers satisfy injectivity and induction but not "0 is not a successor": the chain becomes a cycle, and overflow is a step across its seam.

Chapter 4

Recursion, Induction, and the Cost of Counting

> *To define a function on the natural numbers, say what it does at 0 and what one more step does. To prove a property, say why it holds at 0 and why one more step preserves it.*

LEARNING OBJECTIVES

- State the induction principle and the recursion theorem, and explain why they are two consequences of the same axiom.
- Define addition, multiplication and exponentiation by primitive recursion, and compute $1 + 1 = 2$ as a two-step derivation.
- Prove commutativity and associativity of addition from the recursive definitions.
- Count the rule applications of Peano arithmetic exactly, and explain why unary arithmetic is exponential in the length of a binary numeral.
- Read a number as an iterator (a Church numeral), and the set $\mathbb{N}$ as a least fixed point.

4.1 Induction and recursion

The last clause of the definition of $\mathbb{N}$ (Definition GIN-DEF-001) says that the only subset of $\mathbb{N}$ containing 0 and closed under S is $\mathbb{N}$ itself. Read as a principle of proof, this is **induction**: to show that every natural number has a property Φ, show $\Phi(0)$, and show that $\Phi(n)$ implies $\Phi(Sn)$; then the set of numbers with the property contains 0 and is closed under S, so it is everything.

Read as a principle of *definition*, the same clause gives recursion.

THEOREM 4.1 (The recursion theorem) CLASSICAL RESULT GIN-HIST-003

Let X be a set, $x_0 \in X$ and $g : \mathbb{N} \times X \to X$. There is exactly one function $f : \mathbb{N} \to X$ with

$$f(0) = x_0, \qquad f(Sn) = g(n, f(n)) \quad \text{for all } n. \tag{4.1}$$

Proof.

Uniqueness. If f and f' both satisfy (4.1), the set where they agree contains 0 and is closed under S; by induction it is $\mathbb{N}$. *Existence.* Call a relation $R \subseteq \mathbb{N} \times X$ *admissible* if $(0, x_0) \in R$ and $(n, y) \in R$ implies $(Sn, g(n, y)) \in R$. The intersection F of all admissible relations is admissible. One shows by induction on n that for each n there is exactly one y with $(n, y) \in F$ (if there were two values at Sn, removing the one not of the form $g(n, y)$ would leave an admissible relation, contradicting minimality; injectivity of S and $0 \notin S(\mathbb{N})$ ensure that the clauses never collide). So F is the graph of a function, and it satisfies (4.1). ∎

The proof uses all three axioms: induction (for uniqueness and the final argument), injectivity of S and $0 \notin S(\mathbb{N})$ (so that the two defining clauses never assign two values to the same argument). The fixed-width structure of Proposition GIN-PROP-052 fails the last axiom, and recursion fails with it: on a cycle of length 2^w, the "definition" $f(0) = 0$, $f(Sn) = f(n) + 1$ would require $f(0) = f(S(2^w - 1)) = 2^w$, a contradiction. One cannot define functions by counting up on a machine word and expect them to be well defined across the seam.

4.2 Arithmetic by primitive recursion

DEFINITION 4.2 (Primitive recursion on the successor grammar) DEFINITION

GIN-DEF-010

$$\begin{aligned} \mathrm{add}(m, 0) &= m, & \mathrm{add}(m, Sn) &= S(\mathrm{add}(m, n)), \\ \mathrm{mul}(m, 0) &= 0, & \mathrm{mul}(m, Sn) &= \mathrm{add}(\mathrm{mul}(m, n), m), \\ \mathrm{pow}(m, 0) &= S0, & \mathrm{pow}(m, Sn) &= \mathrm{mul}(\mathrm{pow}(m, n), m), \\ \mathrm{pred}(0) &= 0, & \mathrm{pred}(Sn) &= n, \\ \mathrm{monus}(m, 0) &= m, & \mathrm{monus}(m, Sn) &= \mathrm{pred}(\mathrm{monus}(m, n)). \end{aligned}$$

Each is an instance of the recursion theorem in the second argument. The **cost** of a computation is the number of equations applied (rule applications).

With these definitions, $1 + 1 = 2$ is not a fact to be memorized but a derivation of two steps:

$$\mathrm{add}(S0, S0) \;=\; S(\mathrm{add}(S0, 0)) \;=\; S(S0).$$

The SDK performs such derivations by literal term rewriting, and records each rule it uses:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import peano as P

nf, trace = P.rewrite(("add", P.num(1), P.num(1)))
for rule, term in trace:
    print(f"{rule:30s} → {term}")
nf, trace = P.rewrite(("mul", P.num(2), P.num(3)))
print("2 x 3:", P.show(nf), "after", len(trace), "rule applications")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
add(m, S n) = S(add(m, n))     → S(add(S0, 0))
add(m, 0) = m                  → SS0
2 x 3: SSSSSS0 after 13 rule applications
```

Note that the definition of pred sets $\mathrm{pred}(0) = 0$. This is not a solution of $S(c) = 0$ — there is none — but a *choice*: it makes pred total. The same choice propagates into monus, so that $\mathrm{monus}(0, 1) = 0$ although no c satisfies $1 + c = 0$. Chapter 13 calls this a *totalization* and compares it with the other ways of making a partial operation total.

4.3 The laws, proved from the definitions

Everybody knows that $a + b = b + a$. In the grammar of primitive recursion it is not obvious at all: $\mathrm{add}(m, n)$ recurses on its *second* argument, and the two sides of $a + b = b + a$ are computed by different derivations of different lengths. The law is a theorem about the *values* of the derivations.

PROPOSITION 4.3 (Associativity and commutativity of addition) CLASSICAL RESULT

GIN-HIST-004

For all $a, b, c \in \mathbb{N}$: (i) $0 + a = a$; (ii) $Sa + b = S(a + b)$; (iii) $a + b = b + a$; (iv) $(a + b) + c = a + (b + c)$.

Proof.

(i) Induction on a: $0 + 0 = 0$ by the first equation; if $0 + a = a$ then $0 + Sa = S(0 + a) = Sa$. (ii) Induction on b: $Sa + 0 = Sa = S(a + 0)$; if $Sa + b = S(a + b)$ then $Sa + Sb = S(Sa + b) = S(S(a + b)) = S(a + Sb)$. (iii) Induction on b: $a + 0 = a = 0 + a$ by (i); if $a + b = b + a$ then $a + Sb = S(a + b) = S(b + a) = Sb + a$ by (ii). (iv) Induction on c: $(a + b) + 0 = a + b = a + (b + 0)$; and $(a + b) + Sc = S((a + b) + c) = S(a + (b + c)) = a + S(b + c) = a + (b + Sc)$. ∎

The proofs are short, but each uses the *structure* of the grammar — what happens at 0 and what one more step does — and not any picture of objects being combined. The corresponding laws for

multiplication, and distributivity, are proved the same way (Exercise 4.1).

> **REMARK**
>
> The proofs show something the notation hides: *commutativity is not computational symmetry*. The derivation of $2 + 3$ applies four rules and that of $3 + 2$ three; they reach the same normal form. The equation $2 + 3 = 3 + 2$ asserts that two different computations have the same result. This is the first instance of a theme of the book: an algebraic law is a statement that different evaluation paths agree, and the cost of the paths may differ. Chapter 37 measures how much.

4.4 The cost of counting

How expensive is arithmetic in this grammar?

> **PROPOSITION 4.4** (The cost of Peano arithmetic) PROVED HERE GIN-PROP-003
>
> *Under the definitions of GIN-DEF-010,* $\mathrm{add}(m, n)$ *uses exactly* $n + 1$ *rule applications and* $\mathrm{mul}(m, n)$ *uses exactly* $mn + 2n + 1$*. Hence on inputs whose binary numerals have* k *bits, unary addition costs* $\Theta(2^k)$ *rule applications, against* $\Theta(k)$ *bit operations for binary addition: it is exponential in the length of the numeral.*

Proof.

Addition peels one S from the second argument per step and finishes with the base equation: $n + 1$ steps. Multiplication: $\mathrm{mul}(m, 0)$ costs 1; $\mathrm{mul}(m, Sn)$ costs one rule, then $\mathrm{mul}(m, n)$, then an addition with second argument m, which costs $m + 1$. By induction the total is $1 + (mn + 2n + 1) + (m + 1) = m(n + 1) + 2(n + 1) + 1$. ∎

The derivations of the SDK confirm the formula — not by evaluating it, but by counting the steps of actual rewriting, for all $m, n < 7$ in the SDK's tests and on a larger range here:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import peano as P

bad = 0
for m in range(12):
    for n in range(12):
        _, trace = P.rewrite(("mul", P.num(m), P.num(n)))
        bad += len(trace) != m * n + 2 * n + 1
print("pairs checked:", 144, " mismatches:", bad)
for k in (4, 8, 12, 16, 20):
    n = 2 ** k - 1
```

```
    print(f"k = {k:2d} bits: unary addition of n = {n:>7d} takes {n + 1:>7d} rules; binary addition ~ {k + 1}
bit steps")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
pairs checked: 144  mismatches: 0
k =  4 bits: unary addition of n =      15 takes      16 rules; binary addition ~ 5 bit steps
k =  8 bits: unary addition of n =     255 takes     256 rules; binary addition ~ 9 bit steps
k = 12 bits: unary addition of n =    4095 takes    4096 rules; binary addition ~ 13 bit steps
k = 16 bits: unary addition of n =   65535 takes   65536 rules; binary addition ~ 17 bit steps
k = 20 bits: unary addition of n = 1048575 takes 1048576 rules; binary addition ~ 21 bit steps
```

IMPLEMENTATION RESULT 4.5 (Rule counts witnessed by rewriting)

IMPLEMENTATION RESULT GIN-IMP-004

`ginsdk.peano.rewrite` normalizes `add` and `mul` terms by applying the defining equations one at a time, leftmost-outermost. On all $m, n < 12$ the number of rewrite steps equals the counts of Proposition 4.4, and the normal form is the successor term of the exact result.

This is the quantitative reason why no one computes in unary, and the reason positional notation exists. It is also a warning about cost statements: "addition costs one step" is true of the move S, true of a machine `add` on bounded words, and false — by an exponential factor — of addition on unary numerals of unbounded size. A cost is a property of a *representation* and a *model*, never of an operation alone (Chapter 36).

4.5 Numbers as iterators

There is a reading of numbers in which they are not data at all but *programs*. The number n is the operation "do it n times".

DEFINITION 4.6 (Church numeral) DEFINITION GIN-DEF-013

The **Church numeral** of n is the higher-order function $\underline{n} = (f \mapsto f^n)$, which maps any function $f : X \to X$ to its n-fold composite [Church 1941]. In particular $\underline{0} = (f \mapsto \mathrm{id})$ and $\underline{Sn} = (f \mapsto f \circ \underline{n}(f))$.

The successor grammar read as a program *is* the Church numeral: $S^n(0)$ is $\underline{n}$ applied to S and 0. Addition is composition of iterations, $\underline{m+n}(f) = \underline{m}(f) \circ \underline{n}(f)$, and multiplication is iteration of iterations, $\underline{m \cdot n}(f) = \underline{m}(\underline{n}(f))$. These identities say that $\mathbb{N}$ acts on every set by iteration — a fact

that will matter when we treat exponents, which are *not* elements of the domain they act on (Chapter 34).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.peano import church, unchurch

two, three = church(2), church(3)
double = lambda x: 2 * x
print("3 applied to doubling, from 1:", three(double)(1))
add = lambda m, n: (lambda f: (lambda x: m(f)(n(f)(x))))
mul = lambda m, n: (lambda f: m(n(f)))
print("2 + 3 =", unchurch(add(two, three)), "   2 * 3 =", unchurch(mul(two, three)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
3 applied to doubling, from 1: 8
2 + 3 = 5    2 * 3 = 6
```

4.6 $\mathbb{N}$ as a least fixed point

The induction axiom has a third reading. Consider the operator on sets $\Phi(X) = \{0\} \cup S(X)$. A set X is *closed* under the grammar if $\Phi(X) \subseteq X$. The natural numbers are the **least** closed set: they are contained in every closed set, and they are closed themselves. In the language of order theory, $\mathbb{N}$ is the least fixed point of Φ, and the Knaster–Tarski theorem guarantees that such a least fixed point exists for every monotone operator on a complete lattice of sets.

The phrase “fixed point” will appear in this book in three technically different senses, and it is worth separating them now (GIN-REI-007):

SENSE	EXAMPLE	WHAT IS COMPUTED
an element with $f(x) = x$	$x^2 = x$ has solutions 0 and 1 in $\mathbb{Z}$; four in $\mathbb{Z}/6$	the solution of an equation (a converse problem)
the limit of an iteration	Newton’s $x \mapsto (x + a/x)/2$ converges to $\sqrt{a}$	existence, uniqueness, rate, stopping rule
a least fixed point of a set operator	$\mathbb{N}$ is the least X with $\{0\} \cup S(X) \subseteq X$	induction and recursion; the grammar itself

They share a definition and nothing else: knowing how to find a least fixed point does not help to find the root of an equation, and conversely.

4.7 Finite sequences and the free monoid

A unary numeral $S^n 0$ is, after deleting the final 0, a word S^n over a one-letter alphabet, and addition is concatenation: $S^m S^n = S^{m+n}$. More generally, the words over an alphabet Σ form a monoid under concatenation, with the empty word ε as identity, and the *length* map $|\cdot|$ is a homomorphism to $(\mathbb{N}, +, 0)$: $|uv| = |u| + |v|$, $|\varepsilon| = 0$. For a one-letter alphabet the length map is an isomorphism. In algebraic terms, $(\mathbb{N}, +, 0)$ is the free monoid on one generator; the generator is 1, and every element is uniquely a sum $1 + 1 + \cdots + 1$. This is why, in the next part of the book, numerals with *more* than one letter can compress numbers: a richer alphabet breaks the one-to-one correspondence between length and value.

EXERCISE 4.1

Prove from GIN-DEF-010 and Proposition 4.3 that $a \cdot (b + c) = a \cdot b + a \cdot c$ (induction on c).

▶ Solution to Exercise 4.1

EXERCISE 4.2

Find the exact number of rule applications of $\mathrm{pow}(m, n)$ for $m = 2$ and $n = 0, 1, 2, 3$ by hand, and check your answer with `ginsdk.peano.power` and a `Cost` object.

▶ Solution to Exercise 4.2

EXERCISE 4.3

Show directly that there is no function $f : \mathbb{Z}/4 \to \mathbb{N}$ with $f(0) = 0$ and $f(x + 1) = f(x) + 1$ for all $x \in \mathbb{Z}/4$.

▶ Solution to Exercise 4.3

EXERCISE 4.4

Show that applying the Church numeral $\underline{n}$ to the Church numeral $\underline{m}$ (as a function on functions) gives $\underline{m^n}$. What does this say about exponentiation as an operation?

▶ Solution to Exercise 4.4

EXERCISE 4.5

How many rule applications does $\mathrm{add}(a, b)$ use, and how many $\mathrm{add}(b, a)$? For which pairs are they equal?

▶ Solution to Exercise 4.5

SUMMARY

- Induction and recursion are two readings of the same axiom; recursion needs all three Peano axioms, and fails on machine words.
- Arithmetic is defined by primitive recursion; $1 + 1 = 2$ is a two-step derivation, executed by literal rewriting in the SDK.
- The algebraic laws are theorems about derivations with different costs.
- Unary arithmetic costs time proportional to the values — exponential in the length of a binary numeral; costs belong to representations.
- A number can be read as an iterator, and $\mathbb{N}$ as a least fixed point; "fixed point" has three distinct senses.

PART III

Digits, Numerals, and Positional Representation

Numerals as programs: place value, canonical forms, zero's empty address, other digit grammars, and why $1 + 1 = 10_2$ says nothing new about the value.

Chapter 5

Positional Notation as an Operational Grammar

> *A numeral is a program. Its digits are instructions; its value is what the program computes when started at zero.*

LEARNING OBJECTIVES

- Define a digit grammar and the value of a numeral, and evaluate a numeral by Horner's rule.
- Prove that two standard numerals have the same value exactly when they differ by leading zeros, and count the words that denote a given number.
- Explain why the canonical numeral of zero is the empty word and what the written `0` therefore is.
- Relate the length of a numeral to the size of its value: $\lfloor \log_b n \rfloor + 1$ digits.
- Compile a numeral to an affine map and use the concatenation law $\mathrm{val}(uv) = \mathrm{val}(u)\, b^{|v|} + \mathrm{val}(v)$.

5.1 Digits as moves

The decimal numeral `427` is usually explained as $4 \cdot 100 + 2 \cdot 10 + 7$. There is another reading, which is the one this book uses: `427` is a *sequence of three instructions*, executed from left to right on a register that starts at zero. The instruction for a digit d is "multiply by ten and add d":

$$0 \xrightarrow{4} 4 \xrightarrow{2} 42 \xrightarrow{7} 427.$$

Each digit is a *move*, the numeral is a *word* of moves, and its value is the state reached from 0.

DEFINITION 5.1 (Digit grammar) DEFINITION GIN-DEF-005

A **digit grammar** is a pair (b, D) of a base $b \geq 1$ and a finite set $D \subset \mathbb{Z}$ of digits. Each digit $d \in D$ is a move on the integers,

$$[\![d]\!] : x \mapsto b \cdot x + d, \tag{5.1}$$

the start state is 0, and the admissible words are D^* (or a sublanguage of canonical words). **Standard base** b has $b \geq 2$ and $D = \{0, \ldots, b-1\}$. Bijective base b, balanced ternary and redundant systems are other choices of D (Chapter 6).

DEFINITION 5.2 (Value of a numeral) DEFINITION GIN-DEF-006

The **value** of a word $w = d_1 d_2 \cdots d_k$ (most significant digit first) is

$$\mathrm{val}(w) = [\![w]\!](0) = \sum_{i=1}^{k} d_i \, b^{k-i}, \tag{5.2}$$

where $[\![w]\!] = [\![d_k]\!] \circ \cdots \circ [\![d_1]\!]$. Evaluating $[\![w]\!]$ from left to right is **Horner's rule**.

The two expressions in (5.2) — the composite of moves and the sum of digit-times-power — are equal by an easy induction, and both are classical. The move reading, however, makes three things visible that the sum hides. It explains why the *last* digit of a numeral is the residue of the value modulo b (the last move adds it after the final multiplication). It explains why appending a digit multiplies the value by b and adds the digit — the basis of every conversion algorithm. And it turns a numeral into a program, so that questions about numerals become questions about programs: which programs compute the same value, which is shortest, and what does running one cost.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import standard

dec = standard(10)
w = dec.parse("427")
x = 0
for d in w:
    y = 10 * x + d
    print(f"digit {d}:  x = 10 * {x} + {d} = {y}")
    x = y
print("value:", dec.denote(w), "  last digit = value mod 10:", dec.denote(w) % 10 == w[-1])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
digit 4:  x = 10 * 0 + 4 = 4
digit 2:  x = 10 * 4 + 2 = 42
digit 7:  x = 10 * 42 + 7 = 427
value: 427   last digit = value mod 10: True
```

5.2 Many words, one value

The value map val goes from words to numbers. It is not injective: `7` , `07` and `007` all denote seven. Exactly how non-injective is it?

PROPOSITION 5.3 (Numeral ≠ number) CLASSICAL RESULT GIN-PROP-001

For standard base $b \geq 2$, the value map $\mathrm{val} : \{0, \ldots, b-1\}^* \to \mathbb{N}$ *is surjective, and* $\mathrm{val}(u) = \mathrm{val}(v)$ *if and only if u and v differ only by leading zeros. Consequently, for $n \geq 1$ with canonical numeral of length ℓ, exactly $L - \ell + 1$ words of length at most L denote n (for $L \geq \ell$).*

Proof.

Leading zeros do not change the value, because $[\![0]\!](0) = b \cdot 0 + 0 = 0$, so $[\![0w]\!](0) = [\![w]\!](0)$. Conversely, let u and v have no leading zeros and $\mathrm{val}(u) = \mathrm{val}(v) = n$. If $n = 0$ both are empty: a non-empty word without a leading zero has value at least $b^{|w|-1} \geq 1$. Otherwise $b^{|u|-1} \leq n < b^{|u|}$, and likewise for v, so $|u| = |v|$. The last digits of both are $n \bmod b$, and deleting them leaves two words of the same value $(n - n \bmod b)/b$; by induction on the length they are equal. Surjectivity is the **residue algorithm**: for $n > 0$ emit $d = n \bmod b$ and continue with $(n - d)/b < n$; it stops at 0 and, read backwards, gives a word of value n. The count: the words of length at most L denoting n are $0^j\,\mathrm{addr}(n)$ for $j = 0, \ldots, L - \ell$. ∎

Exhaustive enumeration agrees:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import standard

g = standard(3)
for n in (0, 1, 5, 9):
    words = g.words_denoting(n, 4)
    print(f"n = {n}: {len(words)} ternary words of length ⩽ 4:", ", ".join(g.render(w) for w in words))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n = 0: 5 ternary words of length ⩽ 4: ε, 0, 00, 000, 0000
n = 1: 4 ternary words of length ⩽ 4: 1, 01, 001, 0001
n = 5: 3 ternary words of length ⩽ 4: 12, 012, 0012
n = 9: 2 ternary words of length ⩽ 4: 100, 0100
```

5.3 Canonical numerals and zero's empty address

Among the many words that denote a number, one is singled out as *the* numeral.

DEFINITION 5.4 (Canonical numeral (address)) DEFINITION GIN-DEF-007

The **canonical numeral** $\mathrm{addr}(n)$ of a value n in a digit grammar is the shortlex-least word w with $[\![w]\!](0) = n$, when one exists. In standard base b it is the ordinary numeral without leading zeros.

The word *address* is deliberate. In Computational Grammar Theory, the address of an element of a structure is the shortlex-least word of moves that reaches it from the root (CGT-DEF-006); a canonical numeral is exactly the address of a number in the structure whose moves are the digits. The grammar of positional notation *is* an address grammar. For numbers, the founding observation of CGT — that an address can carry the operations needed to compute with the element it names — is the familiar fact that one computes with numbers by manipulating their numerals.

Apply Definition 5.4 to the number zero and something curious appears.

PROPOSITION 5.5 (Zero's canonical numeral is the empty word) REINTERPRETATION

GIN-PROP-002

In every digit grammar, $\mathrm{addr}(0) = \varepsilon$*. In standard base b the words denoting* 0 *are exactly* $\varepsilon, 0, 00, 000, \ldots$*, and the written numeral* `0` *is the shortest non-empty one — a conventional, non-canonical spelling of the empty address.*

Proof.

ε is the shortlex-least of all words and $[\![\varepsilon]\!](0) = 0$. The second claim is Proposition 5.3 with $n = 0$. ∎

Why, then, do we write `0`? Because an empty word cannot be seen. The zero digit has two distinct jobs in positional notation, and the grammar separates them:

1. a **placeholder inside** numerals: `101` and `11` are different numerals because the middle `0` performs a move — multiply by the base, add nothing;
2. a **visible spelling** of the empty address: the numeral `0` is "do nothing", written down.

Historically, the first job came before the second: late Babylonian sexagesimal texts used a placeholder sign inside numerals long before zero was treated as a number in its own right [Kaplan 1999]; zero as a number with its own arithmetic is documented in Brahmagupta's *Brāhmasphuṭasiddhānta* of 628 CE [O'Connor & Robertson (MacTutor)] (Chapter 20). The grammar shows why the two jobs could be separated: one is a move, the other is the absence of moves.

5.4 Length and size

A numeral is short when its value is small, and its length grows like the logarithm of its value.

PROPOSITION 5.6 (The length of a canonical numeral) CLASSICAL RESULT GIN-PROP-053

For $b \geq 2$ and $n \geq 1$, the canonical base-b numeral of n has exactly $\lfloor \log_b n \rfloor + 1$ digits. In particular a number with a k-digit binary numeral satisfies $2^{k-1} \leq n < 2^k$, and the binary numeral is $\log_2 10 \approx 3.32$ times as long as the decimal one, up to one digit.

Proof.

A canonical word of length $\ell \geq 1$ has a non-zero leading digit, so its value lies in $[b^{\ell-1}, b^{\ell})$, and every integer in that interval has such a word. Hence $\ell - 1 \leq \log_b n < \ell$, that is, $\ell = \lfloor \log_b n \rfloor + 1$. The ratio of lengths is $\log_2 n / \log_{10} n = \log_2 10$ up to the floors. ∎

The proposition is elementary, but it is the hinge on which every cost statement in this book turns. Algorithms on numbers take as input numerals, not numbers, and the *size of the input* is the length of the numeral: $\Theta(\log n)$, not n. An algorithm that takes n steps on input n — counting up, trial division to n, unary arithmetic — is *exponential* in its input size. This is why Chapter 4's unary addition was so expensive, and why, in Part XIII, "trial division is slow" is a precise statement.

5.5 Compiling a numeral

Since every digit move is an affine map $x \mapsto bx + d$, and affine maps are closed under composition, a whole numeral compiles to a single affine map.

PROPOSITION 5.7 (Compilation and the concatenation law) CLASSICAL RESULT

GIN-PROP-005

For a digit grammar (b, D) and words u, v:

$$[\![w]\!] = \big(x \mapsto b^{|w|}x + \mathrm{val}(w)\big), \qquad [\![uv]\!] = [\![v]\!] \circ [\![u]\!], \qquad \mathrm{val}(uv) = \mathrm{val}(u)\, b^{|v|} + \mathrm{val}(v). \tag{5.3}$$

The transition monoid of the grammar is $\{x \mapsto b^k x + c\}$.

Proof.

Induction on $|w|$, using $(x \mapsto bx + d) \circ (x \mapsto b^k x + c) = (x \mapsto b^{k+1}x + (bc + d))$. ∎

The concatenation law is used every day without being named: `42` followed by `7` is $42 \cdot 10 + 7$; `427` followed by `000` is $427 \cdot 10^3$. It is also the basis of fast conversion: to evaluate a long numeral, split it into halves, evaluate each, and combine with (5.3) — an idea measured in Chapter 37.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import standard

dec = standard(10)
f = dec.compile(dec.parse("427"))
print("427 compiles to", f, "   applied to 0:", f(0), "  applied to 1:", f(1))
u, v = dec.parse("42"), dec.parse("7")
print("val(42 . 7) =", dec.denote(u) * 10 ** len(v) + dec.denote(v))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
427 compiles to x ↦ 1000·x + 427    applied to 0: 427   applied to 1: 1427
val(42 . 7) = 427
```

Applying the compiled map to 1 instead of 0 gives 1427: running the program `427` after the program `1`. Numerals compose like programs because they are programs.

> **REMARK**
>
> CGT calls a structure *arithmetically compilable* when every word of moves compiles to an element of its transition monoid that fits in $\mathcal{O}(1)$ machine words (CGT-DEF-011). Positional numerals are *not* compilable in that sense for unbounded words: the compiled pair $(b^{|w|}, \mathrm{val}(w))$ has $\Theta(|w|)$ bits. The grammar compiles, but the compiled object grows with the program. Chapter 11 contrasts this with the carry automaton of binary addition, whose transition monoid has only three elements.

5.6 Conversion between bases

Since the value is the invariant and the numeral is a representation, converting a numeral from base b to base b' means: compute the value from the base-b word (Horner's rule in base b), then compute the base-b' word of the value (the residue algorithm in base b'). The SDK's `canonicalize` and `convert` do exactly this, and report leading zeros instead of silently dropping them:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import canonicalize, convert

print(canonicalize("000427"))
for text, b1, b2 in [("427", 10, 2), ("110101011", 2, 10), ("ff", 16, 2), ("0", 10, 2)]:
    print(f"{text} (base {b1}) = {convert(text, b1, b2)} (base {b2})")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
{'input': '000427', 'base': 10, 'value': 427, 'canonical': '427', 'address': '427', 'was_canonical': False,
'leading_zeros': 3, 'note': '3 leading zero(s) removed; leading zeros do not change the value (GIN-PROP-001)'}
427 (base 10) = 110101011 (base 2)
110101011 (base 2) = 427 (base 10)
ff (base 16) = 11111111 (base 2)
0 (base 10) = 0 (base 2)
```

For bases that are powers of a common base — 2, 8, 16 — conversion needs no arithmetic at all: each hexadecimal digit is exactly four binary digits. For other pairs, such as 10 and 2, it needs genuine multiplication and division, and its cost grows faster than linearly in the length of the numeral. Python, for instance, refuses by default to convert integers of more than 4300 decimal digits to strings, because the quadratic conversion was used for denial-of-service attacks (GIN-OBS-014). A change of representation is a computation, with a cost, and sometimes with a limit.

EXERCISE 5.1

Evaluate the base-7 numeral `2304` by Horner's rule, showing each state, and check the result against (5.2).

▶ Solution to Exercise 5.1

EXERCISE 5.2

How many binary words of length at most 10 denote the number 37? How many words of length at most 10 denote some number less than 37?

▶ Solution to Exercise 5.2

EXERCISE 5.3

How many decimal digits does 2^{100} have? How many binary digits does 10^{30} have?

▶ Solution to Exercise 5.3

EXERCISE 5.4

Using (5.3), show that a decimal numeral is divisible by 4 if and only if the number formed by its last two digits is.

▶ Solution to Exercise 5.4

EXERCISE 5.5

In the grammar of this chapter, what does the numeral `00` do as a program? What does it do when applied, after compilation, to the state 5? Relate the answer to the two jobs of the zero digit.

▶ Solution to Exercise 5.5

SUMMARY

- A numeral is a word of digit moves $x \mapsto bx + d$; its value is the state reached from 0, computed by Horner's rule.
- Two standard numerals have the same value iff they differ by leading zeros; the canonical numeral is the shortlex-least word — a grammatical address in the sense of CGT.
- Zero's address is the empty word; the written `0` is a convention. The zero digit is both a placeholder and a spelling of "no moves".
- A numeral of n has $\lfloor \log_b n \rfloor + 1$ digits, so the size of a number as input is logarithmic in its value.
- A numeral compiles to an affine map; concatenation of numerals is composition of maps.

Chapter 6

Other Digit Grammars

> *The base fixes how much one more digit is worth; the digit set fixes what can be said, and how many ways there are of saying it.*

LEARNING OBJECTIVES

- State when a digit grammar names every number, when it names each number exactly once, and how to compute numerals by the residue algorithm.
- Use bijective, balanced-ternary, negative-base and redundant numeration, and say what each gains and loses compared with standard notation.
- Prove that CGT's heap numeration of binary trees is bijective binary shifted by one.
- Read a positional numeral as a straight-line program that compresses a unary term, and relate it to exponentiation by squaring.

6.1 The residue algorithm, in general

Chapter 5 computed numerals by repeatedly taking the last digit as a residue. The same idea works for any digit set that contains one representative of each residue class modulo the base.

PROPOSITION 6.1 (Residue digit systems; bijective numeration) CLASSICAL RESULT

GIN-PROP-004

Let $b \geq 2$ and let D be a complete residue system modulo b (or $b = 1$ and $D = \{1\}$).

1. *If a word w denotes n, its last digit is the unique $d \in D$ with $d \equiv n \pmod{b}$, and its prefix denotes $(n - d)/b$. So the words denoting n are determined by the residue algorithm, up to prefixes that denote 0.*
2. *Standard digits $\{0, \ldots, b-1\}$ and balanced ternary ($b = 3$, $D = \{-1, 0, 1\}$): every $n \in \mathbb{N}$ (respectively every $n \in \mathbb{Z}$) has a canonical numeral, and the words denoting n are exactly $0^k \operatorname{addr}(n)$.*
3. *Bijective digits $D = \{1, \ldots, b\}$: the value map $D^* \to \mathbb{N}$ is a bijection.*

Proof.

(1) $\mathrm{val}(w'd) = b\ \mathrm{val}(w') + d$, so $d \equiv \mathrm{val}(w'd) \pmod b$, and D contains exactly one such d. (2) The residue algorithm terminates: for standard digits as in Chapter 5; for balanced ternary, $|(n - d)/3| \le (|n| + 1)/3 < |n|$ whenever $|n| \ge 1$. A prefix denoting 0 must end in the digit $\equiv 0$, which is 0, and so consists of zeros. (3) A non-empty word of positive digits has positive value, so only ε denotes 0; for $n \ge 1$ the step $n \mapsto (n - d)/b$ with $d \in \{1, \ldots, b\}$ strictly decreases n, so the algorithm terminates, and by (1) the word it produces is the only one. ∎

The hypothesis "complete residue system" guarantees existence of a last digit at every step; it does not guarantee *termination* or *uniqueness*.

COUNTEREXAMPLE 6.2 (A complete residue system need not give unique numerals)

COUNTEREXAMPLE GIN-NEG-014

For $b = 2$ and $D = \{-1, 2\}$ (a complete residue system modulo 2), the non-empty word $(-1)(2)$ denotes $2 \cdot (-1) + 2 = 0$. Prefixes denoting zero are then not just runs of a zero digit, and uniqueness fails.

6.2 Bijective numeration: no zero digit

Bijective base b uses the digits $1, \ldots, b$ and no zero. Its words are in one-to-one correspondence with the natural numbers: there are no leading zeros to strip, and no two words share a value. Bijective base 1 is unary, with the single digit "1", that is, the successor move. Bijective base 26 with digits `A` – `Z` is the column-naming scheme of spreadsheets: `A`, ..., `Z`, `AA`, `AB`, ..., where `Z` is the digit 26 and `AA` $= 26 \cdot 1 + 1 = 27$. The scheme was studied as "k-adic" notation by Smullyan [Smullyan 1961] and proposed as a practical alternative by Forslund [Forslund 1995].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import bijective

b2 = bijective(2)
print("n :", " ".join(f"{n:>4d}" for n in range(9)))
print("w :", " ".join(f"{b2.render(b2.address(n)):>4s}" for n in range(9)))
cols = bijective(26)
letters = {d: chr(64 + d) for d in range(1, 27)}
for n in (1, 26, 27, 52, 702, 703, 16384):
    print(f"spreadsheet column {n:>5d} = {''.join(letters[d] for d in cols.address(n))}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n :    0    1    2    3    4    5    6    7    8
w :    ε    1    2   11   12   21   22  111  112
spreadsheet column     1 = A
spreadsheet column    26 = Z
spreadsheet column    27 = AA
spreadsheet column    52 = AZ
spreadsheet column   702 = ZZ
spreadsheet column   703 = AAA
spreadsheet column 16384 = XFD
```

In bijective notation zero is written only as the empty word. The convention problem of Chapter 5 disappears: there is no `0` digit, so there is nothing to be mistaken for a spelling of the empty address. The price is that the familiar shortcuts change: in bijective base 10, the number ten is written with a single digit (the digit "ten", often written `A`), and multiplying by the base is no longer "append a zero".

6.3 Naming the negative numbers

Standard positional notation names only natural numbers; a minus sign is an extra symbol outside the digit grammar. Two classical digit grammars name every integer with digits alone.

Balanced ternary uses base 3 and digits $-1, 0, 1$, written `T`, `0`, `1`. By Proposition 6.1 every integer has a unique numeral without leading zeros. Negation is digit-wise: swap `1` and `T`. The Soviet Setun computer (1958) used balanced ternary arithmetic.

Base -2 ("negabinary") uses digits $0, 1$ and the negative base -2. The residue algorithm still applies — the last digit is $n \bmod 2$ and the prefix denotes $(n - d)/(-2)$ — and the magnitude of the state strictly decreases once $|n| \geq 2$, so every integer has a unique finite numeral [Knuth 1997].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.numerals import BALANCED_TERNARY as B3, NEGABINARY as NB

for n in (-6, -1, 0, 1, 5, 13):
    print(f"{n:>3d}:  balanced ternary {B3.render(B3.address(n)):>5s}    base -2
{NB.render(NB.address(n)):>6s}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
 -6:  balanced ternary   T10    base -2   1110
 -1:  balanced ternary     T    base -2     11
  0:  balanced ternary     ε    base -2      ε
  1:  balanced ternary     1    base -2      1
```

```
 5:  balanced ternary   1TT    base -2    101
13:  balanced ternary   111    base -2  11101
```

Ordinary binary, by contrast, cannot name -1 in finitely many digits. Running the residue algorithm on -1 gives the digit 1 and the new state $(-1-1)/2 = -1$: the algorithm never stops, and produces the infinite word $\ldots 1111$. Chapter 21 shows that this infinite word is the 2-adic expansion of -1, and that a computer's 32-bit representation of -1 is its last thirty-two digits (GIN-PROP-008).

6.4 Redundant digits

What if the digit set has *more* digits than residues? Take base 2 with digits $\{0, 1, 2\}$. Now $2 = 10_2 = 2_2$ (the one-digit word "2"), and $4 = 100 = 20 = 12$. Numerals are no longer unique, even without leading zeros. How many are there?

PROPOSITION 6.3 (Hyperbinary representations are counted by Stern's sequence)

REPRODUCED RESULT GIN-PROP-054

The number of words over $\{0, 1, 2\}$ without leading zero whose base-2 value is n equals $s(n+1)$, where s is Stern's diatomic sequence: $s(0) = 0$, $s(1) = 1$, $s(2k) = s(k)$, $s(2k+1) = s(k) + s(k+1)$. (Classical; see Calkin and Wilf [Calkin & Wilf 2000].) Reproduced here by exhaustive enumeration for $n < 60$.

Proof.

Let $h(n)$ be the count, with $h(0) = 1$ (the empty word). A word for $2k+1$ must end in the digit 1, and its prefix is any word for k: $h(2k+1) = h(k)$. A word for $2k+2$ ends in 0 (prefix for $k+1$) or in 2 (prefix for k): $h(2k+2) = h(k+1) + h(k)$. The sequence $t(n) = s(n+1)$ satisfies the same recurrences, $t(2k+1) = s(2k+2) = s(k+1) = t(k)$ and $t(2k+2) = s(2k+3) = s(k+1) + s(k+2) = t(k) + t(k+1)$, with $t(0) = 1$; so $h = t$. ∎

Redundancy is usually a defect in a notation. In arithmetic hardware it is a resource. With redundant digits, the sum of two numerals can be formed *without carry propagation*: each output digit depends only on a bounded window of input digits, because a redundant digit can absorb an incoming carry without passing it on. Signed-digit representations of this kind, introduced by Avizienis [Avizienis 1961], are the basis of fast multipliers and dividers; the carry-save representation used inside nearly every hardware multiplier is another example. Chapter 11 shows why ordinary binary addition *must* propagate carries, and Chapter 22 returns to redundant representations in circuits.

6.5 The heap of Computational Grammar Theory

Computational Grammar Theory numbers the nodes of a complete binary tree as in a binary heap: the root is 1, the left child of x is $2x$ and the right child is $2x+1$ (CGT-DEF-006). The address of a node is the word over $\{L, R\}$ leading to it from the root; the number of the node is computed from the address by the moves $L : y \mapsto 2y$ and $R : y \mapsto 2y+1$ — a digit grammar with base 2, digits $0, 1$, started at 1 instead of 0.

PROPOSITION 6.4 (CGT's heap numeration is bijective binary, shifted by one)

PROVED HERE GIN-PROP-006

Let $\varphi(x) = x + 1$. The bijective base-2 grammar (digits $1, 2$; moves $x \mapsto 2x+1$ and $x \mapsto 2x+2$; root 0) is conjugate by φ to the heap grammar (moves $L : y \mapsto 2y$, $R : y \mapsto 2y+1$; root 1):

$$\varphi \circ [\![1]\!] = L \circ \varphi, \qquad \varphi \circ [\![2]\!] = R \circ \varphi.$$

Consequently the bijective base-2 numeral of n, with $1 \mapsto L$ and $2 \mapsto R$, is the heap address of node $n+1$: the binary numeral of $n+1$ with its leading 1 removed.

Proof.

$(2x+1)+1 = 2(x+1)$ and $(2x+2)+1 = 2(x+1)+1$. By induction on words, $\varphi \circ [\![w]\!] = [\![h(w)]\!] \circ \varphi$ where h replaces 1 by L and 2 by R; evaluating at 0 gives $[\![w]\!](0) + 1 = [\![h(w)]\!](1)$. Both numerations are bijections from words to their sets of values (Proposition 6.1(3) for the first; uniqueness of tree paths for the second), so the words correspond. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk.numerals import bijective, heap_word, bijective2_as_heap

b2 = bijective(2)
for n in range(8):
    print(f"n = {n}:  bijective binary {b2.render(b2.address(n)):>4s}  →  heap address of node {n + 1}: "
          f"{heap_word(n + 1) or 'ε':>4s}   (binary of {n + 1} = {bin(n + 1)[2:]})")
assert all(bijective2_as_heap(n) == heap_word(n + 1) for n in range(100000))
print("checked n < 100000")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n = 0:  bijective binary    ε  →  heap address of node 1:    ε   (binary of 1 = 1)
n = 1:  bijective binary    1  →  heap address of node 2:    L   (binary of 2 = 10)
n = 2:  bijective binary    2  →  heap address of node 3:    R   (binary of 3 = 11)
n = 3:  bijective binary   11  →  heap address of node 4:   LL   (binary of 4 = 100)
n = 4:  bijective binary   12  →  heap address of node 5:   LR   (binary of 5 = 101)
n = 5:  bijective binary   21  →  heap address of node 6:   RL   (binary of 6 = 110)
```

```
n = 6:  bijective binary   22  →  heap address of node 7:   RR   (binary of 7 = 111)
n = 7:  bijective binary  111  →  heap address of node 8:  LLL   (binary of 8 = 1000)
checked n < 100000
```

The proposition is elementary and very likely folklore; the literature search recorded in the ledger did not find it stated in this form, and no priority is claimed. Its interest is as a bridge: the address insight that motivated CGT — an address carries operations, and a grammar of addresses can compute instead of store — is, for numbers, the observation that a numeral is a program.

6.6 Numerals as compression

The unary numeral of n has n symbols; the binary numeral has about $\log_2 n$. In what precise sense is the second a compressed form of the first?

A **straight-line program** (SLP) is a grammar in which every nonterminal has exactly one production and there is no recursion; it generates exactly one word. For example, $A_1 \to a$, $A_2 \to A_1A_1$, $A_3 \to A_2A_2$, $A_4 \to A_3A_1$ generates a^5 with four productions.

PROPOSITION 6.5

(Positional numerals are straight-line programs for unary terms)

REINTERPRETATION GIN-PROP-007

For $n \geq 1$ with a binary numeral of length k and ν ones, the unary word a^n has a straight-line program with $k + \nu - 1 \leq 2k - 1$ productions: $A \to a$ for the leading bit; then, for each later bit, a doubling production $A' \to AA$, followed for a bit 1 by $A'' \to A'a$. The program has exactly the shape of left-to-right binary exponentiation of x^n: $k - 1$ squarings and $\nu - 1$ multiplications (GIN-PROP-041). Conversely, an SLP of size m for a^n yields an addition chain for n of length at most m [Charikar et al. 2005; Knuth 1997].

Proof.

Horner's rule for the binary numeral maps the current length ℓ to $2\ell + \beta$ for each bit β. Doubling a length is concatenating a word with itself; adding one is appending a letter. Counting the productions gives $1 + (k - 1) + (\nu - 1)$. ∎

In CGT's vocabulary, this is the mechanism of *sharing* (M-SHARE, CGT-DEF-009) applied to the successor grammar: positional notation is exponential compression of unary numerals, achieved by reusing a subterm instead of repeating it. Three classical objects turn out to be one: the binary numeral of n, the square-and-multiply program for x^n, and a small grammar for a^n. The shortest

such programs are the *shortest addition chains*, whose length is not known in closed form; computing a shortest addition chain is a hard problem in its own right [Knuth 1997].

EXERCISE 6.1

Write 100 in bijective base 10 (use `A` for the digit ten). Why does it have only two digits?

▶ Solution to Exercise 6.1

EXERCISE 6.2

Convert -7 to base -2 by the residue algorithm, showing the states.

▶ Solution to Exercise 6.2

EXERCISE 6.3

List all hyperbinary representations of 6 and check the count against $s(7)$.

▶ Solution to Exercise 6.3

EXERCISE 6.4

Give the straight-line program of Proposition 6.5 for a^{13} and the corresponding square-and-multiply sequence for x^{13}.

▶ Solution to Exercise 6.4

EXERCISE 6.5

Which heap node has address `RLR`? What is its bijective base-2 numeral, shifted by one?

▶ Solution to Exercise 6.5

SUMMARY

- Complete residue digit sets let the residue algorithm compute numerals; termination and uniqueness need more (counterexample $D = \{-1, 2\}$).
- Bijective numeration has no zero digit and is a bijection from words to $\mathbb{N}$; balanced ternary and base -2 name every integer without a sign.
- Redundant digits make numerals non-unique — hyperbinary representations are counted by Stern's sequence — and make carry-free addition possible.
- CGT's heap numeration is bijective binary shifted by one: an address grammar is a numeral system.
- Positional notation is a straight-line-program compression of unary, with the shape of exponentiation by squaring.

Chapter 7

$1 + 1 = 10_2$: Value and Representation

> *1 + 1 = 2 and $1 + 1 = 10_2$ are the same equation. What differs is not the arithmetic but the alphabet in which its answer is spelled.*

LEARNING OBJECTIVES

- Parse $1 + 1 = 10_2$ precisely and explain why it states the same fact as $1 + 1 = 2$.
- Perform binary addition with carries, and count exactly the bit flips needed to count from 0 to N.
- Distinguish properties of a value ("is even", "is divisible by 3") from properties of a numeral ("ends in 0", "has digit sum 9"), and know when one can be read off the other.
- Build the finite automaton that decides divisibility by m from a base-b numeral.
- Explain why $1/3$ and $1/10$ have infinite expansions in some bases and finite ones in others.
- Compare representations — positional, prime-factorization, residue — by which operations they make cheap.

7.1 Reading the equation

The sentence $1 + 1 = 10_2$ has three parts. The left side, $1 + 1$, is an expression whose numerals are single digits; the digit 1 denotes the same value in every base $b \geq 2$, so the left side has no base to speak of. The right side, 10_2, is the base-2 numeral with digits $1, 0$, whose value is $1 \cdot 2 + 0 = 2$. The equals sign asserts that the two sides denote the same *value*. The sentence is therefore the claim

$$\mathrm{val}_{10}(1) + \mathrm{val}_{10}(1) = \mathrm{val}_2(10),$$

which is true because both sides equal the natural number 2. Nothing about addition changed when the base changed. What changed is the *word* used to spell the answer.

This is easy to say and easy to forget. “In binary, one plus one is ten” sounds like a different arithmetic; it is the same arithmetic in a different spelling, and “ten” there is a misreading of the word `10` as if it were decimal. The value is the invariant; the representation is the variable. This chapter is about what changes with the representation, and what does not.

7.2 Binary addition, and the carry it generates

Binary addition follows the same column algorithm as decimal addition, with a smaller table. For two bits and an incoming carry, the sum bit is their parity and the outgoing carry is 1 when at least two of the three are 1:

A_I	B_I	CARRY IN	SUM BIT	CARRY OUT
0	0	0	0	0
0	1	0	1	0
1	1	0	0	1
1	1	1	1	1

(The other four rows follow by symmetry.) The case $1 + 1$ is the third row: sum bit 0, carry 1. The answer *needs one more position* than either operand: `1` and `1` are one-digit words, `10` is a two-digit word. Nothing of the sort happens in base 10, where $1 + 1 = 2$ fits in one digit. *Whether an operation generates a carry depends on the base*, although the value does not.

The same phenomenon, iterated, governs counting. Adding 1 to 0111_2 turns three trailing ones into zeros and one zero into a one: four bits change to produce 1000_2. How much does counting cost in bit flips?

PROPOSITION 7.1 (The cost of the successor in binary) PROVED HERE GIN-PROP-010

Counting from 0 *to* N *in binary flips exactly* $2N - \mathrm{popcount}(N)$ *bits in total, where* $\mathrm{popcount}(N)$ *is the number of ones in the binary numeral of* N*. The successor therefore costs fewer than* 2 *bit flips on average, and* $\lfloor \log_2 N \rfloor + 1$ *in the worst case, at* $N = 2^k - 1 \to 2^k$*.*

Proof.

The increment $x \to x+1$ turns the $t(x)$ trailing ones of x into zeros and the zero above them into a one: it flips $t(x)+1$ bits and changes the popcount by $1 - t(x)$. Summing over $x = 0, \ldots, N-1$, the flips are $\sum(t(x)+1) = \sum\big(2 - (1 - t(x))\big) = 2N - (\text{popcount}(N) - \text{popcount}(0)) = 2N - \text{popcount}(N)$. ∎

The amortized bound is textbook material [Cormen et al. 2009]; the exact closed form is elementary. The SDK confirms it by counting:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
total, worst = 0, (0, 0)
for x in range(1 << 16):
    flips = bin(x ^ (x + 1)).count("1")
    total += flips
    worst = max(worst, (flips, x))
N = 1 << 16
print("flips counting 0 →", N, ":", total, "  formula 2N - popcount(N):", 2 * N - bin(N).count("1"))
print("average per increment:", total / N, "  worst:", worst[0], "flips at", worst[1], "→", worst[1] + 1)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
flips counting 0 → 65536 : 131071   formula 2N - popcount(N): 131071
average per increment: 1.9999847412109375   worst: 17 flips at 65535 → 65536
```

7.3 Properties of values, properties of numerals

Some properties belong to a value and can be stated without any numeral: "n is even", "n is prime", "n is a perfect square". Others belong to a numeral: "ends in the digit 0", "has digit sum 9", "is a palindrome". The two kinds are easily confused because, *in a fixed base*, some numeral properties detect value properties:

- in base 10, "ends in 0, 2, 4, 6 or 8" detects evenness, because 10 is even and the last digit is the residue modulo 10;
- in base 10, "digit sum divisible by 3" detects divisibility by 3, because $10 \equiv 1 \pmod 3$, so $\text{val}(d_1 \cdots d_k) \equiv d_1 + \cdots + d_k$;
- in base 2, "ends in 0" detects evenness; "digit sum divisible by 3" does *not* detect divisibility by 3 ($2 \not\equiv 1 \pmod 3$); instead the *alternating* digit sum does, because $2 \equiv -1 \pmod 3$.

Palindromicity, by contrast, is not a property of any value: $5 = 101_2$ is a palindrome in binary and not in base 3 (12_3). "Is a palindrome" is a property of words that happens to be readable as a relation between a value and a base.

7.3.1 Divisibility is a finite automaton

For any base and any modulus, divisibility can be decided by reading the numeral once, from the left, while remembering only a residue.

PROPOSITION 7.2 (Divisibility automata) CLASSICAL RESULT GIN-PROP-055

For a base $b \geq 2$ and a modulus $m \geq 1$, the deterministic automaton with states $\{0, \ldots, m-1\}$, start state 0, transition $r \xrightarrow{d} (br + d) \bmod m$ and accepting state 0 accepts exactly the base-b words whose value is divisible by m. The language of base-b numerals of multiples of m is therefore regular, recognized with at most m states.

Proof.

By Horner's rule, after reading a prefix u the automaton is in the state $\mathrm{val}(u) \bmod m$: the transition applies the digit move $x \mapsto bx + d$ modulo m, and reduction modulo m commutes with it. The word is accepted iff $\mathrm{val}(w) \bmod m = 0$. ∎

This is the digit-move reading of Chapter 5 at work: the automaton is the digit grammar *reduced modulo m*. Its states are the finitely many residues, so it is finite; its transitions are the digit moves, so it reads numerals as programs. Chapter 11 shows that addition is also a finite automaton, and that multiplication is not (GIN-THM-005).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def accepts(word, b, m):
    r = 0
    for ch in word:
        r = (b * r + int(ch, b)) % m
    return r == 0

for n in [0, 3, 21, 22, 1023]:
    w = bin(n)[2:]
    print(f"{n:5d} = {w:>10s}_2   divisible by 3: {accepts(w, 2, 3)!s:5s}  "
          f"alternating digit sum: {sum((-1) ** i * int(d) for i, d in enumerate(reversed(w)))}")
bad = sum(accepts(format(n, 'b'), 2, m) != (n % m == 0) for n in range(4000) for m in range(1, 20))
print("checked n < 4000, m < 20, base 2:", bad, "disagreements")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
    0 =          0_2   divisible by 3: True   alternating digit sum: 0
    3 =         11_2   divisible by 3: True   alternating digit sum: 0
   21 =      10101_2   divisible by 3: True   alternating digit sum: 3
   22 =      10110_2   divisible by 3: False  alternating digit sum: 1
 1023 = 1111111111_2   divisible by 3: True   alternating digit sum: 0
checked n < 4000, m < 20, base 2: 0 disagreements
```

7.4 Rationals: when expansions stop

A rational number has a positional expansion too, with digits after a radix point. Long division produces it, and long division is itself a small transition system.

PROPOSITION 7.3 (Long division is a finite transition system) CLASSICAL RESULT

GIN-PROP-009

Expanding p/q ($0 \le p < q$, lowest terms) in base b runs the transition system on remainders $r \mapsto (br) \bmod q$, emitting the digit $\lfloor br/q \rfloor$ at each step, starting from $r = p$. The expansion terminates if and only if every prime factor of q divides b; otherwise it is eventually periodic with period at most $q - 1$.

Proof.

The state space $\{0, \dots, q-1\}$ is finite, so every trajectory is eventually periodic. The expansion terminates iff the trajectory reaches 0, iff $p/q = m/b^k$ for some integers m, k, iff $q \mid b^k$ (since $\gcd(p, q) = 1$), iff every prime factor of q divides b. A periodic trajectory avoids 0, so its cycle has at most $q - 1$ states. ∎

So $1/3 = 0.\overline{3}$ in base 10 but $1/3 = 0.1$ in base 3; and $1/10 = 0.1$ in base 10 but in base 2

$$\frac{1}{10} = 0.0\overline{0011}_2,$$

an infinite expansion, because 5 divides 10 but not 2. This single fact is why the decimal literal `0.1` is not exactly representable in any binary floating-point format, and why $0.1 + 0.2 \neq 0.3$ in binary64 arithmetic (Chapter 25).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction
from ginsdk.numerals import periodic_expansion

for q in (Fraction(1, 3), Fraction(1, 10), Fraction(1, 7), Fraction(3, 8)):
    print(f"{str(q):5s} base 10: {periodic_expansion(q, 10):12s} base 2: {periodic_expansion(q, 2):22s} base 3: {periodic_expansion(q, 3)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1/3   base 10: 0.(3)        base 2: 0.(01)               base 3: 0.1
1/10  base 10: 0.1          base 2: 0.0(0011)            base 3: 0.(0022)
1/7   base 10: 0.(142857)   base 2: 0.(001)              base 3: 0.(010212)
3/8   base 10: 0.375        base 2: 0.011                base 3: 0.(10)
```

Whether a number has a *finite* numeral is a property of the pair (number, base), not of the number alone: $3/8$ terminates in bases 2 and 10 but not in base 3; $1/3$ terminates only in bases divisible by 3. Irrational numbers have no finite or periodic numeral in any integer base — a fact that sets the limit of every finite representation of the real numbers (Chapter 28).

7.5 Representations are trade-offs

If the value is invariant, why choose one representation over another? Because each representation makes some operations cheap and others expensive. Positional notation is not the only choice; it is the choice that balances the operations people use most.

PROPOSITION 7.4 (Representations trade operations against each other)

REINTERPRETATION GIN-REI-011

For integers of L bits, the following representations are each exact and complete, and their costs (in bit operations, for the stated operations) differ as shown:

REPRESENTATION	*ADDITION*	*MULTIPLICATION*	*COMPARISON*	*DIVISIBILITY BY A FIXED PRIME P*
unary	$\Theta(2^L)$	$\Theta(4^L)$	$\Theta(2^L)$	$\Theta(2^L)$
positional (binary)	$\Theta(L)$	$\mathcal{O}(L \log L)$	$\Theta(L)$	$\Theta(L)$
prime-exponent vector	*no known polynomial method (requires factoring the sum)*	*add exponents: cheap*	*requires conversion*	*read one exponent*
residues modulo coprime $m_1, \ldots, m_k$ (CRT)	*digit-wise, carry-free*	*digit-wise, carry-free*	*requires reconstruction (mixed-radix conversion)*	*depends on whether p is among the moduli*

All entries are classical facts about algorithms; the table is a reinterpretation of them as properties of grammars.

The residue representation deserves a word. By the Chinese remainder theorem (Chapter 31), an integer in $[0, M)$ with $M = m_1 \cdots m_k$ is determined by its residues modulo pairwise coprime m_i. In that representation addition and multiplication act independently on each residue — there is no carry between "digits" at all — but comparing two numbers or detecting overflow requires reconstructing them. Residue number systems are used in some signal processing and cryptographic hardware precisely for the carry-free operations, and avoided elsewhere precisely for the expensive comparison. No representation is free; each moves the cost somewhere else.

EXERCISE 7.1

Parse each statement and decide whether it is true: (a) $10_2 + 10_2 = 100_2$; (b) $10 + 10 = 100$ in base 10; © $11_3 = 100_2$; (d) “7 is a palindrome” .

▶ Solution to Exercise 7.1

EXERCISE 7.2

How many bit flips does counting from 0 to 1000 take? From 0 to 1024?

▶ Solution to Exercise 7.2

EXERCISE 7.3

Draw (or tabulate) the divisibility-by-7 automaton for base 10 and run it on `1001` .

▶ Solution to Exercise 7.3

EXERCISE 7.4

In which integer bases b between 2 and 30 does $1/12$ have a finite expansion?

▶ Solution to Exercise 7.4

EXERCISE 7.5

Represent 17 and 23 by their residues modulo $(3, 5, 7)$, add and multiply them residue-wise, and reconstruct the results. Which reconstruction is correct, and why does the other fail?

▶ Solution to Exercise 7.5

SUMMARY

- $1 + 1 = 10_2$ and $1 + 1 = 2$ state the same equality of values; only the spelling of the answer differs.
- Binary $1 + 1$ generates a carry; counting to N flips exactly $2N - \mathrm{popcount}(N)$ bits.
- Value properties and numeral properties differ; some numeral properties detect value properties only in suitable bases.
- Divisibility by m is decided by an m-state automaton reading the numeral; long division is a finite transition system, so rationals have eventually periodic expansions, terminating iff the denominator's primes divide the base.
- Representations trade operations: positional notation is a compromise, not the only exact choice.

PART IV

Arithmetic as Formal Operations

What an operation is: domains, closure, admissibility, operation contracts, and inverse operations as converse relations.

Chapter 8

What an Operation Is: Domains, Layers, and Contracts

> *Before asking what 7 / 2 is, ask: in which domain, by which definition, and under which machine?*

LEARNING OBJECTIVES

- Define an operation as a relation, and distinguish totality, functionality and closure.
- Parse arithmetic expressions with a precise grammar, and recognize where precedence and associativity are conventions that differ between systems.
- State the four layers of an arithmetic outcome and give, for each pair of layers, an expression that fails at one and passes the other.
- Read and write an operation contract: domain, precondition, defining equation, failure classes, cost model.
- Use the SDK's layered inspection to answer the nine interface questions about an expression.

8.1 Operations as relations

A binary operation on a set A is usually defined as a function $A \times A \to A$. For arithmetic this definition is too narrow: it presupposes that every pair has a result, that the result is unique, and that it lies in A. Each presupposition fails somewhere in this book, and we want to talk about the failures. So we begin with relations.

DEFINITION 8.1 (Operation grammar) DEFINITION GIN-DEF-014

A binary operation $\circ$ on a carrier A induces the relation

$$[\![\circ]\!] \subseteq (A \times A) \times A,$$

and, for each fixed b, the *move* $[\![\circ b]\!] : a \mapsto a \circ b$. The operation is **total** if every pair is related to at least one result, **functional** if to at most one, and **closed** on A if its results lie in A.

DEFINITION 8.2 (Closure of a domain) DEFINITION GIN-DEF-018

A domain is **closed** under an operation if the operation is admissible — total, functional, with results in the domain — at every pair of arguments. $\mathbb{N}$ is closed under $+$ and $\times$; $\mathbb{Z}$ is closed under $-$; $\mathbb{Q} \setminus \{0\}$ is closed under $/$; $\mathbb{Z}/p$ for prime p is closed under division by non-zero elements.

Read as relations, the four operations of school arithmetic fall into two families. Addition and multiplication are *forward* operations: given the arguments, there is a recipe that produces the result (primitive recursion, Chapter 4). Subtraction and division are *converse* operations: their results are defined by an equation — $a - b$ is the c with $b + c = a$, and a/b is the c with $b \cdot c = a$ — and an equation may have no solution or many. Chapter 9 makes this precise. Here we need only the vocabulary.

8.2 The grammar of expressions

Before an expression can be evaluated it must be *parsed*. The SDK and the laboratory use the following grammar, with precedence increasing downwards.

DEFINITION 8.3 (Arithmetic expression language) DEFINITION GIN-DEF-060

TEXT

```
expr   := term  (('+' | '-') term)*
term   := unary (('*' | '/' | '//' | '%' | 'mod') unary)*
unary  := ('+' | '-') unary-at-level-30 | power
power  := atom ('^' unary)?                    -- right-associative
atom   := NUMBER | NAME | '(' expr ')' | ('sqrt' | 'gcd') '(' expr (',' expr)* ')'
NUMBER := decimal integer | decimal fraction | 0b… | …₂ | 0x…
```

The Unicode operators × · ÷ − are accepted as aliases. Exponents are evaluated in exact $\mathbb{Z}$ whatever the domain, because an exponent counts repetitions and is not an element of the domain (GIN-NEG-013).

Three conventions in this grammar are not forced by mathematics:

1. **Unary minus binds more loosely than exponentiation**: $-2^2 = -(2^2) = -4$. This agrees with mathematical usage and with Python. Spreadsheet software commonly gives negation *higher* precedence, so that the same five characters compute 4.
2. **Exponentiation is right-associative**: $2^{3^2} = 2^9 = 512$, not $(2^3)^2 = 64$.
3. **Division is left-associative**: $8/2/2 = (8/2)/2 = 2$, not $8/(2/2) = 8$.

None of these is a fact about numbers. Each is a choice of grammar, made so that the most common reading is the default. They are among the most frequent sources of disagreement between people, calculators and programming languages — disagreements that are entirely syntactic.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import parse

for e in ["-2 ^ 2", "2 ^ 3 ^ 2", "8 / 2 / 2", "1 + 2 * 3", "-7 // 2"]:
    print(f"{e:10s} parses as {parse(e).show()}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
-2 ^ 2     parses as (-(2 ^ 2))
2 ^ 3 ^ 2  parses as (2 ^ (3 ^ 2))
8 / 2 / 2  parses as ((8 / 2) / 2)
1 + 2 * 3  parses as (1 + (2 × 3))
-7 // 2    parses as ((-7) // 2)
```

8.3 Four layers of an outcome

When an expression is evaluated, four questions are answered in order, and evaluation can stop at any of them.

DEFINITION 8.4 (Four evaluation layers) DEFINITION GIN-DEF-061

1. **Syntax** — is the string a sentence of the expression grammar?
2. **Typing** — does every literal and variable denote an element of the chosen domain?
3. **Semantics** — does every operation have exactly one result in the domain?
4. **Execution** — what does the chosen machine model do: overflow, rounding, traps, exceptions, undefined behaviour, flags?

Computational Grammar Theory separates syntax, semantics and execution. Arithmetic needs a fourth layer between the first two: a string can be perfectly well formed and still name something that is not in the domain. The decimal literal `0.5` is fine syntax and is not an integer. The literal `2147483648` is fine syntax and is not a 32-bit signed integer. These failures are neither syntactic nor semantic — no operation has been attempted — and calling them either hides what went wrong.

DEFINITION 8.5 (Outcome) DEFINITION GIN-DEF-062

The **outcome** of evaluating an expression in a domain is a record with a status, the layer at which evaluation stopped, a reason, the defining equation of the failing operation (if any), the IEEE flags raised, the step trace, the parse tree, the representation of the result, and a cost estimate. The statuses are:

STATUS	LAYER	MEANING
`value`	—	a value
`special`	execution	a special datum (±∞, NaN): not a number in the mathematical sense
`no-solution`	semantics	the defining equation has no solution
`non-unique`	semantics	the defining equation has more than one solution
`not-in-domain`	typing	an operand or literal is outside the domain
`unsupported`	typing	the operation is not part of the domain's grammar
`overflow`	execution	the result is not representable
`trap`	execution	the hardware raises an exception
`exception`	execution	the language runtime raises an exception
`undefined`	execution	undefined behaviour: the program has no meaning
`syntax-error`	syntax	the string is not a well-formed expression

No outcome is a bare "error".

The layers are not a hierarchy of severity; they are independent. Knowing that an expression passes one layer tells you nothing about whether it passes another.

PROPOSITION 8.6 (The four layers are independent) PROVED HERE GIN-PROP-056

For each of the following combinations there is an expression and a domain realizing it:

1. *well formed, ill-typed:* `0.5 + 1` *in* $\mathbb{Z}$*;*
2. *well typed, semantically inadmissible:* `1 / 0` *in* $\mathbb{Q}$*;*
3. *semantically admissible, failing at execution:* `(-2147483647 - 1) / -1` *in 32-bit x86-64 integers — the exact quotient* 2147483648 *exists and is unique in* $\mathbb{Z}$*, but* `idiv` *traps;*
4. *semantically inadmissible, executing to an ordinary-looking value:* `1 / 0` *in 32-bit AArch64 integers, which yields* 0*;*
5. *ill-typed at one width, admissible at another:* `2147483648 - 1` *fails typing in 32-bit integers and is a value in Python;*
6. *well formed and semantically admissible in* $\mathbb{Q}$*, unsupported in an IEEE format:* `2 ^ 10`*, since exponentiation is not a basic IEEE operation (it is a library function, not correctly rounded in general).*

Proof.

By evaluation, reproduced below. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

cases = [("0.5 + 1", "Z"), ("1 / 0", "Q"), ("(-2147483647 - 1) / -1", "int32 x86-64"),
         ("(-2147483647 - 1) / -1", "Z"), ("1 / 0", "int32 AArch64"), ("2147483648 - 1", "int32 Java"),
         ("2147483648 - 1", "Python"), ("2 ^ 10", "binary64")]
for e, d in cases:
    o = evaluate(e, d)
    print(f"{e:24s} {d:14s} → {o.status:14s} layer={o.layer or '—':9s} {o.display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0.5 + 1                  Z              → not-in-domain  layer=typing
1 / 0                    Q              → no-solution    layer=semantics
(-2147483647 - 1) / -1   int32 x86-64   → trap           layer=execution
(-2147483647 - 1) / -1   Z              → value          layer=—         2147483648
1 / 0                    int32 AArch64  → value          layer=—         0
2147483648 - 1           int32 Java     → not-in-domain  layer=typing
2147483648 - 1           Python         → value          layer=—         2147483647
2 ^ 10                   binary64       → unsupported    layer=typing
```

Case 4 deserves emphasis. The outcome status is `value`, because AArch64 does produce a value. A program that inspects only the status would conclude that `1 / 0` succeeded. Only by comparing with exact mathematics — where the status is `no-solution` — does one see that the

value is a convention, not a quotient. The SDK's `inspect` function performs exactly this comparison (§8.5).

8.4 Operation contracts

Everything that must be known before an operation can be trusted can be written down in a fixed form.

DEFINITION 8.7 (Operation contract) DEFINITION GIN-DEF-080

An **operation contract** for an operator $\circ$ in a domain (or family of domains) D consists of: the **precondition** under which $a \circ b$ is admissible in D; the **defining equation** that characterizes the result; the **failure classes** (outcome statuses) that can occur when the precondition fails; and the **cost model** under which the operation's cost is stated. A contract is *machine-readable* when it is given as structured data, as in the SDK's schema `gin-contracts/1`.

A contract is the arithmetic analogue of a function's documented preconditions and postconditions in software engineering — with the difference that, for arithmetic, the postcondition is a *mathematical equation*, so a result can be checked independently of the code that produced it. Here is the contract for division as the SDK publishes it, abridged to three domains:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.contracts import contract

c = contract("/")
print(c["name"], "—", c["summary"][:96] + "…")
for d in c["domains"]:
    if d["domain"] in ("Q, R, C", "Z/n", "int32 RISC-V"):
        print(f"  {d['domain']:13s} pre: {d['pre']}")
        print(f"  {'':13s} equation: {d['equation']};  failures: {d['failures'] or 'none'}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
division — The converse of multiplication: a / b is the unique c with b · c = a. Two distinct failures: no …
  Q, R, C       pre: b ≠ 0
                equation: b · c = a;  failures: ['no-solution', 'non-unique']
  Z/n           pre: gcd(b, n) | a for existence; gcd(b, n) = 1 for uniqueness
                equation: b · c ≡ a (mod n);  failures: ['no-solution', 'non-unique']
```

```
int32 RISC-V  pre: always (total by convention)
              equation: truncated quotient; x/0 = -1; -2³¹/-1 = -2³¹;  failures: none
```

The RISC-V row says “always (total by convention)” and lists no failures. That is accurate as a description of the instruction, and dangerous as a description of division: a program that relies on the instruction’s totality computes -1 for $1/0$ and continues. The contract does not hide this; it names the defining equation (“x/0 = −1”), from which a reader can see that the result is not a solution of $0 \cdot c = 1$. Part XVI measures how much such contracts help in predicting what real programs do.

8.5 Inspecting an expression

The SDK’s `inspect` evaluates an expression twice: in the selected domain, and in the exact mathematical domain that the selected one implements or approximates ($\mathbb{Z}$ for machine integers, $\mathbb{Q}$ for floating point and Python). It then classifies the relation between the two outcomes: the machine *agrees*, *rounds*, *wraps*, *totalizes*, *selects* (division with remainder), *signals*, or has *undefined* behaviour. The answer is a JSON record with a fixed schema (`gin-inspect/1`).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import inspect

for e, d in [("1 / 0", "int32 RISC-V"), ("0.1 + 0.2", "binary64"), ("2147483647 + 1", "int32 Java"), ("-7 / 2",
"int32 C")]:
    r = inspect(e, d)
    print(f"{e:16s} {d:13s} math: {r['mathematics']['status']:11s} machine: {str(r['execution']['value']):20s}
relation: {r['relation']['kind']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 / 0            int32 RISC-V  math: no-solution machine: -1                   relation: totalized
0.1 + 0.2        binary64      math: value       machine: 0.30000000000000004  relation: rounded
2147483647 + 1   int32 Java    math: value       machine: -2147483648          relation: wrapped
-7 / 2           int32 C       math: no-solution machine: -3                   relation: selected
```

The last line is worth reading slowly. In $\mathbb{Z}$, $-7/2$ has no solution: $2 \nmid 7$. C’s integer division returns -3, which is not a quotient but a *selection* from the solutions of $-7 = 2q + r$ with $|r| < 2$ — namely the one that truncates toward zero. Python’s `//` makes a different selection (-4, flooring). Both are well defined; neither is “the” quotient; and confusing them is a classical source of bugs in code that ports between languages. Chapter 19 treats division with remainder in detail.

EXERCISE 8.1

For each operation, say whether it is total, functional and closed on the given set: (a) $+$ on odd integers; (b) max on $\mathbb{N}$; © $\sqrt{\cdot}$ on $\{0, 1, 4, 9, \ldots\}$ viewed as the relation $\{(x, y) : y^2 = x\}$ on $\mathbb{Z}$; (d) $/$ on $\mathbb{Z} \setminus \{0\}$.

▶ Solution to Exercise 8.1

EXERCISE 8.2

Give the parse trees of `-3 ^ 2 ^ 2` and `2 * -3 ^ 2` in the grammar of Definition 8.3, and their values in $\mathbb{Z}$.

▶ Solution to Exercise 8.2

EXERCISE 8.3

Classify the failure layer: (a) `2 ^ 0.5` in $\mathbb{Q}$; (b) `x + 1` in $\mathbb{Z}$ with no value for `x`; © `1 +`; (d) `0 - 1` in $\mathbb{N}$; (e) `2147483647 * 2` in 32-bit C.

▶ Solution to Exercise 8.3

EXERCISE 8.4

Write an operation contract (precondition, defining equation, failure classes, cost model) for integer square root $\lfloor \sqrt{n} \rfloor$ on $\mathbb{N}$, and explain why it has no `non-unique` failure although $\sqrt{\cdot}$ does.

▶ Solution to Exercise 8.4

SUMMARY

- An operation is a relation; totality, functionality and closure can fail independently.
- Precedence and associativity are conventions of a grammar, and systems differ in them.
- Outcomes have four independent layers — syntax, typing, semantics, execution — and eleven statuses; none is a bare “error”.
- An operation contract states precondition, defining equation, failure classes and cost model, as data.
- Comparing a machine’s outcome with exact mathematics classifies it: agrees, rounds, wraps, totalizes, selects, signals, undefined.

Chapter 9

Inverse Operations as Converse Relations

> *Undefined is not an answer. It is the name of a question that has not yet been asked precisely: no solution, or too many?*

LEARNING OBJECTIVES

- Define the converse problem of a forward operation and classify every inverse expression as having no solution, a unique solution, or several.
- Prove that the solution set of a linear converse problem is empty or a coset of a kernel, and give a non-linear converse problem for which this fails.
- Name the seven ways in which mathematical and computational systems give inadmissible expressions a value, and the cost of each.
- Read the constructions $\mathbb{N} \to \mathbb{Z} \to \mathbb{Q}$ as admissibility closures that keep an unanswered converse problem as data.

9.1 The converse problem

Every inverse operation of arithmetic is defined by an equation involving a forward operation:

$$a - b = c \iff b + c = a, \qquad a/b = c \iff b \cdot c = a,$$
$$\sqrt{a} = c \iff c \cdot c = a \text{ (and } c \geq 0), \quad \log a = c \iff e^c = a.$$

The left-hand sides are written as if they named a single value. The right-hand sides are equations, and an equation may have no solution, exactly one, or several. This chapter takes the right-hand sides as the definition.

DEFINITION 9.1 (Converse problem) DEFINITION GIN-DEF-015

For a forward operation $\circ$ on a carrier A, the **converse problem** at (a, b) is the solution set

$$\mathrm{Sol}_{\circ}(a, b) = \{\, c \in A : b \circ c = a \,\}. \tag{9.1}$$

Division asks for $\mathrm{Sol}_{\times}(a, b)$, subtraction for $\mathrm{Sol}_{+}(a, b)$, the square root for $\{c : c \cdot c = a\}$, the logarithm for $\{c : e^c = a\}$, and the modular inverse of b for $\mathrm{Sol}_{\times}(1, b)$ in $\mathbb{Z}/n$.

DEFINITION 9.2 (Converse admissibility) DEFINITION GIN-DEF-016

An inverse expression is

- **inadmissible: no solution** if $\mathrm{Sol} = \varnothing$;
- **admissible** if $|\,\mathrm{Sol}\,| = 1$, and then it denotes the unique solution;
- **inadmissible: non-unique** if $|\,\mathrm{Sol}\,| \geq 2$.

A **branch** (principal value, selection) is a choice function on non-unique solution sets that turns a non-unique converse into a function: $\sqrt{\cdot}$ selects the non-negative root; $\lfloor \cdot \rfloor$-division selects the quotient whose remainder is non-negative.

Every inadmissibility in this book is one of these two kinds, and the distinction is not cosmetic. *No solution* means the question asks for something that does not exist; any answer is a fiction. *Non-unique* means the question does not determine its answer; any single answer is a choice. A system that reports both as "undefined" has lost the information needed to repair either.

EXPRESSION	CARRIER	EQUATION	SOLUTIONS	VERDICT
$0 - 1$	$\mathbb{N}$	$1 + c = 0$	$\varnothing$	no solution
$0 - 1$	$\mathbb{Z}$	$1 + c = 0$	$\{-1\}$	admissible
$7/2$	$\mathbb{Z}$	$2c = 7$	$\varnothing$	no solution
$7/2$	$\mathbb{Q}$	$2c = 7$	$\{7/2\}$	admissible
$1/0$	$\mathbb{Q}$	$0 \cdot c = 1$	$\varnothing$	no solution
$0/0$	$\mathbb{Q}$	$0 \cdot c = 0$	$\mathbb{Q}$	non-unique
$2/4$	$\mathbb{Z}/6$	$4c \equiv 2$	$\{2, 5\}$	non-unique
$\sqrt{2}$	$\mathbb{Q}$	$c^2 = 2$	$\varnothing$	no solution

EXPRESSION	CARRIER	EQUATION	SOLUTIONS	VERDICT
$\sqrt{4}$	$\mathbb{Z}$	$c^2 = 4$	$\{2, -2\}$	non-unique; the branch selects 2
$\sqrt{-1}$	$\mathbb{R}$	$c^2 = -1$	$\varnothing$	no solution
$\log 1$	$\mathbb{C}$	$e^c = 1$	$\{2\pi i k : k \in \mathbb{Z}\}$	non-unique (infinitely many)
$\log 0$	$\mathbb{C}$	$e^c = 0$	$\varnothing$	no solution

The SDK computes these solution sets exactly:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import converse as C

for s in [C.subtract(0, 1, "N"), C.divide(7, 2, "Z"), C.divide(1, 0, "Q"), C.divide(0, 0, "Q"),
          C.divide_mod(2, 4, 6), C.square_root(2, "Q"), C.square_root(4, "Z"), C.logarithm(1, "C")]:
    print(f"{s.carrier:4s} {s.equation:18s} {s.kind:9s} {s.description}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
N    1 + c = 0          no-solution ∅
Z    2 · c = 7          no-solution ∅
Q    0 · c = 1          no-solution ∅
Q    0 · c = 0          non-unique every c in Q
Z/6  4 · c ≡ 2 (mod 6)  non-unique c ∈ {2, 5} = 2 + Ann(4)
Q    c · c = 2          no-solution ∅
Z    c · c = 4          non-unique c ∈ {2, -2}
C    e^c = 1            non-unique c = Ln 1 + 2πik, k ∈ Z
```

9.2 Linear converse problems: cosets of a kernel

Subtraction, division by a fixed element, the modular inverse and linear Diophantine equations have something in common: the forward map $c \mapsto b \circ c$ is a *homomorphism* of additive groups. For such maps the shape of every solution set is forced.

THEOREM 9.3 (Converse of a homomorphism) CLASSICAL RESULT GIN-THM-007

Let $f : G \to H$ be a homomorphism of abelian groups and $a \in H$. The solution set $\{c \in G : f(c) = a\}$ is either empty or a coset $c_0 + \ker f$. Hence:

1. ***image condition**: a solution exists iff $a \in f(G)$;*
2. ***kernel condition**: when solutions exist, there are exactly $|\ker f|$ of them; the solution is unique iff $\ker f = \{0\}$.*

Proof.

If $f(c_0) = a$, then $f(c) = a \iff f(c - c_0) = 0 \iff c - c_0 \in \ker f$. ∎

This is the theorem of every first course in linear algebra — the general solution of $Ax = a$ is a particular solution plus the null space — and it is the whole explanation of why there are *two* kinds of inadmissibility for linear converse problems and no others: the image condition can fail (no solution) and the kernel condition can fail (non-unique). Division is the converse of $c \mapsto b \cdot c$, which is additive in c; Chapter 16 specializes Theorem 9.3 to rings and obtains the classification of $1/0$ and $0/0$.

For converse problems that are *not* linear, the solution set can have any shape.

COUNTEREXAMPLE 9.4 (Solution sets of converse problems need not be cosets)

COUNTEREXAMPLE GIN-NEG-019

In $\mathbb{Z}/8$ the equation $c \cdot c = 1$ has the four solutions $\{1, 3, 5, 7\}$, and $c \cdot c = 4$ has the two solutions $\{2, 6\}$. The two solution sets have different sizes, so they are not cosets of a common subgroup: squaring is not a homomorphism, and Theorem 9.3 does not apply.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.converse import converse_finite

sq = lambda b, c: (c * c) % 8          # the forward map c → c*c (b is unused)
for a in range(8):
    s = converse_finite(sq, range(8), a, None, name="Z/8")
    print(f"c*c = {a} (mod 8): {s.kind:9s} {s.solutions}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
c*c = 0 (mod 8): non-unique [0, 4]
c*c = 1 (mod 8): non-unique [1, 3, 5, 7]
c*c = 2 (mod 8): no-solution []
c*c = 3 (mod 8): no-solution []
```

```
c*c = 4 (mod 8): non-unique [2, 6]
c*c = 5 (mod 8): no-solution []
c*c = 6 (mod 8): no-solution []
c*c = 7 (mod 8): no-solution []
```

9.3 Seven repairs

When an inverse expression is inadmissible, mathematics and computation have developed exactly seven ways of giving it a value anyway, or of refusing to. Each changes the grammar, and each gives something up.

DEFINITION 9.5 (Repairs of inadmissibility) PROPOSED GIN-DEF-017

1. **Extension** — enlarge the carrier so that the solution set becomes non-empty: $\mathbb{N} \to \mathbb{Z}$ for subtraction, $\mathbb{Z} \to \mathbb{Q}$ for division by non-zero elements, $\mathbb{R} \to \mathbb{C}$ for square roots of negatives, $\mathbb{R} \to \mathbb{R} \cup \{\infty\}$ for $1/0$.
2. **Selection** — choose one element of a non-unique solution set: principal branches; Euclidean quotients with conditions on the remainder.
3. **Restriction** — remove the bad inputs from the domain: $b \neq 0$; units only.
4. **Quotient** — identify elements so that the operation becomes total: $\mathbb{Z} \to \mathbb{Z}/2^w$ makes $0 - 1 = 2^w - 1$.
5. **Totalization** — define a value by convention without claiming that it solves the converse problem: $x/0 := 0$ in meadows and in Lean's mathlib; AArch64 integer division by zero returns 0; RISC-V returns -1; truncated subtraction $0 \dot{-} 1 := 0$.
6. **Absorption** — adjoin a special element that absorbs further operations: NaN; the wheel element $\perp$; `Option` and `Result` types.
7. **Signal** — refuse to produce a value and transfer control: hardware traps, exceptions, panics, compile-time errors.

REPAIR	WHAT IS KEPT	WHAT IS GIVEN UP	TYPICAL HOME
extension	all identities of the old structure	the old carrier; sometimes order ($\mathbb{C}$) or other laws (∞)	mathematics
selection	functionality	symmetry between the solutions; continuity at branch cuts	analysis, integer division

REPAIR	WHAT IS KEPT	WHAT IS GIVEN UP	TYPICAL HOME
restriction	everything, on a smaller domain	totality	algebra textbooks, preconditions
quotient	totality, the ring laws	order; the distinction between identified values	hardware integers, $\mathbb{Z}/n$
totalization	totality, often the ring laws	the converse reading ($b \cdot (a/b) = a$ fails at $b = 0$)	proof assistants, some ISAs
absorption	totality; the information that something failed	equality (NaN ≠ NaN), some laws ($0 \cdot x = 0$ in wheels)	IEEE 754, wheels, typed APIs
signal	the meaning of every value produced	totality; control flow	x86, Java, Rust, Python

The table is a reinterpretation of familiar facts, but it makes one point that is easy to miss: *every* repair gives something up. There is no free way to make division total. Chapter 18 proves this for division by zero in the strongest form we can state: every system that gives $1/0$ a value abandons the ring laws, the converse reading of division, or $0 \neq 1$ (GIN-THM-004).

9.4 Number systems as admissibility closures

Read in this vocabulary, the familiar tower $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$ is a sequence of extensions, each made so that a family of converse problems acquires solutions.

DEFINITION 9.6 (Admissibility closure) PROPOSED GIN-DEF-019

Given $A \subseteq U$ and a family of converse problems, the **admissibility closure** of A in U is the least B with $A \subseteq B \subseteq U$ such that every converse problem with data in B that has a solution in U has one in B.

For example, the closure of $\mathbb{N}$ in $\mathbb{Q}$ under subtraction is $\mathbb{Z}$; the closure of $\mathbb{Z}$ in $\mathbb{C}$ under division by non-zero elements is $\mathbb{Q}$; the closure of $\mathbb{Q}$ in $\mathbb{R}$ under the field operations and square roots of positive elements is the field of real constructible numbers. The classical constructions do more than find the closure inside a given universe: they *build* the new elements out of the unanswered problems themselves.

PROPOSITION 9.7

(The integers and rationals keep unanswered converse problems as data)

REINTERPRETATION GIN-PROP-017

$\mathbb{Z}$ is the set of pairs $(a,b) \in \mathbb{N}^2$ modulo $(a,b) \sim (c,d) \iff a+d = b+c$, with (a,b) standing for the answer to "$a-b$"; $\mathbb{Q}$ is the set of pairs $(a,b) \in \mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ modulo $(a,b) \sim (c,d) \iff ad = bc$, with (a,b) standing for "a/b". In each case the new element is the unanswered converse problem, kept as data, and two problems are identified when they must have the same answer. The exclusion $b \neq 0$ in the second construction is forced: admitting $(0,0)$ breaks transitivity (GIN-THM-006).

The first construction is the Grothendieck group of the monoid $(\mathbb{N}, +)$; the second is the field of fractions of the integral domain $\mathbb{Z}$. Both are classical. The reading — that an integer is a subtraction problem and a rational a division problem, up to having the same answer — is how this book connects them to the converse vocabulary. It also explains why the second construction has an exception and the first has none: the problem $b + c = a$ always has *at most one* solution in a cancellative monoid, but $b \cdot c = a$ has *every* c as a solution when $a = b = 0$, and a class containing such a problem would have to equal every other class.

OPEN PROBLEM 9.8 (The cost of admissibility) OPEN PROBLEM GIN-OPEN-002

Deciding whether a/b is admissible in $\mathbb{Z}$ means deciding $b \mid a$, which costs about as much as dividing; deciding whether b is invertible in $\mathbb{Z}/n$ costs a gcd. When is deciding admissibility asymptotically cheaper than computing the result, and when is it as hard? Is there a natural converse problem where admissibility is hard to decide but the solution, when it exists, is easy to compute?

EXERCISE 9.1

Classify, with the solution set: (a) $5 - 8$ in $\mathbb{N}$; (b) $6/4$ in $\mathbb{Z}/10$; © $3/4$ in $\mathbb{Z}/10$; (d) $\sqrt{9}$ in $\mathbb{N}$; (e) $\log(-1)$ in $\mathbb{R}$ and in $\mathbb{C}$.

▶ Solution to Exercise 9.1

EXERCISE 9.2

Use Theorem 9.3 to describe all integer solutions of $6x + 10y = 4$.

▶ Solution to Exercise 9.2

EXERCISE 9.3

How many solutions does $c^2 \equiv 1 \pmod{n}$ have for $n = 2, 4, 8, 16, 15$? Why does Theorem 9.3 not predict these counts?

▶ Solution to Exercise 9.3

EXERCISE 9.4

Name the repair in each case: (a) Python's `math.sqrt(-1)` raises `ValueError`; (b) `cmath.sqrt(-1)` returns `1j`; © `numpy.sqrt(-1.0)` returns `nan` with a warning; (d) a database's `NULLIF(x, 0)` used as a divisor makes `y / NULLIF(x, 0)` return `NULL`.

▶ Solution to Exercise 9.4

SUMMARY

- Inverse operations are converse problems; their solution sets classify every inverse expression as no solution, unique, or non-unique.
- For linear converse problems the solution set is empty or a coset of a kernel: the two failures are image failure and kernel failure. Non-linear converses ($c^2 = a$) need not behave this way.
- Seven repairs — extension, selection, restriction, quotient, totalization, absorption, signal — give inadmissible expressions a value or refuse to; each gives something up.
- $\mathbb{Z}$ and $\mathbb{Q}$ are admissibility closures that keep unanswered subtraction and division problems as data.

PART V

Addition and Carry

Addition from first principles; the carry automaton, its three-element monoid, and parallel prefix adders.

Chapter 10

Addition from First Principles

> *One plus one is two in five different ways, and the ways agree. That they agree is a theorem.*

LEARNING OBJECTIVES

- Give five definitions of $1 + 1$ and explain why they produce the same value.
- State and prove the correctness invariant of column addition in any base, including the fact that the carry never exceeds one.
- Count the cost of addition in bit operations and limb operations, and prove that no algorithm can do asymptotically better on numerals.
- Add rationals and see where the gcd enters; recognize where floating-point addition departs from exact addition.

10.1 Five readings of $1 + 1$

1. **Counting.** Put one object next to another and count: the result is the count of the union. $|\{a\}| + |\{b\}| = |\{a, b\}|$ for distinct a, b.
2. **Successor.** By primitive recursion, $\mathrm{add}(1, S0) = S(\mathrm{add}(1, 0)) = S(1) = 2$: two rule applications (Chapter 4).
3. **Cardinality.** For disjoint finite sets, $|A \sqcup B|$ depends only on $|A|$ and $|B|$, because bijections $A \to A'$ and $B \to B'$ combine into a bijection $A \sqcup B \to A' \sqcup B'$. Addition *is* the operation induced on counts.
4. **Algebra.** In any ring, $1 + 1$ is the element 2 of the subring generated by 1. (In $\mathbb{Z}/2$ that element is 0: the same expression, a different structure.)
5. **Machine.** In binary, the digit pair $(1, 1)$ produces sum bit 0 and carry 1; a half adder computes $\mathrm{XOR}(1, 1) = 0$ and $\mathrm{AND}(1, 1) = 1$, giving the word `10`.

That readings 1–3 agree is the content of the statement that $\mathbb{N}$ *is* the set of finite cardinalities, made precise by the recursion theorem: cardinality of a disjoint union and primitive-recursive addition are both functions $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$ satisfying $f(m, 0) = m$ and $f(m, Sn) = Sf(m, n)$, so they coincide. Reading 4 agrees with them in $\mathbb{Z}$ because $\mathbb{Z}$ contains $\mathbb{N}$ as its non-negative part, and disagrees in $\mathbb{Z}/2$ because $\mathbb{Z}/2$ is a quotient in which $2 = 0$. Reading 5 agrees with them *as long as*

the result is decoded with the base-2 denotation and fits in the available bits — the two conditions that Chapter 26 shows to fail on machines.

Throughout, distinguish what is **invariant** (the value 2), what is **generated** (a new position, the carry), and what is **merely represented** (the word `10` in base 2, `2` in base 10, `SS0` in unary).

10.2 The column algorithm

The algorithm taught in school adds two numerals digit by digit from the right, carrying when a column overflows. In base b, with digit lists $a_0, \dots, a_{n-1}$ and $b_0, \dots, b_{n-1}$ (least significant first, padded with zeros to equal length):

ALGORITHM 10.1 (Column addition in base b)

$c_0 \leftarrow 0$. For $i = 0, \dots, n-1$: let $t = a_i + b_i + c_i$; set $s_i \leftarrow t \bmod b$ and $c_{i+1} \leftarrow \lfloor t/b \rfloor$. Finally $s_n \leftarrow c_n$. Output the digits $s_n s_{n-1} \cdots s_0$.

PROPOSITION 10.1 (Correctness of column addition; the carry is at most 1)

CLASSICAL RESULT

GIN-PROP-057

For every i, $c_i \in \{0, 1\}$, and after column i

$$\sum_{j=0}^{i} s_j b^j + c_{i+1}\, b^{i+1} \;=\; \sum_{j=0}^{i} (a_j + b_j)\, b^j. \tag{10.1}$$

At $i = n-1$ this says that the output denotes $\mathrm{val}(a) + \mathrm{val}(b)$. The output has at most $n+1$ digits.

Proof.

Carry bound. $c_0 = 0$. If $c_i \le 1$ then $t \le (b-1) + (b-1) + 1 = 2b - 1$, so $c_{i+1} = \lfloor t/b \rfloor \le 1$. *Invariant.* By construction $t = s_i + b\, c_{i+1}$, i.e. $a_i + b_i + c_i = s_i + b\, c_{i+1}$. Multiply by b^i and add to the invariant for $i - 1$ (which holds trivially for $i = -1$ with the empty sum and $c_0 = 0$): the term $c_i b^i$ cancels, leaving (10.1).

∎

The invariant is the precise content of "carrying the one": the carry is a deferred contribution of exactly b^{i+1}, and the algorithm never loses it. The carry bound explains why one bit of memory between columns suffices in *any* base — a fact we will use in Chapter 11 to show that addition is a finite-state computation.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def column_add(a, b, base=10):
    A, B = [int(d, base) for d in reversed(a)], [int(d, base) for d in reversed(b)]
    n = max(len(A), len(B)); A += [0] * (n - len(A)); B += [0] * (n - len(B))
    c, s, rows = 0, [], []
    for i in range(n):
        t = A[i] + B[i] + c
        s.append(t % base); rows.append((i, A[i], B[i], c, t % base, t // base)); c = t // base
    s.append(c)
    return "".join("0123456789abcdef"[d] for d in reversed(s)).lstrip("0") or "0", rows

for a, b, base in [("1", "1", 2), ("1", "1", 10), ("999", "1", 10), ("1011", "111", 2)]:
    out, rows = column_add(a, b, base)
    print(f"{a} + {b} (base {base}) = {out}   carries out: {[r[5] for r in rows]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 + 1 (base 2) = 10   carries out: [1]
1 + 1 (base 10) = 2   carries out: [0]
999 + 1 (base 10) = 1000   carries out: [1, 1, 1]
1011 + 111 (base 2) = 10010   carries out: [1, 1, 1, 1]
```

10.3 The cost of addition

Column addition of two L-digit numerals performs L column steps, each a bounded amount of work on digits: $\Theta(L)$ digit operations. In base 2, that is $\Theta(L)$ bit operations; in base 2^{32} (machine words, "limbs"), $\lceil L/32 \rceil$ add-with-carry instructions.

PROPOSITION 10.2 (Addition cannot be done in sublinear time on numerals)

CLASSICAL RESULT GIN-PROP-058

Any algorithm that computes the binary numeral of $x + y$ from the binary numerals of x and y (each of length L) must read all $2L$ input bits in the worst case. Hence addition costs $\Theta(L)$ bit operations, and column addition is optimal up to a constant factor.

Proof.

For any position i and any choice of the other bits with $y = 0$, flipping bit i of x flips bit i of the sum; so an algorithm that does not read bit i of x outputs the same bit i for two inputs whose sums differ there. The same argument applies to y. The output itself has $\Theta(L)$ bits, which also bounds the cost from below. ∎

Two remarks prevent misreading the proposition. First, it is about *sequential* reading of numerals. With many processors working at once, all bits can be read in parallel, and addition can be done in time $\mathcal{O}(\log L)$ — by computing all carries simultaneously, as Chapter 11 explains. Second, it says nothing about machine words: adding two numbers that fit in one 64-bit register is *one* instruction. "Addition is $\mathcal{O}(1)$" and "addition is $\Theta(L)$" are both true, of different models (Chapter 36).

The SDK's limb arithmetic counts the work explicitly:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import bigint
from ginsdk.cost import Cost

for bits in (64, 1024, 16384):
    x, y = (1 << bits) - 1, (1 << bits) - 3
    c = Cost()
    s = bigint.add(bigint.from_int(x), bigint.from_int(y), c)
    assert bigint.to_int(s) == x + y
    print(f"{bits:6d}-bit operands: {c['add']:4d} limb additions (32-bit limbs), result {len(s)} limbs")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
    64-bit operands:    2 limb additions (32-bit limbs), result 3 limbs
  1024-bit operands:   32 limb additions (32-bit limbs), result 33 limbs
 16384-bit operands:  512 limb additions (32-bit limbs), result 513 limbs
```

10.4 Adding rationals

Rationals are classes of pairs (Chapter 9), and their sum is defined on representatives:

$$\frac{a}{b} + \frac{c}{d} = \frac{ad + bc}{bd}.$$

The definition is well posed — replacing (a, b) by (ka, kb) multiplies numerator and denominator of the sum by k — but it does not return a *canonical* representative: $\frac{1}{6} + \frac{1}{3} = \frac{9}{18}$, which must be reduced to $\frac{1}{2}$ by dividing by $\gcd(9, 18) = 9$. Exact rational arithmetic therefore pays, on every addition, for three multiplications and a gcd, and its operands can grow: the sum of n unit fractions $1/k$ has a denominator that grows exponentially in n. Rational addition is exact and total on $\mathbb{Q}$; it is not cheap.

10.5 Adding floating-point numbers

IEEE 754 defines floating-point addition as exact addition followed by *one* rounding to the format: $x \oplus y = \mathrm{rnd}(x + y)$ (Chapter 25). Commutativity survives, because $x + y = y + x$ before rounding. Associativity does not:

COUNTEREXAMPLE 10.3 (Floating-point addition is associative) COUNTEREXAMPLE

GIN-NEG-001

In binary64, $(0.1 + 0.1) + 0.4 = 0.6000000000000001$ but $0.1 + (0.1 + 0.4) = 0.6$. Of the 729 triples of one-digit tenths $(0.1, \ldots, 0.9)^3$, 242 give different results under the two groupings (GIN-PROP-032).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e in ["(0.1 + 0.1) + 0.4", "0.1 + (0.1 + 0.4)", "(1e16 + 1) - 1e16"]:
    e2 = e.replace("1e16", "10000000000000000")
    print(f"{e:20s} binary64: {evaluate(e2, 'binary64').display:22s} exact: {evaluate(e2, 'Q').display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
(0.1 + 0.1) + 0.4    binary64: 0.6000000000000001     exact: 3/5
0.1 + (0.1 + 0.4)    binary64: 0.6                    exact: 3/5
(1e16 + 1) - 1e16    binary64: 0                      exact: 1
```

The third line is *absorption*: $10^{16} + 1$ is not representable in binary64 (the spacing of representable numbers near 10^{16} is 2), so the 1 is lost entirely. Addition of floating-point numbers is total, commutative and correctly rounded, and it is not the addition of the real numbers it approximates.

EXERCISE 10.1

Add 2121_3 and 1012_3 with Algorithm 10.1, listing the carries. Check by converting to decimal.

▶ Solution to Exercise 10.1

EXERCISE 10.2

Show that if three numerals are added column by column in base b, the carry never exceeds 2. What is the bound for k numerals?

▶ Solution to Exercise 10.2

EXERCISE 10.3

Which of the five readings of §10.1 give $1 + 1 = 0$, and in which structure?

▶ Solution to Exercise 10.3

EXERCISE 10.4

Compute $\sum_{k=1}^{20} 1/k$ exactly with `fractions.Fraction` and report the number of digits of the reduced denominator. What does this say about the cost of exact rational addition?

▶ Solution to Exercise 10.4

SUMMARY

- Counting, successor, cardinality, algebra and machine define $1 + 1$; they agree in $\mathbb{N}$ by the recursion theorem, and the machine reading agrees only within its encoding and width.
- Column addition is correct by an explicit invariant; the carry is at most 1 in every base.
- Addition of L-bit numerals costs $\Theta(L)$ bit operations and no sequential algorithm does better; on bounded machine words it is one instruction.
- Rational addition is exact but needs a gcd to stay canonical; floating-point addition is commutative, correctly rounded, and not associative.

Chapter 11

Carry: Automata, Monoids, and Parallel Prefix

> *A carry is a message from one column to the next. Addition is fast when the messages can be summarized before they are delivered.*

LEARNING OBJECTIVES

- Describe binary addition as a two-state transducer reading digit pairs from least to most significant.
- Compute in the carry monoid $\{\mathsf{K}, \mathsf{P}, \mathsf{G}\}$ and prove that the carry into each position is a prefix product.
- Explain how associativity turns a sequential computation of depth n into a parallel one of depth $\mathcal{O}(\log n)$, and state the gate cost of doing so.
- State, with the right attribution, the classical theorem that addition is finite-state and multiplication is not.
- Relate carry-lookahead to CGT's arithmetic compilation and grammar-shape trade-off.

11.1 The carry automaton

Column addition (Chapter 10) keeps one piece of state between columns: the carry, which is 0 or 1 in every base. Read the two input numerals in parallel, least significant digit first, as a single word of digit *pairs*; then addition is performed by a machine with two states.

DEFINITION 11.1 (Carry automaton and carry monoid) DEFINITION GIN-DEF-041

Binary addition, read least significant digit first, is a two-state transducer whose state is the carry. On the digit pair (a_i, b_i) it outputs $a_i \oplus b_i \oplus c$ and moves to the new carry, which is a function of the old one:

- K (**kill**, $c \mapsto 0$) for the pair $(0,0)$;
- P (**propagate**, $c \mapsto c$) for $(0,1)$ and $(1,0)$;
- G (**generate**, $c \mapsto 1$) for $(1,1)$.

The **carry monoid** is $\{\mathsf{K}, \mathsf{P}, \mathsf{G}\}$ under composition.

The addition $1 + 1 = 10_2$ is a single generate: the pair $(1,1)$ maps any incoming carry to 1. The successor's worst case, $0111_2 + 1 = 1000_2$, is a generate at position 0 followed by two propagates. The laboratory shows the classes position by position.

LABORATORY (WEBSITE EDITION)

Text alternative: two numbers are written in binary. Each position is classified K (both bits 0), P (exactly one bit 1) or G (both bits 1); the carry into each position is obtained by composing the classes from the lowest position upward. For $11 + 1$ in 8 bits: G at position 0, P at positions 1 and 3, giving carries into positions 1, 2 and the result $1100_2 = 12$.

11.2 The monoid and its prefix products

Composing the effect of a lower position x with the effect of the next higher position y gives a single map; write $x \,;\, y$ for "first x, then y".

PROPOSITION 11.2 (The carry monoid; carries are prefix products) CLASSICAL RESULT

GIN-PROP-020

In the carry monoid, $x \,;\, y = y$ if $y \in \{\mathsf{K}, \mathsf{G}\}$ and $x \,;\, y = x$ if $y = \mathsf{P}$:

$x \,;\, y$	$y = \mathsf{K}$	$y = \mathsf{P}$	$y = \mathsf{G}$
$x = \mathsf{K}$	K	K	G
$x = \mathsf{P}$	K	P	G
$x = \mathsf{G}$	K	G	G

P *is the identity, every element is idempotent, and the operation is associative. The carry into position $i + 1$ is the image of the input carry 0 under the prefix product $x_0 \,;\, x_1 \,;\, \cdots \,;\, x_i$ of the classes of positions $0, \ldots, i$.*

Proof.

K and G are constant maps on $\{0, 1\}$ and P is the identity, so a composite is determined by its last non-P factor: that gives the table. Composition of functions is associative. By induction, the carry after position i is the composite map applied to $c_0 = 0$.

∎

This is the whole theory of carry-lookahead in one table. It is known in substance from the parallel-prefix literature of Ladner and Fischer [Ladner & Fischer 1980] and Brent and Kung [Brent & Kung 1982], who formulated the carry computation with exactly such an associative operator on (generate, propagate) pairs.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.circuits import carry_class, carry_compose, carries
from functools import reduce

x, y, n = 0b01110110, 0b00011011, 8
classes = [carry_class(x >> i & 1, y >> i & 1) for i in range(n)]
prefix = [reduce(carry_compose, classes[:i + 1]) for i in range(n)]
from_prefix = [0] + [1 if p == "G" else 0 for p in prefix]     # image of carry-in 0
print("classes (pos 0..7):", " ".join(classes))
print("prefix products   :", " ".join(prefix))
print("carries via prefix:", from_prefix)
print("carries sequential:", carries(x, y, n))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
classes (pos 0..7): P G P P G P P K
prefix products   : P G G G G G G K
carries via prefix: [0, 0, 1, 1, 1, 1, 1, 1, 0]
carries sequential: [0, 0, 1, 1, 1, 1, 1, 1, 0]
```

11.3 From sequential to parallel

The sequential algorithm computes the prefix products one after another: depth n. But because ; is associative, the product $x_0 \,;\cdots;\, x_i$ can be bracketed in any way — in particular, as a balanced tree of depth $\lceil \log_2(i+1) \rceil$. Computing *all* n prefix products at once is the **parallel prefix** problem, solvable for any associative operation in depth $\mathcal{O}(\log n)$ [Ladner & Fischer 1980; Blelloch 1990]. Different networks trade the number of operations against the depth:

NETWORK	IDEA	COMBINE OPERATIONS	DEPTH
ripple (sequential)	left to right	$n-1$	$n-1$
Sklansky (1960) [Sklansky 1960]	divide and conquer; each half's last prefix broadcast to the other half	$\frac{n}{2}\log_2 n$	$\log_2 n$
Kogge–Stone (1973) [Kogge & Stone 1973]	doubling: at round j combine with the element 2^j positions down	$n\log_2 n - n + 1$	$\log_2 n$
Brent–Kung (1982) [Brent & Kung 1982]	up-sweep a tree, then down-sweep	$\leq 2n - 2 - \log_2 n$	$2\log_2 n - 1$

The SDK builds each adder as an explicit gate-level circuit (gates AND, OR, XOR with two inputs) and counts gates and depth:

EMPIRICAL OBSERVATION 11.3 (Gate counts and depths of four adders)

EMPIRICAL OBSERVATION GIN-EXP-006

For n-bit addition, the circuits of `ginsdk.circuits` have the following (size, depth). All are correct exhaustively for $n \leq 8$ and on random inputs to $n = 64$.

N	RIPPLE-CARRY	SKLANSKY	KOGGE–STONE	BRENT–KUNG
4	17, 7	20, 6	23, 5	20, 6
16	77, 31	128, 10	179, 9	110, 14
64	317, 127	704, 14	1091, 13	488, 22
256	1277, 511	3584, 18	5891, 17	2018, 30

PROPOSITION 11.4 (Ripple-carry cost) PROVED HERE GIN-PROP-021

The n-bit ripple-carry adder of `ginsdk.circuits` (a half adder at position 0, full adders built from XOR, XOR, AND, AND, OR above it) has exactly $5n - 3$ gates and depth $2n - 1$.

Proof.

$2 + 5(n - 1)$ gates. The carry out of position 0 has depth 1, and each full adder adds an AND followed by an OR to the carry path, so the carry out of position $n - 1$ has depth $1 + 2(n - 1)$; the sum bits are no deeper. ∎

How much better can depth get? Not much better than the prefix networks:

PROPOSITION 11.5 (Addition needs logarithmic depth) CLASSICAL RESULT GIN-PROP-059

Every circuit of fan-in-2 gates computing the carry out of n-bit addition has depth at least $\lceil \log_2(2n) \rceil = 1 + \lceil \log_2 n \rceil$.

Proof.

The carry out depends on all $2n$ input bits (Proposition GIN-PROP-058's adversary argument). A gate of depth d with fan-in 2 depends on at most 2^d inputs, so $2^d \geq 2n$. ∎

At $n = 256$ the lower bound is 9; Kogge–Stone achieves 17 — within a factor of two — at the price of more than four times as many gates as the ripple adder. Brent–Kung achieves 30 with fewer than twice as many. This is the typical shape of a trade-off between *work* and *depth*, and it is the arithmetic instance of a theorem of the parent program: the same semantics computed by grammars of different shape has different cost (CGT-THM-004).

11.4 How long are carry chains?

The ripple adder is slow only if carries actually travel far. On random inputs they usually do not.

EMPIRICAL OBSERVATION 11.6 (Mean longest carry chain) EMPIRICAL OBSERVATION

GIN-OBS-002

For random pairs of n-bit numbers, the mean length of the longest carry chain (a generate followed by the propagates it travels through) measured over 3,000 samples (1,000 for $n > 1024$) is $\log_2 n - c$ with c between 0.66 and 0.84 for $n = 4, \ldots, 4096$; the class frequencies are $\mathsf{K} \approx \mathsf{P}/2 \approx \mathsf{G} \approx 1/4$. This agrees with the classical estimate $\log_2 n + \mathcal{O}(1)$ of Burks, Goldstine and von Neumann and of Knuth [Burks, Goldstine & von Neumann 1946; Knuth 1978]; the constant depends on how a chain is counted.

The reason is visible in the monoid: a chain continues only through propagates, each occurring with probability $1/2$, so a chain of length ℓ has probability about $2^{-\ell}$, and among n starting points the longest is about $\log_2 n$. Asynchronous adders that detect completion exploit exactly this: on average they finish in $\mathcal{O}(\log n)$ gate delays even with a ripple structure. Worst-case design, however, must budget for a chain of length n.

11.5 Addition is finite-state; multiplication is not

The carry automaton has two states, whatever the length of the numbers. This puts addition in a very small class of computations.

THEOREM 11.7 (Addition is finite-state; multiplication is not) CLASSICAL RESULT

GIN-THM-005

Fix a base $b \geq 2$ and read numerals least significant digit first, padded to equal length. (1) The graph of addition $\{(x, y, x + y)\}$ is recognized by a synchronous finite automaton, so $(\mathbb{N}, +)$ is an automatic structure. (2) The graph of multiplication is not synchronous-rational in any presentation in which addition is: otherwise $(\mathbb{N}, +, \times)$ would be automatic; every automatic structure has a decidable first-order theory; and the first-order theory of $(\mathbb{N}, +, \times)$ is undecidable.

Proof.

(1) The carry automaton, reading triples of digits and checking the output digit, recognizes the graph. (2) Decidability of automatic structures is due to Hodgson [Hodgson 1983] and Khoussainov and Nerode [Khoussainov & Nerode 1995] (see also Blumensath and Grädel [Blumensath & Grädel 2000]); the decidability of the theory of $(\mathbb{N}, +)$ via automata goes back to Büchi [Büchi 1960]. The undecidability of the theory of $(\mathbb{N}, +, \times)$ follows from Gödel's and Church's work [Gödel 1931; Church 1936]. ∎

The theorem has classical refinements. Multiplication by a *fixed* constant k is finite-state: the carry is bounded by k. Reduction modulo a fixed m is finite-state, read most significant digit first (the divisibility automaton of Chapter 7). Comparison $x < y$ is finite-state. What is not finite-state is the product of two *unbounded* numbers: the "carry" of schoolbook multiplication grows with the length of the operands. In this book's vocabulary, the successor, addition, comparison, multiplication and division by constants, and reduction modulo constants live in the class of **regular** operational grammars over numerals; general multiplication does not.

11.6 The CGT reading

Computational Grammar Theory calls a structure *arithmetically compilable* when every word of moves compiles to an element of a transition monoid that fits in $\mathcal{O}(1)$ machine words (CGT-DEF-011). The carry automaton is the cleanest arithmetic example: any word of digit pairs, however long, compiles to one of three elements. Carry-lookahead *is* this compilation, performed in parallel. Positional numerals themselves (Chapter 5) are the contrasting case: they compile to affine maps whose coefficients grow with the word. The difference between a finite and an infinite transition monoid is the difference between addition, which a finite automaton can do, and multiplication, which it cannot (GIN-REI-005).

CONJECTURE 11.8 (Re-association as a mechanism) CONJECTURE GIN-CONJ-001

CGT's taxonomy of mechanisms of advantage (CGT-DEF-009) should include **M-ASSOC**: when a computation is a product in an associative operation, re-associating a right-linear evaluation into a balanced tree trades work for depth (prefix adders, balanced numeral evaluation, parallel prefix, repeated squaring), and changes *total work* only together with a cost model that rewards balanced operands. Whether M-ASSOC is genuinely distinct from CGT's grammar-shape effect or merely names it is open (GIN-OPEN-004).

EXERCISE 11.1

Classify the positions of $x = 10110111_2$, $y = 01001001_2$ and compute all carries by prefix products. What is the sum?

▶ Solution to Exercise 11.1

EXERCISE 11.2

Verify associativity of ; by checking all 27 triples, and show that the monoid has no inverses except for P.

▶ Solution to Exercise 11.2

EXERCISE 11.3

Design a finite automaton that multiplies by 3: reading x least significant bit first, it outputs the bits of $3x$. How many states does it need?

▶ Solution to Exercise 11.3

EXERCISE 11.4

What is the lower bound of Proposition 11.5 for $n = 64$, and how far from it are the four adders of Empirical observation 11.3?

▶ Solution to Exercise 11.4

SUMMARY

- Binary addition is a two-state transducer; its transitions form the carry monoid $\{\mathsf{K}, \mathsf{P}, \mathsf{G}\}$, in which P is the identity and every element is idempotent.
- Carries are prefix products; associativity lets them be computed in parallel in depth $\mathcal{O}(\log n)$, at the price of more gates. Depth below $1 + \log_2 n$ is impossible.
- Measured: ripple $5n - 3$ gates, depth $2n - 1$; at $n = 256$, Kogge–Stone 5891 gates, depth 17.
- Random carry chains are about $\log_2 n$ long.
- Addition is finite-state and $(\mathbb{N}, +)$ is automatic; multiplication is not finite-state (classical).

PART VI

Subtraction and Inverses

Why $0 - 1$ has no answer in $\mathbb{N}$ and one in $\mathbb{Z}$, and the other ways systems make subtraction total.

Chapter 12

Subtraction and the Construction of the Integers

> *Asked for 0 − 1, the natural numbers have nothing to say. The integers answer −1. A 32-bit register answers 4294967295. All three are correct about their own structure.*

LEARNING OBJECTIVES

- Classify $0 - 1$ in $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Z}/2^w$ and under truncated subtraction, and say what changed between them: the structure, not the operation.
- Construct $\mathbb{Z}$ from pairs of natural numbers, prove that the construction is well defined, and show why it requires the cancellation law.
- State the invariant of column subtraction with borrows.
- Derive hardware subtraction $a - b = a + \bar{b} + 1$ and prove that its carry out is the comparison $a \geq b$.

12.1 $0 - 1$ in four structures

Subtraction is the converse of addition: $a - b$ is the c with $b + c = a$ (Chapter 9).

STRUCTURE	EQUATION $1 + C = 0$	OUTCOME	WHAT KIND OF ANSWER
$\mathbb{N}$	no solution: $1 + c \geq 1$	inadmissible	none
$\mathbb{Z}$	$c = -1$	admissible	the converse solution, after **extension**
$\mathbb{Z}/2^w$ (unsigned machine words)	$c = 2^w - 1$	admissible	the converse solution, after a **quotient**

STRUCTURE	EQUATION $1 + C = 0$	OUTCOME	WHAT KIND OF ANSWER
$\mathbb{N}$ with truncated subtraction $\dot{-}$	—	$0 \dot{-} 1 = 0$	a **totalization**: $1 + 0 \neq 0$

The grammar of the operation — "the c with $b + c = a$" — is the same in the first three rows. What changes is the *structure* in which the equation is solved. Extension enlarges the carrier until a solution exists; quotient identifies numbers modulo 2^w until one does. The fourth row changes the operation instead: $\dot{-}$ is total on $\mathbb{N}$ and agrees with subtraction wherever subtraction is admissible, but at $0 \dot{-} 1$ it returns a value that solves nothing.

PROPOSITION 12.1 (Truncated subtraction is a totalization, not a converse)

PROVED HERE GIN-PROP-016

$m \dot{-} n = \max(m - n, 0)$ is total on $\mathbb{N}$, agrees with subtraction whenever $m \geq n$, and satisfies $(m + n) \dot{-} n = m$; but $(m \dot{-} n) + n = m$ fails whenever $m < n$, so $m \dot{-} n$ is not a solution of $n + c = m$ there.

Proof.

Immediate from the definition: for $m < n$, $(m \dot{-} n) + n = n \neq m$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate, peano

for d in ["N", "Z", "Z/256", "Python", "int32 Java"]:
    o = evaluate("0 - 1", d)
    print(f"{d:10s} {o.status:12s} {o.display or o.reason}")
print("monus(0, 1) =", peano.monus(0, 1), "  and 1 + monus(0, 1) =", 1 + peano.monus(0, 1))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
N          no-solution  b + c ≥ b = 1 > 0 for every natural c (the only integer solution -1 is negative)
Z          value        -1
Z/256      value        255
Python     value        -1
int32 Java value        -1
monus(0, 1) = 0   and 1 + monus(0, 1) = 1
```

12.2 The integers as answers to subtraction problems

How does one build a structure in which every subtraction problem has an answer, without assuming negative numbers exist? The classical answer keeps the unanswered problem as data.

PROPOSITION 12.2 (The Grothendieck construction of $\mathbb{Z}$) CLASSICAL RESULT

GIN-PROP-017

On $\mathbb{N} \times \mathbb{N}$ define $(a,b) \sim (c,d) \iff a+d = b+c$. Then $\sim$ is an equivalence relation; addition $(a,b)+(c,d) = (a+c, b+d)$ and negation $-(a,b) = (b,a)$ are well defined on classes; the classes form an abelian group $\mathbb{Z}$ with zero $[(0,0)]$; and $n \mapsto [(n,0)]$ embeds $(\mathbb{N},+)$ in it. In $\mathbb{Z}$, the class $[(a,b)]$ solves $b + c = a$.

Proof.

Reflexivity and symmetry are clear. Transitivity: $a+d = b+c$ and $c+f = d+e$ give $a+d+f = b+c+f = b+d+e$, and cancelling d gives $a+f = b+e$. Well-definedness of $+$: if $(a,b) \sim (a',b')$ then $(a+c)+(b'+d) = (a'+c)+(b+d)$. The inverse of $[(a,b)]$ is $[(b,a)]$ since $(a+b, b+a) \sim (0,0)$. The embedding is injective: $(m,0) \sim (n,0)$ means $m+0 = 0+n$. Finally $[(b,0)]+[(a,b)] = [(a+b,b)] = [(a,0)]$. ∎

The proof of transitivity used the **cancellation law** $x+d = y+d \Rightarrow x = y$ in $\mathbb{N}$. That use is essential. For a commutative monoid without cancellation, the same construction (with $(a,b) \sim (c,d)$ iff $a+d+k = b+c+k$ for some k) still produces a group — but the map from the monoid into it can collapse elements, sometimes all of them.

PROPOSITION 12.3 (Adjoining negatives to saturating addition destroys it)

PROVED HERE

GIN-PROP-060

Let $M_K = \{0, 1, \dots, K\}$ with saturating addition $x \boxplus y = \min(x+y, K)$. Every homomorphism h from $(M_K, \boxplus, 0)$ to a group sends every element to the identity. In particular the group completion of saturating arithmetic is the trivial group: no structure in which saturating addition has inverses can distinguish any two values.

Proof.

For every x, $K \boxplus x = K$, so $h(K) + h(x) = h(K)$ in the group, hence $h(x) = 0$. ∎

So the integers cannot be obtained from a machine's saturating arithmetic by "adding negatives": the information has already been destroyed. They can be obtained from wraparound arithmetic, because $\mathbb{Z}/2^w$ already is a group — but then they are not the integers.

12.3 Column subtraction and the borrow

The school algorithm for $a - b$ with $a \geq b$ mirrors column addition. Working from the least significant digit with a borrow $\beta_i \in \{0, 1\}$ ($\beta_0 = 0$): let $t = a_i - b_i - \beta_i$; if $t < 0$ write $s_i = t + b$ and borrow $\beta_{i+1} = 1$, else write $s_i = t$ and $\beta_{i+1} = 0$. The invariant is

$$\sum_{j \leq i} s_j b^j - \beta_{i+1}\, b^{i+1} = \sum_{j \leq i} (a_j - b_j)\, b^j, \tag{12.1}$$

proved exactly like Proposition GIN-PROP-057. At the end, $\beta_n = 0$ if and only if $a \geq b$; a final borrow means that the converse problem has no solution in $\mathbb{N}$. The SDK's limb subtraction raises an error in precisely that case, saying so in the vocabulary of this book ("$a < b$ has no solution in N").

12.4 Subtraction in hardware

Processors do not contain a separate subtraction circuit. They add the complement.

PROPOSITION 12.4

(Subtraction as addition of the complement; the carry is the comparison)

PROVED HERE GIN-PROP-061

For w-bit unsigned operands $0 \leq a, b < 2^w$, let $\bar{b} = 2^w - 1 - b$ be the bitwise complement of b. Then

$$a + \bar{b} + 1 = (a - b) + 2^w.$$

Consequently the low w bits of $a + \bar{b} + 1$ are $(a - b) \bmod 2^w$, and the carry out of the top position is 1 exactly when $a \geq b$.

Proof.

$a + (2^w - 1 - b) + 1 = a - b + 2^w$. If $a \geq b$, this is in $[2^w, 2^{w+1})$: carry 1, low bits $a - b$. If $a < b$, it is in $[0, 2^w)$: carry 0, low bits $a - b + 2^w = (a - b) \bmod 2^w$. ∎

The carry out, which addition uses to signal overflow, becomes in subtraction a "no borrow" flag: the hardware answer to the question "does $b + c = a$ have a solution in $\mathbb{N}$?". Instruction sets differ in how they expose it — x86 sets its carry flag to the *borrow* (the complement of this carry), while ARM sets it to the carry itself — another case of one mathematical fact with two machine

encodings. The SDK's gate-level subtractor implements the proposition and is checked exhaustively:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import circuits as K

n = 6
c = K.subtractor(n)
bad = 0
for a in range(2 ** n):
    for b in range(2 ** n):
        out = K.apply_words(c, a=(a, n), b=(b, n))
        value = sum(bit << i for i, bit in enumerate(out[:n]))
        bad += value ≠ (a - b) % 2 ** n or out[n] ≠ (a ⩾ b)
print(f"{n}-bit subtractor: {c.size} gates, depth {c.depth()}; {4 ** n} cases, {bad} errors")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
6-bit subtractor: 36 gates, depth 14; 4096 cases, 0 errors
```

12.5 Signed subtraction and its overflow

In two's complement the same circuit subtracts signed numbers: the bit patterns of $a - b \bmod 2^w$ are the same whether the operands are read as signed or unsigned (Chapter 21). What differs is the overflow condition. Unsigned subtraction overflows ("borrows") when $a < b$; signed subtraction overflows when the exact difference leaves $[-2^{w-1}, 2^{w-1})$, which happens exactly when the operands have different signs and the result's sign differs from a's. The expression $-2147483647 - 2$ in 32-bit arithmetic has the exact value -2147483649, one below the range; wraparound gives 2147483647, ISO C declares it undefined, Rust's debug build panics. These are the execution-layer outcomes of a subtraction that is perfectly admissible in $\mathbb{Z}$.

EXERCISE 12.1

Which pairs in $\mathbb{N}^2$ represent the integer -3? Show that $[(2,5)] + [(7,1)] = [(3,0)]$ directly from the definitions.

▶ Solution to Exercise 12.1

EXERCISE 12.2

Which of these hold for all $m, n, p \in \mathbb{N}$? (a) $(m \mathbin{\dot{-}} n) \mathbin{\dot{-}} p = m \mathbin{\dot{-}} (n + p)$; (b) $m \mathbin{\dot{-}} (n \mathbin{\dot{-}} p) = (m \mathbin{\dot{-}} n) + p$; © $m \mathbin{\dot{-}} m = 0$.

▶ Solution to Exercise 12.2

EXERCISE 12.3

Using Proposition 12.4, compute the 8-bit result and carry for $a = 5$, $b = 9$. What does the result mean as a signed number?

▶ Solution to Exercise 12.3

EXERCISE 12.4

Find a commutative monoid that is not cancellative but whose group completion is not trivial.

▶ Solution to Exercise 12.4

SUMMARY

- $0 - 1$ has no solution in $\mathbb{N}$, the solution -1 in $\mathbb{Z}$ (extension), $2^w - 1$ in $\mathbb{Z}/2^w$ (quotient); truncated subtraction gives 0 without solving anything (totalization).
- $\mathbb{Z}$ is built from pairs (a, b) standing for "$a - b$"; the construction needs cancellation, and saturating addition, which lacks it, collapses completely.
- Column subtraction tracks a borrow with an invariant; a final borrow signals that the converse problem has no natural solution.
- Hardware computes $a - b$ as $a + \bar{b} + 1$; the carry out is the comparison $a \geq b$.

Chapter 13

Closure, Partiality, and the Price of Totality

> *A total operation is a promise that an answer always exists. Every way of keeping that promise is paid for in some other law.*

LEARNING OBJECTIVES

- Tabulate the closure of $+, -, \times, /, \sqrt{}$ over the main number systems, and name the kind of failure where an operation is not closed.
- Compare four totalizations of fixed-width subtraction by the laws they preserve.
- Prove, by exhaustive computation, that saturating 8-bit addition is commutative but not associative, and quantify how often associativity fails.
- Explain why the choice of totalization matters to compilers and to programmers.

13.1 A closure table

For each number system and operation, the entry gives the outcome of the converse classification of Chapter 9: total (✓), or which kind of failure occurs and where.

	$+$	$\times$	$-$	$/$	$\sqrt{}$ (NON-NEGATIVE BRANCH)
$\mathbb{N}$	✓	✓	no solution when $a < b$	no solution unless $b \neq 0$, $b \mid a$; non-unique at $0/0$	no solution unless a is a square
$\mathbb{Z}$	✓	✓	✓	as in $\mathbb{N}$	no solution for $a < 0$ or non-squares
$\mathbb{Q}$	✓	✓	✓	no solution at $b = 0$, $a \neq 0$; non-unique at $0/0$	no solution unless a is a rational square

	$+$	$\times$	$-$	$/$	$\sqrt{}$ (NON-NEGATIVE BRANCH)
$\mathbb{R}$	✓	✓	✓	as in $\mathbb{Q}$	no solution for $a < 0$; the branch selects ≥ 0
$\mathbb{C}$	✓	✓	✓	as in $\mathbb{Q}$	✓ with a branch (two roots for $a \neq 0$)
$\mathbb{Z}/n$	✓	✓	✓	no solution or non-unique when $\gcd(b, n) > 1$	depends on a and n (Chapter 29)
$\mathbb{Z}/p$, p prime	✓	✓	✓	as in $\mathbb{Q}$	about half the non-zero elements are squares

Two patterns stand out. Subtraction stops failing as soon as negative numbers exist, and never fails again; division by zero fails in every row; and the *kind* of failure of division by zero — no solution for $a \neq 0$, non-unique for $a = 0$ — is the same in every row. Chapter 16 proves that it must be.

13.2 Four ways to make subtraction total

On a machine, numbers have a fixed width, and even $\mathbb{Z}$-subtraction can leave the representable range. The systems in use handle this in four ways.

TOTALIZATION	W-BIT RESULT OF $A - B$ WHEN THE EXACT RESULT DOES NOT FIT	USED BY
wraparound (quotient)	the exact result reduced modulo 2^w	hardware `sub` ; Java, Go; Rust release builds; C unsigned
saturation (clamp)	the nearest representable value	DSP and SIMD saturating instructions; some graphics code
truncation ($\dot{-}$, for unsigned)	0 when $a < b$	Peano arithmetic; `saturating_sub` on unsigned values
signal	no value: exception, trap or panic	Rust debug builds; checked arithmetic; Python never needs it

(A fifth answer, ISO C's for *signed* overflow, is not a totalization at all: the behaviour is undefined, so a program that overflows has no meaning.)

Each is reasonable, and each preserves different laws. Which laws?

PROPOSITION 13.1 (Laws kept by wraparound and by saturation (8-bit census))

PROVED HERE

GIN-PROP-062

On 8-bit signed integers $[-128, 127]$*:*

1. *wraparound addition* $a \oplus b = \mathrm{wrap}(a+b)$ *forms an abelian group, and wraparound* $+$ *and* $\times$ *form a commutative ring: it is the ring* $\mathbb{Z}/256$ *with the signed labels;*
2. *saturating addition* $a \boxplus b = \max(-128, \min(127, a+b))$ *is commutative and has identity* 0*, but is* ***not associative****: exactly* $4\,177\,792$ *of the* 2^{24} *triples* (a,b,c) *— about* 24.9% *— satisfy* $(a \boxplus b) \boxplus c \neq a \boxplus (b \boxplus c)$*;*
3. *saturating addition has no inverse for* -128*, and saturating multiplication does not distribute over saturating addition.*

Proof.

(1) Reduction modulo 256 is a ring homomorphism $\mathbb{Z} \to \mathbb{Z}/256$, and the signed labels are a bijection with $\mathbb{Z}/256$. (2) Commutativity and identity are inherited from $\mathbb{Z}$, since clamping is applied to the same exact sum. The associativity count is by exhaustive enumeration of all 2^{24} triples (reproduced below). (3) $-128 \boxplus b = 0$ requires $b = 128$, which is not representable; a counterexample to distributivity is $a = 2$, $b = 100$, $c = -100$: $2 \cdot (100 \boxplus -100) = 0$ but $(2 \cdot 100) \boxplus (2 \cdot -100)$ clamps to $127 \boxplus -128 = -1$. ∎

COUNTEREXAMPLE 13.2 (Saturating addition is associative) COUNTEREXAMPLE

GIN-NEG-021

In 8-bit signed saturating arithmetic, $(100 \boxplus 100) \boxplus (-100) = 127 \boxplus (-100) = 27$, while $100 \boxplus (100 \boxplus (-100)) = 100 \boxplus 0 = 100$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
lo, hi = -128, 127
sat = lambda x: max(lo, min(hi, x))
wrap = lambda x: (x - lo) % 256 + lo
R = range(lo, hi + 1)
bad_sat = sum(sat(sat(a + b) + c) != sat(a + sat(b + c)) for a in R for b in R for c in R)
bad_wrap = sum(wrap(wrap(a + b) + c) != wrap(a + wrap(b + c)) for a in R for b in R for c in R)
print(f"associativity failures, saturating: {bad_sat} of {256 ** 3} ({bad_sat / 256 ** 3:.2%})")
```

```
print(f"associativity failures, wraparound: {bad_wrap}")
print("example:", sat(sat(100 + 100) - 100), "vs", sat(100 + sat(100 - 100)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
associativity failures, saturating: 4177792 of 16777216 (24.90%)
associativity failures, wraparound: 0
example: 27 vs 100
```

13.3 Why the choice matters

The census is not a curiosity. Associativity is what allows a compiler to evaluate $a + b + c + d$ as $(a + b) + (c + d)$ and use two adders in parallel, or to vectorize a sum. For wraparound integers the transformation is always valid, which is why compilers may freely reassociate unsigned C arithmetic and Java `int` arithmetic. For saturating arithmetic it is invalid, and code using saturating instructions must be evaluated in exactly the written order. For IEEE floating point it is also invalid (Chapter 25), which is why compilers do not reassociate floating-point sums unless explicitly permitted to (for example with "fast-math" options that trade exactness for speed).

ISO C's choice for signed integers is the most interesting. By making signed overflow *undefined*, the standard allows a compiler to assume it never happens, and therefore to treat signed arithmetic as the exact arithmetic of $\mathbb{Z}$ — reassociating, simplifying $x + 1 > x$ to *true*, and eliminating code that tests for overflow after the fact. The totalization is replaced by a promise from the programmer. Chapter 24 measures a compiler acting on such a promise.

EXERCISE 13.1

Fill in the closure table row for the even integers $2\mathbb{Z}$ under $+$, $\times$, $-$, $/$.

▶ Solution to Exercise 13.1

EXERCISE 13.2

For 2-bit signed saturating addition on $\{-2, -1, 0, 1\}$, count the non-associative triples by hand or by program.

▶ Solution to Exercise 13.2

EXERCISE 13.3

Is truncated subtraction associative on $\mathbb{N}$? Is it commutative? Give the laws it does satisfy that ordinary subtraction on $\mathbb{Z}$ also satisfies.

▶ Solution to Exercise 13.3

SUMMARY

- Subtraction is closed from $\mathbb{Z}$ upward; division by zero fails in every number system, always in the same two ways.
- Machines make subtraction total by wraparound, saturation, truncation or signal; ISO C instead makes signed overflow undefined.
- Wraparound is the ring $\mathbb{Z}/2^w$ and keeps every ring law; saturating 8-bit addition is commutative but fails associativity on 24.9% of triples, and has no inverse for -128.
- Associativity is what licenses reassociation and parallel evaluation; a totalization that loses it constrains compilers and programmers.

PART VII

Multiplication

Multiplication as recursion, product and scaling; why ×0 gives 0; and the algorithms from shift-and-add to O(n log n).

Chapter 14

Multiplication: Recursion, Products, and Scaling

> *2 + 2 + 2 and 3 + 3 are different computations. That they agree is not arithmetic's notation speaking but its structure.*

LEARNING OBJECTIVES

- Define multiplication in six ways and say which assumptions each makes.
- Prove commutativity of multiplication by induction and by a bijection, and explain the difference between the two proofs.
- Derive $a \cdot 0 = 0$ in four frameworks, and identify the framework in which it is an axiom rather than a theorem.
- State the distributive law as the bridge between addition and multiplication, and see where it fails.

14.1 Six readings of $2 \times 3 = 6$

1. **Repeated addition (primitive recursion).** $\mathrm{mul}(2,3) = \mathrm{mul}(2,2) + 2 = (\mathrm{mul}(2,1) + 2) + 2 = ((0 + 2) + 2) + 2$: "three twos".
2. **Cardinality of a product.** $|A \times B| = |A| \cdot |B|$: two rows of three seats.
3. **Scaling.** The map $x \mapsto 3x$ applied to 2; multiplication by a number is a *move*, $[\![\times 3]\!]$.
4. **Composition of scalings.** $x \mapsto 2x$ followed by $x \mapsto 3x$ is $x \mapsto 6x$: multiplication of numbers is composition of the maps they define. This reading extends to matrices, where composition is not commutative.
5. **Iteration.** The Church numeral $\underline{2}$ applied to $\underline{3}(f)$ iterates f six times (Chapter 4).
6. **Machine.** $10_2 \times 11_2$: two shifted copies of 10_2 (one for each 1 in 11_2) added: $10_2 + 100_2 = 110_2$.

The readings have different domains. Repeated addition needs a natural-number multiplier; cardinality needs finite sets; scaling and composition work for any ring; iteration needs a natural number acting on a set; the machine reading needs fixed-width binary words. They agree where their domains overlap — and showing that they agree is the content of several classical theorems.

14.2 Commutativity is a theorem

By reading 1, 2×3 is $2 + 2 + 2$ and 3×2 is $3 + 3$. The derivations have different shapes and lengths:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import peano as P

for m, n in [(2, 3), (3, 2)]:
    nf, trace = P.rewrite(("mul", P.num(m), P.num(n)))
    print(f"mul({m}, {n}) → {P.show(nf)} in {len(trace)} rule applications")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
mul(2, 3) → SSSSSS0 in 13 rule applications
mul(3, 2) → SSSSSS0 in 11 rule applications
```

That they reach the same normal form is not visible in either derivation. It is a theorem.

PROPOSITION 14.1 (Why 2 × 3 = 3 × 2) CLASSICAL RESULT GIN-PROP-013

For all $m, n \in \mathbb{N}$, $m \cdot n = n \cdot m$. Two proofs: (i) by induction from the primitive-recursive definitions; (ii) by the bijection $(x, y) \mapsto (y, x)$ between $A \times B$ and $B \times A$.

Proof.

(i) First, $0 \cdot n = 0$ by induction on n. Then $Sm \cdot n = m \cdot n + n$ by induction on n: for $n = 0$ both sides are 0; and $Sm \cdot Sn = Sm \cdot n + Sm = (m \cdot n + n) + Sm = m \cdot n + m + Sn = m \cdot Sn + Sn$ using Proposition GIN-HIST-004. Finally $m \cdot n = n \cdot m$ by induction on m: $0 \cdot n = 0 = n \cdot 0$, and $Sm \cdot n = m \cdot n + n = n \cdot m + n = n \cdot Sm$. (ii) If $|A| = m$ and $|B| = n$, the swap is a bijection $A \times B \to B \times A$, so the two products have equal cardinality. ∎

The two proofs illustrate a recurring choice. Proof (i) stays inside the grammar of numerals and derivations, and is mechanical: it could be checked by a proof assistant step by step. Proof (ii) leaves the grammar for a different structure — finite sets — where the law is *visible*, and transfers it back through the correspondence between numbers and cardinalities. Neither proof is more correct. They show the same equality from two sides: the computational and the semantic.

14.3 Why $a \cdot 0 = 0$: four derivations

That anything times zero is zero is learned early and rarely questioned. It is worth questioning, because the answer depends on the framework, and because Part VIII's analysis of division by zero rests on it.

PROPOSITION 14.2 (Why a · 0 = 0, derived four ways) CLASSICAL RESULT GIN-PROP-012

1. ***Ring axioms.*** *In any ring,* $0 \cdot a = (0 + 0) \cdot a = 0 \cdot a + 0 \cdot a$*; adding* $-(0 \cdot a)$ *to both sides gives* $0 \cdot a = 0$*. The derivation uses distributivity and the existence of additive inverses (cancellation).*
2. ***Peano recursion.*** $a \cdot 0 = 0$ *is the base equation of the definition of* mul*;* $0 \cdot a = 0$ *follows by induction on* a*.*
3. ***Cardinality.*** $|\varnothing \times A| = 0$*, because there are no pairs whose first component lies in the empty set.*
4. ***Repeated addition.*** $a \cdot 0$ *is the sum of zero copies of* a *— the empty sum, which is the additive identity* 0*.*

In a **semiring** — a structure with addition and multiplication but without additive inverses, such as $\mathbb{N}$, or the "tropical" semiring $(\mathbb{R} \cup \{\infty\}, \min, +)$ used in shortest-path algorithms — derivation 1 is unavailable, because one cannot subtract $0 \cdot a$. There, $0 \cdot a = 0$ (absorption) must be *assumed as an axiom*, and there are natural structures in which it is false. The law that "zero annihilates" is therefore a theorem in rings, a definition in recursive arithmetic, and an extra assumption in semirings. Chapter 18 shows that the systems which give $1/0$ a value are exactly those that weaken one of the ingredients of derivation 1.

14.4 Distributivity, the bridge

Addition and multiplication are connected by one law:

$$a \cdot (b + c) = a \cdot b + a \cdot c. \tag{14.1}$$

Every algorithm for multiplication uses it. Long multiplication writes $a \cdot b = a \cdot \sum_i b_i 10^i = \sum_i (a \cdot b_i)\, 10^i$; binary shift-and-add is the same with base 2; Karatsuba's algorithm (Chapter 15) is a clever regrouping of the same expansion. Without distributivity, no multiplication algorithm based on digits would be correct.

Distributivity is exactly the law that fails in many machine arithmetics. In floating point, $a \otimes (b \oplus c)$ and $(a \otimes b) \oplus (a \otimes c)$ round at different points and generally differ. In saturating

arithmetic, the clamps happen at different points (Chapter 13). In wraparound arithmetic, by contrast, distributivity holds exactly, because $\mathbb{Z}/2^w$ is a ring.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for d in ["Q", "binary64", "int32 Java"]:
    lhs = evaluate("0.1 * (0.2 + 0.3)" if d ≠ "int32 Java" else "65536 * (65536 + 1)", d)
    rhs = evaluate("0.1 * 0.2 + 0.1 * 0.3" if d ≠ "int32 Java" else "65536 * 65536 + 65536 * 1", d)
    print(f"{d:11s} a(b+c) = {lhs.display:22s} ab+ac = {rhs.display:22s} equal: {lhs.display == rhs.display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
Q           a(b+c) = 1/20                  ab+ac = 1/20                  equal: True
binary64    a(b+c) = 0.05                  ab+ac = 0.05                  equal: True
int32 Java  a(b+c) = 65536                 ab+ac = 65536                 equal: True
```

The binary64 line shows that the law *can* hold for particular values even where it is not a law. It fails for others. The first guess an author might make, $0.1 \cdot (0.1 + 0.2)$, happens to satisfy the law in binary64; a search over the 512 triples drawn from $\{0.1, 0.2, 0.3, 0.5, 0.7, 1.1, 3, 100\}$ finds 116 that do not, the first being $0.1 \cdot (0.1 + 0.3)$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate
for e in ["0.1 * (0.1 + 0.2)", "0.1 * 0.1 + 0.1 * 0.2", "0.1 * (0.1 + 0.3)", "0.1 * 0.1 + 0.1 * 0.3"]:
    print(f"{e:22s} = {evaluate(e, 'binary64').display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0.1 * (0.1 + 0.2)      = 0.030000000000000006
0.1 * 0.1 + 0.1 * 0.2  = 0.030000000000000006
0.1 * (0.1 + 0.3)      = 0.04000000000000001
0.1 * 0.1 + 0.1 * 0.3  = 0.04
```

The Java line deserves a second look: $65536 \cdot 65537 = 4295032832 = 2^{32} + 65536$, which wraps to 65536 — on both sides. Wraparound arithmetic gets the *wrong* answer, but it gets it *consistently*: the law holds in the ring $\mathbb{Z}/2^{32}$ even though neither side equals the integer product.

14.5 Multiplication by constants

Multiplying by a fixed number is a much simpler operation than multiplying two variable numbers. In binary, multiplication by 2^k is a shift by k positions — no arithmetic at all, just a change of the numeral. Multiplication by a small constant c is a few shifts and additions: $10x = 8x + 2x = (x \ll 3) + (x \ll 1)$. And, as Chapter 11 noted, multiplication by a constant is a finite-state operation on numerals, while general multiplication is not (GIN-THM-005). Compilers exploit both facts constantly: no compiler emits a multiply instruction for `x * 8`, and division by a constant is routinely replaced by a multiplication by a precomputed reciprocal and a shift (Chapter 19).

EXERCISE 14.1

Count the rule applications of $\mathrm{mul}(m, n)$ and $\mathrm{mul}(n, m)$ for $(m, n) = (5, 1)$ and $(1, 5)$. Which order is cheaper, and why?

▶ Solution to Exercise 14.1

EXERCISE 14.2

In the tropical semiring $(\mathbb{R} \cup \{\infty\}, \oplus = \min, \otimes = +)$, what is the additive identity "zero", what is the multiplicative identity "one", and does "zero annihilates" hold?

▶ Solution to Exercise 14.2

EXERCISE 14.3

Express multiplication by 15 and by 100 with as few shifts and additions/subtractions as you can.

▶ Solution to Exercise 14.3

SUMMARY

- Multiplication has six readings — repeated addition, product cardinality, scaling, composition, iteration, shift-and-add — with different domains that agree where they overlap.
- $2 \times 3 = 3 \times 2$ is a theorem about values; the derivations differ in shape and cost.
- $a \cdot 0 = 0$ is a theorem in rings (using inverses), a definition in Peano arithmetic, and an axiom in semirings.
- Distributivity connects $+$ and $\times$ and underlies every digit-based algorithm; it holds in wraparound arithmetic and fails in floating point.

Chapter 15

Algorithms for Multiplication

> *Four products of halves give the product of wholes. Karatsuba noticed that three are enough — and the exponent of multiplication fell for the first time in four thousand years.*

LEARNING OBJECTIVES

- Analyse long multiplication: its invariant, its $\Theta(n^2)$ digit operations, and its gate-level form, the array multiplier.
- Derive Karatsuba's algorithm from distributivity and solve its recurrence.
- Read measured operation counts and log–log slopes, and relate them to the theoretical exponents.
- State what is known about the complexity of integer multiplication, separating proved upper bounds from conjectured lower bounds.

15.1 Long multiplication

To multiply an n-digit numeral a by an m-digit numeral b, long multiplication forms the partial products $a \cdot b_j$ (one digit of b at a time), shifts the j-th by j places, and adds them. The algorithm is distributivity applied to the positional expansion of b:

$$a \cdot b = a \cdot \sum_j b_j \, \beta^j = \sum_j (a \cdot b_j) \, \beta^j,$$

where β is the base. In the usual organization each digit product $a_i b_j$ is formed once and added into position $i + j$ with carries. Its invariant after processing digit j of b is that the accumulator equals $a \cdot \sum_{k \le j} b_k \beta^k$. It performs nm digit multiplications and about as many additions: $\Theta(nm)$ digit operations, $\Theta(n^2)$ for equal lengths.

In binary, each digit b_j is 0 or 1, so each partial product is either 0 or a itself: **shift and add**. As a circuit, every product bit $a_i b_j$ is one AND gate, and the shifted partial products are summed by rows of adders — the **array multiplier**.

IMPLEMENTATION RESULT 15.1 (The array multiplier: size and depth)

IMPLEMENTATION RESULT

GIN-IMP-006

The $n \times n \to 2n$-bit array multiplier of `ginsdk.circuits` (AND partial products summed by ripple-carry rows) has exactly $6n^2 - 8n$ gates and depth $6n - 8$ for $n = 3, \ldots, 32$ (for $n = 2$: 8 gates, depth 3), and computes the product correctly for all inputs with $n \leq 6$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import circuits as K

for n in (4, 8, 16, 32):
    c = K.array_multiplier(n)
    print(f"n = {n:2d}: {c.size:5d} gates (6n²-8n = {6*n*n - 8*n:5d}), depth {c.depth():3d} (6n-8 = {6*n -
8})")
n, bad = 5, 0
c = K.array_multiplier(n)
for x in range(2 ** n):
    for y in range(2 ** n):
        out = K.apply_words(c, a=(x, n), b=(y, n))
        bad += sum(bit << i for i, bit in enumerate(out)) != x * y
print(f"exhaustive check, n = {n}: {bad} errors in {4 ** n} products")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n =  4:    64 gates (6n²-8n =    64), depth  16 (6n-8 = 16)
n =  8:   320 gates (6n²-8n =   320), depth  40 (6n-8 = 40)
n = 16:  1408 gates (6n²-8n =  1408), depth  88 (6n-8 = 88)
n = 32:  5888 gates (6n²-8n =  5888), depth 184 (6n-8 = 184)
exhaustive check, n = 5: 0 errors in 1024 products
```

The quadratic size is unavoidable for this design: it forms all n^2 bit products explicitly. Real multipliers reduce the *depth* — not the size — by summing partial products with trees of carry-save adders (Wallace and Dadda trees), which use the redundant representations of Chapter 6 to postpone carry propagation until a single final addition. Depth then falls to $\mathcal{O}(\log n)$; size stays $\Theta(n^2)$.

15.2 Karatsuba: three products instead of four

For a long time it was assumed that multiplying n-digit numbers needs about n^2 digit operations. In 1960 Karatsuba found otherwise [Karatsuba & Ofman 1962]. Split each operand in half, $a = a_1\beta^k + a_0$ and $b = b_1\beta^k + b_0$. Distributivity gives

$$ab = a_1 b_1 \, \beta^{2k} + (a_1 b_0 + a_0 b_1) \, \beta^{k} + a_0 b_0,$$

four half-size products. But the middle coefficient can be obtained from *one* further product:

$$a_1 b_0 + a_0 b_1 = (a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0. \tag{15.1}$$

So three half-size products, and a constant number of additions, suffice.

PROPOSITION 15.2 (Karatsuba's bound) CLASSICAL RESULT GIN-PROP-063

Karatsuba multiplication of two n-digit numbers uses $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ digit operations.

Proof.

The cost satisfies $T(n) = 3\,T(n/2) + \Theta(n)$ (three recursive products of half size, plus additions and shifts of linear cost; the sums $a_1 + a_0$ may have one extra digit, which does not change the asymptotics). Unfolding $\log_2 n$ levels gives $3^{\log_2 n} = n^{\log_2 3}$ leaf multiplications and a geometric sum of linear work dominated by the last level. ∎

The algorithm is the clearest example in this book of a gain obtained *from algebra alone*: identity (15.1) is a rearrangement of the distributive law, and it changes the exponent of the cost. The SDK counts the 32-bit limb operations of both algorithms on random operands:

EMPIRICAL OBSERVATION 15.3

(Measured exponents of schoolbook and Karatsuba multiplication)

EMPIRICAL OBSERVATION GIN-OBS-010

Counted 32-bit limb operations (GIN-EXP-008), random operands, Karatsuba threshold 16 limbs:

BITS	SCHOOLBOOK LIMB PRODUCTS	KARATSUBA LIMB PRODUCTS
1,024	2,048	1,796
4,096	32,768	18,409
16,384	524,288	172,598

The log–log slope of the Karatsuba counts between successive sizes falls from 2.0 (below the threshold) to 1.604 at 16,384 bits, approaching $\log_2 3 \approx 1.585$; schoolbook stays at 2.0. At 16,384 bits Karatsuba uses 3.04 times fewer limb operations.

The threshold matters. Below about sixteen limbs, the additions of Karatsuba cost more than the product they save, and every practical library — GMP, the multiplication inside CPython's `int` — switches between algorithms at tuned thresholds [Brent & Zimmermann 2010].

15.3 Toward $\mathcal{O}(n \log n)$

Karatsuba's idea generalizes. Splitting into three parts and evaluating at five points gives Toom–Cook multiplication, with exponent $\log_3 5 \approx 1.465$; more parts give exponents approaching 1. The decisive step treats the digits of a number as the coefficients of a polynomial, multiplies the polynomials by evaluating them at roots of unity with the fast Fourier transform, and carries at the end. Schönhage and Strassen's algorithm of 1971 achieved $\mathcal{O}(n \log n \log \log n)$; after a sequence of improvements, Harvey and van der Hoeven proved in 2021 that n-bit integers can be multiplied in $\mathcal{O}(n \log n)$ bit operations [Harvey & van der Hoeven 2021; Knuth 1997; Brent & Zimmermann 2010].

What is *not* known is a matching lower bound. No proof shows that multiplication requires more than linear time in general models of computation; the $\mathcal{O}(n \log n)$ bound is conjectured to be optimal, and is known to be optimal only under additional assumptions. In the summary table of costs (Chapter 36) the entry for multiplication therefore has a proved upper bound and an open lower bound — unlike addition, whose $\Theta(n)$ is settled (GIN-PROP-058).

ALGORITHM	DIGIT-OPERATION COUNT	IN PRACTICE
repeated addition	$\Theta(\beta^n \cdot n)$ — exponential	never
long multiplication	$\Theta(n^2)$	small operands
Karatsuba	$\Theta(n^{1.585})$	from tens of machine words
Toom–Cook 3-way	$\Theta(n^{1.465})$	hundreds of words
FFT-based (Schönhage–Strassen)	$\mathcal{O}(n \log n \log \log n)$	thousands of words and up
Harvey–van der Hoeven	$\mathcal{O}(n \log n)$	a theoretical result; the constant is astronomically large

15.4 Multiplication in machines

A processor's multiply instruction computes a product of two words in a few cycles with a dedicated tree multiplier; on 64-bit machines it typically produces either the low 64 bits (enough for wraparound arithmetic) or the full 128-bit product split over two registers. Wraparound multiplication is multiplication in $\mathbb{Z}/2^{64}$, a ring, so all ring laws hold for it — including the ones that make overflow invisible: $2^{32} \cdot 2^{32} = 0$ in 64-bit unsigned arithmetic. Multiplication is where overflow happens soonest: the product of two w-bit numbers needs up to $2w$ bits, so even moderately sized operands overflow a word. Arbitrary-precision libraries exist largely because of this.

EXERCISE 15.1

Multiply 47×83 by Karatsuba's method with $\beta = 10$, $k = 1$, showing the three products.

▶ Solution to Exercise 15.1

EXERCISE 15.2

Solve $T(n) = 4T(n/2) + n$ and $T(n) = 3T(n/2) + n$ for n a power of two with $T(1) = 1$, exactly.

▶ Solution to Exercise 15.2

EXERCISE 15.3

Show that in $\mathbb{Z}/2^{64}$ the product of two non-zero elements can be zero, and find all pairs of powers of two with that property.

▶ Solution to Exercise 15.3

SUMMARY

- Long multiplication is distributivity over digits: $\Theta(n^2)$; its circuit form, the array multiplier, has $6n^2 - 8n$ gates in this book's construction.
- Karatsuba rearranges distributivity to use three half-size products: $\Theta(n^{\log_2 3})$; measured slopes approach 1.585.
- FFT methods reach $\mathcal{O}(n \log n)$ (Harvey–van der Hoeven 2021); no superlinear lower bound is known.
- Machine multiplication is multiplication in $\mathbb{Z}/2^w$: a ring with zero divisors, where overflow is silent.

PART VIII

Division and the Boundary of Zero

The central part: division as a converse, the image/kernel theorem, 1/0 versus 0/0, every way of giving 1/0 a value and what it costs, division with remainder, and the grammar of zero.

Chapter 16

Division as the Converse of Multiplication

> *To divide a by b is to ask a question about multiplication: which c, multiplied by b, gives a? Everything about division follows from how that question can fail.*

LEARNING OBJECTIVES

- Give six readings of $6/3 = 2$ and identify the one that defines division in every ring.
- State and prove the image/kernel theorem: the solutions of $b\,c = a$ form either the empty set or a coset of the annihilator of b.
- Analyse $0/1 = 0$ completely, and explain why it is not the mirror image of $1/0$.
- Decide admissibility of every division in $\mathbb{Z}/n$, and count exactly how many pairs have no quotient, one quotient, or several.
- Characterize the rings in which division, when possible, is unique.

16.1 Six readings of $6/3 = 2$

1. **Inverse multiplication.** $6/3$ is the number c with $3 \cdot c = 6$; $c = 2$.
2. **Quotient and remainder.** $6 = 2 \cdot 3 + 0$: the quotient 2 with remainder 0.
3. **Partition.** Six objects in three equal groups: two per group.
4. **Repeated subtraction (measurement).** $6 - 3 - 3 = 0$: three can be taken away twice.
5. **Ratio.** $6 : 3 = 2 : 1$.
6. **Field operation.** $6 \cdot 3^{-1}$, where 3^{-1} is the multiplicative inverse of 3 in $\mathbb{Q}$.

And the machine readings: binary long division $110_2 \div 11_2 = 10_2$ as two compare-and-subtract rows of a restoring divider; x86's `idiv`, AArch64's `sdiv`, RISC-V's `div`; Knuth's Algorithm D on multi-word integers.

The readings agree on $6/3$ and disagree elsewhere. Reading 2 gives a value for $7/3$ (quotient 2, remainder 1) where reading 1 gives none in $\mathbb{Z}$. Reading 6 requires an inverse, which 0 never has, while reading 1 merely requires a solution, which $0/0$ has in abundance. Reading 3 makes no sense for $6/0$ ("zero groups") and reading 4 never terminates for it. To analyse division by zero

precisely we need one reading that works in every ring and does not presuppose the answer. Reading 1 is that reading.

DEFINITION 16.1 (Division as a converse problem) DEFINITION GIN-DEF-015

In a commutative ring R, the **quotient** a/b is defined when the converse problem

$$\mathrm{Sol}(a,b) = \{\, c \in R : b \cdot c = a \,\}$$

has exactly one element, and then a/b is that element. Otherwise a/b is inadmissible: with *no solution* if $\mathrm{Sol}(a,b) = \varnothing$, *non-unique* if $|\,\mathrm{Sol}(a,b)| \geq 2$.

16.2 The image/kernel theorem

The map $c \mapsto b \cdot c$ is additive: $b(c + c') = bc + bc'$. By the converse theorem for homomorphisms (GIN-THM-007), its solution sets are cosets of its kernel. The kernel of $c \mapsto bc$ has a classical name.

DEFINITION 16.2 (Annihilator) DEFINITION GIN-DEF-091

The **annihilator** of b in a commutative ring R is $\mathrm{Ann}(b) = \{\, t \in R : bt = 0 \,\}$. An element $b \neq 0$ with $\mathrm{Ann}(b) \neq \{0\}$ is a **zero divisor**.

THEOREM 16.3 (The structure of a converse in a commutative ring) CLASSICAL RESULT GIN-THM-001

Let R be a commutative ring and $a, b \in R$. Then $\mathrm{Sol}(a,b)$ is either empty or a coset $c_0 + \mathrm{Ann}(b)$. Hence:

1. ***image condition**: $\mathrm{Sol}(a,b) \neq \varnothing$ if and only if $a \in bR$;*
2. ***kernel condition**: when $\mathrm{Sol}(a,b) \neq \varnothing$, it has exactly $|\,\mathrm{Ann}(b)|$ elements; the quotient is unique if and only if $\mathrm{Ann}(b) = \{0\}$.*

Proof.

If $c_0 \in \mathrm{Sol}(a,b)$, then $bc = a \iff bc = bc_0 \iff b(c - c_0) = 0 \iff c - c_0 \in \mathrm{Ann}(b)$. ∎

The theorem says that a division can fail in exactly two ways, and that the two ways are independent of each other:

- an **image failure**: a is not a multiple of b, so *nothing* solves the equation;
- a **kernel failure**: b annihilates something non-zero, so *if* anything solves the equation, many things do.

The research ledger checks the theorem exhaustively on 30 finite commutative rings — $\mathbb{Z}/n$ for $n \le 24$, five products $\mathbb{Z}/m \times \mathbb{Z}/k$, and $\mathbb{F}_2[x]/(x^3)$ — over all 5737 pairs (a, b) (GIN-THM-001).

16.3 $0/1 = 0$, completely

Many people hesitate at $0/1$. The converse reading removes the hesitation, and shows that $0/1$ and $1/0$ are not mirror images of each other.

PROPOSITION 16.4 (0 / 1 = 0 as a complete case) CLASSICAL RESULT GIN-PROP-011

In every commutative ring with 1, $0/1$ *is admissible and equals* 0*: the image condition holds because* $0 = 1 \cdot 0 \in 1 \cdot R$*, and the kernel condition holds because* $\mathrm{Ann}(1) = \{0\}$*. More generally* $0/b = 0$ *whenever* $\mathrm{Ann}(b) = \{0\}$.

Proof.

$1 \cdot t = t$, so $1 \cdot t = 0$ only for $t = 0$; and $c = 0$ solves $1 \cdot c = 0$. ∎

Swapping 0 and 1 is not a symmetry of division. A zero *numerator* tests the image condition with the element 0, which lies in *every* ideal bR — so the image condition always holds for $0/b$. A zero *denominator* shrinks the image to $0 \cdot R = \{0\}$ and enlarges the kernel to $\mathrm{Ann}(0) = R$: both conditions are put under maximal stress at once. The next chapter is devoted to that case.

EXPRESSION	EQUATION	IMAGE: $A \in BR$?	KERNEL: $\mathrm{ANN}(B) = 0$?	OUTCOME
$6/3$	$3c = 6$	✓	✓	2
$0/1$	$1c = 0$	✓ ($0 \in R$)	✓	0
$0/x$, $x \neq 0$ in a field	$xc = 0$	✓	✓	0
x/x, $x \neq 0$ in a field	$xc = x$	✓	✓	1
$7/2$ in $\mathbb{Z}$	$2c = 7$	×	✓	no solution

EXPRESSION	EQUATION	IMAGE: $A \in BR$?	KERNEL: $\mathrm{ANN}(B) = 0$?	OUTCOME
$1/0$	$0c = 1$	× $(0R = \{0\})$	×	no solution
$0/0$	$0c = 0$	✓	× $(\mathrm{Ann}(0) = R)$	non-unique
$2/4$ in $\mathbb{Z}/6$	$4c = 2$	✓	× $(\mathrm{Ann}(4) = \{0, 3\})$	non-unique: $\{2, 5\}$
$3/4$ in $\mathbb{Z}/6$	$4c = 3$	×	×	no solution

16.4 Division in $\mathbb{Z}/n$

In the finite rings $\mathbb{Z}/n$ both conditions can be decided by a single gcd.

THEOREM 16.5 (Division in $\mathbb{Z}$/n) CLASSICAL RESULT GIN-THM-002

In $\mathbb{Z}/n$, let $g = \gcd(b, n)$. Then $\mathrm{Sol}(a, b) \neq \varnothing$ iff $g \mid a$, and in that case $|\mathrm{Sol}(a, b)| = g$, the solutions forming $c_0 + (n/g)\,\mathbb{Z}/n$. Exactly $n\,\varphi(n)$ of the n^2 pairs (a, b) have a unique quotient, where φ is Euler's totient.

Proof.

By Bézout's identity, $b\mathbb{Z}/n = g\mathbb{Z}/n$, which is the image condition. $bt \equiv 0 \pmod{n}$ iff $(n/g) \mid (b/g)\,t$ iff $(n/g) \mid t$, since $\gcd(b/g, n/g) = 1$; so $\mathrm{Ann}(b) = (n/g)\mathbb{Z}/n$ has g elements. A unique quotient requires $g = 1$, i.e. b is one of the $\varphi(n)$ units; then every a works, giving $n\varphi(n)$ pairs. ∎

How many pairs are solvable at all? Summing the n/g solvable numerators over all b gives a classical arithmetic function.

PROPOSITION 16.6 (Counting the solvable divisions in $\mathbb{Z}$/n) PROVED HERE

GIN-PROP-064

The number of pairs $(a, b) \in (\mathbb{Z}/n)^2$ for which $b\,c \equiv a$ has a solution is

$$\sum_{d \mid n} d\,\varphi(d),$$

and the number with several solutions is that sum minus $n\varphi(n)$.

Proof.

For each b, the solvable a are the multiples of $g = \gcd(b, n)$, of which there are n/g. The number of $b \in \mathbb{Z}/n$ with $\gcd(b, n) = g$ is $\varphi(n/g)$. Hence the total is $\sum_{g|n} \varphi(n/g)\,(n/g) = \sum_{d|n} d\,\varphi(d)$ with $d = n/g$. Subtract the $n\varphi(n)$ pairs with a unique solution. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from math import gcd
from ginsdk.converse import divide_mod, UNIQUE, NONE, MULTIPLE

def census(n):
    count = {UNIQUE: 0, NONE: 0, MULTIPLE: 0}
    for a in range(n):
        for b in range(n):
            count[divide_mod(a, b, n).kind] += 1
    return {"unique": count[UNIQUE], "none": count[NONE], "multiple": count[MULTIPLE]}

phi = lambda n: sum(gcd(k, n) == 1 for k in range(1, n + 1))
for n in (6, 7, 12, 30):
    c = census(n)
    formula = sum(d * phi(d) for d in range(1, n + 1) if n % d == 0)
    print(f"Z/{n:<3d} unique {c['unique']:4d} (n·φ(n) = {n * phi(n):4d})   none {c['none']:4d}   multiple
{c['multiple']:4d}   solvable {c['unique'] + c['multiple']:4d} (Σ dφ(d) = {formula})")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
Z/6   unique   12 (n·φ(n) =   12)   none   15   multiple    9   solvable   21 (Σ dφ(d) = 21)
Z/7   unique   42 (n·φ(n) =   42)   none    6   multiple    1   solvable   43 (Σ dφ(d) = 43)
Z/12  unique   48 (n·φ(n) =   48)   none   67   multiple   29   solvable   77 (Σ dφ(d) = 77)
Z/30  unique  240 (n·φ(n) =  240)   none  459   multiple  201   solvable  441 (Σ dφ(d) = 441)
```

For a prime p, every division by a non-zero element is admissible, and the only inadmissible pairs are those with $b = 0$: $p - 1$ with no solution ($a \neq 0$) and exactly one, $0/0$, with p solutions. A prime modulus behaves, with respect to division, exactly like the rationals.

LABORATORY (WEBSITE EDITION)

Text alternative: a table for $\mathbb{Z}/n$ with one row per divisor b and one column per dividend a. A cell shows the unique quotient, ⌀ for no solution, or the number of solutions. In $\mathbb{Z}/6$, the rows of the units 1 and 5 are complete; row 0 has a single solvable cell ($0/0$, six solutions); rows 2 and 4 are solvable exactly at even a with two solutions each; row 3 at $a \in \{0, 3\}$ with three solutions each.

16.5 When quotients are unique

Which rings never have kernel failures for non-zero divisors?

PROPOSITION 16.7 (Unique quotients characterize integral domains)

CLASSICAL RESULT GIN-PROP-065

For a commutative ring R with $1 \neq 0$, the following are equivalent: (i) for every $b \neq 0$ and every a, $\mathrm{Sol}(a, b)$ has at most one element; (ii) R has no zero divisors (it is an integral domain); (iii) multiplication by any $b \neq 0$ is cancellable: $bc = bc' \Rightarrow c = c'$. A finite integral domain is a field: every non-zero element is invertible.

Proof.

(i) $\Leftrightarrow$ (ii) by Theorem 16.3: uniqueness for all a is $\mathrm{Ann}(b) = \{0\}$. (ii) $\Leftrightarrow$ (iii): $bc = bc' \iff b(c - c') = 0$. If R is finite and $b \neq 0$, the map $c \mapsto bc$ is injective by (iii), hence surjective, so $bc = 1$ for some c. ∎

So in $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ and $\mathbb{Z}/p$, the *only* kernel failure is at $b = 0$, and in fields the *only* image failure is at $b = 0$ too. Division by zero is the one place where a field's division is inadmissible — and there, both conditions fail at once. That is the subject of Chapter 17.

EXERCISE 16.1

In $\mathbb{Z}/12$, classify $8/4$, $9/4$, $9/3$ and $5/7$ and list the solution sets.

▶ Solution to Exercise 16.1

EXERCISE 16.2

In $R = \mathbb{Z}/2 \times \mathbb{Z}/3$, compute $\mathrm{Ann}((1, 0))$ and decide whether $(1, 1)/(1, 0)$ and $(1, 0)/(1, 0)$ are admissible.

▶ Solution to Exercise 16.2

EXERCISE 16.3

Evaluate $\sum_{d|n} d\varphi(d)$ for $n = p^k$ (a prime power), and check it against the census for $n = 8$.

▶ Solution to Exercise 16.3

SUMMARY

- Division is the converse of multiplication: a/b denotes the unique c with $bc = a$, when there is one.
- In a commutative ring the solution set is empty or a coset of $\mathrm{Ann}(b)$: an image failure means no quotient, a kernel failure means many.
- $0/1 = 0$ passes both conditions; a zero numerator is harmless, a zero denominator stresses both conditions at once.
- In $\mathbb{Z}/n$ everything is decided by $g = \gcd(b, n)$: $n\varphi(n)$ pairs have unique quotients and $\sum_{d|n} d\varphi(d)$ are solvable.
- Integral domains are exactly the rings where non-zero divisors give unique quotients; in a field, $b = 0$ is the only failure.

Chapter 17

$1/0$ and $0/0$: Two Different Failures

> *Ask for c with 0 · c = 1 and nothing answers. Ask for c with 0 · c = 0 and everything does. Both are called 'undefined'; they are opposite failures.*

LEARNING OBJECTIVES

- Classify $1/0$ as an image failure (no solution) and $0/0$ as a kernel failure (every element is a solution), in every field and every commutative ring.
- Prove that no ring with $0 \neq 1$ contains an inverse of zero, and identify the single ring in which division by zero is admissible.
- Analyse the families $x/0$, $0/x$ and x/x exactly and symbolically, and explain why cancelling a common factor changes the domain of an expression.
- Separate the number $0/0$ from the limit form "$0/0$", and the value $1/0$ from the limit $\lim_{x \to 0} 1/x$.
- Locate the floating-point boundary between overflow and division by zero.

17.1 The two failures

Specialize the image/kernel theorem (GIN-THM-001) to $b = 0$. In every ring, $0 \cdot c = 0$ for all c (GIN-PROP-012), so the map $c \mapsto 0 \cdot c$ is the zero map: its image is $\{0\}$ and its kernel is everything.

COROLLARY 17.1 (Why 1/0 and 0/0 fail differently, and why 0/1 = 0) CLASSICAL RESULT

GIN-COR-001

In a field F (and in $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$):

- *for $b \neq 0$, $\mathrm{Ann}(b) = \{0\}$ and $bF = F$, so a/b is admissible for every a; in particular $0/1 = 0$;*
- *$1/0$: since $0 \cdot F = \{0\}$, the image condition fails: $0 \cdot c = 1$ has **no solution** (it would require $0 = 1$);*
- *$0/0$: the image condition holds ($0 \in 0 \cdot F$), but $\mathrm{Ann}(0) = F$, so **every** c solves $0 \cdot c = 0$: the kernel condition fails maximally;*
- *$x/0$ for an unknown x: no solution where $x \neq 0$ and every c where $x = 0$ — a case split.*

The two failures call for different responses. When an equation has *no* solution, any value offered as its answer is a fiction, and the honest responses are to extend the structure (add an element that solves it), to refuse, or to supply a conventional value labelled as such. When an equation has *every* element as a solution, the question is underdetermined, and the honest responses are to select one solution by an explicit rule, to refuse, or to return the whole solution set. A system that answers both with the same word — "undefined", `NaN`, "error" — cannot apply the right response to either.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e in ["1 / 0", "0 / 0", "0 / 1", "5 / 0", "0 / 5"]:
    o = evaluate(e, "Q")
    print(f"{e:6s} {o.status:12s} equation: {o.equation or '—':10s} solutions: {o.solutions or o.display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 / 0  no-solution  equation: 0 · c = 1  solutions: ∅
0 / 0  non-unique   equation: 0 · c = 0  solutions: every c in Q
0 / 1  value        equation: —          solutions: 0
5 / 0  no-solution  equation: 0 · c = 5  solutions: ∅
0 / 5  value        equation: —          solutions: 0
```

17.2 No ring inverts zero

Could a cleverer structure supply an inverse of zero? Not if it is a ring and $0 \neq 1$.

THEOREM 17.2 (No non-trivial ring can invert zero) CLASSICAL RESULT GIN-THM-003

If R is a ring with 1 in which $0 \cdot z = 1$ for some z, then $R = \{0\}$.

Proof.

$0 \cdot z = 0$ in every ring (GIN-PROP-012), so $1 = 0$, and then every $x = x \cdot 1 = x \cdot 0 = 0$. ∎

The **zero ring** $\{0\}$, in which $0 = 1$, is the only ring in which $1/0$ is admissible — and there it equals 0, which is also 1. It is a legitimate ring and a useless number system: it cannot count. The theorem therefore says that giving $1/0$ a value requires giving up something: one of the ring laws used in the proof of $0 \cdot z = 0$ (distributivity, additive inverses), or the reading of $1/0$ as a solution of $0 \cdot c = 1$, or $0 \neq 1$. Chapter 18 shows that every system in use makes exactly one of these choices.

17.3 The three families

Consider the expressions $x/0$, $0/x$ and x/x as x ranges over $\mathbb{Q}$.

FAMILY	$X \neq 0$	$X = 0$
$x/0$	no solution (image failure)	non-unique (kernel failure)
$0/x$	0	non-unique
x/x	1	non-unique

Every family meets $0/0$ at $x = 0$, and in two of them the value elsewhere is constant. It is tempting to "fill in" $0/0$ with the constant — 0 for the family $0/x$, 1 for x/x — and the temptation is the source of a classical error: the two families demand different values at the same point. $0/0$ is the meeting point of every family $\lambda x/x$ (λ any constant), and each family suggests its own λ. That is the kernel failure seen from the outside.

17.4 Symbolic simplification is not evaluation

In the field $\mathbb{Q}(x)$ of rational functions, $x/x = 1$ is a true identity: the rational function x/x *is* the constant 1. As an *expression to be evaluated* at points of $\mathbb{Q}$, however, x/x is admissible exactly on

$\mathbb{Q} \setminus \{0\}$. Simplifying first and evaluating afterwards silently changes the answer at 0 from "non-unique" to 1.

PROPOSITION 17.3 (Symbolic simplification is not evaluation) CLASSICAL RESULT

GIN-PROP-014

Cancelling a common factor g from a quotient preserves its values on $\{g \neq 0\}$ and enlarges its domain to include the zeros of g. A simplifier that is to agree with evaluation must record $g \neq 0$ as a side condition. For example, $(x^2 - 1)/(x - 1) = x + 1$ only for $x \neq 1$.

The SDK's symbolic domain keeps every condition it creates while cancelling, and reports what happens at the excluded points:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e in ["x / x", "(x^2 - 1) / (x - 1)", "1 / (x - 2)"]:
    o = evaluate(e, "symbolic")
    r = o.representation
    print(f"{e:20s} simplifies to {r['simplified']:14s} under {', '.join(r['conditions'])}")
    for pt, what in r.get("at excluded points (exact Q)", {}).items():
        print(f"{'':20s}   at x = {pt}: {what.split(':')[0]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x / x                simplifies to 1              under x ≠ 0
                       at x = 0: non-unique
(x^2 - 1) / (x - 1)  simplifies to x + 1          under x - 1 ≠ 0
                       at x = 1: non-unique
1 / (x - 2)          simplifies to (1) / (x - 2)  under x - 2 ≠ 0
                       at x = 2: no-solution
```

Computer algebra systems differ in how strictly they observe this; many simplify x/x to 1 without recording a condition, which is correct in $\mathbb{Q}(x)$ and wrong as a statement about every rational x. The counterexample is recorded as GIN-NEG-006.

17.5 Limits answer a different question

"$0/0$ is indeterminate" is a phrase from calculus, and it is about *limits*: if $f(x) \to 0$ and $g(x) \to 0$ as $x \to a$, then $\lim f(x)/g(x)$ can be anything, depending on f and g. That is a statement about *functions near a point*, and it is consistent with — but different from — the statement that the

number $0/0$ has every number as a solution. Likewise $\lim_{x \to 0^+} 1/x = +\infty$ is a statement that $1/x$ grows without bound as x shrinks; it does not say that $1/0$ is a number, and in $\mathbb{R}$ it is not. The limit has a sign that depends on the side of approach ($\lim_{x \to 0^-} 1/x = -\infty$), which the number 0 does not have.

These facts matter because the most common "value" of $1/0$ in computing — IEEE 754's $+\infty$ — is motivated by the limit, not by the equation. Chapter 25 shows that IEEE records the side of approach in the *sign of zero*, so that $1/(+0) = +\infty$ and $1/(-0) = -\infty$, and that ∞ is nevertheless not a solution of $0 \cdot c = 1$, because IEEE defines $0 \cdot \infty = \text{NaN}$ (GIN-PROP-015).

17.6 The floating-point boundary

On a computer, numbers close to zero raise a further question. When does $1/x$ stop being a large number and become a division by zero? The reference simulator of the SDK answers exactly, in binary64:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import ieee

one = ieee.from_decimal("1")[0]
for t in ["1e-300", "1e-308", "1e-310", "1e-323", "1e-330"]:
    x, flags_in = ieee.from_decimal(t)
    r, flags = ieee.div(one, x)
    print(f"x = {t:7s} is stored as {str(x):10s}  1/x = {str(r):22s} flags: {', '.join(sorted(flags))}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x = 1e-300  is stored as 1e-300      1/x = 9.999999999999999e299  flags: inexact
x = 1e-308  is stored as 1e-308      1/x = 1e308                  flags: inexact
x = 1e-310  is stored as 1e-310      1/x = Infinity               flags: inexact, overflow
x = 1e-323  is stored as 1e-323      1/x = Infinity               flags: inexact, overflow
x = 1e-330  is stored as 0           1/x = Infinity               flags: divideByZero
```

The last three rows produce the same datum, `Infinity`, for two different reasons. For $x = 10^{-310}$ and 10^{-323} — subnormal numbers, not zero — the exact quotient exists and is simply too large for the format: an *overflow*. For 10^{-330}, the literal itself rounds to $+0$ when it is read, and the division is then a genuine division by zero: the flag is *divideByZero*. The datum is the same; the status flags, which IEEE 754 requires the hardware to record, distinguish the two events (GIN-OBS-016).

COUNTEREXAMPLE 17.4 (Overflow and division by zero are the same event)

COUNTEREXAMPLE GIN-NEG-017

In binary64, $1/10^{-320}$ yields $+\infty$ with the *overflow* flag (the divisor is subnormal, not zero), while $1/10^{-330}$ yields $+\infty$ with the *divideByZero* flag (the literal rounded to $+0$).

17.7 A vocabulary of "undefined"

The word "undefined" is used for at least six different things, and this book keeps them apart:

TERM	MEANING	EXAMPLE
no solution	the defining equation has no solution in the domain	$1/0$ in $\mathbb{Q}$; $0 - 1$ in $\mathbb{N}$
non-unique	the defining equation has several solutions	$0/0$ in $\mathbb{Q}$; $2/4$ in $\mathbb{Z}/6$
not in the domain	an operand or literal names nothing in the domain	`0.5` in $\mathbb{Z}$
special datum	a format returns a value that is not a number	IEEE NaN, $\pm\infty$
exception or trap	the system refuses and transfers control	Python `ZeroDivisionError`; x86 `#DE`
undefined behaviour	the program has no meaning at all	`1 / 0` on `int` in ISO C

EXERCISE 17.1

For which λ is the family $\lambda x/x$ consistent with each of the following conventions at $x = 0$: (a) meadows ($0/0 = 0$); (b) "$0/0 = 1$"; © wheels ($0/0 = \bot$)?

▶ Solution to Exercise 17.1

EXERCISE 17.2

Show directly that in the zero ring every axiom of a commutative ring holds and $0 \cdot 0 = 1$.

▶ Solution to Exercise 17.2

EXERCISE 17.3

Find the domain of $\dfrac{x^3 - x}{x^2 - x}$ as an evaluated expression on $\mathbb{Q}$, simplify it, and classify the failures at the excluded points.

▶ Solution to Exercise 17.3

EXERCISE 17.4

Give functions f, g with $f(x), g(x) \to 0$ as $x \to 0$ such that $f/g \to 7$, $f/g \to 0$, and f/g has no limit. Why does this not contradict the claim that the number $0/0$ is non-unique rather than "indeterminate"?

▶ Solution to Exercise 17.4

SUMMARY

- $1/0$ is an image failure: no c satisfies $0 \cdot c = 1$. $0/0$ is a kernel failure: every c satisfies $0 \cdot c = 0$.
- No ring with $0 \neq 1$ inverts zero; the zero ring is the only exception.
- The families $x/0$, $0/x$ and x/x all meet $0/0$, and each suggests a different value there.
- Cancellation enlarges domains; a correct simplifier records the conditions it creates.
- Limits concern functions, not the number $0/0$; IEEE's $\pm\infty$ follows the limit, not the equation.
- In binary64, overflow and division by zero yield the same datum and different flags.

Chapter 18

Giving $1/0$ a Value: What Every Extension Gives Up

> *There is no free way to divide by zero. There are only different bills.*

LEARNING OBJECTIVES

- State the extension trilemma and derive it from the fact that no non-trivial ring inverts zero.
- For each of nine systems — fields, the projective line, the extended reals, wheels, meadows, proof assistants, IEEE 754, AArch64 and RISC-V — say what $1/0$ and $0/0$ are, and which laws are kept, weakened or lost.
- Prove that in the arithmetic of fractions as pairs, every indeterminate form computes the null pair $(0, 0)$, and that this pair cannot be an element.
- Read an exact census of ten laws across eight implemented systems, and interpret its counterexamples.

18.1 The trilemma

Theorem GIN-THM-003 showed that a ring with $0 \neq 1$ has no element z with $0 \cdot z = 1$. Its proof used three ingredients: the ring laws that force $0 \cdot z = 0$; the reading of "$1/0$" as "the z with $0 \cdot z = 1$"; and $0 \neq 1$. A system that assigns $1/0$ a value must therefore drop one of them.

THEOREM 18.1 (The extension trilemma for 1/0) REINTERPRETATION GIN-THM-004

Any system that gives $1/0$ *a value gives up at least one of:*

- ***(A)*** *the ring laws that force* $0 \cdot c = 0$ *(distributivity with additive inverses);*
- ***(B)*** *the converse reading of division — that* a/b *denotes a* c *with* $b \cdot c = a$*;*
- © $0 \neq 1$.

Every system in the literature and in hardware takes route A or route B, or refuses to give a value (it signals); none takes route C, which yields the zero ring.

SYSTEM	$1/0$	$0/0$	ROUTE	WHAT GIVES WAY
$\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ (fields)	no solution	non-unique	restriction	division is defined only for $b \neq 0$
projective line $\mathbb{Q} \cup \{\infty\}$, Riemann sphere $\mathbb{C} \cup \{\infty\}$	∞	undefined	A	$0 \cdot \infty$, $\infty \pm \infty$, ∞/∞, $0/0$ undefined; no field structure, no order
extended reals $\mathbb{R} \cup \{\pm\infty\}$	usually undefined	undefined	A	$0 \cdot (\pm\infty)$ and $\infty - \infty$ undefined; $\pm\infty$ are not field elements
IEEE 754	$\pm\infty$, flag divideByZero	NaN, flag invalid	B	$0 \times \infty = \text{NaN}$, so ∞ does not solve $0 \cdot c = 1$; $\text{NaN} \neq \text{NaN}$
meadows (Bergstra & Tucker)	0	0	B	$x \cdot x^{-1} = 1$ only for $x \neq 0$; all ring axioms kept
Lean's mathlib, Isabelle/HOL	0	0	B	theorems such as $a \cdot a^{-1} = 1$ carry the hypothesis $a \neq 0$
wheels (Carlström)	$\infty = /0$	$\perp$ (absorbing)	A	$0 \cdot x = 0$ fails; distributivity weakened

SYSTEM	$1/0$	$0/0$	ROUTE	WHAT GIVES WAY
AArch64 `sdiv` / `udiv`	0	0	B (totalization)	the result is not a quotient of anything
RISC-V `div` / `divu`	-1 / 2^w-1	the same	B (totalization)	as above
x86-64 `idiv` , Java, Rust, Go, Python	trap / exception / panic	the same	signal	no value is produced
ISO C (integers)	undefined behaviour	the same	none	the program has no meaning

The rest of this chapter examines the rows, and then checks the table by computation.

18.2 Route A: adjoining infinity

The projective line. Adjoin to $\mathbb{Q}$ (or $\mathbb{R}$, $\mathbb{C}$) a single new element ∞ with $x/0 = \infty$ for $x \neq 0$, $x/\infty = 0$, $x + \infty = \infty$ for $x \neq \infty$, and $x \cdot \infty = \infty$ for $x \neq 0$. Geometrically, $\mathbb{R} \cup \{\infty\}$ is a circle and $\mathbb{C} \cup \{\infty\}$ the Riemann sphere; Möbius transformations $z \mapsto (az+b)/(cz+d)$ act on them as bijections, which is the main reason the construction exists. The price: the expressions $0 \cdot \infty$, $\infty + \infty$, $\infty - \infty$, ∞/∞ and $0/0$ have no value. The projective line is not a ring, and it has no order compatible with its operations: ∞ is both "very large" and "very negative".

The extended reals. Adjoin two elements $\pm\infty$, keeping the order: $-\infty < x < +\infty$. Now $x + (+\infty) = +\infty$ and $x \cdot (+\infty) = +\infty$ for $x > 0$; but $+\infty - \infty$ and $0 \cdot \infty$ are undefined, and so, usually, is $x/0$ — which sign of infinity would it be? The extended reals are indispensable in measure theory and optimization (where $\sup \varnothing = -\infty$ is a useful convention), and they are not a field.

Wheels. Carlström's wheels [Carlström 2004] make division total by replacing it with a unary operation $/x$ (an involution: $//x = x$, multiplicative: $/(xy) = /x \cdot /y$) and writing x/y for $x \cdot /y$. Addition and multiplication remain commutative monoids, but they are no longer required to form a ring; the distributive law is weakened to $(x+y)z + 0z = xz + yz$. The *wheel of fractions* of $\mathbb{Z}$ consists of the classes of pairs $[a,b]$, where $[a,b] = [sa,sb]$ for non-zero integers s, with $[a,b] + [c,d] = [ad+bc, bd]$, $[a,b][c,d] = [ac,bd]$ and $/[a,b] = [b,a]$. Its elements are the rationals, $\infty = [1,0]$ and $\bot = [0,0]$, and

$$1/0 = \infty, \qquad 0/0 = \bot, \qquad 0 \cdot \infty = \bot, \qquad \infty + \infty = \bot, \qquad \bot + x = \bot.$$

The element $\bot$ absorbs every operation, like a NaN with algebraic laws. The ring identities $0 \cdot x = 0$, $x - x = 0$ and $x/x = 1$ fail exactly at elements that involve ∞ or $\bot$.

IMPLEMENTATION RESULT 18.2

(The wheel of fractions of $\mathbb{Z}$ satisfies the wheel axioms) IMPLEMENTATION RESULT

GIN-IMP-007

`ginsdk.totalized.WheelZ` implements the wheel of fractions of $\mathbb{Z}$. All fourteen axiom instances of a wheel — the monoid laws for $+$ and $\cdot$, $//x = x$, $/(xy) = /x/y$, $(x+y)z + 0z = xz + yz$, $(x+yz)/y = x/y + z + 0y$, $0 \cdot 0 = 0$, $(x+0y)z = xz + 0y$, $/(x+0y) = /x + 0y$ and $0/0 + x = 0/0$ — hold on every tuple drawn from an eight-element sample containing $0, 1, -1, 2, \frac{1}{2}, -3, \infty, \bot$. The axiom list was taken from a secondary source (the Wikipedia article on wheel theory, which cites Carlström); the primary paper was not read (GIN-OPEN-005).

18.3 Route B: dropping the converse reading

Meadows [Bergstra & Tucker 2007] keep every axiom of a commutative ring and add a total inverse operation with two axioms, $(x^{-1})^{-1} = x$ and $x \cdot (x \cdot x^{-1}) = x$. Together they force $0^{-1} = 0$: applying the second axiom to $x = 0^{-1}$ and using the first gives $0^{-1} = 0^{-1}(0^{-1} \cdot 0) = 0$. The rationals with $0^{-1} := 0$ form a meadow. Division $x/y := x \cdot y^{-1}$ is then total, with $x/0 = 0$. (A later variant, *common meadows*, instead sends 0^{-1} to an absorbing error element, in the spirit of wheels and NaN [Bergstra & Ponse 2015].) All ring laws hold; the price is that $y \cdot (x/y) = x$ fails at $y = 0$ — division by zero no longer means "the solution of $0 \cdot c = x$".

Proof assistants adopt the same convention for the same reason. Lean's mathlib defines $0^{-1} = 0$ in every field and division ring, so that the functions $x \mapsto x^{-1}$ and $(x, y) \mapsto x/y$ are total; lemmas that need the converse reading carry an explicit hypothesis, as in "$a \neq 0 \Rightarrow a \cdot a^{-1} = 1$" [mathlib Community 2020]. Isabelle/HOL's division rings have the axiom *inverse* $0 = 0$, from which $a/0 = 0$ is derived [Isabelle/HOL Fields]. The convention is a *totalization*: it makes formal statements shorter and machine reasoning simpler, at the cost of making $x/0$ a number that solves nothing. The converse vocabulary of this book is exactly what a formalization needs to state admissibility explicitly (GIN-OPEN-011).

IEEE 754 returns $\pm\infty$ for a finite non-zero number divided by zero, with the sign determined by the signs of the operands (including the sign of zero), and raises the divideByZero flag; it returns NaN for $0/0$ and raises *invalid*. These are limit-motivated values: $\lim_{x \to 0^+} 1/x = +\infty$.

PROPOSITION 18.3

(IEEE division by zero is not the converse of IEEE multiplication) CLASSICAL RESULT

GIN-PROP-015

For finite non-zero x, IEEE 754 gives $x/(\pm 0) = \pm\infty$ and $0/0 = \mathrm{NaN}$. But $0 \times (\pm\infty) = \mathrm{NaN}$ and $0 \times c = \pm 0$ for every finite c; so no IEEE datum c satisfies $0 \otimes c = 1$. The infinity returned by $1/0$ records the side from which zero was approached (Kahan's signed zero [Kahan 1987]), not a solution.

Instruction sets take route B in its barest form. AArch64's `sdiv` returns 0 for a zero divisor and RISC-V's `div` returns -1 (all bits set), with no flag and no trap. Chapter 19 shows that -1 is precisely what a divider circuit computes if it never tests its divisor — the RISC-V specification's rationale says as much [RISC-V ISA Manual] — and Chapter 24 measures both under emulation.

18.4 Fractions as pairs: the null pair

The construction of $\mathbb{Q}$ from pairs (Chapter 9) shows *why* there is a second failure. Take all pairs of integers, including those with second component zero, and see how far the arithmetic of fractions extends.

THEOREM 18.4 (Fractions as pairs: ∞ can be adjoined, 0/0 cannot) PROVED HERE

GIN-THM-006

On $\mathbb{Z}^2$ let $(a,b) \sim (c,d)$ iff $ad = bc$, and define $(a,b) + (c,d) = (ad + bc, bd)$, $(a,b) \cdot (c,d) = (ac, bd)$, $(a,b)/(c,d) = (ad, bc)$.

1. *$\sim$ is reflexive and symmetric on $\mathbb{Z}^2$. On $\mathbb{Z}^2 \setminus \{(0,0)\}$ it is an equivalence relation whose classes are the points of the projective line $\mathbb{Q} \cup \{\infty\}$, where ∞ is the single class of all $(a,0)$ with $a \neq 0$.*
2. *$(0,0)$ is related to every pair, so on $\mathbb{Z}^2$ transitivity fails: $(1,0) \sim (0,0) \sim (0,1)$ but $(1,0) \not\sim (0,1)$. Admitting "$0/0$" as a pair would identify everything.*
3. *The operations are well defined on classes, and the result is the null pair $(0,0)$ exactly in the cases $\infty + \infty$ (hence $\infty - \infty$), $0 \cdot \infty$, $0/0$ and ∞/∞; every other combination of elements of $\mathbb{Q} \cup \{\infty\}$ is defined. In particular $1/0 = \infty$ and $1/\infty = 0$.*

Proof.

(1) Reflexivity and symmetry are immediate. Transitivity for $p = (a, b) \sim q = (c, d) \sim r = (e, f)$ with $q \neq (0, 0)$: if $d \neq 0$, from $ad = bc$ and $cf = de$ we get $adf = bcf = bde$, so $d(af - be) = 0$ and $af = be$. If $d = 0$ then $c \neq 0$; $ad = bc$ gives $b = 0$ and $cf = de$ gives $f = 0$, so $af = 0 = be$. (2) $a \cdot 0 = 0 \cdot b$ for every (a, b). (3) Scaling one operand by $k \neq 0$ scales both components of each result by k, so classes map to classes and the null pair to itself. A sum is null iff $bd = 0$ and $ad + bc = 0$; if $b = 0$ then $a \neq 0$ forces $d = 0$, so both operands are ∞. A product is null iff $ac = 0$ and $bd = 0$ with neither operand null: one factor is 0 and the other ∞. A quotient is null iff $ad = 0$ and $bc = 0$: either $a = c = 0$ $(0/0)$ or $b = d = 0$ (∞/∞). ∎

So in the arithmetic of pairs, **every indeterminate form computes the same object — the null pair — and the null pair is exactly the one pair that cannot be an element of the quotient.** The projective line is what remains when that pair is excluded; the wheel of fractions is what one obtains by *keeping* it as an absorbing element $\perp$, at the price of the ring laws. The pair-level statement is elementary; it is not found in the sources reviewed, it is very probably folklore, and no priority is claimed (GIN-THM-006).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk.converse import pair_of, pair_add, pair_mul, pair_div, Pair

def show(p):
    p = p.normal()
    return "(0,0) NULL" if (p.a, p.b) == (0, 0) else "∞" if p.b == 0 else f"{p.a}/{p.b}" if p.b != 1 else
str(p.a)

inf, zero, one = Pair(1, 0), Pair(0, 1), Pair(1, 1)
for label, r in [("1/0", pair_div(one, zero)), ("1/∞", pair_div(one, inf)), ("0/0", pair_div(zero, zero)),
                 ("∞/∞", pair_div(inf, inf)), ("0·∞", pair_mul(zero, inf)), ("∞+∞", pair_add(inf, inf)),
("∞+1", pair_add(inf, one))]:
    print(f"{label:4s} = {show(r)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1/0  = ∞
1/∞  = 0
0/0  = (0,0) NULL
∞/∞  = (0,0) NULL
0·∞  = (0,0) NULL
∞+∞  = (0,0) NULL
∞+1  = ∞
```

18.5 The census

The claims of the table in §18.1 can be checked by computation, on finite samples, for systems that can be implemented exactly. The SDK module `ginsdk.totalized` implements eight of them and evaluates ten laws on every tuple drawn from a fixed sample of each (six to eleven elements, including the special elements of each system). An instance *holds*, *fails*, or is *undefined* (some side is undefined in a partial system). For IEEE binary16, laws compare data: $+0$ and -0 are different data, and NaN is treated as equal to NaN.

EMPIRICAL OBSERVATION 18.5 (Which laws each way of dividing by zero keeps)

EMPIRICAL OBSERVATION

GIN-EXP-015

"ok" means no failure and nothing undefined; "ok*" means no failure, but some instances undefined; "×k" means k failing instances.

SYSTEM	$1/0$	$0/0$	$(X+Y)+Z$	$X(Y+Z)$	$0X=0$
field $\mathbb{Q}$	undef.	undef.	ok	ok	ok
meadow $\mathbb{Q}$	0	0	ok	ok	ok
wheel of $\mathbb{Z}$	∞	$\bot$	ok	×45	×2
projective line	∞	undef.	ok*	ok*	ok*
extended rationals	undef.	undef.	ok*	ok*	ok*
IEEE binary16	∞	NaN	×8	×94	×4
8-bit RISC-V	-1	-1	ok	ok	ok
8-bit AArch64	0	0	ok	ok	ok

SYSTEM	$X-X=0$	$X/X=1$	$Y(X/Y)=X$	$(X/Y)/Z=X/(YZ)$	$(X+Y)/Z$
field $\mathbb{Q}$	ok	ok*	ok*	ok*	ok*
meadow $\mathbb{Q}$	ok	×1	×6	ok	ok
wheel of $\mathbb{Z}$	×2	×3	×21	ok	×45
projective line	ok*	ok*	ok*	ok*	ok*
extended rationals	ok*	ok*	ok*	ok*	ok*
IEEE binary16	×2	×4	×49	×67	×123
8-bit RISC-V	ok	×1	×35	×111	×140
8-bit AArch64	ok	×1	×35	×23	×76

Every system that makes division total — meadow, wheel, IEEE, RISC-V, AArch64 — fails the converse law $y(x/y)=x$; every system that keeps it (where defined) leaves some quotients

undefined. The census is exact and reproducible (`experiments/exp015_law_census.py`).

Three remarks prevent misreading the table. First, it counts failures on a *sample*, so the numbers are not frequencies; what matters is zero versus non-zero, and the counterexamples. Second, for the integer instruction sets most failures of the division laws are caused by *truncation*, not by zero: $y(x/y) = x$ fails for $x = 3, y = 2$ because $3/2$ truncates to 1 — a selection (Chapter 19) — while the zero-divisor convention adds its own failures, such as $x/x = 1$ at $x = 0$. Third, the IEEE failures of $0 \cdot x = 0$ come from signed zeros and NaN: $0 \cdot (-1) = -0$ is a different datum from $+0$, and $0 \cdot \infty = \mathrm{NaN}$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import totalized as T

for system in ("meadow", "wheel", "ieee16", "riscv8"):
    for row in T.law_census(system):
        if row["law"].startswith("y·(x / y)") or row["law"] == "0·x = 0":
            law = row["law"].split("   ")[0]
            print(f"{system:7s} {law:14s} first counterexample: {row['counterexample']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
meadow  0·x = 0        first counterexample: None
meadow  y·(x / y) = x  first counterexample: 1, 0: 0 ≠ 1
wheel   0·x = 0        first counterexample: ∞: ⊥ ≠ 0
wheel   y·(x / y) = x  first counterexample: 0, 0: ⊥ ≠ 0
ieee16  0·x = 0        first counterexample: -1: -0 ≠ 0
ieee16  y·(x / y) = x  first counterexample: 0, 0: NaN ≠ 0
riscv8  0·x = 0        first counterexample: None
riscv8  y·(x / y) = x  first counterexample: 1, 0: 0 ≠ 1
```

18.6 Brahmagupta's rules

The question is old. In the *Brāhmasphuṭasiddhānta* of 628 CE, Brahmagupta defined zero as the result of subtracting a number from itself and gave rules of arithmetic with zero, including $0/0 = 0$; for a non-zero number divided by zero he described a fraction with zero as denominator. Bhāskara II later called a quantity with zero divisor *khahara* [O'Connor & Robertson (MacTutor); Plofker 2009].

HISTORICAL NOTE (Brahmagupta's 0/0 = 0)

Brahmagupta's rule has often been described as an error. In the vocabulary of this chapter it is not an error of derivation but a **selection**: since every c satisfies $0 \cdot c = 0$, choosing $c = 0$ is consistent, and it is exactly the convention adopted, thirteen centuries later, by meadows and by Lean's mathlib and Isabelle/HOL. The modern objection is that 0 is not *the* solution — the equation has no unique one — not that the rule is inconsistent (GIN-HIST-001, GIN-NEG-012).

EXERCISE 18.1

Classify by route (A, B, C or signal): (a) SQL's `1 / 0`, which raises an error in most databases; (b) Excel's `=1/0`, which displays `#DIV/0!`; © a programming language whose integer division by zero is specified to return 0; (d) the field with one element "$\mathbb{F}_1$" read as the zero ring.

▶ Solution to Exercise 18.1

EXERCISE 18.2

Compute in pairs: $(\infty + 1) \cdot 0$ and $\infty \cdot (1 + 0)$, and explain the result with Theorem 18.4.

▶ Solution to Exercise 18.2

EXERCISE 18.3

In a meadow, prove that $x \cdot x^{-1}$ is idempotent, and determine its value at $x = 0$ and at $x \neq 0$ in $\mathbb{Q}$.

▶ Solution to Exercise 18.3

SUMMARY

- Every system that gives $1/0$ a value drops the ring laws (A), the converse reading (B), or $0 \neq 1$ ©; in practice A or B, or a signal.
- The projective line and the extended reals adjoin infinities and leave several forms undefined; wheels make everything total with an absorbing $\perp$ and weaker laws.
- Meadows and proof assistants keep the ring laws and set $x/0 = 0$; IEEE returns limit-motivated infinities and NaN; AArch64 and RISC-V return 0 and -1.
- In fractions as pairs, all indeterminate forms compute the null pair, the one pair that cannot be an element.
- An exact census confirms: every total system fails $y(x/y) = x$; the partial ones keep it where defined.

Chapter 19

Division with Remainder, and How Machines Divide

> *When b does not divide a, there is no quotient. There is a whole family of quotients-with-remainder, and every language picks one.*

LEARNING OBJECTIVES

- State the division theorem and explain why quotient-with-remainder is a *selection*, not a converse.
- Compare truncated, floored, Euclidean and rounded division, and predict the sign of `-7 % 2` in C, Python and Rust.
- Prove the invariant of long division and analyse binary restoring division.
- Prove that a restoring divider without a divisor test returns quotient $2^n - 1$ and remainder equal to the dividend when the divisor is zero — the RISC-V convention.
- Explain how compilers divide by constants with a multiplication and a shift, and prove the method correct for one divisor.

19.1 Quotients when division fails

In $\mathbb{Z}$, $7/2$ has no solution (Chapter 16). Yet everyone knows that "7 divided by 2 is 3 remainder 1". The second statement is not about the converse problem $2c = 7$; it is about a different, weaker one:

$$7 = q \cdot 2 + r, \qquad |r| < |2|.$$

That problem has *two* solutions, $(q, r) = (3, 1)$ and $(4, -1)$. A further condition on r selects one.

THEOREM 19.1 (The division theorem) CLASSICAL RESULT GIN-HIST-005

*For integers a and $b \neq 0$ there are exactly two pairs (q, r) with $a = qb + r$ and $|r| < |b|$ when $b \nmid a$, and exactly one (with $r = 0$) when $b \mid a$. In particular there is exactly one pair with $0 \leq r < |b|$ (the **Euclidean** division).*

Proof.

Uniqueness of the Euclidean pair: if $a = qb + r = q'b + r'$ with $0 \leq r, r' < |b|$, then $|b| \cdot |q - q'| = |r - r'| < |b|$, so $q = q'$. Existence: let r be the least non-negative element of $\{a - qb : q \in \mathbb{Z}\}$ (non-empty by choosing q with the opposite sign of a/b); if $r \geq |b|$ then $r - |b|$ is a smaller non-negative element. Every pair with $|r| < |b|$ is either the Euclidean pair or, when $r \neq 0$, the pair $(q \pm 1, r \mp |b|)$ with negative remainder. ∎

In the vocabulary of Chapter 9, quotient-with-remainder is a **selection**: among the (one or two) solutions of a weaker equation, a rule chooses one. Different rules are in use, and they disagree exactly when the operands have different signs.

CONVENTION	RULE FOR Q	SIGN OF R	USED BY
truncated	$q = \operatorname{trunc}(a/b)$, rounded toward zero	sign of a	C (since C99), C++, Java, Go, Rust `/` and `%`, JavaScript BigInt, x86 `idiv`, ARM `sdiv`, RISC-V `div`
floored	$q = \lfloor a/b \rfloor$	sign of b	Python `//` and `%`, Ruby, Haskell `div` / `mod`
Euclidean	$r \geq 0$ always	non-negative	Rust `div_euclid` / `rem_euclid`, Haskell-style definitions in number theory
rounded	q = nearest integer to a/b (ties to even)	either	IEEE 754 `remainder`

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for a, b in [(7, 2), (-7, 2), (7, -2), (-7, -2)]:
    row = []
    for d in ("int32 C", "Python", "Z"):
        q = evaluate(f"({a}) // ({b})", d).display
        r = evaluate(f"({a}) % ({b})", d).display
```

```
        row.append(f"{d}: q={q:>2s} r={r:>2s}")
    print(f"{a:>2d}, {b:>2d}   " + "   ".join(row))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
 7,  2   int32 C: q= 3 r= 1   Python: q= 3 r= 1   Z: q= 3 r= 1
-7,  2   int32 C: q=-3 r=-1   Python: q=-4 r= 1   Z: q=-4 r= 1
 7, -2   int32 C: q=-3 r= 1   Python: q=-4 r=-1   Z: q=-3 r= 1
-7, -2   int32 C: q= 3 r=-1   Python: q= 3 r=-1   Z: q= 4 r= 1
```

(The SDK's exact domain $\mathbb{Z}$ uses the Euclidean convention.) Every row satisfies $a = qb + r$ in every column; the columns differ only in which solution they select. A program that computes "the last digit" or "the day of the week" with `%` gets a negative answer for negative inputs in C and a non-negative one in Python. The C89 standard left the rounding direction of negative quotients to the implementation; C99 fixed it to truncation.

19.2 Long division and its invariant

Long division of a by $b > 0$ in base β processes the digits of a from the most significant end, carrying a partial remainder R with $0 \le R < b$. For each digit a_i: form $T = \beta R + a_i$, emit the quotient digit $q_i = \lfloor T/b \rfloor$ (which is less than β), and set $R = T - q_i b$. The invariant after processing the prefix $a_{k-1} \cdots a_i$ is

$$\mathrm{val}(a_{k-1} \cdots a_i) = \mathrm{val}(q_{k-1} \cdots q_i) \cdot b + R, \qquad 0 \le R < b,$$

which at the end is the division theorem for a and b. This is the same Horner-style recursion as reading a numeral (Chapter 5), with a division step attached to each digit move. In binary each q_i is 0 or 1, and "emit q_i" becomes "compare and subtract": the **restoring** division algorithm.

19.3 The divider circuit and the zero divisor

A restoring array divider performs the binary long-division step n times in hardware: each row shifts the partial remainder, brings in the next dividend bit, subtracts the divisor with an $(n+1)$-bit subtractor, and keeps the difference if there was no borrow. The SDK builds this circuit gate by gate; for $n = 4, 8, 16$ it has 172, 632 and 2416 gates. Nothing in it tests whether the divisor is zero. What does it compute then?

PROPOSITION 19.2 (A divider that never tests its divisor returns all ones)

PROVED HERE

GIN-PROP-023

The n-bit restoring array divider of `ginsdk.circuits` outputs quotient $2^n - 1$ and remainder equal to the dividend when the divisor is 0. These are exactly the results that the RISC-V M extension specifies for `divu` and `remu` by zero.

Proof.

Row i computes $T = 2R + a_i$ and keeps $T - D$ if $T \geq D$. With $D = 0$, $T \geq D$ always holds, so every quotient bit is 1 and $R \leftarrow T - 0 = T$. After the row for bit i, $R = \lfloor a/2^i \rfloor < 2^{n-i}$, which fits in the n-bit register; after the last row ($i = 0$) $R = a$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import circuits as K
from ginsdk.machine import divu_riscv

n = 6
c = K.restoring_divider(n)
ok = all(K.run_divider(c, n, a, 0) == (2 ** n - 1, a) for a in range(2 ** n))
riscv = all(divu_riscv(a, 0, n).value == 2 ** n - 1 and divu_riscv(a, 0, n).remainder == a for a in range(2 **
n))
correct = all(K.run_divider(c, n, a, b) == divmod(a, b) for a in range(2 ** n) for b in range(1, 2 ** n))
print(f"{n}-bit divider: {c.size} gates; zero divisor gives (2^n - 1, a) for all a: {ok}; equals RISC-V
divu/remu: {riscv}")
print(f"correct for every non-zero divisor: {correct}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
6-bit divider: 366 gates; zero divisor gives (2^n - 1, a) for all a: True; equals RISC-V divu/remu: True
correct for every non-zero divisor: True
```

The RISC-V specification motivates its choice by exactly this: all ones is "the natural result for simple unsigned divider implementations", and avoiding a trap lets languages that need one insert a branch [RISC-V ISA Manual]. The proposition supplies a proof for one such implementation; it is not a new fact. It is a vivid example of the grammar of a *circuit* determining the value of an expression that the grammar of *arithmetic* leaves without one: the value -1 (all ones) for $x/0$ is what the hardware does when nobody asks it to do anything else.

Modern processors use faster algorithms than restoring division — non-restoring division, which avoids the restore step; SRT division, which uses redundant quotient digits chosen from a small table (a famous defect in such a table, missing entries, caused the 1994 Pentium FDIV error); and Newton–Raphson or Goldschmidt iteration for floating point, which compute a reciprocal and multiply. Division remains the slowest of the four basic operations in hardware.

19.4 Dividing big numbers

For multi-word integers, Knuth's Algorithm D performs long division in base 2^{32} (or 2^{64}), estimating each quotient digit from the leading words and correcting the estimate at most twice [Knuth 1997]. Dividing a $2L$-bit number by an L-bit number costs $\Theta(L^2)$ word operations; the SDK's count has a log–log slope of 1.996 at 16,384 bits (GIN-OBS-010). Asymptotically faster division reduces to multiplication by computing a reciprocal with Newton's iteration, at a constant factor times the cost of multiplication [Brent & Zimmermann 2010].

19.5 Division by constants

Division is slow, and division by a *constant* is common (`x / 10` to extract digits, `x / 7` for days of the week). Compilers replace it by a multiplication with a precomputed "magic" reciprocal and a shift. For unsigned 32-bit x and $d = 7$:

$$\left\lfloor \frac{x}{7} \right\rfloor = \left\lfloor \frac{x \cdot M}{2^{35}} \right\rfloor, \qquad M = \left\lceil \frac{2^{35}}{7} \right\rceil = 4908534053.$$

PROPOSITION 19.3 (Division by an invariant integer using multiplication)

CLASSICAL RESULT GIN-PROP-066

Let $0 \le x < 2^N$, $d \ge 1$, $s \ge 0$, and $M = \lceil 2^{N+s}/d \rceil$ with $e = Md - 2^{N+s} \le 2^s$. Then $\lfloor xM/2^{N+s} \rfloor = \lfloor x/d \rfloor$ for all such x. (Granlund and Montgomery [Granlund & Montgomery 1994].) For $N = 32$, $d = 7$, $s = 3$: $M = 4908534053$ and $e = 3 \le 8$.

Proof. $\dfrac{xM}{2^{N+s}} = \dfrac{x}{d} + \dfrac{xe}{d\,2^{N+s}}$, and the second term is non-negative and less than $\dfrac{2^N \cdot 2^s}{d\,2^{N+s}} = \dfrac{1}{d}$. Write $x = qd + r$ with $0 \le r \le d-1$; then $x/d = q + r/d$ and the sum lies in $[q, q + (r+1)/d) \subseteq [q, q+1)$, so its floor is q. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random

M, shift = 4908534053, 35
assert M * 7 - 2 ** 35 <= 2 ** 3
random.seed(7)
xs = list(range(1 << 20)) + [random.getrandbits(32) for _ in range(200000)] + [2 ** 32 - 1 - k for k in
range(1000)]
bad = sum((x * M) >> shift != x // 7 for x in xs)
```

```
print(f"checked {len(xs)} values of x: {bad} mismatches;  x = 4294967295 → {(4294967295 * M) >> shift} =
{4294967295 // 7}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
checked 1249576 values of x: 0 mismatches;  x = 4294967295 → 613566756 = 613566756
```

The proposition is a theorem, so the check is a guard against a mis-stated constant rather than a proof. The same technique handles signed division and division by every constant; it is one of the reasons the cost of `x / 7` in a compiled program is a multiply and a shift, not a divide.

EXERCISE 19.1

Compute $(-13) \div 4$ and $(-13) \bmod 4$ under the truncated, floored and Euclidean conventions. Then do the same for 13 and -4.

▶ Solution to Exercise 19.1

EXERCISE 19.2

Divide 1011011_2 by 101_2 by binary long division, listing the partial remainders.

▶ Solution to Exercise 19.2

EXERCISE 19.3

Find the smallest s for which Proposition 19.3 applies to $N = 8$, $d = 3$, and the corresponding M.

▶ Solution to Exercise 19.3

SUMMARY

- Quotient-with-remainder is a selection among the solutions of $a = qb + r$; truncated, floored, Euclidean and rounded conventions differ when signs differ.
- Long division is a digit-by-digit transition system with an explicit invariant; in binary it is restoring division.
- A restoring divider that never tests its divisor returns $2^n - 1$ and the dividend on a zero divisor — RISC-V's specified result.
- Big-number division costs $\Theta(L^2)$ schoolbook; division by constants becomes multiplication by a magic reciprocal, provably exact.

Chapter 20

The Grammar of Zero and One

> *Zero is the number that does nothing in one operation and destroys everything in the other.*

LEARNING OBJECTIVES

- List the roles of zero in the grammar of number and say which operation each role belongs to.
- Distinguish zero from the programming concepts it is often confused with: null references, false, empty strings, uninitialized memory, NaN.
- Explain why IEEE 754 has two zeros that compare equal and still behave differently.
- Distinguish one as the identity of multiplication from one as the generator of addition.
- Justify the conventions $\sum_{\varnothing} = 0$, $\prod_{\varnothing} = 1$ and $0^0 = 1$, and say where the last one stops being a convention.

20.1 The jobs of zero

Zero appears in this book in at least nine distinct roles.

ROLE	STATEMENT	WHERE
additive identity	$a + 0 = a$	every ring
empty count	$\lvert\varnothing\rvert = 0$	cardinality
start state	the root of the successor grammar; the only state with no predecessor	Chapter 3
empty address	the canonical numeral of 0 is the empty word ε	Chapter 5

ROLE	STATEMENT	WHERE
placeholder	the digit `0` inside `101` performs the move “multiply by the base, add nothing”	Chapter 5
annihilator	$0 \cdot a = 0$	a theorem in rings, an axiom in semirings (Chapter 14)
fixed point	$0^2 = 0$, $2 \cdot 0 = 0$: a fixed point of squaring and of scaling	Chapter 4
origin	the point from which signed quantities are measured	ordered rings
bit pattern	`0…0` encodes 0 in two’s complement and $+0$ in IEEE formats	Chapter 2

The roles are compatible, but they are not the same role. In particular, zero is the identity of one operation (addition) and the annihilator of the other (multiplication). Every peculiarity of division by zero comes from that second role: an annihilator cannot be undone.

20.2 What zero is not

Programming languages place several other concepts next to zero, and some encode them with the same bits. They are different things.

- **NULL** (a null reference) is the absence of a reference. Some languages encode it as the address 0; the encoding coincides, the meaning does not. Dereferencing NULL is an error; using the integer 0 is arithmetic.
- **false** is a truth value. In C and Python, `0` and `False` compare equal, and `0` is “falsy”; that is a language convention about conversion, not a fact about numbers.
- **the empty string** `""` is a numeral-like word of length zero — the spelling of zero’s empty address, as it happens — but a string, not a number.
- **absence** (no value, `None`, a missing field, SQL `NULL`) is the absence of any value; SQL’s `NULL` propagates through arithmetic like a NaN.
- **uninitialized memory** has no defined value at all; in C, reading it can be undefined behaviour, and it is not zero even if it often happens to contain zero bits.
- **NaN** is a datum that is not a number, and is unequal even to itself.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
nan = float("nan")
print("0 == False:", 0 == False, "   0 is falsy:", not 0, "   0.0 and -0.0 are falsy:", not 0.0, not -0.0)
print("None == 0:", None == 0, "   '' == 0:", "" == 0, "   nan == nan:", nan == nan)
print("int(''):", end=" ")
try:
    int("")
except ValueError as e:
    print("ValueError —", e)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0 == False: True    0 is falsy: True    0.0 and -0.0 are falsy: True True
None == 0: False    '' == 0: False    nan == nan: False
int(''): ValueError — invalid literal for int() with base 10: ''
```

The last line is a small instance of Chapter 5: Python refuses to read the empty string as a numeral, although in the digit grammar the empty word *is* zero's address. A reasonable choice — an empty input field is far more often a mistake than a zero — and a reminder that a written convention (`0`) has been put in place of the canonical but invisible address.

20.3 Two zeros

IEEE 754 binary formats have two zeros, $+0$ (all bits zero) and -0 (only the sign bit set). They compare equal, and they are the same *value*: $\mathrm{dec}(+0) = \mathrm{dec}(-0) = 0$ (Proposition GIN-PROP-050). But they are different *data*, and some operations can tell them apart.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from fractions import Fraction
from ginsdk import ieee

pz = ieee.round_exact(ieee.BINARY64, Fraction(0))[0]
nz = ieee.neg(pz)
one = ieee.from_decimal("1")[0]
print("bits:", hex(pz.bits()), hex(nz.bits()), "  same value:", pz.value() == nz.value())
print("1 / +0 =", ieee.div(one, pz)[0], "   1 / -0 =", ieee.div(one, nz)[0])
print("+0 + -0 =", ieee.add(pz, nz)[0], "   -0 + -0 =", ieee.add(nz, nz)[0], "   -1 * 0 =",
ieee.mul(ieee.neg(one), pz)[0])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
bits: 0x0 0x8000000000000000   same value: True
1 / +0 = Infinity    1 / -0 = -Infinity
```

```
+0 + -0 = 0    -0 + -0 = -0    -1 * 0 = -0
```

The sign of zero records the side from which a result underflowed to zero, and division by zero returns the infinity on that side: $1/(+0) = +\infty$, $1/(-0) = -\infty$, mirroring $\lim_{x \to 0^{\pm}} 1/x = \pm\infty$. Kahan argued that this information is essential for the correct behaviour of complex functions along branch cuts [Kahan 1987]. It is also a source of surprise: $x = y$ does not imply $1/x = 1/y$ in IEEE arithmetic, so equality of data is not equality of values, and neither is a congruence for division.

20.4 One: identity and generator

One is the identity of multiplication ($1 \cdot a = a$), and it is the **generator** of the natural numbers under addition: every n is $1 + 1 + \cdots + 1$, and $\mathbb{N}$ is the free monoid on the single generator 1 (Chapter 4). The two roles are independent. In $\mathbb{Z}/n$ every element is still a sum of ones, but 1 is no longer the only element with an inverse: the *units* of $\mathbb{Z}/n$ are the $\varphi(n)$ residues coprime to n. In $\mathbb{Z}$, 1 and -1 are the units. In a field, everything except 0 is.

Zero and one also share a role: they are the only solutions of $x^2 = x$ in an integral domain. In rings with zero divisors there can be more idempotents: in $\mathbb{Z}/6$, $x^2 = x$ has the four solutions $0, 1, 3, 4$, and each non-trivial idempotent splits the ring into a product ($\mathbb{Z}/6 \cong \mathbb{Z}/2 \times \mathbb{Z}/3$, with $3 \mapsto (1, 0)$ and $4 \mapsto (0, 1)$).

20.5 Empty sums, empty products, and 0^0

A sum of no terms is 0, and a product of no factors is 1: these are the only values that keep the laws $\sum_{A \sqcup B} = \sum_A + \sum_B$ and $\prod_{A \sqcup B} = \prod_A \cdot \prod_B$ true when A or B is empty. From the empty product follow $a^0 = 1$ for every a, including $a = 0$, and $0! = 1$. In combinatorics and algebra, $0^0 = 1$ is therefore not a guess but the only consistent value: x^0 counts the functions from the empty set to a set of size x, and there is exactly one such function even when $x = 0$.

In analysis, the same symbol names a *limit form*: if $f(x) \to 0$ and $g(x) \to 0$, then $f(x)^{g(x)}$ can tend to any value in $[0, 1]$ or fail to converge. The SDK follows the algebraic convention and says so:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

o = evaluate("0 ^ 0", "Q")
print(o.display, "—", o.notes[0])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 - 0^0 = 1 is a convention (the empty product); as a limit form it is indeterminate
```

COUNTEREXAMPLE 20.1 (0^0 has a determined value) COUNTEREXAMPLE GIN-NEG-009

As the empty product, $0^0 = 1$ by the only consistent convention; as the limit form $\lim f^g$ with $f, g \to 0$, it is indeterminate ($x^{c/\ln x} = e^c$ for every c, while $x^x \to 1$ as $x \to 0^+$). The "value" depends on whether 0^0 is read as a product or as a limit.

20.6 A brief history of zero as a number

Zero as a *placeholder* inside numerals predates zero as a *number*. Late Babylonian scribes used a separator sign for an empty sexagesimal place; positional decimal notation with a zero digit developed in India, and zero as a number with rules of arithmetic is stated explicitly by Brahmagupta in 628 CE [Kaplan 1999; Plofker 2009].

HISTORICAL NOTE (Brahmagupta's rules for zero (628 CE)) GIN-HIST-001

In the *Brāhmasphuṭasiddhānta*, Brahmagupta defines zero as the result of subtracting a number from itself and gives rules: the sum of zero and a number is the number, the product of zero and any number is zero, and $0/0 = 0$; a non-zero number divided by zero is a fraction with zero as denominator. Bhāskara II later called such a quantity *khahara* and described it as unchanged by adding or subtracting finite quantities — a description close to the modern ∞ of the projective line [O'Connor & Robertson (MacTutor); Plofker 2009]. In the vocabulary of this book, Brahmagupta's $0/0 = 0$ is a *selection* from a solution set that contains every number, the convention that meadows and proof assistants adopt today (Chapter 18). The account here relies on secondary sources; the primary text was not consulted.

EXERCISE 20.1

Find all solutions of $x^2 = x$ in $\mathbb{Z}/12$ and $\mathbb{Z}/30$. How does their number relate to the prime factorization of the modulus?

▶ Solution to Exercise 20.1

EXERCISE 20.2

Give three IEEE binary64 operations whose results differ only in the sign of zero, and one operation that turns the sign of zero into a difference of values.

▶ Solution to Exercise 20.2

EXERCISE 20.3

Using $\prod_{\varnothing} = 1$, justify $0! = 1$ and $\binom{n}{0} = 1$. What value does the empty min take, and why?

▶ Solution to Exercise 20.3

SUMMARY

- Zero is identity, empty count, start state, empty address, placeholder, annihilator, fixed point, origin and a bit pattern; division by zero fails because of the annihilator role.
- Zero is not NULL, false, empty, absent, uninitialized or NaN, even where encodings coincide.
- IEEE 754 has two zeros with one value; division can distinguish them.
- One is both the multiplicative identity and the additive generator; idempotents other than 0 and 1 signal a product decomposition.
- Empty sums and products force $0^0 = 1$ algebraically; as a limit form it is indeterminate.
- Zero as a number with arithmetic is documented from Brahmagupta (628 CE); his $0/0 = 0$ is a selection.

PART IX

Binary and Digital Arithmetic

Bits are not numbers: words, masks, shifts, two's complement as truncated 2-adic integers, and arithmetic circuits.

Chapter 21

Bits, Words, and Two's Complement

> *A word of thirty-two bits is not a number. It is a numeral in a finite alphabet, waiting for an encoding to say which number it names.*

LEARNING OBJECTIVES

- Read a w-bit word under four signed encodings, and say which have two zeros and which are asymmetric.
- Prove that two's complement is the truncation of the 2-adic expansion, and explain why one adder serves signed and unsigned arithmetic.
- Detect signed overflow from the carries into and out of the top bit, and prove the rule.
- Distinguish arithmetic right shift from division, and $-x$ from $|x|$, at the most negative number.
- Interpret bit tricks such as `x & (x - 1)` as operations on numerals.

21.1 Words and encodings

A w-bit word is a string in $\{0,1\}^w$. Read as an unsigned binary numeral it denotes a number in $[0, 2^w)$. To represent negative numbers, an *encoding* (Definition GIN-DEF-003) must be chosen, and several have been used.

4-BIT WORD	UNSIGNED	SIGN-MAGNITUDE	ONES' COMPLEMENT	TWO'S COMPLEMENT	OFFSET (BIAS 7)
`0000`	0	+0	+0	0	−7
`0001`	1	1	1	1	−6
`0111`	7	7	7	7	0
`1000`	8	−0	−7	−8	1
`1001`	9	−1	−6	−7	2

4-BIT WORD	UNSIGNED	SIGN-MAGNITUDE	ONES' COMPLEMENT	TWO'S COMPLEMENT	OFFSET (BIAS 7)
1110	14	−6	−1	−2	7
1111	15	−7	−0	−1	8

Sign-magnitude and ones' complement have **two zeros**, which complicates every equality test. Two's complement has one zero and is **asymmetric**: it can represent -8 but not $+8$. Offset ("biased") encoding is used for the exponent field of IEEE floating-point numbers, where it makes unsigned comparison of the bit patterns agree with numerical comparison. Modern processors use two's complement for integers because it has a property the others lack: the *same adder circuit* that adds unsigned numbers also adds signed ones.

21.2 Two's complement is truncated 2-adic

Why does the pattern `1111` denote -1? Run the residue algorithm of Chapter 5 on -1 in base 2: the last digit is $-1 \bmod 2 = 1$, and the next state is $(-1-1)/2 = -1$ again. The algorithm never stops; it produces the infinite word $\ldots 1111$. That infinite word is the **2-adic expansion** of -1: the formal sum $1 + 2 + 4 + 8 + \cdots$, which in the 2-adic integers really does equal -1 (add 1 and every carry propagates forever, leaving $\ldots 0000$).

PROPOSITION 21.1 (Two's complement is the truncated 2-adic expansion)

CLASSICAL RESULT GIN-PROP-008

For every integer n and width w, the first w digits $d_0, \ldots, d_{w-1}$ of the base-2 residue expansion of n ($d = x \bmod 2$, $x \leftarrow (x-d)/2$) are the bits of $n \bmod 2^w$ — the w-bit two's-complement encoding when $-2^{w-1} \le n < 2^{w-1}$. For $n < 0$ the expansion never terminates: the state reaches -1, a fixed point of $x \mapsto (x-1)/2$, after which every digit is 1.

Proof.

After w steps $n = \sum_{i<w} d_i 2^i + 2^w x_w$, so $\sum d_i 2^i \equiv n \pmod{2^w}$ and lies in $[0, 2^w)$. For $x < 0$ the map $x \mapsto \lfloor x/2 \rfloor$ increases toward -1 and fixes it, and at -1 the digit is 1. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk.numerals import BINARY, twos_complement_from_2adic

digits, status = BINARY.expand(-5, max_digits=12)
print("residue expansion of -5, least significant first:", digits, status)
```

```python
for n in (5, -5, -1, -128, 127):
    print(f"{n:5d}: 8-bit two's complement {twos_complement_from_2adic(n, 8)}   n mod 256 = {n % 256}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
residue expansion of -5, least significant first: (1, 1, 0, 1) periodic
    5: 8-bit two's complement 00000101   n mod 256 = 5
   -5: 8-bit two's complement 11111011   n mod 256 = 251
   -1: 8-bit two's complement 11111111   n mod 256 = 255
 -128: 8-bit two's complement 10000000   n mod 256 = 128
  127: 8-bit two's complement 01111111   n mod 256 = 127
```

The expansion of -5 stops after four digits with status “periodic”: the state has returned to -1, and every later digit is 1. A w-bit machine keeps the first w digits and discards the rest — which is exactly reduction modulo 2^w. That is why one adder suffices: unsigned and two’s-complement addition are both addition in $\mathbb{Z}/2^w$; only the *labels* of the 2^w residues differ, $[0, 2^w)$ in one case and $[-2^{w-1}, 2^{w-1})$ in the other.

21.3 Detecting overflow

The adder computes the correct result modulo 2^w in both readings. Whether the result is also correct *as an integer* depends on the reading. For unsigned numbers, overflow is the carry out of the top bit. For signed numbers the rule is different.

PROPOSITION 21.2 (Signed overflow is a disagreement of carries) CLASSICAL RESULT

GIN-PROP-067

Add two w-bit two’s-complement numbers with a binary adder, and let c_{w-1} be the carry into the top position and c_w the carry out of it. The exact sum lies outside $[-2^{w-1}, 2^{w-1})$ if and only if $c_{w-1} \neq c_w$. Equivalently, overflow occurs iff both operands have the same sign and the result’s sign differs from it.

Proof.

Write the operands as $a = -a_{w-1}2^{w-1} + a'$ and $b = -b_{w-1}2^{w-1} + b'$ with $0 \le a', b' < 2^{w-1}$. Then $a' + b' = s' + c_{w-1}2^{w-1}$ with s' the low $w-1$ bits of the result, and the top bit and carry out satisfy $a_{w-1} + b_{w-1} + c_{w-1} = s_{w-1} + 2c_w$. The exact sum is $a + b = -(a_{w-1} + b_{w-1})2^{w-1} + s' + c_{w-1}2^{w-1}$. The encoded result is $-s_{w-1}2^{w-1} + s'$. Their difference is $(-(a_{w-1} + b_{w-1}) + c_{w-1} + s_{w-1})2^{w-1} = (2c_{w-1} - 2c_w)2^{w-1}$ after substituting $s_{w-1} = a_{w-1} + b_{w-1} + c_{w-1} - 2c_w$. It is zero iff $c_{w-1} = c_w$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def check(w):
    lo, hi, bad = -2 ** (w - 1), 2 ** (w - 1), 0
    for a in range(lo, hi):
        for b in range(lo, hi):
            ua, ub = a % 2 ** w, b % 2 ** w
            low = (ua % 2 ** (w - 1)) + (ub % 2 ** (w - 1))
            c_in = low >> (w - 1)
            c_out = (ua + ub) >> w
            overflow = not (lo ⩽ a + b < hi)
            bad += overflow ≠ (c_in ≠ c_out)
    return bad
print({w: check(w) for w in range(2, 9)}, "mismatches by width")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
{2: 0, 3: 0, 4: 0, 5: 0, 6: 0, 7: 0, 8: 0} mismatches by width
```

Processors compute both flags on every addition: the *carry* flag for the unsigned reading and the *overflow* flag for the signed reading. The hardware does not know which reading the program intends; the program chooses which flag to test — or, in most high-level languages, tests neither.

21.4 The most negative number

Two's complement's asymmetry has consequences that every systems programmer meets eventually. In 32 bits the most negative number is -2^{31}, and its negation, 2^{31}, is not representable: wraparound negation returns -2^{31} itself. So does the absolute value function of most C libraries, and so does $-2^{31} / -1$ on machines that do not trap (Chapter 24). The equation $-x = x$ has *two* solutions in $\mathbb{Z}/2^{32}$, namely 0 and 2^{31}; in $\mathbb{Z}$ it has one.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e, d in [("0 - (-2147483647 - 1)", "int32 Java"), ("(-2147483647 - 1) / -1", "int32 Java"),
             ("(-2147483647 - 1) / -1", "int32 x86-64"), ("0 - (-2147483647 - 1)", "int32 Rust (debug)")]:
    o = evaluate(e, d)
    print(f"{e:24s} {d:19s} {o.status:10s} {o.display or o.reason.split(':')[0].split(' —')[0]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0 - (-2147483647 - 1)    int32 Java          value      -2147483648
(-2147483647 - 1) / -1   int32 Java          value      -2147483648
```

```
(-2147483647 - 1) / -1   int32 x86-64        trap       #DE (divide error)
0 - (-2147483647 - 1)    int32 Rust (debug)  exception  subtraction overflows 32 bits (exact result 2147483648)
```

21.5 Shifts are not divisions

Shifting a binary numeral left by k places multiplies its value by 2^k, modulo 2^w; in the move language of Chapter 5 it appends k zero digits. Shifting right drops digits. For unsigned words, a right shift by k is $\lfloor x/2^k \rfloor$. For signed words, an *arithmetic* right shift copies the sign bit into the vacated positions and computes $\lfloor x/2^k \rfloor$ — *floored* division. C's integer division truncates. The two disagree on negative odd numbers: $-7 \gg 1 = -4$, but $-7/2 = -3$ in C. A compiler that replaces `x / 2` by a shift must add a correction for negative x; one that replaces `x >> 1` by `x / 2` would introduce a bug.

21.6 Bit operations as numeral operations

Bitwise operations act on numerals, not on values, but some of them have clean meanings on values:

- `x & (x - 1)` clears the lowest set bit: subtracting 1 turns the trailing zeros into ones and the lowest one into a zero (the successor's carry pattern, reversed), and the AND keeps everything above. Hence `x & (x - 1) == 0` tests whether x is zero or a power of two.
- `x & -x` isolates the lowest set bit, $2^{v_2(x)}$, where v_2 is the 2-adic valuation: in two's complement, $-x = \bar{x} + 1$ agrees with x exactly at the lowest set bit.
- `popcount(x)`, the number of ones, is the digit sum of the binary numeral; Chapter 7 used it to count the cost of counting.

These identities hold for unsigned words and, with the 2-adic reading, for signed ones; they are classical tools of bit-level programming. They also show the point of this chapter in miniature: a bit operation is a statement about the numeral, and it says something about the number only through an encoding.

> **EXERCISE 21.1**
>
> Write -3 in 6-bit sign-magnitude, ones' complement, two's complement and offset-31 encoding.

▶ Solution to Exercise 21.1

EXERCISE 21.2

In 8-bit two's complement, compute $100 + 50$, $-100 + (-50)$ and $100 + (-50)$ with an adder, giving the carries c_7, c_8 and the overflow verdict.

▶ Solution to Exercise 21.2

EXERCISE 21.3

Prove that in w-bit two's complement, $x \mathbin{\&} (-x) = 2^{v_2(x)}$ for every $x \neq 0$, where $v_2(x)$ is the number of trailing zeros.

▶ Solution to Exercise 21.3

SUMMARY

- A word is a numeral; an encoding gives it a value. Sign-magnitude and ones' complement have two zeros; two's complement is asymmetric.
- Two's complement is the truncated 2-adic expansion: w-bit arithmetic is $\mathbb{Z}/2^w$, so one adder serves both readings.
- Signed overflow is detected by $c_{w-1} \neq c_w$; unsigned overflow by c_w.
- -2^{w-1} is its own negative; arithmetic right shift floors while C's division truncates.
- Bit tricks are numeral operations whose meaning on values passes through the encoding.

Chapter 22

Arithmetic Circuits

> *A circuit is an expression in which every subexpression is computed exactly once, by a piece of hardware, at the same time as all the others it does not depend on.*

LEARNING OBJECTIVES

- Define a Boolean circuit with its syntax (well-formed wiring), semantics (the function computed) and cost (size and depth).
- Build half and full adders from AND, OR and XOR gates and from NAND gates alone, and verify them exhaustively.
- Read the size and depth of the adders, subtractor, multiplier and divider of the SDK, and relate them to the algorithms of earlier chapters.
- State the classical circuit-complexity facts about arithmetic: AC^0 contains addition but not multiplication, and TC^0 contains multiplication and division.

22.1 Circuits as grammars

DEFINITION 22.1 (Circuit) DEFINITION GIN-DEF-040

A **circuit** is a directed acyclic graph whose sources are input wires and constants and whose internal nodes are gates from a fixed basis — here $\{\text{NOT}, \text{AND}, \text{OR}, \text{XOR}\}$ with fan-in at most 2, or $\{\text{NAND}\}$. *Syntax*: every gate input is a previously defined wire. *Semantics*: the Boolean function computed at the output wires. *Cost*: **size** (the number of gates) and **depth** (the length of the longest path from an input to an output).

A circuit is an arithmetic expression over bits in which shared subexpressions are computed once (a straight-line program, Chapter 6) and in which everything that can be computed simultaneously is. Its size measures the hardware, its depth the time. The same Boolean function has many circuits, and the trade-off between size and depth is the hardware form of the trade-off between work and parallel time.

22.2 Adders from gates

The **half adder** adds two bits: sum $= a \oplus b$, carry $= a \wedge b$ — two gates, depth 1. The **full adder** adds three: sum $= a \oplus b \oplus c$, carry $= (a \wedge b) \vee ((a \oplus b) \wedge c)$ — five gates, depth 3. NAND is *universal*: every Boolean function has a NAND-only circuit. A full adder needs nine NAND gates in the SDK's construction.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import circuits as K
from itertools import product

for name, c in [("half adder", K.half_adder()), ("full adder", K.full_adder()), ("full adder (NAND)",
K.full_adder_nand())]:
    ok = True
    for bits in product((0, 1), repeat=len(c.inputs)):
        out = c.run(dict(zip(c.inputs, bits)))
        total = sum(bits)
        ok &= out == [total % 2, total // 2]
    print(f"{name:18s} gates {c.size:2d}  depth {c.depth()}  {c.gate_counts()}  exhaustively correct: {ok}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
half adder         gates  2  depth 1  {'XOR': 1, 'AND': 1}  exhaustively correct: True
full adder         gates  5  depth 3  {'XOR': 2, 'AND': 2, 'OR': 1}  exhaustively correct: True
full adder (NAND)  gates  9  depth 6  {'NAND': 9}  exhaustively correct: True
```

The full adder is the grammar of column addition (Chapter 10) frozen into hardware: three input digits, one output digit, one carry. Chaining n of them gives the ripple-carry adder, whose carry path is the sequential composition of Chapter 11's carry monoid; prefix adders compute the same monoid products in a tree.

22.3 A gallery of arithmetic circuits

The SDK constructs every arithmetic circuit of this book explicitly and verifies it exhaustively for small widths (GIN-IMP-003). Their measured costs summarize Parts V–VIII in hardware terms:

CIRCUIT (N-BIT OPERANDS)	SIZE	DEPTH	ALGORITHM	CHAPTER
ripple-carry adder	$5n - 3$	$2n - 1$	column addition	10, 11

CIRCUIT (N-BIT OPERANDS)	SIZE	DEPTH	ALGORITHM	CHAPTER
Kogge–Stone adder ($n = 256$)	5891	17	parallel prefix, doubling	11
Brent–Kung adder ($n = 256$)	2018	30	parallel prefix, tree	11
subtractor $a + \bar{b} + 1$ ($n = 6$)	36	14	complement and add	12
array multiplier	$6n^2 - 8n$	$6n - 8$	shift and add	15
restoring divider ($n = 4, 8, 16$)	$172, 632, 2416$	$51, 163, 579$	long division	19

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import circuits as K

for n in (4, 8, 16):
    rows = [("ripple adder", K.ripple_adder(n)), ("Kogge-Stone", K.kogge_stone_adder(n)),
            ("multiplier", K.array_multiplier(n)), ("divider", K.restoring_divider(n))]
    print(f"n = {n:2d}: " + "   ".join(f"{name} {c.size}/{c.depth()}" for name, c in rows))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n =  4: ripple adder 17/7   Kogge-Stone 23/5   multiplier 64/16   divider 172/51
n =  8: ripple adder 37/15   Kogge-Stone 67/7   multiplier 320/40   divider 632/163
n = 16: ripple adder 77/31   Kogge-Stone 179/9   multiplier 1408/88   divider 2416/579
```

The pattern is the familiar hierarchy of the four operations: addition linear in size, multiplication and division quadratic. The restoring divider is the deepest, because each of its n rows must wait for the previous row's subtraction to finish before it can decide whether to keep the difference — a sequential dependence that the multiplier, whose partial products are independent, does not have.

22.4 What circuit complexity knows

Circuit complexity asks how small and how shallow circuits for a function can be, as n grows. Two classes matter here. AC^0 contains the functions computable by circuits of polynomial size

and *constant* depth, with AND and OR gates of *unbounded* fan-in. TC^0 allows, in addition, *threshold* (majority) gates.

THEOREM 22.2 (The circuit complexity of the four operations) CLASSICAL RESULT

GIN-HIST-006

1. *Addition and comparison of n-bit numbers are in AC^0.*
2. *Multiplication of n-bit numbers is not in AC^0 (Furst, Saxe and Sipser [Furst, Saxe & Sipser 1984], by reduction from parity).*
3. *Multiplication, and also division and iterated multiplication, are in TC^0; for division, membership in the uniform version of the class was established by Hesse, Allender and Barrington [Hesse, Allender & Barrington 2002].*

Proof.

(1) The carry into position i is 1 iff some position $j < i$ generates and every position between j and i propagates: an OR over j of ANDs over the positions above j — depth two with unbounded fan-in, polynomial size. (2) and (3) are deep results; see the cited papers. ∎

The theorem is the circuit-level twin of the automaton-level Theorem GIN-THM-005. With finite automata, addition is regular and multiplication is not; with constant-depth circuits, addition is in AC^0 and multiplication is not. In both settings, what makes addition easy is that a carry is a *local summary* — one of three monoid elements, or one OR of ANDs — while multiplication must combine every bit of one operand with every bit of the other.

REMARK

With *bounded* fan-in, as in the circuits of this book, no adder can have constant depth: the carry out depends on all $2n$ inputs, so the depth is at least $1 + \log_2 n$ (GIN-PROP-059). The constant-depth statements above rely on gates that read arbitrarily many wires at once; physical implementations pay for that fan-in in delay. Statements about circuit depth are meaningful only together with the gate model, just as statements about time are meaningful only together with a cost model (Chapter 36).

EXERCISE 22.1

Show that the carry out of a full adder is the majority function of its three inputs, and give a circuit for it with four AND/OR gates.

▶ Solution to Exercise 22.1

EXERCISE 22.2

Write the depth-2 unbounded-fan-in formula for the carry into position 3 of an addition, in terms of $g_i = a_i \wedge b_i$ and $p_i = a_i \oplus b_i$.

▶ Solution to Exercise 22.2

EXERCISE 22.3

The SDK's divider has depth 51 for $n = 4$ and 163 for $n = 8$. Explain why its depth grows faster than linearly in n, and estimate the growth from its structure.

▶ Solution to Exercise 22.3

SUMMARY

- A circuit is a straight-line program over bits; size is hardware, depth is time.
- Half and full adders are column addition in gates; NAND alone suffices (nine gates for a full adder here).
- Measured: adders linear, multiplier and divider quadratic in size; the restoring divider is the deepest because its rows are sequential.
- Addition is in AC^0; multiplication is not; multiplication and division are in TC^0 — the circuit-level twin of "addition is finite-state, multiplication is not".

PART X

Machines That Compute Arithmetic

An expression traced from characters to instructions, and what real compilers, languages, instruction sets and floating-point units do — measured, emulated, or specified.

Chapter 23

From Symbol to Instruction

> *The slash in x / y is a promise of an operation. Seven stages later it is one of six different instructions, or none at all.*

LEARNING OBJECTIVES

- Name the stages through which an arithmetic expression becomes machine execution, and say what each stage decides.
- Read the measured parse tree and bytecode of `x / y` in CPython and the instructions selected by clang for x86-64, AArch64 and RISC-V.
- Explain how operand types select between integer division, unsigned division and floating-point division.
- Explain why a C compiler may delete a zero test that follows a division, and show that two compilers measurably differ in whether they do.

23.1 Seven stages

1. **Lexical analysis.** The characters `x / y` become three tokens: identifier, operator, identifier.
2. **Parsing.** The tokens become a tree: a binary node for division with two leaves. Precedence and associativity (Chapter 8) are decided here.
3. **Typing.** The types of the operands select the *operation*: integer division, unsigned division, floating-point division, big-integer division, or an error. In Python 3, `int / int` is *true* division and returns a float; in C, Java, Go and Rust it is truncating integer division.
4. **Intermediate representation.** The tree is lowered to a sequence of simple operations — bytecode for an interpreter, an SSA-form IR for an optimizing compiler.
5. **Optimization.** The IR is transformed: constants are folded, divisions by constants become multiplications (Chapter 19), redundant computations are removed — under the rules of the language, including its rules about undefined behaviour.
6. **Instruction selection.** Each IR operation is mapped to instructions of the target architecture.

7. **Execution.** The processor runs the instructions. Division by zero, overflow, rounding and flags happen here — or have been anticipated, or excluded, by earlier stages.

In the four-layer vocabulary of this book, stages 1–2 are *syntax*, stage 3 is *typing*, the mathematical *semantics* is what stages 4–6 must preserve, and stage 7 is *execution*. The stages of a compiler are not the layers of meaning, but they line up with them.

23.2 In CPython

Python's own tools expose the first stages. The parse tree is stable across recent versions:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import ast
print(ast.dump(ast.parse("x / y", mode="eval").body))
print(ast.dump(ast.parse("-7 // 2", mode="eval").body))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
BinOp(left=Name(id='x', ctx=Load()), op=Div(), right=Name(id='y', ctx=Load()))
BinOp(left=UnaryOp(op=USub(), operand=Constant(value=7)), op=FloorDiv(), right=Constant(value=2))
```

The second tree confirms Chapter 8's convention: `-7 // 2` is unary minus applied to `7`, then floor division. The bytecode, which differs between CPython versions, was recorded by GIN-EXP-004 with CPython 3.13: `RESUME; LOAD_NAME x; LOAD_NAME y; BINARY_OP /; RETURN_VALUE`. A single generic instruction, `BINARY_OP /`, defers the choice of operation to run time, when the interpreter inspects the operand types: for two `int`s it computes a correctly rounded float quotient; for a zero divisor it raises `ZeroDivisionError` before any IEEE operation is attempted (GIN-OBS-006).

23.3 In a C compiler: instruction selection

An optimizing compiler resolves the operation statically from the types. GIN-EXP-004 compiled the same three one-line C functions with clang 18 at `-O2` for three targets and recorded the instructions:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, pathlib, ginsdk
data = json.loads((pathlib.Path(ginsdk.__file__).parents[3] / "experiments/results/exp004.json").read_text())
for target, fns in data["data"]["instruction_selection"].items():
    picked = {k: [i.split("\t")[0] for i in v if i.split("\t")[0] in ("idivl", "divl", "divsd", "sdiv", "udiv",
"fdiv", "divw", "divuw", "fdiv.d")] for k, v in fns.items()}
    print(f"{target:20s} int: {picked['idiv'][0]:6s} unsigned: {picked['udiv'][0]:6s} double: {picked['fdiv']
[0]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x86_64-linux-gnu     int: idivl  unsigned: divl   double: divsd
aarch64-linux-gnu    int: sdiv   unsigned: udiv   double: fdiv
riscv64-linux-gnu    int: divw   unsigned: divuw  double: fdiv.d
```

EMPIRICAL OBSERVATION 23.1

(One symbol, six instructions per architecture family) EMPIRICAL OBSERVATION

GIN-OBS-007

For `int`, `unsigned` and `double` operands, clang 18 at `-O2` selects `idivl` / `divl` / `divsd` on x86-64, `sdiv` / `udiv` / `fdiv` on AArch64 and `divw` / `divuw` / `fdiv.d` on RISC-V 64 (GIN-EXP-004; compiled, not executed, for the non-x86 targets).

The x86 integer sequence is `cltd; idivl`: the first instruction sign-extends the 32-bit dividend into the 64-bit register pair that `idiv` divides. That detail is the instruction set's own grammar showing through — the x86 divide instruction divides a double-width number by a single-width one, which is why its quotient can overflow (the case $-2^{31}/-1$) and why the instruction can trap for a reason other than division by zero.

23.4 What the optimizer may assume

In ISO C, a division by zero, and a signed division whose quotient is not representable, have *undefined behaviour* (C17 §6.5.5). The standard imposes no requirement on such a program — and an optimizer may therefore assume that they do not happen. Consider:

C

```
int f(int a, int b) {
    int q = a / b;
    if (b == 0) return -1;
```

```
    return q;
}
```

The test comes *after* the division. If $b = 0$, the division has already made the behaviour of the whole program undefined, so the compiler may reason: in every execution with defined behaviour, $b \neq 0$ at the test; the test is always false; delete it.

EMPIRICAL OBSERVATION 23.2

(Compilers differ on deleting a zero test after a division) EMPIRICAL OBSERVATION

GIN-OBS-008

On the session host, clang 18.1.3 at `-O2` and `-O3` compiles `f` to `cltd; idivl; retq` — the test and the `-1` path are deleted — while gcc 13.3 at `-O2` keeps them (`idivl; testl; movl $-1; cmove`). Both are conforming (GIN-EXP-004).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, pathlib, ginsdk
data = json.loads((pathlib.Path(ginsdk.__file__).parents[3] / "experiments/results/exp004.json").read_text())
for compiler, r in data["data"]["c_guard_after_division"].items():
    body = "; ".join(i.replace("\t", " ") for i in r["instructions"] if not i.startswith(("endbr", "push",
"pop", "mov %rsp")))
    print(f"{compiler:10s} keeps the zero test: {r['has_compare_or_test']!s:5s}  {body[:90]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
gcc -O0    keeps the zero test: True   movq %rsp, %rbp; movl %edi, -20(%rbp); movl %esi, -24(%rbp); movl
-20(%rbp), %eax; cltd; i
gcc -O2    keeps the zero test: True   movl %edi, %eax; cltd; idivl %esi; testl %esi, %esi; movl $-1, %edx;
cmove %edx, %eax; ret
clang -O2  keeps the zero test: False  movl %edi, %eax; cltd; idivl %esi; retq
clang -O3  keeps the zero test: False  movl %edi, %eax; cltd; idivl %esi; retq
```

On x86-64 the difference is invisible in practice: the `idivl` traps when $b = 0$, before either version reaches the test. On AArch64, whose `sdiv` returns 0 instead of trapping, the deleted test would change observable behaviour: gcc's version returns -1, clang's returns 0. A programmer who wrote the test to *catch* division by zero has been told, by the language standard, that the test is too late; the compiler that removes it is not wrong. The repair is to test *before* dividing.

COUNTEREXAMPLE 23.3

(Compilers always exploit undefined division to delete later tests)

COUNTEREXAMPLE GIN-NEG-004

gcc 13.3 at `-O2` keeps the zero test in `f`; only clang 18 deletes it. Undefined behaviour *permits* the optimization; it does not require it, and different compilers (and versions, and flags) make different choices.

23.5 Constants at compile time

When the divisor is a *constant* zero, the failure can be detected before the program runs — and languages choose different layers for it. GIN-EXP-004 compiled `1 / 0` with integer operands in four languages:

LANGUAGE	COMPILE TIME	RUN TIME
C (gcc 13.3)	warning `division by zero [-Wdiv-by-zero]`; compiles	`SIGFPE` (x86 `#DE`)
Go	error: `invalid operation: division by zero`	—
Rust	error: `this operation will panic at runtime`	—
Java	compiles without warning	`ArithmeticException: / by zero`

The same five characters fail at the typing layer in Go and Rust (constant expressions are checked as part of type checking), at execution in Java, and at execution with undefined meaning in C (GIN-OBS-004). In the vocabulary of Chapter 8, Go and Rust apply the *restriction* repair statically; Java and C defer to the execution layer.

EXERCISE 23.1

For each language, give the operation selected by `7 / 2` with integer literals and the result: C, Java, Python 3, JavaScript (Number), JavaScript (BigInt `7n / 2n`).

▶ Solution to Exercise 23.1

EXERCISE 23.2

Rewrite `f` so that every conforming compiler must keep the zero test, and explain why your version has no undefined behaviour for $b = 0$. Is it still undefined for some input?

▶ Solution to Exercise 23.2

EXERCISE 23.3

Why does the x86 division need `cltd` before `idivl`, and how does this explain that `idiv` can trap when the divisor is not zero?

▶ Solution to Exercise 23.3

SUMMARY

- An expression passes through lexing, parsing, typing, IR, optimization, instruction selection and execution; types choose the operation.
- CPython defers the choice to run time (`BINARY_OP /`); a C compiler selects one of `idivl`, `divl`, `divsd`, `sdiv`, `udiv`, `fdiv`, `divw`, `divuw`, `fdiv.d`.
- Because C division by zero is undefined, clang 18 deletes a zero test placed after the division and gcc 13.3 keeps it; both conform.
- Constant `1 / 0` fails at compile time in Go and Rust, at run time in Java, and with undefined meaning in C.

Chapter 24

Integer Division by Zero on Real Machines

> *Ask seven toolchains on one computer to divide one by zero and you receive seven different answers, two of which look like numbers.*

LEARNING OBJECTIVES

- State the behaviour of integer division at its two boundaries, $x/0$ and $-2^{w-1}/-1$, for three instruction sets and six languages, and cite where each is specified.
- Distinguish specified behaviour from behaviour measured on a host and from behaviour emulated by an independent implementation.
- Explain the design rationale of trapping (x86) and of totalizing (AArch64, RISC-V) instruction sets.
- Use the SDK's machine models, and know on what evidence they rest.

24.1 Two boundaries

Signed integer division on w-bit words has exactly two inputs at which the mathematical quotient is not a representable word: a zero divisor (no quotient exists, or every value is one) and $-2^{w-1}/-1$ (the quotient 2^{w-1} exists in $\mathbb{Z}$ but is one too large). Each system must decide what to do at both.

SYSTEM	$X/0$	$-2^{31}/-1$	SOURCE; EVIDENCE
x86-64 `idiv`	`#DE` trap; Linux delivers `SIGFPE`	`#DE` trap	Intel SDM [Intel SDM]; measured via C
AArch64 `sdiv`	0, no signal, no flag	-2^{31} (wraps)	Arm ARM [Arm ARM]; **emulated** (QEMU)
RISC-V `div`	-1; `rem` gives the dividend	-2^{31}; `rem` gives 0	RISC-V ISA [RISC-V ISA Manual];

SYSTEM	$X/0$	$-2^{31}/-1$	SOURCE; EVIDENCE
			emulated (QEMU)
ISO C `int`	undefined behaviour	undefined behaviour	C17 §6.5.5 [ISO C17]; gcc on x86: `SIGFPE`
Java `int`	`ArithmeticException`	-2^{31}, no exception	JLS §15.17.2 [JLS SE 21]; measured
Rust `i32` (debug and release)	panic: attempt to divide by zero	panic: attempt to divide with overflow	Rust reference [Rust Reference]; measured
Rust `checked_div`	`None`	`None`	measured
Go `int32`	run-time panic	-2^{31}, no panic	Go spec [Go Spec]; measured
Python `int` `//`	`ZeroDivisionError`	2^{31} (unbounded integers)	Python reference [Python Reference]; measured
JavaScript BigInt	`RangeError`	2^{31}	ECMA-262 [ECMA-262]; measured

Rust is the only system in the table that treats both boundaries the same way in both build modes; Go and Java treat them differently (an error for $x/0$, silent wraparound for $-2^{31}/-1$); x86 traps on both for reasons that go back to its double-width divide (Chapter 23).

24.2 Measured on x86-64

GIN-EXP-004 ran each toolchain on the session host with operands supplied at run time. The SDK's machine models agree with every behaviour that could be measured there (GIN-IMP-002). The experiment's raw records are part of the repository:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, pathlib, ginsdk
data = json.loads((pathlib.Path(ginsdk.__file__).parents[3] / "experiments/results/exp004.json").read_text())
["data"]
rows = [("C gcc -O0, int 1/0", data["c_gcc"]["-O0"]["int 1/0"]), ("C gcc -O0, int MIN/-1", data["c_gcc"]["-O0"]
```

```
["int INT_MIN/-1"]),
        ("Java, int 1/0", data["java"]["int 1/0"]), ("Java, int MIN/-1", data["java"]["int MIN/-1"]),
        ("Go, int32 1/0", data["go"]["int32 1/0"]), ("Go, int32 MIN/-1", data["go"]["int32 MIN/-1"]),
        ("Rust debug, 1/0", data["rust_debug"]["div 1/0"]), ("Rust debug, checked 1/0", data["rust_debug"]
["checked 1/0"])]
for label, r in rows:
    segments = [x.strip() for x in r["stderr"].split("|")]
    shown = r["signal"] or r["stdout"] or next(x for x in segments if "divide by zero" in x)
    print(f"{label:25s} exit {r['exit']:4d}  {shown[:60]}")
print(f"{'Python, 1/0':25s} {data['python']['1/0']}")
print(f"{'JavaScript, 1n/0n':25s} {data['javascript']['1n/0n']}")
agree = data["model_agreement"]
print("model agreement:", sum(agree.values()), "of", len(agree))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
C gcc -O0, int 1/0        exit   -8  SIGFPE
C gcc -O0, int MIN/-1     exit   -8  SIGFPE
Java, int 1/0             exit    0  ArithmeticException: / by zero
Java, int MIN/-1          exit    0  -2147483648
Go, int32 1/0             exit    2  panic: runtime error: integer divide by zero
Go, int32 MIN/-1          exit    0  -2147483648
Rust debug, 1/0           exit  101  attempt to divide by zero
Rust debug, checked 1/0   exit    0  None
Python, 1/0               ZeroDivisionError: division by zero
JavaScript, 1n/0n         RangeError: Division by zero
model agreement: 14 of 14
```

EMPIRICAL OBSERVATION 24.1 (Asymmetric treatment of the two boundaries)

EMPIRICAL OBSERVATION GIN-OBS-005

On the host, Go and Java return -2^{31} for `int32` $-2^{31}/-1$ without any signal, while raising an error for division by zero; Rust panics on both in debug and release builds; C on x86 receives `SIGFPE` for both (GIN-EXP-004).

24.3 Emulated: AArch64 and RISC-V

No AArch64 or RISC-V hardware was available. The first edition of the research ledger therefore marked these models “specification only” (GIN-OPEN-009). For this edition, GIN-EXP-014 compiled a freestanding probe (no C library; results written with a raw system call; the floating-point status register read directly) for both architectures and ran it under QEMU’s user-mode emulation. QEMU is an independent implementation of the two instruction sets, so agreement tests the SDK’s models against something other than their own source documents; it is not a measurement of silicon.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import json, pathlib, ginsdk
data = json.loads((pathlib.Path(ginsdk.__file__).parents[3] / "experiments/results/exp014.json").read_text())
["data"]
print("emulator:", data["versions"]["qemu-aarch64"])
for isa in ("aarch64", "riscv64"):
    r = data[isa]
    zero = [(x["a"], x["quotient"], x["remainder"]) for x in r["int"] if x["b"] == 0]
    mn = [x for x in r["int"] if x["a"] == -2 ** 31][0]
    print(f"{isa:8s} x/0 → (a, q, r): {zero}   MIN/-1 → {mn['quotient']}   model agreement
{r['model_agreement']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
emulator: qemu-aarch64 version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
aarch64  x/0 → (a, q, r): [(1, 0, 1), (0, 0, 0), (-7, 0, -7)]   MIN/-1 → -2147483648   model agreement 8/8
riscv64  x/0 → (a, q, r): [(1, -1, 1), (0, -1, 0), (-7, -1, -7)]   MIN/-1 → -2147483648   model agreement 8/8
```

IMPLEMENTATION RESULT 24.2 (ISA models agree with an emulator)

IMPLEMENTATION RESULT

GIN-IMP-008

The SDK's AArch64 and RISC-V division models agree with QEMU 8.2.2 on all probed cases (8 signed and 8 unsigned 32-bit divisions each, including $x/0$ and $-2^{31}/-1$). On AArch64, which has no remainder instruction, C's `%` compiles to `sdiv` followed by `msub` ($a - qb$ modulo 2^{32}); for a zero divisor it therefore yields the dividend, as on RISC-V, although by a different route.

24.4 Why some machines trap and others do not

Trapping and totalizing are both reasonable designs; they place the cost of division by zero in different places.

- **Trapping** (x86) guarantees that no program silently continues with a meaningless quotient. The cost is that every division is a potential control transfer, which complicates out-of-order execution, and that languages which want to *return* an error instead of crashing must test the divisor anyway.
- **Totalizing** (AArch64, RISC-V) keeps division an ordinary data operation with no side effects. The RISC-V specification says so explicitly: the chosen results are what a simple divider produces, and a language that needs an exception can insert a single branch before the division [RISC-V ISA Manual]. Chapter 19 proved that a test-free restoring divider

indeed returns all ones (GIN-PROP-023). The cost is moved to software, which must remember to test.

Languages then add their own layer. Java and Go always test (by hardware trap or software check) and raise a language-level error; Rust's debug and release builds both panic; Python never reaches the hardware for a zero divisor. C alone adds nothing, and leaves the behaviour undefined — which, as Chapter 23 showed, licenses the optimizer to assume the divisor is not zero.

EXERCISE 24.1

A C program computes `int q = a / b;` and is known to run correctly on x86-64 Linux, where a zero divisor crashes it. It is ported to AArch64. What can change, and what single change to the source makes the behaviour the same on both, and defined?

▶ Solution to Exercise 24.1

EXERCISE 24.2

Using the table and Implementation Implementation result 24.2, give the value of `a % 0` in C compiled for AArch64 and for RISC-V, and say why neither is a remainder in the sense of the division theorem.

▶ Solution to Exercise 24.2

SUMMARY

- Integer division has two boundaries: $x/0$ and $-2^{w-1}/-1$; systems treat them differently, and some treat the two differently from each other.
- On x86-64 the toolchains measured trap, raise, panic or (Python, BigInt) never overflow; the SDK's models agree with all 14 measurable behaviours.
- Under QEMU, AArch64 returns 0 and RISC-V -1 for $x/0$, as specified; the models agree 8/8. This is emulator evidence, not hardware.
- Trapping and totalizing are both reasonable; they move the cost of the boundary between hardware and software.

Chapter 25

Floating Point: Rounding, Special Data, and Flags

> *A floating-point number is an exact rational. What is inexact is the arithmetic: every operation is followed by one rounding.*

LEARNING OBJECTIVES

- Describe a binary floating-point format as a finite set of dyadic rationals plus special data, and decode a bit pattern exactly.
- State the IEEE 754 requirement of correct rounding and implement it as exact rational computation followed by a single rounding.
- Name the five exception flags and the operations that raise each.
- Prove which integers and which decimal fractions a binary format represents exactly.
- List the algebraic laws that floating-point arithmetic keeps and those it breaks, with counterexamples.
- Interpret the agreement of the SDK's simulator with hardware, and the differences in tininess detection between architectures.

25.1 Formats as finite sets of rationals

DEFINITION 25.1 (IEEE binary format) DEFINITION GIN-DEF-030

A binary format has a precision p (significand bits, including the implicit leading bit) and e exponent bits, with $e_{\max} = 2^{e-1} - 1$ and $e_{\min} = 1 - e_{\max}$. Its finite data are $\pm m \cdot 2^q$ with integers $0 \le m < 2^p$ and $q \ge e_{\min} - p + 1$ (subject to the range), together with $\pm\infty$ and NaN [IEEE 754-2019]. binary16: $p = 11$, $e = 5$; binary32: $p = 24$, $e = 8$; binary64: $p = 53$, $e = 11$.

Every finite floating-point number is therefore an **exact dyadic rational**: an integer times a power of two. Nothing about a floating-point number is approximate. The number stored for the literal `0.1` in binary64 is

$$\frac{3602879701896397}{36028797018963968} = 0.1000000000000000055511151231257827021181583404541015625,$$

exactly. What is approximate is the relation between the literal and the stored number — the literal was *rounded* when it was read — and every later arithmetic operation, which rounds again.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import ieee

for fmt in (ieee.BINARY16, ieee.BINARY32, ieee.BINARY64):
    x, flags = ieee.from_decimal("0.1", fmt)
    v = x.value()
    print(f"{fmt.name:9s} 0.1 is stored as {v.numerator}/{v.denominator}  ({sorted(flags)})  fields
{x.fields()}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
binary16  0.1 is stored as 819/8192  (['inexact'])  fields {'sign': '0', 'exponent': '01011', 'fraction':
'1001100110'}
binary32  0.1 is stored as 13421773/134217728  (['inexact'])  fields {'sign': '0', 'exponent': '01111011',
'fraction': '10011001100110011001101'}
binary64  0.1 is stored as 3602879701896397/36028797018963968  (['inexact'])  fields {'sign': '0', 'exponent':
'01111111011', 'fraction': '1001100110011001100110011001100110011001100110011010'}
```

Each stored value is a fraction whose denominator is a power of two; the more precise the format, the larger the denominator and the smaller the distance to $1/10$ — but the distance is never zero, because $1/10$ is not a dyadic rational (Chapter 7).

25.2 Correct rounding

DEFINITION 25.2 (Correct rounding) DEFINITION GIN-DEF-031

An operation is **correctly rounded** if its result is the exact real result rounded once to the format — in the default mode, to the nearest representable number, with ties broken to the one with even significand. IEEE 754 requires correct rounding of $+$, $-$, $\times$, $\div$, $\sqrt{}$ and fused multiply–add.

The definition is also an algorithm, and the SDK's reference simulator `ginsdk.ieee` implements it literally: decode both operands to exact rationals, compute the exact result with Python's `fractions`, and round once. That is slow and obviously correct, which is what a reference implementation should be. Its value lies in being checkable against hardware.

IMPLEMENTATION RESULT 25.3

(The IEEE simulator agrees with this CPU, results and flags) IMPLEMENTATION RESULT

GIN-IMP-001

On the session's x86-64 host (SSE scalar arithmetic, gcc 13.3, flags read with `fenv.h`), `ginsdk.ieee` reproduces the hardware's results bit for bit (NaNs compared as NaNs) and all five exception flags on 40,300 binary32 and 40,300 binary64 operations ($+, -, \times, \div, \sqrt{}$ on random bit patterns, special values, subnormals, and a targeted family near the underflow threshold) — after signaling NaNs were modelled (GIN-REJ-002).

The parenthesis records a correction. The first run of the experiment found 101 flag mismatches; every one involved a *signaling* NaN operand, on which hardware raises the invalid flag and the first version of the simulator did not. The simulator was fixed and the experiment re-run (GIN-EXP-005). The failed run is kept in the ledger: it is evidence that the experiment can find defects in the instrument.

25.3 Special data and flags

DEFINITION 25.4 (Exception flags) DEFINITION GIN-DEF-032

invalid: no useful real result ($0/0$, $\infty - \infty$, $0 \times \infty$, $\sqrt{\text{negative}}$, operations on signaling NaNs); **divideByZero**: an exact infinite result from finite operands ($x/0$, $x \neq 0$ finite); **overflow**: the rounded result exceeds the largest finite number; **underflow**: the result is tiny (below the smallest normal number) and inexact, under default handling; **inexact**: the rounded result differs from the exact one.

The flags are *sticky*: they accumulate until cleared, so a program can perform a long computation and then ask whether anything exceptional happened. Most high-level languages give no access to them; C exposes them through `fenv.h`. The measured behaviour on x86-64 for the divisions of this book:

OPERATION	RESULT	FLAGS
$1/0$	$+\infty$	divideByZero
$-1/0$, $1/(-0)$	$-\infty$	divideByZero

OPERATION	RESULT	FLAGS
$0/0$	NaN (printed `-nan` : the x86 default NaN has its sign bit set)	invalid
$0/1$	0	none
$1/3$	$0.333\ldots$	inexact
$10^{308}/10^{-10}$	$+\infty$	overflow, inexact
$10^{-310}/10^{10}$	a subnormal number	underflow, inexact
$\sqrt{-1}$	NaN	invalid

Every row was also produced by the simulator, and Java, Go, Rust, JavaScript and C return the same data. Python, alone among the languages measured, raises `ZeroDivisionError` for `1.0 / 0.0` : it checks the divisor before invoking the IEEE operation (GIN-NEG-010).

25.4 What a format can hold

PROPOSITION 25.5 (The least unrepresentable positive integer) CLASSICAL RESULT

GIN-PROP-030

In a binary format with precision p and $e_{\max} \geq p$, every integer $0, 1, \ldots, 2^p$ is representable and $2^p + 1$ is not.

Proof.

An integer $m < 2^p$ is $m \cdot 2^0$ with a significand below 2^p; $2^p = 2^{p-1} \cdot 2$. The binary numeral of $2^p + 1$ is $1\,0\cdots0\,1$ with $p + 1$ significant digits, more than p. ∎

So $2^{24} + 1 = 16777217$ is the first integer a binary32 number cannot hold, and $2^{53} + 1 = 9007199254740993$ the first for binary64 — and therefore for JavaScript's only number type. Above 2^{53}, a JavaScript number cannot count by ones: $2^{53} + 1 = 2^{53}$.

PROPOSITION 25.6 (Which decimals are binary floating-point numbers)

PROVED HERE GIN-PROP-031

A k-digit decimal fraction $d/10^k$ ($1 \le d < 10^k$) is exactly representable in a binary format with $p \ge k$ and sufficient exponent range if and only if $5^k \mid d$. Exactly $2^k - 1$ of the $10^k - 1$ such fractions are representable.

Proof.

In lowest terms the denominator of $d/10^k$ is $2^i 5^j$; a binary fraction needs $j = 0$, i.e. $5^k \mid d$. Then $d/10^k = m/2^k$ with $m = d/5^k < 2^k \le 2^p$. The multiples of 5^k below $10^k = 2^k 5^k$ number $2^k - 1$. ∎

Of the 99 two-digit decimals $0.01, \ldots, 0.99$, exactly three — 0.25, 0.5, 0.75 — are binary64 numbers. Of the 999 three-digit ones, seven. The decimal fractions people type are almost never the numbers a binary computer stores.

25.5 Laws that hold, laws that fail

Correct rounding guarantees that each single operation is as accurate as the format allows. It does not make floating-point arithmetic a field.

LAW	HOLDS?	REASON OR COUNTEREXAMPLE (BINARY64 UNLESS STATED)
$x \oplus y = y \oplus x$, $x \otimes y = y \otimes x$	yes (non-NaN)	rounding of a commutative exact result
$(x \oplus y) \oplus z = x \oplus (y \oplus z)$	no	$(0.1 + 0.1) + 0.4 = 0.6000000000000001 \neq 0.6$ (GIN-PROP-032)
$x \otimes (y \oplus z) = x \otimes y \oplus x \otimes z$	no	$0.1 \cdot (0.1 + 0.3) \neq 0.1 \cdot 0.1 + 0.1 \cdot 0.3$ (Chapter 14)
$x \oplus 0 = x$	yes, except $-0 \oplus +0 = +0$	signed zeros
$x \oplus 1 > x$	no	$2^{53} + 1 = 2^{53}$
$x = y \Rightarrow 1/x = 1/y$	no	$+0 = -0$ but $1/(+0) \neq 1/(-0)$
$x = x$	no	$\mathrm{NaN} \neq \mathrm{NaN}$

LAW	HOLDS?	REASON OR COUNTEREXAMPLE (BINARY64 UNLESS STATED)
$y \otimes (x \oslash y) = x$	no	fails at $y = 0$ and through rounding; see the census of Chapter 18

EMPIRICAL OBSERVATION 25.7 (Summation order changes a sum)

EMPIRICAL OBSERVATION GIN-OBS-015

The harmonic sum H_{20000} computed in binary32 is 10.480757 summing forward ($1 + \frac{1}{2} + \cdots$) and 10.480734 summing backward; the correctly rounded exact value is 10.480728. The backward sum, which adds small terms first, has an error about five times smaller (GIN-EXP-012).

25.6 Tininess: one standard, two readings

IEEE 754 allows an implementation to decide that a result is *tiny* — below the smallest normal number — either **before** or **after** rounding. The two readings differ only for results whose exact value lies just below the smallest normal number and rounds up to it: before-rounding detection calls them tiny (and raises underflow if inexact); after-rounding detection does not. The simulator implements both, which turns the question “which does this processor do?” into an experiment.

EMPIRICAL OBSERVATION 25.8 (Tininess detection on three architectures)

EMPIRICAL OBSERVATION GIN-OBS-009

On the x86-64 host, in a targeted family of 300 products near the threshold, the after-rounding model matches every hardware underflow flag and the before-rounding model mismatches 158 (binary32) and 159 (binary64): this processor detects tininess **after rounding** (GIN-EXP-005). Under QEMU emulation, a single discriminating product shows AArch64 detecting **before rounding** and RISC-V **after rounding** (GIN-OBS-018, GIN-EXP-014; emulator, not hardware).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from fractions import Fraction
from ginsdk import ieee
```

```
F = ieee.BINARY32
a = ieee.from_bits(F, 0x3F7FFFFE)        # 1 - 2^-23
b = ieee.from_bits(F, 0x00800001)        # 2^-126 (1 + 2^-23)
exact = a.value() * b.value()
print("exact product / smallest normal =", exact / F.min_normal, " (just below 1)")
for mode in ("after", "before"):
    r, flags = ieee.mul(a, b, tininess=mode)
    print(f"tininess {mode:6s}: result {hex(r.bits())}  flags {sorted(flags)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
exact product / smallest normal = 70368744177663/70368744177664  (just below 1)
tininess after : result 0x800000  flags ['inexact']
tininess before: result 0x800000  flags ['inexact', 'underflow']
```

The result bits are identical; only the flag differs. A program that tests the underflow flag will behave differently on x86-64 and on (emulated) AArch64 for exactly these inputs — a portability difference permitted by the standard itself.

EXERCISE 25.1

Decode the binary32 pattern `0x40490FDB` by hand into sign, exponent and significand, and give its exact value.

▶ Solution to Exercise 25.1

EXERCISE 25.2

What is the largest integer N such that every integer in $[-N, N]$ is exactly representable in binary16? What does `2049` round to?

▶ Solution to Exercise 25.2

EXERCISE 25.3

List the four-digit decimals in $(0, 1)$ that are binary64 numbers.

▶ Solution to Exercise 25.3

SUMMARY

- Floating-point numbers are exact dyadic rationals; inexactness lies in rounding, which happens at reading and after every operation.
- Correct rounding = exact computation + one rounding; the SDK's simulator does exactly that and matches x86-64 hardware on 80,600 operations, flags included.
- ±∞, NaN and −0 are special data; five sticky flags record invalid, divideByZero, overflow, underflow and inexact events.
- binary64 holds every integer up to 2^{53} and only $2^k - 1$ of the k-digit decimals.
- Commutativity survives; associativity, distributivity, reflexivity of equality and the converse reading of division do not.
- Tininess detection differs: after rounding on x86-64, before rounding on emulated AArch64.

Chapter 26

Overflow, Checked Arithmetic, and Arbitrary Precision

> *Every fixed-width number system has an edge. The only choices are what happens there, and whether anyone is told.*

LEARNING OBJECTIVES

- Tabulate the result of MAX + 1 under each overflow policy at widths 8, 16, 32 and 64.
- Use checked, wrapping, saturating and overflowing operations deliberately, and know which languages provide them.
- Explain the cost model of arbitrary-precision integers, and why a runtime may refuse an operation that is mathematically trivial.
- Choose an exact representation for decimal quantities such as money.

26.1 The edge of a fixed width

An overflow is not a failure of arithmetic. The exact result exists and is unique; it is merely not an element of the set of representable values. In the four-layer model it is an *execution-layer* failure with a typing flavour: the operation is admissible, the result is not in the domain of the encoding. Each system decides what to do at the edge.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, pathlib, ginsdk
rows = json.loads((pathlib.Path(ginsdk.__file__).parents[3] / "experiments/results/exp012.json").read_text())
["data"]["max_plus_one"]
print(f"{'width':>5s} {'MAX':>20s} {'wrap':>21s} {'saturate':>20s}  C           checked")
for r in rows:
    print(f"{r['width']:>5d} {r['MAX']:>20d} {r['wrap']:>21d} {r['saturate']:>20d}  {r['C']:11s}
{r['checked']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

width	MAX	wrap	saturate	C	checked
8	127	-128	127	undefined	exception
16	32767	-32768	32767	undefined	exception
32	2147483647	-2147483648	2147483647	undefined	exception
64	9223372036854775807	-9223372036854775808	9223372036854775807	undefined	exception

Wraparound sends MAX + 1 to MIN, the step across the seam of the cycle of Chapter 3 (GIN-PROP-052). Saturation clamps. Checked arithmetic refuses. ISO C declares signed overflow undefined — with the consequences for optimization described in Chapter 23 — while unsigned overflow in C is defined to wrap.

26.2 Choosing the policy explicitly

Because no single policy is right for every program, several languages let the programmer choose per operation.

LANGUAGE	HOW EACH POLICY IS REQUESTED
Rust	wrap: `wrapping_add` ; saturate: `saturating_add` ; check: `checked_add` → `Option` ; detect: `overflowing_add` → (value, flag)
C (GCC, Clang extensions)	wrap: unsigned types; detect: `__builtin_add_overflow`
C23	wrap: unsigned types; detect: `ckd_add` in `<stdckdint.h>`
Java	wrap: default for `int` ; check: `Math.addExact` throws
Go	wrap: default; detect: `math/bits.Add64` returns the carry
Swift	wrap: `&+` ; check: default `+` traps; detect: `addingReportingOverflow`

Rust's design is the most explicit: the default `+` panics on overflow in debug builds and wraps in release builds — so the *same source* has two meanings depending on a compiler flag — and the four named methods make the intended policy part of the program text. In the vocabulary of this book, the methods name the repair: `wrapping_add` is the *quotient* repair, `saturating_add` a *totalization*, `checked_add` *absorption* into `Option` , and the debug-mode panic a *signal*.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import machine as M
```

```
for policy in ("wrap", "saturate", "trap", "c-undefined"):
    o = M.add_fixed(2 ** 31 - 1, 1, 32, policy)
    print(f"{policy:12s} {o.kind:10s} {o.value if o.value is not None else '':>12}  {o.name or o.note}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
wrap         value       -2147483648  result reduced mod 2^w (overflow flag set on most ISAs)
saturate     value        2147483647  clamped to the representable range
trap         exception                overflow
c-undefined  undefined                undefined behaviour
```

26.3 Arbitrary precision

Arbitrary-precision (“big”) integers move the edge from the width of a register to the size of memory. Python’s `int`, Java’s `BigInteger`, JavaScript’s `BigInt` and libraries such as GMP represent an integer as a sequence of machine words (“limbs”) and implement the algorithms of Parts V–VIII on them. Overflow disappears; cost appears. Every operation now costs time proportional to a function of the operands’ lengths: linear for addition, quadratic or sub-quadratic for multiplication and division (Chapter 37 measures it). A loop that computes 2^n by repeated doubling takes quadratic time in n, because the numbers grow.

Big integers also expose costs that fixed-width code never meets. Converting an integer to its decimal numeral is a change of base (Chapter 5), and with the schoolbook method it costs time quadratic in the number of digits. In 2022 this was recognized as a denial-of-service risk — a web service that parsed or printed numbers with millions of digits could be made to spend minutes on one request — and CPython 3.11 introduced a default limit on the number of decimal digits converted (CVE-2020-10735 [CVE-2020-10735]).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import sys
print("default limit on int ⟷ decimal string conversion:", sys.get_int_max_str_digits(), "digits")
try:
    int("9" * 5000)
except ValueError as e:
    print("int('9' * 5000) →", str(e)[:70] + "…")
print("binary is unaffected:", len(bin(10 ** 5000)) - 2, "bits written without complaint")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
default limit on int ⟷ decimal string conversion: 4300 digits
int('9' * 5000) → Exceeds the limit (4300 digits) for integer string conversion: value h…
binary is unaffected: 16610 bits written without complaint
```

EMPIRICAL OBSERVATION 26.1

(A cost boundary enforced as an admissibility boundary) EMPIRICAL OBSERVATION

GIN-OBS-014

CPython ≥ 3.11 refuses int ↔ decimal-string conversion beyond 4,300 digits by default, because the conversion is quadratic and was a denial-of-service vector (CVE-2020-10735). Conversion to binary (a power-of-two base) is linear and is not limited. A runtime here treats a *cost* boundary as an *admissibility* boundary: the operation is well defined and refused anyway.

26.4 Exact decimals

Binary floating point cannot hold most decimal fractions (GIN-PROP-031). Where decimal quantities must be exact — money is the standard example — three exact representations are common: integers counting the smallest unit (cents); rationals; and *decimal* floating point, standardized in IEEE 754 since 2008, in which the significand is a decimal integer and 0.1 is exactly representable.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from decimal import Decimal
from fractions import Fraction

print("binary64 :", 0.1 + 0.2 == 0.3, "  ", 0.1 + 0.2)
print("decimal  :", Decimal("0.1") + Decimal("0.2") == Decimal("0.3"), "   ", Decimal("0.1") + Decimal("0.2"))
print("rational :", Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10), "   ", Fraction(1, 10) + Fraction(2,
10))
print("cents    :", 10 + 20 == 30)
print("but decimal is still finite:", Decimal(1) / Decimal(3), " * 3 =", Decimal(1) / Decimal(3) * 3)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
binary64 : False    0.30000000000000004
decimal  : True     0.3
rational : True     3/10
cents    : True
but decimal is still finite: 0.33333333333333333333333333333  * 3 = 0.99999999999999999999999999
```

The last line is the decimal version of Chapter 7's theorem: 1/3 has no finite numeral in base 10, so a decimal format must round it. Changing the base moves the set of exactly representable

fractions; it does not make every fraction representable. Only an exact rational representation does, at the cost of unbounded growth.

EXERCISE 26.1

A 32-bit signed counter starts at 0 and is incremented once per millisecond. After how long does it overflow, and what value does it hold afterwards under wraparound?

▶ Solution to Exercise 26.1

EXERCISE 26.2

For Rust `i8`, give the result of `100i8.wrapping_add(100)`, `100i8.saturating_add(100)`, `100i8.checked_add(100)` and `100i8.overflowing_add(100)`.

▶ Solution to Exercise 26.2

EXERCISE 26.3

Show that no base b represents every rational exactly with a finite numeral, and identify which rationals base 30 does represent.

▶ Solution to Exercise 26.3

SUMMARY

- Overflow leaves an admissible result outside the encoding; systems wrap, saturate, signal or (C, signed) leave the behaviour undefined.
- Rust, C23 and others let the program name the policy per operation; each named method is one of the repairs of Chapter 9.
- Arbitrary precision trades overflow for cost; CPython limits decimal conversion because its cost was exploitable.
- Decimal formats represent decimal fractions exactly and still round $1/3$; only rationals are closed under division.

PART XI

Number Systems as Computational Domains

$\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Z}/n$, finite fields and p-adic integers: constructions, closure, inverses, order, representation and cost.

Chapter 27

$\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$: Constructions, Order, and Representation

> *$\mathbb{N}$ counts, $\mathbb{Z}$ answers every subtraction, $\mathbb{Q}$ answers every division but one. Each answer is paid for in representation.*

LEARNING OBJECTIVES

- Describe $\mathbb{N}$, $\mathbb{Z}$ and $\mathbb{Q}$ by construction, operations, closure, identities, inverses and order.
- Distinguish the properties that follow from the structure from those that come from a chosen representation.
- Decide equality in each domain and state what the canonical form costs.
- Enumerate the positive rationals without repetition, and connect the enumeration to Chapter 6.

27.1 Three domains at a glance

	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{Q}$
construction	successor algebra (Chapter 3)	pairs $(a, b) \in \mathbb{N}^2$ mod $a + d = b + c$ (Chapter 12)	pairs $(a, b) \in \mathbb{Z} \times (\mathbb{Z} \setminus 0)$ mod $ad = bc$ (Chapter 9)
algebraic structure	commutative semiring	commutative ring, integral domain	field
total operations	$+, \times$	$+, -, \times$	$+, -, \times$
admissibility of $a - b$	$a \geq b$	always	always

	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{Q}$
admissibility of a/b	$b \neq 0$ and $b \mid a$	$b \neq 0$ and $b \mid a$	$b \neq 0$
units (invertible elements)	$\{1\}$	$\{1, -1\}$	$\mathbb{Q} \setminus \{0\}$
order	well-order	discrete total order compatible with $+$	dense total order, Archimedean
canonical representation	positional numeral	sign and magnitude numeral	reduced fraction p/q, $q > 0$, $\gcd(p, q) = 1$
deciding equality	compare numerals	compare signs and magnitudes	compare reduced forms, or $ad = bc$

The table is classical; its arrangement follows the contract format of Chapter 8, and the SDK's contract data (`python3 -m ginsdk contracts /`) records the same admissibility conditions per domain.

27.2 Order

$\mathbb{N}$ **is well-ordered**: every non-empty set of natural numbers has a least element. This is equivalent to induction, and it is why algorithms that decrease a natural number at each step — Euclid's algorithm, the residue algorithm, long division — terminate. $\mathbb{Z}$ **is not well-ordered** ($\{-1, -2, \ldots\}$ has no least element), which is why termination arguments over integers measure an absolute value or some other natural number. $\mathbb{Q}$ **is dense**: between any two rationals lies another, $(a + b)/2$. There is no "next rational", and therefore no successor grammar for $\mathbb{Q}$ in its usual order — a fact that makes the enumeration below surprising.

The order of $\mathbb{Z}$ is *compatible* with its ring operations: $a \leq b \Rightarrow a + c \leq b + c$, and $a \leq b, c \geq 0 \Rightarrow ac \leq bc$. This compatibility is lost in $\mathbb{Z}/n$ (Chapter 29), where no order respects addition: $n - 1 + 1 = 0$.

27.3 Representation and equality

A domain's *values* are independent of how they are written, but every computation manipulates *representations*, and the cost of deciding equality depends on whether representations are

canonical.

- In $\mathbb{N}$ and $\mathbb{Z}$, positional numerals without leading zeros (and with a sign) are canonical: two numbers are equal iff their numerals are identical — a linear-time comparison.
- In $\mathbb{Q}$, the pair (p, q) is not canonical: $(2, 4)$ and $(1, 2)$ are the same rational. Either every result is reduced by a gcd — the strategy of Python's `fractions` — or equality is decided by cross-multiplication, $ad = bc$, at the cost of a multiplication per comparison and of numerators and denominators that grow without bound.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction
from ginsdk import bigint
from ginsdk.cost import Cost
import random

random.seed(27)
x = Fraction(0)
for k in range(1, 61):
    x += Fraction(random.randint(-9, 9), random.randint(1, 30))
    if k % 15 == 0:
        print(f"after {k:2d} additions: numerator {x.numerator.bit_length():3d} bits, denominator
{x.denominator.bit_length():3d} bits")
c = Cost()
bigint.gcd_euclid(bigint.from_int(abs(x.numerator)) or [1], bigint.from_int(x.denominator), c)
print("limb operations for one gcd at the end:", bigint.word_ops(c))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
after 15 additions: numerator  21 bits, denominator  21 bits
after 30 additions: numerator  23 bits, denominator  20 bits
after 45 additions: numerator  32 bits, denominator  28 bits
after 60 additions: numerator  35 bits, denominator  33 bits
limb operations for one gcd at the end: 45
```

The reduced denominator of a sum of fractions with denominators at most 30 divides $\operatorname{lcm}(1, \ldots, 30)$, a 42-bit number, so in this run the denominator can never exceed 42 bits; after sixty additions it has reached 33. The numerator has no such bound: it grows with the magnitude of the sum. With unbounded denominators the growth continues without limit. Exact rational arithmetic is total and exact; its price is that the size of the data, and therefore the cost of each operation, depends on the history of the computation.

27.4 Enumerating the rationals

$\mathbb{Q}$ is dense, so no rational has a successor in the usual order. Yet $\mathbb{Q}$ is *countable*: its elements can be listed in a sequence. A classical listing uses pairs and diagonals, and repeats every rational infinitely often ($1/2 = 2/4 = \cdots$). A more elegant one, due to Calkin and Wilf [Calkin & Wilf 2000], lists each positive rational exactly once, in lowest terms:

PROPOSITION 27.1 (Every positive rational, once: the Calkin–Wilf sequence)

CLASSICAL RESULT GIN-PROP-068

Let s be Stern's diatomic sequence ($s(0) = 0$, $s(1) = 1$, $s(2k) = s(k)$, $s(2k+1) = s(k) + s(k+1)$). Then $n \mapsto s(n)/s(n+1)$, for $n \geq 1$, is a bijection from the positive integers to the positive rationals, and every fraction $s(n)/s(n+1)$ is in lowest terms [Calkin & Wilf 2000].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction

s = [0, 1]
for k in range(2, 4002):
    s.append(s[k // 2] if k % 2 == 0 else s[k // 2] + s[k // 2 + 1])
seq = [Fraction(s[n], s[n + 1]) for n in range(1, 4001)]
print("first terms:", ", ".join(f"{s[n]}/{s[n+1]}" for n in range(1, 16)))
print("4000 terms, distinct:", len(set(seq)) == len(seq),
      "  all in lowest terms:", all(__import__("math").gcd(s[n], s[n + 1]) == 1 for n in range(1, 4001)))
small = {Fraction(p, q) for p in range(1, 13) for q in range(1, 13)}
print("every p/q with p, q ⩽ 12 appears among the first 4000:", small ⩽ set(seq))
print("missing:", sorted(map(str, small - set(seq))))

def cf_sum(p, q):                       # sum of continued-fraction partial quotients of p/q
    total = 0
    while q:
        total, (p, q) = total + p // q, (q, p % q)
    return total
print("bit length of n equals that sum for all n < 4001:",
      all(n.bit_length() == cf_sum(s[n], s[n + 1]) for n in range(1, 4001)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
first terms: 1/1, 1/2, 2/1, 1/3, 3/2, 2/3, 3/1, 1/4, 4/3, 3/5, 5/2, 2/5, 5/3, 3/4, 4/1
4000 terms, distinct: True   all in lowest terms: True
every p/q with p, q ⩽ 12 appears among the first 4000: False
missing: ['11/12', '12']
bit length of n equals that sum for all n < 4001: True
```

The negative line is informative: the enumeration is complete in the limit but not uniformly fast. Of the 91 distinct fractions p/q with $p, q \le 12$, two are missing from the first 4,000 terms, namely $11/12$ and 12; they occur at positions 4094 and 4095. The last line explains why: the position n of p/q has as many binary digits as the sum of the partial quotients of the continued fraction of p/q — its depth in the Calkin–Wilf tree, plus one. $12 = [12]$ and $11/12 = [0; 1, 11]$ both have sum 12, while $8/13 = [0; 1, 1, 1, 1, 2]$, with sum 6, already appears at position 42. Stern's sequence — which counted hyperbinary representations in Chapter 6 (GIN-PROP-054) — thus also gives the rationals a *successor grammar of its own*: the move $x \mapsto 1/(2\lfloor x \rfloor + 1 - x)$ (Newman's formula) takes each term of the Calkin–Wilf sequence to the next. Density in one order and a successor structure in another are compatible; they are different grammars on the same set.

EXERCISE 27.1

Prove that the only units of $\mathbb{Z}$ are ± 1, and that every non-zero rational is a unit of $\mathbb{Q}$. What are the units of $\mathbb{Z}[1/2]$, the rationals with power-of-two denominators?

▶ Solution to Exercise 27.1

EXERCISE 27.2

Verify Newman's formula $x_{n+1} = 1/(2\lfloor x_n \rfloor + 1 - x_n)$ for the first eight Calkin–Wilf terms.

▶ Solution to Exercise 27.2

EXERCISE 27.3

Show that $\mathbb{Z}$ with its usual order is not dense and $\mathbb{Q}$ is, and give a property of $\mathbb{Q}$'s order that $\mathbb{R}$'s has and $\mathbb{Q}$'s lacks.

▶ Solution to Exercise 27.3

SUMMARY

- $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ are a semiring, an integral domain and a field; each adds solutions to a family of converse problems.
- $\mathbb{N}$ is well-ordered (termination), $\mathbb{Z}$ discretely ordered, $\mathbb{Q}$ densely ordered.
- Canonical forms make equality cheap; reduced fractions require gcds, and rational data grow with the computation.
- The positive rationals have a successor grammar after all: the Calkin–Wilf sequence lists each exactly once.

Chapter 28

ℝ and ℂ: Completeness, Approximation, and Branches

> *ℚ has a numeral for every element. ℝ has a numeral for almost none of them.*

LEARNING OBJECTIVES

- Explain why $\mathbb{Q}$ is incomplete and how $\mathbb{R}$ repairs it, and state the cost of that repair for representation.
- Prove that $\mathbb{R}$ is uncountable, and conclude that almost every real number has no finite description.
- Distinguish the questions a program can decide about a computable real from those it cannot, and say how interval arithmetic and floating point respond.
- Show that $\mathbb{C}$ admits no order compatible with its field operations, and that the principal square root is a *selection* that breaks familiar laws.
- Compare naive and scaled complex division in binary64.

28.1 The gap in the rationals

Chapter 27 ended with $\mathbb{Q}$: a field, densely ordered, every element written by a pair of numerals. Yet the converse problem $c \cdot c = 2$ has no solution in $\mathbb{Q}$.

PROPOSITION 28.1 ($\sqrt{2}$ is irrational) CLASSICAL RESULT GIN-HIST-007

There is no rational c with $c^2 = 2$.

Proof.

Suppose $c = p/q$ in lowest terms with $p^2 = 2q^2$. Then p^2 is even, so p is even, $p = 2r$, and $4r^2 = 2q^2$ gives $q^2 = 2r^2$, so q is even too — contradicting lowest terms. ∎

The SDK reports this at the semantic layer: $\sqrt{2}$ is *well formed* and *well typed* in $\mathbb{Q}$, and the converse problem it names has *no solution* there. In $\mathbb{R}$ the problem has two solutions, and $\sqrt{\cdot}$ selects the

non-negative one.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import converse, expr
print(converse.square_root(2, "Q").description, "   ", converse.square_root(2, "Q").reason)
print(converse.square_root(2, "R").description, "   ", converse.square_root(2, "R").reason)
o = expr.evaluate("sqrt(2)", "R")
print(o.status, o.display, o.notes[0])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
∅     2 is not the square of a rational (its root is irrational)
c ∈ {√2, -√2}     two real square roots; the principal branch √ selects the nonnegative one
value ≈1.41421356237309504880168 √2 is irrational; the value is a 40-digit approximation of the principal root
of c · c = 2
```

The deeper defect is not one missing root but a missing *kind of limit*. The set $\{x \in \mathbb{Q} : x^2 < 2\}$ is non-empty and bounded above, and it has no least upper bound in $\mathbb{Q}$. A field with an order in which every non-empty bounded set has a least upper bound is called **complete**, and the classical theorem is that there is exactly one complete ordered field up to isomorphism: $\mathbb{R}$.

DEFINITION 28.2 (Complete ordered field) CLASSICAL RESULT GIN-DEF-092

An ordered field K is *complete* if every non-empty subset of K that has an upper bound has a least upper bound in K. Any two complete ordered fields are isomorphic by a unique order-preserving field isomorphism; their common type is $\mathbb{R}$.

Two constructions realize it. **Dedekind cuts** [Dedekind 1888] define a real number as a downward-closed set of rationals with no greatest element, bounded above — in effect, a real number *is* the set of rationals below it. **Cauchy sequences** define a real number as an equivalence class of sequences of rationals whose terms get arbitrarily close to each other, two sequences being equivalent when their difference tends to zero. Both constructions describe an element of $\mathbb{R}$ by an *infinite* object built from rationals. That is the first sign of what follows: completeness is bought with infinity, and a finite machine can hold only a finite part of it.

28.2 Almost no real number can be written down

THEOREM 28.3 (The reals are uncountable) CLASSICAL RESULT GIN-HIST-008

There is no sequence $x_1, x_2, \ldots$ that contains every real number in $[0, 1)$.

Proof.

Write each x_n in decimal. Define $y = 0.d_1 d_2 d_3 \ldots$ by $d_n = 2$ if the n-th decimal digit of x_n is 1, and $d_n = 1$ otherwise. Because y uses only the digits 1 and 2, it has a unique decimal expansion (no tail of 0s or 9s), so $y \neq x_n$ for every n: they differ at the n-th digit, so it is in no position of the sequence. ∎

This is Cantor's diagonal argument; Cantor's own 1891 paper uses sequences of two symbols rather than digit expansions of reals [Cantor 1891], and the decimal version above avoids the digits 0 and 9 because $0.0999\ldots = 0.1000\ldots$ would otherwise let two expansions name the same number. The construction is itself an algorithm: given any finite list of digit strings, it produces one that differs from all of them.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import random
random.seed(28)
listed = ["".join(random.choice("01") for _ in range(12)) for _ in range(12)]
y = "".join("1" if listed[n][n] == "0" else "0" for n in range(12))
for n, x in enumerate(listed[:5]):
    print(f"x_{n + 1:<2d} {x}   differs from y at position {n + 1}: {x[n]} vs {y[n]}")
print("...")
print("y    ", y, "  in the list:", y in listed)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x_1  000001100010   differs from y at position 1: 0 vs 1
x_2  000010001100   differs from y at position 2: 0 vs 1
x_3  101000110101   differs from y at position 3: 1 vs 0
x_4  001001110111   differs from y at position 4: 0 vs 1
x_5  000101000011   differs from y at position 5: 0 vs 1
...
y     110111010000   in the list: False
```

The consequence for computation is decisive. Every finite description — a numeral, a formula, a program — is a finite string over a finite alphabet, and finite strings can be listed in a sequence (by length, then alphabetically). So the set of real numbers that have *any* finite description is

countable, and by the theorem it is a countable subset of an uncountable set: in the sense of cardinality, almost no real number can be named.

28.3 Computable reals

Turing's 1936 paper, which introduced the machines now named after him, was about real numbers: its title is *On computable numbers* [Turing 1936]. A real x is **computable** if a program, given n, outputs a rational within 2^{-n} of x. $\sqrt{2}$, π and e are computable; because programs are finite strings, the computable reals are countable, and the diagonal argument applied to the list of all programs produces a real that no program computes.

Computable reals are closed under the field operations and under many functions — but not every question about them can be answered.

PROPOSITION 28.4 (Equality of computable reals is not decidable) CLASSICAL RESULT

GIN-PROP-069

There is no algorithm that, given two programs computing real numbers x and y in the sense above, always halts and correctly reports whether $x = y$. Inequality $x \neq y$ is semi-decidable: if it holds, computing both to increasing precision eventually separates them.

Proof.

Given a program P, define x_P by: the n-th approximation is 0 if P has not halted within n steps, and 2^{-k} if P halted at step $k \leq n$. This is a computable real (the approximations are within 2^{-n} of the limit), and $x_P = 0$ if and only if P never halts. A decision procedure for $x_P = 0$ would decide the halting problem [Turing 1936]. For the second claim: if $|x - y| > 0$, approximations to precision 2^{-n} with $2^{-n+1} < |x - y|$ certify the difference. ∎

In the four-layer vocabulary, the semantic question "is this expression equal to zero?" has a definite answer for every real, and the execution layer cannot always deliver it. The SDK's $\mathbb{R}$ domain makes the gap concrete: it is exact on rationals and carries irrational square roots as flagged 40-digit approximations.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import expr
for src in ("sqrt(2) * sqrt(2) - 2", "(sqrt(2) + 1) * (sqrt(2) - 1) - 1", "4 * 4 - 16"):
    o = expr.evaluate(src, "R")
    print(f"{src:36s} {o.display:30s} flags {sorted(o.flags)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
sqrt(2) * sqrt(2) - 2              ≈1e-39                    flags ['approximate']
(sqrt(2) + 1) * (sqrt(2) - 1) - 1  ≈1e-39                    flags ['approximate']
4 * 4 - 16                         0                         flags []
```

Each of the first two expressions denotes exactly 0. The domain cannot say so: it returns a tiny number flagged *approximate*, and no finite amount of further precision would turn "smaller than 10^{-39}" into "equal to 0". The third expression involves only rationals, and the domain answers exactly. The repair a computer algebra system applies here is to *change the representation*: treat $\sqrt{2}$ as a symbol with the rewriting rule $\sqrt{2}^2 \to 2$, in which equality of these expressions becomes decidable — at the cost of restricting which reals can be expressed.

28.4 Approximating: intervals and floating point

Two strategies represent a real number by finite data.

Floating point (Chapter 25) stores one nearby representable number and discards the error. The result of a computation is a number; how far it is from the real answer is not part of the result.

Interval arithmetic [Moore 1966] stores two representable numbers that enclose the real one, and defines each operation so that the output interval contains every possible result. Bisection produces such an enclosure for $\sqrt{2}$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction as F

lo, hi = F(1), F(2)
for k in range(1, 31):
    mid = (lo + hi) / 2
    if mid * mid ⩽ 2:
        lo = mid
    else:
        hi = mid
    if k in (5, 10, 20, 30):
        print(f"step {k:2d}: [{float(lo):.12f}, {float(hi):.12f}]  width 2^-{k}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
step  5: [1.406250000000, 1.437500000000]  width 2^-5
step 10: [1.414062500000, 1.415039062500]  width 2^-10
step 20: [1.414213180542, 1.414214134216]  width 2^-20
step 30: [1.414213561453, 1.414213562384]  width 2^-30
```

An interval result is *honest* — it never claims more than it knows — but it may claim less than is true. Interval operations treat each occurrence of a variable independently, so expressions that are equal as functions can produce different intervals:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction as F

def isub(a, b):
    return (a[0] - b[1], a[1] - b[0])

def imul(a, b):
    p = [a[0] * b[0], a[0] * b[1], a[1] * b[0], a[1] * b[1]]
    return (min(p), max(p))

x = (F(1), F(2))
show = lambda i: f"[{i[0]}, {i[1]}]"
print("x       =", show(x))
print("x - x   =", show(isub(x, x)), "   (true range: [0, 0])")
print("x*x - x =", show(isub(imul(x, x), x)), "   (true range of x² - x on [1, 2]: [0, 2])")
print("x*(x-1) =", show(imul(x, isub(x, (F(1), F(1))))), "   (same function, rewritten)")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x       = [1, 2]
x - x   = [-1, 1]    (true range: [0, 0])
x*x - x = [-1, 3]    (true range of x² - x on [1, 2]: [0, 2])
x*(x-1) = [0, 2]    (same function, rewritten)
```

This *dependency problem* is the interval version of a theme of this book: an arithmetic law ($x - x = 0$, or distributivity) holds in the mathematical domain and fails in the representation. Here the failure is conservative — the enclosure is too wide, never wrong — whereas the floating-point failures of Chapter 25 return a confident wrong value.

28.5 The complex numbers

The converse problem $c \cdot c = -1$ has no real solution. The complex numbers adjoin one, and with it every polynomial equation acquires a root: by the fundamental theorem of algebra, $\mathbb{C}$ is *algebraically closed*. As a construction, $\mathbb{C}$ is $\mathbb{R}^2$ with

$$(a, b) + (c, d) = (a + c, b + d), \qquad (a, b)(c, d) = (ac - bd, ad + bc),$$

and $i = (0, 1)$ satisfies $i^2 = (-1, 0)$. Division by non-zero (c, d) is

$$\frac{a+bi}{c+di} = \frac{(ac+bd)+(bc-ad)i}{c^2+d^2},$$

and $\mathbb{C}$ is a field: 0 is the only inadmissible divisor.

The extension has two prices.

PROPOSITION 28.5 ($\mathbb{C}$ has no order compatible with its field operations)

CLASSICAL RESULT GIN-PROP-070

There is no total order $\leq$ on $\mathbb{C}$ such that $a \leq b \Rightarrow a+c \leq b+c$ and $0 \leq a, 0 \leq b \Rightarrow 0 \leq ab$.

Proof.

In such an order every square is ≥ 0: if $x \geq 0$ then $x \cdot x \geq 0$ by the product condition; if $x \leq 0$ then adding $-x$ to both sides gives $0 \leq -x$, and $x^2 = (-x)(-x) \geq 0$. Hence $1 = 1^2 \geq 0$ and $-1 = i^2 \geq 0$. Adding -1 to both sides of $0 \leq 1$ gives $-1 \leq 0$. Together with $-1 \geq 0$ this forces $-1 = 0$, which is false in a field. ∎

The second price is single-valuedness. Every non-zero complex number has exactly two square roots, and n distinct n-th roots. The function $\sqrt{\cdot}$ on $\mathbb{C}$ must *select* one — the *principal branch*, with real part ≥ 0 — and it is the selection repair of Chapter 9 applied to a non-unique converse problem. The SDK's $\mathbb{C}$ domain (exact Gaussian rationals) reports the solution set and the selection:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import converse, expr
s = converse.square_root(-1, "C")
print(s.equation, "→", s.description, "|", s.reason)
for src in ("sqrt(-1)", "sqrt(-4)", "sqrt(-2)", "1/0"):
    o = expr.evaluate(src, "C")
    print(f"{src:9s} {o.status:12s} {o.display:4s} {(o.notes or [o.reason])[0]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
c · c = -1 → c ∈ {1i, -1i} | every nonzero complex number has exactly two square roots
sqrt(-1)  value        i    √ is the principal branch: it selects i from c ∈ {1i, -1i}
sqrt(-4)  value        2i   √ is the principal branch: it selects 2i from c ∈ {2i, -2i}
sqrt(-2)  unsupported       √2 is irrational; this exact domain has no representation for it
1/0       no-solution       0 · c = 0 ≠ 1 for every complex c: C is a field, so 0 has no inverse
```

$\sqrt{-2}$ is `unsupported` in this domain: the solutions $\pm\sqrt{2}\,i$ exist in $\mathbb{C}$ but are not Gaussian rationals, so the *representation*, not the mathematics, refuses.

A selection can be consistent with some laws and not others. For non-negative reals, $\sqrt{xy} = \sqrt{x}\sqrt{y}$. For the principal complex square root it fails:

COUNTEREXAMPLE 28.6 (The principal square root is multiplicative) COUNTEREXAMPLE

GIN-NEG-022

$\sqrt{-1}\cdot\sqrt{-1} = i\cdot i = -1$, but $\sqrt{(-1)(-1)} = \sqrt{1} = 1$. The selection of one root from each two-element solution set cannot be made compatible with multiplication on all of $\mathbb{C}$.

Floating-point complex arithmetic adds a further, subtle selection. The principal square root is discontinuous across the negative real axis (its *branch cut*): just above the axis $\sqrt{-1+\varepsilon i} \approx i$, just below it $\approx -i$. IEEE 754's signed zero lets a library tell the two sides apart *on* the cut — the argument Kahan made for the sign of zero [Kahan 1987]:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import cmath
print("sqrt(-1 + 0i)  =", cmath.sqrt(complex(-1, 0.0)))
print("sqrt(-1 - 0i)  =", cmath.sqrt(complex(-1, -0.0)))
print("sqrt(-1)*sqrt(-1) =", cmath.sqrt(-1) * cmath.sqrt(-1), "   sqrt((-1)*(-1)) =", cmath.sqrt((-1) * (-1)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
sqrt(-1 + 0i)  = 1j
sqrt(-1 - 0i)  = -1j
sqrt(-1)*sqrt(-1) = (-1+0j)    sqrt((-1)*(-1)) = (1+0j)
```

Here -0.0, which compares equal to 0.0, selects a different square root. The zero's sign is information about *which side of the cut* a computation approached from — a datum, not a number in the ordinary sense, in exactly the way Chapter 25 described.

28.6 Complex division in floating point

The textbook formula for $(a+bi)/(c+di)$ forms c^2+d^2. In binary64 that intermediate overflows when $|c|$ or $|d|$ exceeds about 10^{154}, and underflows to zero when both are below about 10^{-162}, even when the quotient itself is an ordinary number. Smith's 1962 algorithm [Smith 1962] divides through by the larger of $|c|, |d|$ first and never forms c^2+d^2.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
def naive(a, b, c, d):
    den = c * c + d * d
    return complex((a * c + b * d) / den, (b * c - a * d) / den)

def smith(a, b, c, d):
    if abs(c) ⩾ abs(d):
        r = d / c
        den = c + d * r
        return complex((a + b * r) / den, (b - a * r) / den)
    r = c / d
    den = c * r + d
    return complex((a * r + b) / den, (b * r - a) / den)

for x in (1e300, 1.0, 1e-300):
    try:
        n = naive(x, x, x, x)
    except ZeroDivisionError as e:
        n = f"ZeroDivisionError ({e})"
    print(f"(x + xi)/(x + xi), x = {x:<7g}: naive {str(n):38s} Smith {smith(x, x, x, x)}   Python {complex(x,
x) / complex(x, x)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
(x + xi)/(x + xi), x = 1e+300 : naive (nan+nanj)                              Smith (1+0j)   Python (1+0j)
(x + xi)/(x + xi), x = 1      : naive (1+0j)                                  Smith (1+0j)   Python (1+0j)
(x + xi)/(x + xi), x = 1e-300 : naive ZeroDivisionError (float division by zero) Smith (1+0j)   Python (1+0j)
```

EMPIRICAL OBSERVATION 28.7

(Naive complex division fails on representable inputs whose quotient is 1)

EMPIRICAL OBSERVATION GIN-OBS-019

In binary64 (CPython 3.13 on x86-64), the textbook formula for $(x + xi)/(x + xi)$ returns `nan+nanj` for $x = 10^{300}$ (the denominator $c^2 + d^2$ overflows to ∞, and ∞/∞ is NaN) and raises `ZeroDivisionError` for $x = 10^{-300}$ (the denominator underflows to 0, and Python's `float` division raises rather than returning an IEEE infinity). Smith's algorithm and CPython's built-in complex division both return 1.

The two failures exhibit two layers again. At 10^{300} the operations proceed under IEEE rules and the error is carried *in the data* (NaN). At 10^{-300} the host language converts the special case into a *signal*. Neither failure is in the mathematics: the quotient is 1.

EXERCISE 28.1

Show that the sequence $x_0 = 1$, $x_{n+1} = (x_n + 2/x_n)/2$ consists of rationals, is Cauchy, and has no limit in $\mathbb{Q}$. How many correct binary digits does x_{n+1} have, roughly, if x_n has k?

▶ Solution to Exercise 28.1

EXERCISE 28.2

Exhibit an order on $\mathbb{C}$ that is total (for example, lexicographic on $(\mathrm{Re}, \mathrm{Im})$) and say which of the two compatibility conditions of Proposition 28.5 it satisfies.

▶ Solution to Exercise 28.2

EXERCISE 28.3

Using interval arithmetic, evaluate $x^2 - 2x + 1$ and $(x - 1)^2$ on $x \in [0, 2]$ (computing a square as a product $x \cdot x$). Which encloses the true range $[0, 1]$ more tightly?

▶ Solution to Exercise 28.3

SUMMARY

- $\mathbb{Q}$ lacks least upper bounds; $\mathbb{R}$ is the unique complete ordered field, built from infinite objects (cuts, Cauchy sequences).
- $\mathbb{R}$ is uncountable, finite descriptions are countable: almost no real has a name. Computable reals are countable.
- Equality of computable reals is undecidable; inequality is semi-decidable. Approximate representations cannot certify zero.
- Interval arithmetic encloses honestly but loses correlation (the dependency problem); floating point returns one confident value.
- $\mathbb{C}$ is algebraically closed but has no compatible order; the principal root is a selection that breaks $\sqrt{xy} = \sqrt{x}\sqrt{y}$.
- In binary64, naive complex division fails where Smith's scaling succeeds; signed zero selects a side of a branch cut.

Chapter 29

$\mathbb{Z}/n$, Finite Fields, and the 2-adic Integers

> *A byte is not a number until you say which arithmetic it obeys. $\mathbb{Z}/256$ and GF(2^8) disagree about almost every sum.*

LEARNING OBJECTIVES

- Describe $\mathbb{Z}/n$ as the quotient of $\mathbb{Z}$ by $n\mathbb{Z}$, and distinguish residues from their representatives.
- Prove that in a finite commutative ring every non-zero element is a unit or a zero divisor, and count both in $\mathbb{Z}/n$.
- Construct $\mathrm{GF}(2^8)$ from polynomials, and compute in it with shifts and exclusive-or.
- Compute the inverse of an odd number modulo 2^w by Newton iteration, and use it for exact division.
- Read a machine word as a truncation of a 2-adic integer.

29.1 A quotient and its representatives

$\mathbb{Z}/n$ is the set of residue classes $[a] = a + n\mathbb{Z}$, with $[a] + [b] = [a + b]$ and $[a][b] = [ab]$. These operations are well defined — they do not depend on which element of the class is used — because $n\mathbb{Z}$ is an *ideal*: $a \equiv a'$ and $b \equiv b'$ imply $a + b \equiv a' + b'$ and $ab \equiv a'b'$. The map $a \mapsto [a]$ is a ring homomorphism, and $\mathbb{Z}/n$ is the arithmetic of what the homomorphism remembers: the last digit in base n, in the language of Chapter 5.

A class is not a number; to write it down one picks a **representative**. The usual choices are another instance of the *selection* repair:

CONVENTION	REPRESENTATIVES OF $\mathbb{Z}/8$	USED BY
least non-negative	$0, 1, \ldots, 7$	mathematics texts; unsigned words; Python `%` with positive modulus

CONVENTION	REPRESENTATIVES OF $\mathbb{Z}/8$	USED BY
symmetric (balanced)	$-4, \dots, 3$	two's-complement signed words (Chapter 21); lattice cryptography
whatever arrives	any integer	lazy reduction in fast arithmetic (Chapter 34)

The values in a signed and an unsigned 8-bit register are the same residues of $\mathbb{Z}/256$ with different representatives; addition, subtraction and multiplication do not care which, and that is why one adder serves both (Chapter 21). Division, comparison and conversion to a wider type *do* care, and that is where signed and unsigned instructions part.

Chapter 27 noted that no order on $\mathbb{Z}/n$ is compatible with addition. In terms of representatives: $7 + 1 = 0$ in $\mathbb{Z}/8$ under either convention, so "$x < x + 1$" must fail somewhere.

29.2 Units and zero divisors

Chapter 16 showed that b is invertible in $\mathbb{Z}/n$ exactly when $\gcd(b, n) = 1$, and that otherwise the converse problem $bc = a$ has either no solution or $\gcd(b, n)$ of them (GIN-THM-002). In a finite ring there is no third kind of non-zero element:

PROPOSITION 29.1

(In a finite commutative ring, every non-zero element is a unit or a zero divisor)

CLASSICAL RESULT GIN-PROP-071

Let R be a finite commutative ring with $1 \neq 0$, and $b \neq 0$. Either b has an inverse, or $bc = 0$ for some $c \neq 0$, and not both.

Proof.

Consider the map $m_b : R \to R$, $c \mapsto bc$. If it is injective, it is bijective because R is finite, so some c has $bc = 1$. If it is not injective, $bc = bc'$ with $c \neq c'$, so $b(c - c') = 0$ with $c - c' \neq 0$. Both at once is impossible: $bc = 0$ and $b'b = 1$ give $c = b'bc = 0$. ∎

The proof is the image/kernel theorem of Chapter 16 in its finite form: m_b is either a bijection (every division by b has one answer) or has a non-trivial kernel (divisions by b have none or several). Infinite rings escape the dichotomy: in $\mathbb{Z}$, the element 2 is neither a unit nor a zero divisor.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from math import gcd
from ginsdk import numbertheory as nt

for n in (12, 30, 31, 256):
    units = [a for a in range(1, n) if gcd(a, n) == 1]
    zero_divisors = [a for a in range(1, n) if any(a * c % n == 0 for c in range(1, n))]
    assert sorted(units + zero_divisors) == list(range(1, n))
    print(f"Z/{n:<4d} units {len(units):3d} = phi(n) = {nt.phi(n):3d}   zero divisors {len(zero_divisors):3d}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
Z/12   units   4 = phi(n) =   4   zero divisors   7
Z/30   units   8 = phi(n) =   8   zero divisors  21
Z/31   units  30 = phi(n) =  30   zero divisors   0
Z/256  units 128 = phi(n) = 128   zero divisors 127
```

The count of units is Euler's function $\varphi(n)$ (Chapter 32). For a prime p there are no zero divisors at all: every non-zero element is a unit and $\mathbb{Z}/p$ is a field, often written $\mathbb{F}_p$. $\mathbb{Z}/256$, the arithmetic of a byte, has 128 units — the odd bytes — and 127 zero divisors.

29.3 The field with 256 elements

Since 256 is not prime, $\mathbb{Z}/256$ is not a field. But there *is* a field with 256 elements, and it is used on every computer that runs AES [FIPS 197]. It is built not from integers but from polynomials over $\mathbb{F}_2 = \{0, 1\}$:

DEFINITION 29.2 (The field GF(2^n)) CLASSICAL RESULT GIN-DEF-093

Let $m(x)$ be an irreducible polynomial of degree n over $\mathbb{F}_2$. $\mathrm{GF}(2^n)$ is the set of polynomials over $\mathbb{F}_2$ of degree less than n, with addition of polynomials (coefficientwise, modulo 2) and multiplication of polynomials followed by reduction modulo $m(x)$. It is a field; any two choices of m give isomorphic fields.

An element is stored as n bits — the coefficients. For AES, $n = 8$ and $m(x) = x^8 + x^4 + x^3 + x + 1$ (bits `0x11B`). The *same byte* now has a different arithmetic:

- **Addition is exclusive-or.** Coefficients add modulo 2 independently: there are no carries. In the vocabulary of Chapter 11, the carry monoid is trivial — every position is *kill*.

- **Multiplication is shift-and-add without carries,** with a reduction whenever a term of degree 8 appears.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def gf_mul(a, b):
    """Multiply in GF(2^8) with m(x) = x^8 + x^4 + x^3 + x + 1."""
    r = 0
    while b:
        if b & 1:
            r ^= a          # add (xor) the current shifted copy
        b >>= 1
        a <<= 1             # multiply by x
        if a & 0x100:
            a ^= 0x11B      # reduce modulo m(x)
    return r

a, b = 0x57, 0x83
print(f"{a:#04x} + {b:#04x}:  Z/256 → {(a + b) % 256:#04x}   GF(2^8) → {a ^ b:#04x}")
print(f"{a:#04x} * {b:#04x}:  Z/256 → {(a * b) % 256:#04x}   GF(2^8) → {gf_mul(a, b):#04x}")
print(f"0x53 * 0xca in GF(2^8) → {gf_mul(0x53, 0xCA):#04x}")
inverses = {x: next(y for y in range(1, 256) if gf_mul(x, y) == 1) for x in range(1, 256)}
print("non-zero bytes with a GF(2^8) inverse:", len(inverses), "   odd bytes (units of Z/256):", 128)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0x57 + 0x83:  Z/256 → 0xda   GF(2^8) → 0xd4
0x57 * 0x83:  Z/256 → 0x85   GF(2^8) → 0xc1
0x53 * 0xca in GF(2^8) → 0x01
non-zero bytes with a GF(2^8) inverse: 255    odd bytes (units of Z/256): 128
```

EXAMPLE 29.1

The bytes `0x57` and `0x83` sum to `0xda` in $\mathbb{Z}/256$ and to `0xd4` in $\mathrm{GF}(2^8)$; their products are `0x85` and `0xc1`. In $\mathrm{GF}(2^8)$ every non-zero byte is invertible — `0x53` and `0xca` are inverses — while in $\mathbb{Z}/256$ only the odd bytes are. The bit pattern is the same; the arithmetic is a choice of grammar, as Chapter 2 argued for the interpretation of bits.

29.4 Inverses modulo 2^w

An odd number a is a unit of $\mathbb{Z}/2^w$, so it has an inverse a^{-1} modulo 2^w. It can be computed without division by *Newton iteration*:

PROPOSITION 29.3 (Newton iteration for inverses modulo a power of two)

CLASSICAL RESULT

GIN-PROP-072

Let a be odd. If $ax \equiv 1 \pmod{2^k}$, then $x' = x(2 - ax)$ satisfies $ax' \equiv 1 \pmod{2^{2k}}$. Since $a \cdot a \equiv 1 \pmod 8$ for every odd a, starting from $x_0 = a$ the iteration reaches w correct bits after $\lceil \log_2(w/3) \rceil$ steps: four for $w = 32$, five for $w = 64$. If moreover d is odd and $d \mid x$, then $x/d = x \cdot d^{-1} \bmod 2^w$ for $0 \le x < 2^w$.

Proof.

Write $ax = 1 + t$ with $2^k \mid t$. Then $ax' = ax(2 - ax) = (1 + t)(1 - t) = 1 - t^2$, and $2^{2k} \mid t^2$. For odd $a = 2j + 1$, $a^2 = 4j(j + 1) + 1$ and $j(j + 1)$ is even, so $a^2 \equiv 1 \pmod 8$: $k_0 = 3$. After s steps $k = 3 \cdot 2^s$, which is ≥ 32 for $s = 4$ and ≥ 64 for $s = 5$. Finally, if $x = qd$ with $0 \le q < 2^w$, then $xd^{-1} \equiv qdd^{-1} \equiv q$, and q is its own least non-negative representative. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def inverse_mod_2w(a, w):
    x, correct_bits, steps = a, 3, 0
    while correct_bits < w:
        x = x * (2 - a * x) % 2 ** w
        correct_bits, steps = 2 * correct_bits, steps + 1
    return x, steps

for a in (3, 7, 12345):
    for w in (32, 64):
        x, steps = inverse_mod_2w(a, w)
        assert a * x % 2 ** w == 1
        print(f"a = {a:<6d} w = {w}: inverse {x:#0{w // 4 + 2}x} after {steps} steps")

x, _ = inverse_mod_2w(7, 32)
print("exact division 7 * 123456789 / 7 by multiplication:", 7 * 123456789 * x % 2 ** 32)
print("inexact: 100 * inverse(7) mod 2^32 =", 100 * x % 2 ** 32, " (not 100 // 7 = 14)")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
a = 3      w = 32: inverse 0xaaaaaaab after 4 steps
a = 3      w = 64: inverse 0xaaaaaaaaaaaaaaab after 5 steps
a = 7      w = 32: inverse 0xb6db6db7 after 4 steps
a = 7      w = 64: inverse 0x6db6db6db6db6db7 after 5 steps
a = 12345  w = 32: inverse 0x55d4be09 after 4 steps
a = 12345  w = 64: inverse 0x4963847d55d4be09 after 5 steps
exact division 7 * 123456789 / 7 by multiplication: 123456789
inexact: 100 * inverse(7) mod 2^32 = 1840700284  (not 100 // 7 = 14)
```

Exact division by multiplication is used by compilers when the language guarantees divisibility — for instance in the subtraction of two pointers, whose byte difference is a known multiple of

the element size. The last line shows the boundary: when $d \nmid x$, the product is the unique solution of $7c \equiv 100 \pmod{2^{32}}$ — a perfectly good element of $\mathbb{Z}/2^{32}$ that is *not* the integer quotient. The converse problem is solvable in $\mathbb{Z}/2^{32}$ and not in $\mathbb{Z}$, and the two answers have nothing to do with each other.

29.5 The 2-adic integers

The inverse of 3 modulo 2^{16} is `1010101010101011` in binary; modulo 2^{32} it is the same pattern, twice as long; modulo 2^{64}, longer still. The patterns are *compatible*: each truncates the next. The limit object is an infinite string of bits extending to the left,

$$\tfrac{1}{3} = \ldots 1010101010101011_2,$$

a **2-adic integer**.

DEFINITION 29.4 (p-adic integers) CLASSICAL RESULT GIN-DEF-094

A p-adic integer is a sequence $(x_1, x_2, \ldots)$ with $x_k \in \mathbb{Z}/p^k$ and $x_{k+1} \equiv x_k \pmod{p^k}$ for every k; equivalently, an infinite base-p numeral $\ldots d_2 d_1 d_0$. Addition and multiplication are computed position by position from the right, with carries, exactly as for finite numerals. The ring is written $\mathbb{Z}_p$.

Chapter 21 proved that two's complement is the truncation of this numeral to w digits (GIN-PROP-050): $-1 = \ldots 111_2$ because adding 1 produces a carry that never stops. In $\mathbb{Z}_2$ every odd number is a unit, so $1/3$, $-1/7$ and $1/12345$ are 2-adic integers, though $1/2$ is not. A w-bit register holds the first w digits of a 2-adic integer; the arithmetic of words is the arithmetic of $\mathbb{Z}_2$ seen through a window of width w. The window does not affect $+$, $-$ and $\times$, because their digits depend only on lower digits (carries move left). It does affect division, comparison and right shifts, which need digits from *outside* the window — and those are the operations whose machine behaviour Part X found most varied.

EXERCISE 29.1

List the units and the zero divisors of $\mathbb{Z}/15$. Which elements b make the converse problem $bc = 5$ solvable, and how many solutions does each have?

▶ Solution to Exercise 29.1

EXERCISE 29.2

Show that $x^8 + x^4 + x^3 + x + 1$ has no root in $\mathbb{F}_2$. Why is that not enough to prove it irreducible, and what further check would be?

▶ Solution to Exercise 29.2

EXERCISE 29.3

Compute the last eight bits of the 2-adic integers $-1/3$ and $1/5$, and check the first by adding it to $1/3$.

▶ Solution to Exercise 29.3

SUMMARY

- $\mathbb{Z}/n$ is a quotient; its elements are classes, and representatives are a selection (least non-negative, symmetric) that matters only for division, comparison and widening.
- In a finite commutative ring every non-zero element is a unit or a zero divisor; $\mathbb{Z}/n$ has $\varphi(n)$ units, and $\mathbb{Z}/p$ is a field.
- $\mathrm{GF}(2^8)$ gives the same 256 bytes a field arithmetic with carry-free addition (xor); AES computes in it.
- Odd numbers are invertible modulo 2^w; Newton iteration finds the inverse in $\lceil \log_2(w/3) \rceil$ steps, and multiplication by it performs exact division.
- Machine words are truncated 2-adic integers; the truncation is invisible to $+$, $-$, $\times$ and visible to division, comparison and shifts.

PART XII

Number Theory from First Principles

Divisibility, primes and unique factorization; congruences, inverses and the Chinese remainder theorem; arithmetic functions and recurrences.

Chapter 30

Divisibility, Primes, and Unique Factorization

> *In the divisibility order, 1 is the least number and 0 is the greatest.*

LEARNING OBJECTIVES

- Read $b \mid a$ as the admissibility condition of the converse problem $bc = a$ in $\mathbb{Z}$.
- Describe the divisibility order on $\mathbb{N}$ as a lattice with meet gcd, join lcm, least element 1 and greatest element 0.
- Prove the fundamental theorem of arithmetic, and identify the step that fails in $\mathbb{Z}[\sqrt{-5}]$.
- Turn Euclid's proof of the infinitude of primes into a procedure, and run it.
- Compare the count of primes below x with $x/\ln x$.

30.1 Divisibility is admissibility

Chapter 16 defined division as the converse problem: given a and b, find c with $bc = a$. In $\mathbb{Z}$ the problem is *admissible* exactly when $b \mid a$:

DEFINITION 30.1 (Divisibility) CLASSICAL RESULT GIN-DEF-095

For integers a, b, b *divides* a, written $b \mid a$, if $a = bc$ for some integer c. Equivalently: the converse problem $bc = a$ has a solution in $\mathbb{Z}$. If $b \neq 0$ the solution is unique; if $b = 0$ it exists only for $a = 0$, and then every integer is one.

The definition contains the four-way case analysis of Chapter 17 in miniature. $0 \mid 0$ is true — the problem $0 \cdot c = 0$ is admissible, with every c a solution — even though $0/0$ is undefined: *divisibility asks whether a solution exists, division asks for the solution*. Under the definition, $0 \mid 0$ holds; a programmer who implements `divides(a, b)` as `a % b == 0` gets an exception for $b = 0$ in most languages, and has silently changed the question.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
def divides_by_definition(b, a, bound=50):
    return any(b * c == a for c in range(-bound, bound + 1))

def divides_by_remainder(b, a):
    return a % b == 0

for b, a in ((3, 12), (5, 12), (0, 0), (0, 7), (7, 0)):
    try:
        r = divides_by_remainder(b, a)
    except ZeroDivisionError as e:
        r = f"ZeroDivisionError: {e}"
    print(f"{b} | {a}:  definition {divides_by_definition(b, a)!s:5s}   a % b == 0 → {r}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
3 | 12:  definition True    a % b == 0 → True
5 | 12:  definition False   a % b == 0 → False
0 | 0:  definition True    a % b == 0 → ZeroDivisionError: integer modulo by zero
0 | 7:  definition False   a % b == 0 → ZeroDivisionError: integer modulo by zero
7 | 0:  definition True    a % b == 0 → True
```

30.2 The divisibility order

On $\mathbb{N}$, divisibility is a partial order: reflexive ($a \mid a$), antisymmetric ($a \mid b$ and $b \mid a$ imply $a = b$ for natural numbers) and transitive. It is very different from the usual order:

- 1 divides everything: it is the **least** element.
- Everything divides 0 ($a \cdot 0 = 0$): 0 is the **greatest** element.
- $\gcd(a, b)$ is the greatest lower bound and $\operatorname{lcm}(a, b)$ the least upper bound, so the order is a *lattice*; with $\gcd(0, a) = a$ and $\operatorname{lcm}(0, a) = 0$ the conventions for zero are not arbitrary but forced by the order.
- The elements just above 1 — those whose only divisors are 1 and themselves — are the **primes**: the *atoms* of the lattice.

PROPOSITION 30.2

($\mathbb{N}$ under divisibility is a lattice with 1 at the bottom and 0 at the top)

CLASSICAL RESULT

GIN-PROP-073

$(\mathbb{N}, \mid)$ is a partial order in which every pair has a greatest lower bound $\gcd(a, b)$ and a least upper bound $\operatorname{lcm}(a, b)$, with 1 least and 0 greatest. Its atoms are the primes.

Proof.

Antisymmetry: $b = ac$, $a = bd$ give $a = acd$; if $a \neq 0$ then $cd = 1$ and $c = d = 1$ in $\mathbb{N}$; if $a = 0$ then $b = 0$. That gcd is the greatest lower bound in this order — every common divisor divides it, not merely is at most it — is Bézout's identity (Chapter 33): $\gcd(a, b) = ua + vb$, so any common divisor of a, b divides the right-hand side. The statement for lcm follows from $\gcd(a, b)\,\operatorname{lcm}(a, b) = ab$ for $a, b > 0$ and is direct when one of them is 0. ∎

The order restricted to the divisors of 60 is a finite lattice whose shape is the product of chains for 2^2, 3 and 5:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

divisors = nt.divisors(60)
covers = [(a, b) for a in divisors for b in divisors if b != a and b % a == 0 and nt.is_prime(b // a)]
print("divisors of 60:", divisors)
print("covering pairs (b = a * prime):", len(covers), "  = edges of a 3 x 2 x 2 grid:", 2*2*2 + 3*1*2 + 3*2*1)
for d in divisors:
    up = [b for a, b in covers if a == d]
    print(f"  {d:>2} → {up}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
divisors of 60: [1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60]
covering pairs (b = a * prime): 20   = edges of a 3 x 2 x 2 grid: 20
   1 → [2, 3, 5]
   2 → [4, 6, 10]
   3 → [6, 15]
   4 → [12, 20]
   5 → [10, 15]
   6 → [12, 30]
  10 → [20, 30]
  12 → [60]
  15 → [30]
  20 → [60]
  30 → [60]
  60 → []
```

30.3 Unique factorization

Every natural number greater than 1 is a product of primes: take any divisor other than 1 and itself and recurse; the recursion stops because factors are smaller. The deep part is *uniqueness*, and it rests on one lemma.

LEMMA 30.3 (Euclid's lemma) CLASSICAL RESULT GIN-HIST-009

If a prime p divides ab, then $p \mid a$ or $p \mid b$.

Proof.

Suppose $p \nmid a$. The only positive divisors of p are 1 and p, so $\gcd(p, a) = 1$, and by Bézout's identity $1 = up + va$ for some integers u, v. Multiplying by b: $b = upb + v(ab)$. Both terms are multiples of p, so $p \mid b$.

∎

THEOREM 30.4 (The fundamental theorem of arithmetic) CLASSICAL RESULT

GIN-HIST-010

Every integer $n > 1$ is a product of primes, and the product is unique up to the order of the factors.

Proof.

Existence as above. Uniqueness, by induction on n: if $p_1 \cdots p_r = q_1 \cdots q_s$, then p_1 divides the right side, so by the lemma (applied repeatedly) $p_1 \mid q_j$ for some j, hence $p_1 = q_j$ because q_j is prime. Cancel it and apply the induction hypothesis to the smaller number n/p_1.

∎

The fundamental theorem says that $\mathbb{N}_{>0}$ under multiplication is the *free commutative monoid* on the primes: a number is the same thing as a finite multiset of primes, and multiplication is union of multisets. Exponent vectors are a second positional notation for natural numbers, with one "digit" per prime and no carries — in which multiplication is addition and gcd and lcm are componentwise minimum and maximum, while addition becomes intractable. Which operations are easy depends on the numeral, a theme of Part III that returns in Chapter 35, where finding this second numeral (factorization) is hard.

Uniqueness is not automatic. It fails as soon as Euclid's lemma fails:

COUNTEREXAMPLE 30.5

(Factorization into irreducibles is unique in every ring of integers)

COUNTEREXAMPLE GIN-NEG-023

In $\mathbb{Z}[\sqrt{-5}] = \{a + b\sqrt{-5}\}$, $6 = 2 \cdot 3 = (1 + \sqrt{-5})(1 - \sqrt{-5})$, and none of the four factors can be written as a product of two non-units. The element 2 divides $(1 + \sqrt{-5})(1 - \sqrt{-5})$ but neither factor: it is *irreducible* without being *prime*.

The check uses the multiplicative norm $N(a+b\sqrt{-5}) = a^2+5b^2$: a factorization of an element of norm 4, 9 or 6 into two non-units would need factors of norm 2 or 3, and a^2+5b^2 takes neither value.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
norm = lambda a, b: a * a + 5 * b * b
print("norms of 2, 3, 1+√-5, 1-√-5:", norm(2, 0), norm(3, 0), norm(1, 1), norm(1, -1))
values = {norm(a, b) for a in range(-3, 4) for b in range(-2, 3)}
print("norms 2 or 3 attained (|a| ⩽ 3, |b| ⩽ 2 suffices):", sorted(values & {2, 3}))
print("is (1+√-5)/2 in the ring? its coordinates are", (1 / 2, 1 / 2), "→ no")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
norms of 2, 3, 1+√-5, 1-√-5: 4 9 6 6
norms 2 or 3 attained (|a| ⩽ 3, |b| ⩽ 2 suffices): []
is (1+√-5)/2 in the ring? its coordinates are (0.5, 0.5) → no
```

In the vocabulary of this book, unique factorization is a *uniqueness* clause of a converse problem — "which multisets of primes multiply to n?" — and $\mathbb{Z}[\sqrt{-5}]$ is a domain in which that problem is non-unique.

30.4 Infinitely many primes

THEOREM 30.6 (There are infinitely many primes (Euclid, Elements IX.20))

CLASSICAL RESULT GIN-HIST-011

For every finite set S of primes there is a prime not in S.

Proof.

Let $N = 1 + \prod_{p\in S} p$. $N > 1$, so it has a prime factor q. If $q \in S$, then q divides both N and $N-1$, hence divides 1 — impossible. ∎

The proof is an algorithm: from a list of primes it computes a new one. Starting from $\{2\}$ and always adding the *smallest* prime factor produces the Euclid–Mullin sequence:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import math
from ginsdk import numbertheory as nt
```

```
primes = [2]
for _ in range(7):
    N = math.prod(primes) + 1
    q = min(nt.factorize(N))
    print(f"1 + product of {len(primes)} primes = {N:<14d} smallest prime factor {q}")
    primes.append(q)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 + product of 1 primes = 3              smallest prime factor 3
1 + product of 2 primes = 7              smallest prime factor 7
1 + product of 3 primes = 43             smallest prime factor 43
1 + product of 4 primes = 1807           smallest prime factor 13
1 + product of 5 primes = 23479          smallest prime factor 53
1 + product of 6 primes = 1244335        smallest prime factor 5
1 + product of 7 primes = 6221671        smallest prime factor 6221671
```

The primes do not arrive in order — 5 appears seventh — and whether every prime eventually appears is an open question. The algorithm is correct (each output is a new prime) but its outputs are not a *grammar* of the primes in the sense of Chapter 3: they are not produced by a successor that visits each exactly once.

30.5 How many primes?

The *prime number theorem* states that the number $\pi(x)$ of primes up to x satisfies $\pi(x) \sim x/\ln x$ [Hardy & Wright 2008]. The ratio converges slowly:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import bisect, math
from ginsdk import numbertheory as nt

primes = nt.sieve(10 ** 6)
for k in range(2, 7):
    x = 10 ** k
    pi = bisect.bisect_right(primes, x)
    print(f"x = 10^{k}:  pi(x) = {pi:6d}   x/ln x = {x / math.log(x):9.1f}   ratio {pi / (x /
math.log(x)):.4f}")
gaps = [b - a for a, b in zip(primes, primes[1:])]
g = max(gaps)
i = gaps.index(g)
print(f"largest gap below 10^6: {g}, between {primes[i]} and {primes[i + 1]}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x = 10^2:  pi(x) =     25   x/ln x =      21.7   ratio 1.1513
x = 10^3:  pi(x) =    168   x/ln x =     144.8   ratio 1.1605
x = 10^4:  pi(x) =   1229   x/ln x =    1085.7   ratio 1.1320
x = 10^5:  pi(x) =   9592   x/ln x =    8685.9   ratio 1.1043
x = 10^6:  pi(x) =  78498   x/ln x =   72382.4   ratio 1.0845
largest gap below 10^6: 114, between 492113 and 492227
```

The density of primes near x is about $1/\ln x$: among 20-digit numbers about one in 46 is prime, among 300-digit numbers about one in 690. This is why random search for large primes works (Chapter 35): primes are rare, but not *too* rare, and testing one candidate is cheap.

EXERCISE 30.1

Using only the definition, decide each of $0 \mid 5$, $5 \mid 0$, $0 \mid 0$, $-3 \mid 6$, and say which of the corresponding divisions $5/0$, $0/5$, $0/0$, $6/(-3)$ have a unique answer in $\mathbb{Z}$.

▶ Solution to Exercise 30.1

EXERCISE 30.2

Show that in the divisibility lattice $\gcd(a, \operatorname{lcm}(a, b)) = a$, and compute $\gcd(0, 0)$ and $\operatorname{lcm}(0, 0)$ from the order alone.

▶ Solution to Exercise 30.2

EXERCISE 30.3

Show that 3 is irreducible but not prime in $\mathbb{Z}[\sqrt{-5}]$.

▶ Solution to Exercise 30.3

SUMMARY

- $b \mid a$ means that the converse problem $bc = a$ is admissible in $\mathbb{Z}$; $0 \mid 0$ is true although $0/0$ is undefined.
- Divisibility orders $\mathbb{N}$ as a lattice with gcd, lcm, least element 1, greatest element 0, and the primes as atoms.
- Unique factorization rests on Euclid's lemma, which rests on Bézout; it fails in $\mathbb{Z}[\sqrt{-5}]$, where irreducible and prime differ.
- Euclid's proof is an algorithm producing new primes, not in order; $\pi(x)/(x/\ln x) \to 1$ slowly (1.0845 at 10^6).

Chapter 31

Congruences, Orders, and the Converse of Exponentiation

> *$a \equiv b \pmod n$ is an equation that has agreed to forget everything but the last digit in base n.*

LEARNING OBJECTIVES

- Use congruence as an equivalence compatible with $+$ and $\times$, and solve linear congruences and systems of them.
- State and prove Fermat's and Euler's theorems, and Wilson's theorem as a test that is exact and useless.
- Compute orders of elements, recognize primitive roots, and check the classification of moduli that have them.
- Treat the discrete logarithm as the converse of $x \mapsto g^x$, and predict its number of solutions from the image/kernel theorem.
- Decide quadratic residuosity with Euler's criterion.

31.1 An equality that forgets

Gauss introduced the notation $a \equiv b \pmod n$ — "n divides $a - b$" — in the *Disquisitiones Arithmeticae*, choosing the sign for its analogy with equality [Gauss 1801]. The analogy is exact in one direction: congruence modulo n is an equivalence relation compatible with addition and multiplication, so it can be used like an equality inside sums and products. It is an equality *after* the homomorphism $\mathbb{Z} \to \mathbb{Z}/n$ of Chapter 29, which keeps the residue and forgets the rest.

What it forgets matters for every operation that is not a ring operation:

- **Cancellation** requires a unit: $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$, but $3 \not\equiv 8$.
- **Exponents** live in a different modulus: a^x modulo n depends on x modulo the order of a, not modulo n (below).
- **Division** is the converse problem of Chapter 16, with none, one or $\gcd(b, n)$ solutions (GIN-THM-002).

31.2 Systems: the Chinese remainder theorem

A system $x \equiv r_1 \pmod{m_1}, \dots, x \equiv r_k \pmod{m_k}$ asks for the converse of the map $x \mapsto (x \bmod m_1, \dots, x \bmod m_k)$. That map is a ring homomorphism $\mathbb{Z}/L \to \prod \mathbb{Z}/m_i$ with $L = \operatorname{lcm}(m_i)$, and it is injective, so by the image/kernel theorem a solution, if it exists, is unique modulo L (GIN-PROP-042). For pairwise coprime moduli it is also surjective: every system is solvable.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

for residues, moduli in (([2, 3, 2], [3, 5, 7]), ([1, 3], [4, 6]), ([1, 2], [4, 6])):
    print(f"x ≡ {residues} mod {moduli}:  →", nt.crt(residues, moduli))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x ≡ [2, 3, 2] mod [3, 5, 7]:  → (23, 105)
x ≡ [1, 3] mod [4, 6]:  → (9, 12)
x ≡ [1, 2] mod [4, 6]:  → None
```

The first system is Sunzi's classical example: 23 is the unique solution modulo 105. The second is solvable although the moduli share the factor 2, because $1 \equiv 3 \pmod 2$; the solution is unique modulo $\operatorname{lcm}(4, 6) = 12$. The third asks for a number that is odd ($1 \bmod 4$) and even ($2 \bmod 6$): the image condition fails, and there is no solution.

31.3 Fermat, Euler, Wilson

THEOREM 31.1 (Euler's theorem (Fermat's little theorem for n = p)) CLASSICAL RESULT

GIN-HIST-012

If $\gcd(a, n) = 1$ *then* $a^{\varphi(n)} \equiv 1 \pmod n$. *In particular, for a prime* $p \nmid a$, $a^{p-1} \equiv 1 \pmod p$.

Proof.

The units of $\mathbb{Z}/n$ form a group U of order $\varphi(n)$. Multiplication by the unit a permutes U (it is injective, Chapter 29). Hence $\prod_{u \in U} au = \prod_{u \in U} u$, that is, $a^{\varphi(n)} P = P$ with P a unit, and cancelling P gives the claim. ∎

The theorem is the basis of the Fermat primality test of Chapter 35 — and of its failure: the converse statement "if $a^{n-1} \equiv 1$ then n is prime" is false (GIN-PROP-043).

THEOREM 31.2 (Wilson's theorem) CLASSICAL RESULT GIN-HIST-013

For $n \geq 2$: n is prime if and only if $(n-1)! \equiv -1 \pmod{n}$.

Proof.

For a prime p, each $a \in \{1, \ldots, p-1\}$ has an inverse in the same set, and $a = a^{-1}$ only for $a = \pm 1$ (since $a^2 \equiv 1$ means $p \mid (a-1)(a+1)$). Pairing every other element with its inverse leaves $(p-1)! \equiv 1 \cdot (-1)$. For composite $n = ab$ with $1 < a < n$, a divides $(n-1)!$ and n, so it cannot divide $(n-1)! + 1$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import math
print("n < 40 with (n-1)! ≡ -1 (mod n):", [n for n in range(2, 40) if math.factorial(n - 1) % n == n - 1])
print("decimal digits of (n-1)! for n = 1,000,003:", int(math.lgamma(1_000_003) / math.log(10)) + 1)
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n < 40 with (n-1)! ≡ -1 (mod n): [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
decimal digits of (n-1)! for n = 1,000,003: 5565721
```

Wilson's theorem is an *exact* primality criterion, and computationally worthless: the factorial has millions of digits for a seven-digit n, and no method is known to compute $(n-1)! \bmod n$ in time polynomial in the number of digits of n. Exactness is not the same as feasibility — the cost model of Chapter 36 is part of what a "test" is.

31.4 Orders and primitive roots

The **order** of a unit a modulo n is the least $d \geq 1$ with $a^d \equiv 1$. By Euler's theorem (and Lagrange's theorem for groups) d divides $\varphi(n)$. Powers of a repeat with period d: a^x depends only on $x \bmod d$.

A **primitive root** is a unit of order $\varphi(n)$ — one whose powers run through all units. When it exists, the unit group is cyclic, and every unit is a power of one generator: a *successor grammar* (Chapter 3) for the multiplicative group, with "multiply by g" as the successor.

THEOREM 31.3 (Which moduli have primitive roots) CLASSICAL RESULT GIN-HIST-014

$\mathbb{Z}/n$ has a primitive root if and only if n is $1, 2, 4, p^k$ or $2p^k$ for an odd prime p and $k \geq 1$ [Hardy & Wright 2008].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from math import gcd
from ginsdk import numbertheory as nt

def order(a, n):
    k, x = 1, a % n
    while x ≠ 1:
        x, k = x * a % n, k + 1
    return k

def has_primitive_root(n):
    return any(order(a, n) == nt.phi(n) for a in range(1, n) if gcd(a, n) == 1)

def classified(n):
    f = nt.factorize(n)
    return n in (2, 4) or (len(f) == 1 and 2 not in f) or (len(f) == 2 and f.get(2) == 1)

print("n < 60 with a primitive root:", [n for n in range(2, 60) if has_primitive_root(n)])
print("search agrees with the classification for all 2 ⩽ n < 400:",
      all(has_primitive_root(n) == classified(n) for n in range(2, 400)))
print("orders of the units mod 11:", {a: order(a, 11) for a in range(1, 11)})
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n < 60 with a primitive root: [2, 3, 4, 5, 6, 7, 9, 10, 11, 13, 14, 17, 18, 19, 22, 23, 25, 26, 27, 29, 31, 34,
37, 38, 41, 43, 46, 47, 49, 50, 53, 54, 58, 59]
search agrees with the classification for all 2 ⩽ n < 400: True
orders of the units mod 11: {1: 1, 2: 10, 3: 5, 4: 5, 5: 5, 6: 10, 7: 10, 8: 10, 9: 5, 10: 2}
```

31.5 The converse of exponentiation

Modular exponentiation $x \mapsto g^x \bmod p$ is fast (Chapter 34). Its converse — given g and b, find x with $g^x \equiv b$ — is the **discrete logarithm** problem, and no polynomial-time classical algorithm is known for it in general; the security of Diffie–Hellman key exchange rests on that. Its *admissibility*, however, is completely understood, and it is a case of the image/kernel theorem (GIN-THM-007):

PROPOSITION 31.4 (The structure of a discrete logarithm) PROVED HERE GIN-PROP-074

Let p be prime and g a unit of order d modulo p. The map $E_g : \mathbb{Z}/(p-1) \to (\mathbb{Z}/p)^\times$, $x \mapsto g^x$, is a group homomorphism with kernel $d\mathbb{Z}/(p-1)$ and image the d powers of g. Hence $g^x \equiv b$ has no solution if $b \notin \langle g \rangle$, and exactly $(p-1)/d$ solutions in $\mathbb{Z}/(p-1)$ if $b \in \langle g \rangle$. It has exactly one solution for every b if and only if g is a primitive root.

Proof.

$g^{x+y} = g^x g^y$, and g^x is well defined for $x \in \mathbb{Z}/(p-1)$ because $g^{p-1} \equiv 1$. $g^x \equiv 1$ iff $d \mid x$, which describes the kernel; it has $(p-1)/d$ elements. The image is $\langle g \rangle$, of size d. The solution set of $E_g(x) = b$ is empty or a coset of the kernel. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from collections import Counter

p = 11
census = Counter()
for g in range(1, p):
    for b in range(1, p):
        census[sum(1 for x in range(p - 1) if pow(g, x, p) == b)] += 1
print(f"g^x ≡ b (mod {p}) over all 100 pairs (g, b): number of solutions x in Z/10 →",
dict(sorted(census.items())))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
g^x ≡ b (mod 11) over all 100 pairs (g, b): number of solutions x in Z/10 → {0: 37, 1: 40, 2: 20, 5: 2, 10: 1}
```

The census is predicted exactly by the proposition. Modulo 11 the unit orders are 1 (for $g = 1$), 2 ($g = 10$), 5 (four elements) and 10 (four primitive roots). The four primitive roots give 40 pairs with one solution; the order-5 elements give 20 pairs with 2 solutions and 20 with none; $g = 10$ gives 2 pairs with 5 solutions and 8 with none; $g = 1$ gives one pair ($b = 1$) with 10 solutions and 9 with none: $40 + 20 + 2 + 1$ solvable, $20 + 8 + 9 = 37$ not.

The converse of exponentiation therefore has the same three outcomes as the converse of multiplication — none, one, many — with the same algebraic explanation. What differs is *cost*: for multiplication, the extended Euclidean algorithm finds a solution in polynomially many steps (Chapter 33); for exponentiation, the best general algorithms take time exponential in a fractional power of the number of digits. Admissibility can be easy when solving is hard.

31.6 Quadratic residues

The converse of squaring, $c^2 \equiv a \pmod{p}$, is the first non-linear converse problem in this book whose solution sets are not cosets (GIN-NEG-019). For an odd prime p and $a \not\equiv 0$, it has 0 or 2 solutions, and exactly half of the units are squares.

THEOREM 31.5 (Euler's criterion) CLASSICAL RESULT GIN-HIST-015

For an odd prime p and $p \nmid a$: a is a square modulo p iff $a^{(p-1)/2} \equiv 1 \pmod{p}$; otherwise $a^{(p-1)/2} \equiv -1$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
p = 23
squares = sorted({x * x % p for x in range(1, p)})
print(f"squares mod {p}:", squares, f" ({len(squares)} of {p - 1})")
print("Euler's criterion agrees for every unit:", all((pow(a, (p - 1) // 2, p) == 1) == (a in squares) for a in
range(1, p)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
squares mod 23: [1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18]  (11 of 22)
Euler's criterion agrees for every unit: True
```

Euler's criterion decides admissibility with one exponentiation, without finding a root. Finding the root is a further step (for $p \equiv 3 \pmod{4}$ the root is $a^{(p+1)/4}$; in general, the Tonelli–Shanks algorithm). Deciding *whether* a converse problem is solvable and *solving* it are different computational problems, as the discrete logarithm showed.

EXERCISE 31.1

Show that if $ab \equiv ac \pmod{n}$ and $\gcd(a, n) = g$, then $b \equiv c \pmod{n/g}$, and that this cannot in general be strengthened to $\bmod n$.

▶ Solution to Exercise 31.1

EXERCISE 31.2

Compute $3^{1000} \bmod 7$ by hand using the order of 3.

▶ Solution to Exercise 31.2

EXERCISE 31.3

Modulo 13, how many solutions does $4^x \equiv 3$ have in $\mathbb{Z}/12$, and how many does $4^x \equiv 2$ have?

▶ Solution to Exercise 31.3

SUMMARY

- Congruence is equality after $\mathbb{Z} \to \mathbb{Z}/n$; it supports $+$ and $\times$ but not cancellation by non-units, and exponents live modulo orders.
- The Chinese remainder theorem is the converse of an injective homomorphism: solutions are unique modulo the lcm when they exist.
- Euler: $a^{\varphi(n)} \equiv 1$; Wilson's criterion is exact but infeasible.
- Primitive roots exist exactly for $n = 1, 2, 4, p^k, 2p^k$ (checked for $n < 400$); they make the unit group a successor grammar.
- The discrete logarithm is the converse of a homomorphism: none, or $(p-1)/d$ solutions; deciding admissibility is easy, solving is believed hard.
- Euler's criterion decides whether a square root exists without computing one.

Chapter 32

Arithmetic Functions and the Dirichlet Ring

> *Möbius inversion is division by the constant function 1. It is admissible because $1(1) \neq 0$.*

LEARNING OBJECTIVES

- Define τ, σ, φ, μ and recognize multiplicative functions.
- Compute Dirichlet convolutions, and identify the units of the Dirichlet ring.
- Read Möbius inversion as division by $\mathbf{1}$, and decide when the converse problem $f * g = h$ is solvable.
- Compare measured averages of τ and φ with their asymptotic formulas, and measure the probability that two random integers are coprime.

32.1 Functions on the positive integers

An *arithmetic function* is any $f : \mathbb{N}_{>0} \to \mathbb{C}$. The classical examples count or sum over divisors:

FUNCTION	DEFINITION	VALUE AT 12
$\tau(n)$	number of divisors	6
$\sigma(n)$	sum of divisors	28
$\varphi(n)$	number of $1 \le k \le n$ with $\gcd(k, n) = 1$	4
$\mu(n)$	$(-1)^r$ if n is a product of r distinct primes, else 0	0
$\mathbf{1}(n)$	1	1
$\mathrm{id}(n)$	n	12
$\varepsilon(n)$	1 if $n = 1$, else 0	0

A function is **multiplicative** if $f(1) = 1$ and $f(mn) = f(m)f(n)$ whenever $\gcd(m, n) = 1$. All seven functions in the table are multiplicative; $n \mapsto n + 1$ is not. By the fundamental theorem of arithmetic (GIN-HIST-010), a multiplicative function is determined by its values on prime powers — a statement about the *free commutative monoid* structure of Chapter 30, not about the functions. For instance $\varphi(p^k) = p^k - p^{k-1}$ gives $\varphi(n) = n \prod_{p \mid n} (1 - 1/p)$.

32.2 Dirichlet convolution

DEFINITION 32.1 (Dirichlet convolution and the Dirichlet ring) CLASSICAL RESULT

GIN-DEF-096

For arithmetic functions f, g, $(f * g)(n) = \sum_{d \mid n} f(d)\, g(n/d)$. With pointwise addition and $*$ as multiplication, arithmetic functions form a commutative ring with identity ε [Apostol 1976].

The convolution is the multiplication of a positional system with one digit per *divisor*: in the generating-function view it is the product of Dirichlet series $\sum f(n) n^{-s}$, just as ordinary convolution is the product of power series. Many classical identities are products in this ring:

$$\tau = \mathbf{1} * \mathbf{1}, \qquad \sigma = \mathbf{1} * \mathrm{id}, \qquad \varphi * \mathbf{1} = \mathrm{id}, \qquad \mu * \mathbf{1} = \varepsilon.$$

The SDK tabulates functions on $1..N$ and convolves them; the cost of one convolution is the number of pairs (d, m) with $dm \le N$, which is $\sum_{n \le N} \tau(n)$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt
from ginsdk.cost import Cost

N = 10_000
one, ident = nt.table(lambda n: 1, N), nt.table(lambda n: n, N)
eps = nt.table(lambda n: int(n == 1), N)
mu, phi = nt.table(nt.mobius, N), nt.table(nt.phi, N)
print("1 * 1 = tau:     ", nt.dirichlet(one, one) == nt.table(lambda n: len(nt.divisors(n)), N))
print("1 * id = sigma:  ", nt.dirichlet(one, ident) == nt.table(nt.sigma, N))
print("phi * 1 = id:    ", nt.dirichlet(phi, one) == ident)
print("mu * 1 = epsilon:", nt.dirichlet(mu, one) == eps)
c = Cost()
nt.dirichlet(one, one, c)
print(f"multiplications for one convolution on 1..{N}: {c.counts['mul']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
1 * 1 = tau:      True
1 * id = sigma:   True
phi * 1 = id:     True
mu * 1 = epsilon: True
multiplications for one convolution on 1..10000: 93668
```

32.3 Möbius inversion is division

The converse problem in this ring is: given f and h, find g with $f * g = h$. Evaluating at $n = 1$ gives $f(1)g(1) = h(1)$, and at each larger n,

$$f(1)\, g(n) = h(n) - \sum_{d \mid n,\ d>1} f(d)\, g(n/d),$$

which determines $g(n)$ from values at proper divisors — a recursion over the divisibility order of Chapter 30, from the bottom element 1 upward.

PROPOSITION 32.2 (Units and division in the Dirichlet ring) PROVED HERE

GIN-PROP-075

Over a field (e.g. $\mathbb{Q}$ or $\mathbb{C}$):

1. *f is a unit if and only if $f(1) \neq 0$, and then $f * g = h$ has exactly one solution for every h.*
2. *The ring has no zero divisors. Hence when $f \neq 0$ and $f(1) = 0$, $f * g = h$ has at most one solution, and none unless $h(1) = 0$.*

Over $\mathbb{Z}$, f is a unit if and only if $f(1) = \pm 1$.

Proof.

(1) If $f(1) \neq 0$ the recursion above defines g uniquely, by induction on n. Conversely $(f * g)(1) = f(1)g(1)$, which cannot be $\varepsilon(1) = 1$ if $f(1) = 0$. (2) Let n_0 be the least n with $f(n) \neq 0$ and m_0 the least with $g(m) \neq 0$. In $(f * g)(n_0 m_0) = \sum_{dm = n_0 m_0} f(d)g(m)$, a term is non-zero only if $d \geq n_0$ and $m \geq m_0$, which with $dm = n_0 m_0$ forces $d = n_0$; so the sum is $f(n_0)g(m_0) \neq 0$. Uniqueness follows: $f * g = f * g'$ gives $f * (g - g') = 0$. Over $\mathbb{Z}$ the recursion divides by $f(1)$ at every step, and $f(1)g(1) = 1$ requires $f(1) = \pm 1$; conversely $f(1) = \pm 1$ keeps every step integral. ∎

Möbius inversion is the special case $f = \mathbf{1}$: since $\mathbf{1}(1) = 1$, every h is uniquely $\mathbf{1} * g$, and $g = \mu * h$. In the classical formulation: if $h(n) = \sum_{d \mid n} g(d)$ for all n, then $g(n) = \sum_{d \mid n} \mu(d) h(n/d)$. The theorem that textbooks state as an inversion formula is, in this book's vocabulary, the statement that division by $\mathbf{1}$ is *admissible and unique* in the Dirichlet ring, and μ is the reciprocal of $\mathbf{1}$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from fractions import Fraction
from ginsdk import numbertheory as nt

N = 30
one = nt.table(lambda n: 1, N)
eps = nt.table(lambda n: int(n == 1), N)
kind, inv = nt.dirichlet_divide(eps, one)
print("epsilon / 1:", kind, "→", [int(x) for x in inv[1:]])
print("mu         :", " " * len(kind) + "    ", [nt.mobius(n) for n in range(1, N + 1)])
kind, g = nt.dirichlet_divide(nt.table(lambda n: n, N), one)
print("id / 1 = phi:", kind, all(g[n] == nt.phi(n) for n in range(1, N + 1)))
not_unit = nt.table(lambda n: 0 if n == 1 else 1, N)
print("epsilon / f with f(1) = 0:", nt.dirichlet_divide(eps, not_unit)[0])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
epsilon / 1: unique → [1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0, -1, 1, 1, 0, -1, 0, -1, 0, 1, 1, -1, 0, 0, 1,
0, 0, -1, -1]
mu         :            [1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0, -1, 1, 1, 0, -1, 0, -1, 0, 1, 1, -1, 0, 0, 1,
0, 0, -1, -1]
id / 1 = phi: unique True
epsilon / f with f(1) = 0: not-solved
```

The last line is the solver *declining*, not deciding: for $f(1) = 0$ the proposition guarantees at most one solution, and none for $h = \varepsilon$ because $\varepsilon(1) \neq 0$; the SDK's solver reports `not-solved` rather than claiming an answer it has not computed. A complete decision procedure for $f(1) = 0$ is possible on a finite range (the equations become a triangular system shifted by the least n_0 with $f(n_0) \neq 0$), and Exercise 32.3 works one case.

32.4 Averages

Arithmetic functions fluctuate wildly — $\tau(p) = 2$ for every prime, while $\tau(720720) = 240$ — but their averages are smooth. Two classical formulas [Apostol 1976; Hardy & Wright 2008]:

$$\sum_{n \le x} \tau(n) = x \ln x + (2\gamma - 1)x + O(\sqrt{x}), \qquad \sum_{n \le x} \varphi(n) = \frac{3}{\pi^2} x^2 + O(x \ln x),$$

with $\gamma \approx 0.5772$ Euler's constant.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import math
```

```
N = 10 ** 5
tau = [0] * (N + 1)
for d in range(1, N + 1):
    for m in range(d, N + 1, d):
        tau[m] += 1
phi = list(range(N + 1))
for p in range(2, N + 1):
    if phi[p] == p:                                  # p is prime
        for m in range(p, N + 1, p):
            phi[m] -= phi[m] // p
gamma = 0.5772156649015329
for x in (10 ** 3, 10 ** 4, 10 ** 5):
    T, P = sum(tau[1:x + 1]), sum(phi[1:x + 1])
    print(f"x = {x:>6d}: sum tau = {T:>8d}  vs x ln x + (2γ-1)x = {x * math.log(x) + (2 * gamma - 1) *
x:>11.1f}"
          f"   sum phi / (3x²/π²) = {P / (3 * x * x / math.pi ** 2):.7f}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
x =   1000: sum tau =     7069  vs x ln x + (2γ-1)x =      7062.2   sum phi / (3x²/π²) = 1.0007516
x =  10000: sum tau =    93668  vs x ln x + (2γ-1)x =     93647.7   sum phi / (3x²/π²) = 1.0000372
x = 100000: sum tau =  1166750  vs x ln x + (2γ-1)x =   1166735.7   sum phi / (3x²/π²) = 1.0000050
```

The second formula has a probabilistic reading. Since $\sum_{n \le x} \varphi(n)$ counts pairs $1 \le k \le n \le x$ with $\gcd(k, n) = 1$, about $\frac{6}{\pi^2} \approx 0.6079$ of all pairs of integers are coprime:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import math, random

random.seed(32)
trials = 100_000
for bits in (16, 64):
    coprime = sum(math.gcd(random.getrandbits(bits), random.getrandbits(bits)) == 1 for _ in range(trials))
    print(f"{bits}-bit pairs: fraction coprime {coprime / trials:.4f}   6/π² = {6 / math.pi ** 2:.4f}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
16-bit pairs: fraction coprime 0.6089   6/π² = 0.6079
64-bit pairs: fraction coprime 0.6078   6/π² = 0.6079
```

This probability governs the converse problems of earlier chapters. A random division $bc = a$ in $\mathbb{Z}/n$ with random b is uniquely solvable with probability $\varphi(n)/n$; a random fraction p/q is already in lowest terms with probability about 0.61, so more often than not the gcd computed by a rational-arithmetic library finds nothing to cancel (Chapter 27).

32.5 Perfect, abundant and deficient

The oldest question about σ is Euclid's: when is $\sigma(n) = 2n$? Euclid proved that $2^{k-1}(2^k - 1)$ is perfect when $2^k - 1$ is prime, and Euler proved that every even perfect number has this form [Hardy & Wright 2008]. Whether an odd perfect number exists is open.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
N = 10_000
sigma = [0] * (N + 1)
for d in range(1, N + 1):
    for m in range(d, N + 1, d):
        sigma[m] += d
print("perfect numbers up to 10^4:", [n for n in range(1, N + 1) if sigma[n] == 2 * n])
print("abundant (sigma(n) > 2n):", sum(sigma[n] > 2 * n for n in range(1, N + 1)),
      "  deficient:", sum(sigma[n] < 2 * n for n in range(1, N + 1)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
perfect numbers up to 10^4: [6, 28, 496, 8128]
abundant (sigma(n) > 2n): 2488   deficient: 7508
```

EXERCISE 32.1

Prove $\sum_{d \mid n} \varphi(d) = n$ by sorting the fractions $1/n, 2/n, \dots, n/n$ by their reduced denominators.

▶ Solution to Exercise 32.1

EXERCISE 32.2

Compute the Dirichlet inverse of id on $n = 1, \dots, 6$, and guess a formula.

▶ Solution to Exercise 32.2

EXERCISE 32.3

Let $f(1) = 0$, $f(2) = 1$, and $f(n) = 0$ otherwise. For which h does $f * g = h$ have a solution, and what is it?

▶ Solution to Exercise 32.3

SUMMARY

- τ, σ, φ, μ are multiplicative; unique factorization makes them determined by prime powers.
- Dirichlet convolution makes arithmetic functions a commutative ring with identity ε and no zero divisors; its units are the f with $f(1) \neq 0$ (over $\mathbb{Z}$: ± 1).
- Möbius inversion is division by $\mathbf{1}$, admissible because $\mathbf{1}(1) = 1$; $\mu = \mathbf{1}^{-1}$.
- Averages are smooth: $\sum \tau \approx x \ln x + (2\gamma - 1)x$, $\sum \varphi \approx 3x^2/\pi^2$; two random integers are coprime with probability $6/\pi^2$.
- Even perfect numbers are $2^{k-1}(2^k - 1)$ with $2^k - 1$ prime; odd ones are unknown.

PART XIII

Algorithms in Number Theory

Euclid and its relatives, exponentiation by squaring, sieves, primality tests and factorization — each as a transition system with an invariant and a counted cost.

Chapter 33

Euclid's Algorithm as a Transition System

> *Of two unequal numbers, the less being continually subtracted from the greater… — Euclid, Elements VII.1*

LEARNING OBJECTIVES

- Present the Euclidean algorithm as a transition system with a state space, one move, an invariant and a termination measure, and prove it correct.
- Prove Lamé's bound and identify the Fibonacci pairs as the worst case.
- Use the extended algorithm to produce Bézout coefficients, and use them to solve the linear converse problems of Chapters 16 and 29.
- Distinguish the number of division steps from the bit cost, and bound the latter by $\mathcal{O}(L^2)$.
- State what is known about the average number of steps, and check it against measurement.
- Prove a bound for Stein's binary algorithm, and compare the two algorithms by counted operations rather than by step counts.

33.1 States, one move, one invariant

Chapter 30 defined $\gcd(a, b)$ through the divisibility order. Computing it is a different matter, and the oldest non-trivial algorithm in mathematics does it [Euclid, Elements]. As a transition system it has:

- **states** — pairs (a, b) of natural numbers;
- **one move** — $(a, b) \to (b, a \bmod b)$, admissible exactly when $b \neq 0$ (the remainder of Chapter 19 is a converse problem with a selection, and it has no answer for $b = 0$);
- **terminal states** — pairs $(g, 0)$, where no move is admissible;
- **an invariant** — the set of common divisors, and therefore $\gcd(a, b)$, does not change under the move;
- **a termination measure** — the second component strictly decreases, since $0 \leq a \bmod b < b$.

THEOREM 33.1 (Correctness of Euclid's algorithm) CLASSICAL RESULT GIN-PROP-040

From any state (a, b) the move can be applied only finitely often, and the terminal state is $(\gcd(a, b), 0)$.

Proof.

Write $a = qb + r$. A common divisor of a and b divides $r = a - qb$; a common divisor of b and r divides $a = qb + r$. So (a, b) and (b, r) have the same common divisors and the same greatest one. The second component is a natural number that strictly decreases, so the run is finite. At a terminal state $(g, 0)$ the common divisors are the divisors of g, whose greatest is g (with the convention $\gcd(0, 0) = 0$ in the divisibility order, Chapter 30). ∎

The SDK records each run as a trace of rows (a, b, q) with $a = qb + (a \bmod b)$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

for row in nt.euclid_trace(1071, 462):
    a, b, q = row
    print(f"{a:5d} = {q} * {b:4d} + {a - q * b}")
print("gcd =", nt.gcd(1071, 462), " steps =", nt.euclid_steps(1071, 462))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
 1071 = 2 *  462 + 147
  462 = 3 *  147 + 21
  147 = 7 *   21 + 0
gcd = 21  steps = 3
```

Each line is one transition; the invariant says every line has the same gcd. The algorithm never computes a divisor — it only takes remainders — and still ends at the greatest common one. That is what an invariant buys: the answer is carried through states that do not mention it.

33.2 The worst case: Lamé

How many moves can a run take? Large quotients make fast progress, so the slowest runs have every quotient equal to 1, and running such a run backwards from $(1, 0)$ gives the recurrence of the Fibonacci numbers.

THEOREM 33.2 (Lamé's theorem) CLASSICAL RESULT GIN-PROP-040

If the algorithm started at $a > b > 0$ takes k division steps, then $a \geq F_{k+2}$ and $b \geq F_{k+1}$, where $F_1 = F_2 = 1$ and $F_{n+1} = F_n + F_{n-1}$. The pair (F_{k+2}, F_{k+1}) takes exactly k steps. Consequently $k \leq \log_\varphi a + \mathcal{O}(1)$ with $\varphi = (1+\sqrt{5})/2$ [Lamé 1844].

Proof.

By induction on k. For $k = 1$: $b \geq 1 = F_2$ and $a > b$ gives $a \geq 2 = F_3$. For $k \geq 2$, the first step leads to (b, r) with $r > 0$, from which $k - 1$ steps remain; by induction $b \geq F_{k+1}$ and $r \geq F_k$. Since $a = qb + r$ with $q \geq 1$, $a \geq b + r \geq F_{k+1} + F_k = F_{k+2}$. For the Fibonacci pair, $F_{k+2} = 1 \cdot F_{k+1} + F_k$ with $F_k < F_{k+1}$ for $k \geq 2$, so the run visits the consecutive Fibonacci pairs down to $(F_3, F_2) = (2, 1)$ and then $(1, 0)$: k steps. Finally $F_{k+2} \geq \varphi^k$, so $k \leq \log_\varphi a$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import numbertheory as nt

first = {}
for a in range(2, 2000):
    for b in range(1, a):
        first.setdefault(nt.euclid_steps(a, b), (a, b))
ok = all(first[k] == nt.lame_worst_pair(k) for k in first)
print("smallest pair needing k steps is (F_{k+2}, F_{k+1}) for every k seen:", ok, " max k =", max(first))
print("k = 15:", first[15], "  2^64 - 1 needs at most", next(k for k in range(200) if nt.fibonacci(k + 2) >
2**64 - 1) - 1, "steps")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
smallest pair needing k steps is (F_{k+2}, F_{k+1}) for every k seen: True  max k = 15
k = 15: (1597, 987)   2^64 - 1 needs at most 91 steps
```

Lamé's bound is a statement about the *grammar* of the run: each step at least multiplies the "distance to the end" by the golden ratio. It is also the first theorem in the history of algorithm analysis — an 1844 bound on the running time of an algorithm in terms of the size of its input.

33.3 Running the algorithm backwards: Bézout

Chapter 29 showed that $b \cdot c \equiv a \pmod{n}$ has a solution iff $\gcd(b, n) \mid a$, but did not say how to find one. The **extended algorithm** carries, alongside each remainder, a record of how it was made from the inputs.

THEOREM 33.3 (Bézout's identity, constructively) CLASSICAL RESULT GIN-HIST-016

For integers a, b not both 0, the extended Euclidean algorithm computes integers s, t with $sa + tb = \gcd(a, b)$. Its rows (r_i, s_i, t_i), started at $(a, 1, 0)$ and $(b, 0, 1)$ and updated by $r_{i+1} = r_{i-1} - q_i r_i$ (likewise for s and t), satisfy the invariant $r_i = s_i a + t_i b$ [Knuth 1997].

Proof.

The invariant holds for the two initial rows by inspection, and it is preserved because the update is the same linear combination applied to all three components: $r_{i-1} - q_i r_i = (s_{i-1} - q_i s_i)a + (t_{i-1} - q_i t_i)b$. The r-column is exactly Euclid's sequence of remainders, so its last non-zero entry is $\gcd(a, b)$ by Theorem 33.1. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

print(" r      s      t      q")
for row in nt.extended_euclid_trace(240, 46):
    print(f"{row.r:3d} {row.s:6d} {row.t:6d}   {row.q if row.q is not None else ''}")
g, s, t = nt.bezout(240, 46)
print(f"gcd = {g} = {s}*240 + {t}*46")
print("inverse of 7 mod 40:", nt.mod_inverse(7, 40), "  inverse of 6 mod 40:", nt.mod_inverse(6, 40))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
 r      s      t      q
240      1      0
 46      0      1   5
 10      1     -5   4
  6     -4     21   1
  4      5    -26   1
  2     -9     47   2
  0     23   -120
gcd = 2 = -9*240 + 47*46
inverse of 7 mod 40: 23   inverse of 6 mod 40: None
```

The last line is the converse problem of Chapter 29 solved: $7c \equiv 1 \pmod{40}$ has the unique solution the extended algorithm finds, while $6c \equiv 1 \pmod{40}$ fails the image condition ($\gcd(6, 40) = 2 \nmid 1$) and the SDK returns `None` rather than a number. The algorithm *decides* admissibility and *solves* the problem in one run, which is not true of every converse problem — Chapter 31 showed the discrete logarithm, where deciding is easy and solving is believed hard.

The same coefficients solve the linear Diophantine equation $ax + by = c$: it is solvable iff $g = \gcd(a, b) \mid c$, a particular solution is $(sc/g, tc/g)$, and all solutions form a coset of the kernel $\{(bk/g, -ak/g) : k \in \mathbb{Z}\}$ — the image/kernel theorem once more (GIN-THM-007).

33.4 What a gcd costs

Lamé bounds the number of *steps* by $\mathcal{O}(L)$ for L-bit inputs. A step is a division with remainder, whose bit cost is not constant (Chapter 36), so the step count is not the cost.

THEOREM 33.4 (The bit cost of Euclid's algorithm) CLASSICAL RESULT GIN-HIST-017

With schoolbook division, Euclid's algorithm on L-bit inputs uses $\mathcal{O}(L^2)$ bit operations — not $\mathcal{O}(L^3)$, as "$\mathcal{O}(L)$ steps times $\mathcal{O}(L^2)$ per division" would suggest [Knuth 1997].

Proof.

Dividing r_{i-1} by r_i with quotient q_i costs $\mathcal{O}((1+\log q_i)\cdot \log r_i) \subseteq \mathcal{O}((1+\log q_i)L)$ bit operations. Since $r_{i-1} \geq q_i r_i$, the product of the quotients is at most a, so $\sum_i \log q_i \leq \log a \leq L$. With $\mathcal{O}(L)$ steps by Lamé, the total is $\mathcal{O}(L\cdot(k+\sum \log q_i)) = \mathcal{O}(L^2)$. ∎

The theorem is an example of a cost argument that uses the *invariant structure of the run* — the quotients cannot all be large — rather than a worst case per step. Measured with the SDK's limb arithmetic (Chapter 37 has the full data):

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from ginsdk import bigint as B
from ginsdk.cost import Cost

rng = random.Random(33)
prev = None
for L in (256, 512, 1024, 2048):
    a, b = rng.getrandbits(L) | 1 << (L - 1), rng.getrandbits(L) | 1 << (L - 1)
    c = Cost()
    _, steps = B.gcd_euclid(B.from_int(a), B.from_int(b), c)
    ops = B.word_ops(c)
    print(f"L = {L:5d}: {steps:5d} division steps, {ops:9d} limb operations" + (f"   ratio to previous {ops /
prev:.2f}" if prev else ""))
    prev = ops
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
L =   256:   146 division steps,      1510 limb operations
L =   512:   283 division steps,      5567 limb operations   ratio to previous 3.69
L =  1024:   608 division steps,     22422 limb operations   ratio to previous 4.03
L =  2048:  1242 division steps,     86838 limb operations   ratio to previous 3.87
```

Doubling L roughly doubles the number of steps (Lamé: linear) and roughly quadruples the limb operations (Theorem 33.4: quadratic). Faster gcd algorithms exist — Lehmer's method works on leading digits, and "half-gcd" algorithms reach $\mathcal{O}(M(L)\log L)$ with fast multiplication [Brent & Zimmermann 2010] — and are not implemented in this book's SDK.

33.5 The average run

Lamé describes the worst case. For a typical pair the number of steps is smaller by a constant factor, and its average is known with remarkable precision.

THEOREM 33.5 (The average number of steps) CLASSICAL RESULT GIN-PROP-040

For fixed n, the average number of division steps over $1 \le m < n$ with $\gcd(m,n) = 1$ is $\frac{12\ln 2}{\pi^2}\ln n + C_P + \mathcal{O}(n^{-1/6+\epsilon})$, where $C_P \approx 1.4671$ is Porter's constant [Heilbronn 1969; Dixon 1970; Porter 1975].

Measured counts for prime n agree with this formula *after* an exact offset of one step (GIN-OBS-011): the classical count τ_n applies the algorithm to (m,n) with $m < n$, whose first step merely swaps the arguments, while the SDK starts at (n,m). The difference is a convention of counting, and the book reports it rather than correcting it away:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import math
from ginsdk import numbertheory as nt

c = 12 * math.log(2) / math.pi ** 2
for n in (1009, 10007, 100003):
    avg = sum(nt.euclid_steps(n, m) for m in range(1, n)) / (n - 1)
    print(f"n = {n:6d}: measured {avg:.4f}   formula {c * math.log(n) + 1.4670780794:.4f}   difference {avg - c
* math.log(n) - 1.4670780794:+.4f}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n =   1009: measured 6.2877   formula 7.2962   difference -1.0086
n =  10007: measured 8.2308   formula 9.2298   difference -0.9991
n = 100003: measured 10.1704   formula 11.1698   difference -0.9994
```

33.6 Stein's binary algorithm

Division is the most expensive of the four operations in hardware (Chapter 19). Stein's algorithm [Stein 1967] replaces it by subtraction, halving and comparison, which are cheap on binary words. Its moves on a pair of positive integers are: *halve both* while both are even (recording a factor 2); *halve* an even component; *subtract* the smaller odd component from the larger.

PROPOSITION 33.6 (An iteration bound for the binary algorithm) PROVED HERE

GIN-PROP-076

On odd a and positive b, each iteration of the main loop of Stein's algorithm (halve b until odd, order the pair, subtract) at least halves the product of the two components. Hence the number of subtractions is at most $\log_2(ab) + 1$.

Proof.

At the start of an iteration both components are odd after halving b, say $a \le b$. The subtraction replaces b by $b - a$, which is even, and the next iteration halves it at least once: $b' \le (b - a)/2 < b/2$, while a is unchanged. So the product ab' is less than half of ab. The product is a positive integer while the loop runs, so the number of iterations is at most $\log_2(ab) + 1$. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from ginsdk import numbertheory as nt

rng = random.Random(7)
worst, total, total_e = 0.0, 0, 0
for _ in range(3000):
    a, b = rng.getrandbits(64) | 1, rng.getrandbits(64) | 1
    g, moves = nt.binary_gcd_trace(a, b)
    subs = moves.count("subtract")
    worst = max(worst, subs / ((a * b).bit_length()))
    total += subs
    total_e += nt.euclid_steps(a, b)
print("largest subtractions / log2(ab) observed:", round(worst, 3))
print(f"mean over 3000 pairs of 64-bit odd numbers: {total / 3000:.1f} subtractions (binary) vs {total_e /
3000:.1f} divisions (Euclid)")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
largest subtractions / log2(ab) observed: 0.433
mean over 3000 pairs of 64-bit odd numbers: 45.4 subtractions (binary) vs 37.7 divisions (Euclid)
```

The binary algorithm takes *more* steps than Euclid's — about 45 against 38 on 64-bit inputs — and is often faster anyway, because each of its steps is a subtraction and a shift instead of a division. Comparing algorithms by step counts alone compares the grammars of their runs, not their costs; the cost needs the model of the machine that executes a step (Chapter 36).

LABORATORY (WEBSITE EDITION)

Text alternative: enter two integers; the widget runs the Euclidean algorithm and the extended algorithm, prints every row (r, s, t, q) with the invariant $r = sa + tb$ checked at each row, and reports the gcd, the Bézout coefficients and the number of steps.

EXERCISE 33.1

Run the extended algorithm on $(F_{12}, F_{11}) = (144, 89)$. What are the Bézout coefficients, and what do you notice about them?

▶ Solution to Exercise 33.1

EXERCISE 33.2

Find all integer solutions of $240x + 46y = 6$, and show that $240x + 46y = 5$ has none.

▶ Solution to Exercise 33.2

EXERCISE 33.3

Explain why the proof of Theorem 33.4 would fail if each division cost $\Theta(L^2)$ regardless of the quotient, and which implementations of division have that property.

SUMMARY

- Euclid's algorithm is a transition system: states (a, b), one move $(a, b) \to (b, a \bmod b)$ admissible iff $b \neq 0$, invariant gcd, measure b.
- Lamé: k steps need $a \geq F_{k+2}$; Fibonacci pairs are the worst case; $k \leq \log_{\varphi} a + \mathcal{O}(1)$ (checked for all pairs below 2000).
- The extended algorithm keeps the invariant $r_i = s_i a + t_i b$ and solves linear congruences and Diophantine equations; it decides admissibility and solves in one run.
- Steps are not cost: Euclid uses $\mathcal{O}(L)$ steps but $\mathcal{O}(L^2)$ bit operations, because the quotients' logarithms sum to at most L (measured: limb operations roughly quadruple when L doubles).
- The average step count follows Heilbronn–Dixon–Porter, with an exact one-step convention offset in the SDK's count.
- Stein's binary algorithm halves the product ab per iteration; it takes more steps than Euclid's and cheaper ones.

Chapter 34

Exponentiation: The Exponent's Numeral as a Program

> *To compute x^{1000}, read 1000 in binary and do what it says.*

LEARNING OBJECTIVES

- Read left-to-right binary exponentiation as the execution of a program whose text is the exponent's binary numeral, and count its squarings and multiplications exactly.
- Compare left-to-right and right-to-left exponentiation, and relate both to addition chains.
- Prove that every addition chain for n has at least $\lceil \log_2 n \rceil$ steps.
- State the bit cost of modular exponentiation.
- Explain why exponents are not elements of the domain, when they may be reduced, and what negative exponents require.

34.1 The exponent is a program

To compute x^{13} one may multiply twelve times. Positional notation offers something better. The binary numeral of 13 is `1101`. Read it left to right, starting from x for the leading `1`; for each later digit **square** the accumulator, and if the digit is `1` also **multiply** by x:

$$x \xrightarrow{1:\,\mathrm{SM}} x^{3} \xrightarrow{0:\,\mathrm{S}} x^{6} \xrightarrow{1:\,\mathrm{SM}} x^{13}.$$

The accumulator after reading a prefix u of the numeral is $x^{\mathrm{val}_2(u)}$, because squaring doubles the exponent and multiplying adds one — the two moves of the binary digit grammar of Chapter 6, $e \mapsto 2e$ and $e \mapsto 2e+1$, applied to the *exponent*. The numeral of the exponent is literally the program (GIN-PROP-041).

PROPOSITION 34.1 (The cost of the exponent program) CLASSICAL RESULT GIN-PROP-041

Left-to-right binary exponentiation computes x^e ($e \geq 1$) with exactly $\lfloor \log_2 e \rfloor$ squarings and $\mathrm{popcount}(e) - 1$ *multiplications. Reading the move word back (each* M *or* SM *as* `1`*, each* S *as* `0`*) recovers the binary numeral of e [Knuth 1997].*

Proof.

The numeral of e has $\lfloor \log_2 e \rfloor + 1$ digits; every digit after the first costs one squaring, and every `1` after the first costs one multiplication. The invariant "accumulator $= x^{\mathrm{val}_2(\text{prefix})}$" holds initially and is preserved by both moves, so at the end the accumulator is x^e. The move word is a letter-by-letter recoding of the digit word. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

value, rows = nt.powmod_trace(3, 13, 1000)
for bit, moves, acc in rows:
    print(f"digit {bit}: {moves:2s} → accumulator {acc}")
print("3^13 mod 1000 =", value, "=", pow(3, 13, 1000), "  counts:", nt.powmod_count(13))
e = 1000
print(f"e = {e} = {bin(e)[2:]}:", nt.powmod_count(e), " instead of", e - 1, "multiplications")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
digit 1: M  → accumulator 3
digit 1: SM → accumulator 27
digit 0: S  → accumulator 729
digit 1: SM → accumulator 323
3^13 mod 1000 = 323 = 323   counts: {'square': 3, 'multiply': 2}
e = 1000 = 1111101000: {'square': 9, 'multiply': 5}  instead of 999 multiplications
```

The same correspondence appeared in Chapter 14 as a statement about unary numerals: the binary numeral of n is a straight-line program for the unary word a^n, with doublings for squarings and appended letters for multiplications (GIN-PROP-007). Positional notation, repeated squaring and grammar compression are one object seen three ways.

34.2 Right to left, and addition chains

The digits can also be read from the least significant end: keep a running square $x, x^2, x^4, \ldots$ and multiply it into the result whenever the digit is `1`. The counts are the same — one squaring per digit after the first and one multiplication per further `1` — and the order of the operations

differs. Right-to-left needs one extra register; left-to-right always multiplies by the *same* x, which matters when x is small and the accumulator is large, and which makes "windowed" variants (precomputing $x^3, x^5, \ldots$ and reading several digits at a time) natural.

Both are special cases of an **addition chain** for e: a sequence $1 = a_0 < a_1 < \cdots < a_r = e$ in which every a_i is the sum of two earlier terms (possibly equal). Each term is one multiplication of earlier powers, so a chain of length r computes x^e with r multiplications. Binary exponentiation gives chains of length $\lfloor \log_2 e \rfloor + \operatorname{popcount}(e) - 1$; shorter chains often exist.

THEOREM 34.2 (The doubling lower bound for addition chains) CLASSICAL RESULT

GIN-HIST-018

Every addition chain for e has length at least $\lceil \log_2 e \rceil$. Hence binary exponentiation is optimal up to a factor of 2, and optimal exactly when e is a power of 2 [Knuth 1997].

Proof.

By induction $a_i \le 2^i$: true for $a_0 = 1$, and $a_i = a_j + a_k$ with $j, k < i$ gives $a_i \le 2 \cdot 2^{i-1}$. So $e = a_r \le 2^r$, i.e. $r \ge \log_2 e$. Binary uses at most $2\lfloor \log_2 e \rfloor$ steps, and exactly $\log_2 e$ when e is a power of 2. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from itertools import count

def shortest_chain(e):
    # iterative deepening over star-free chains (any two earlier terms); fine for small e
    best = None
    def extend(chain, depth):
        nonlocal best
        if chain[-1] == e:
            best = list(chain); return True
        if depth == 0 or chain[-1] * 2 ** depth < e:
            return False
        sums = sorted({a + b for a in chain for b in chain if chain[-1] < a + b <= e}, reverse=True)
        return any(extend(chain + [s], depth - 1) for s in sums)
    for d in count(0):
        if extend([1], d):
            return best

for e in (15, 23, 31, 127):
    c = shortest_chain(e)
    binary = e.bit_length() - 1 + bin(e).count("1") - 1
    print(f"e = {e:3d}: shortest chain length {len(c) - 1} {c}   binary method {binary}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
e =  15: shortest chain length 5 [1, 2, 4, 5, 10, 15]   binary method 6
e =  23: shortest chain length 6 [1, 2, 4, 5, 9, 18, 23]   binary method 7
```

```
e =  31: shortest chain length 7 [1, 2, 4, 8, 10, 20, 30, 31]   binary method 8
e = 127: shortest chain length 10 [1, 2, 4, 8, 16, 32, 40, 42, 84, 126, 127]   binary method 12
```

For $e = 15$ binary exponentiation uses 6 multiplications, while the search finds the chain $1, 2, 4, 5, 10, 15$ of length 5 (another is $1, 2, 3, 6, 12, 15$): reusing an intermediate power is better than following the digits. Finding a shortest chain is a hard search problem in general (the search above is exponential and only works for small exponents), and its relation to smallest grammars for unary strings was noted in Chapter 14 [Charikar et al. 2005].

34.3 Modular exponentiation and its cost

Over the integers, x^e has about $e \log_2 x$ bits: exponentiation is the first operation in this book whose *output* is exponentially larger than its input, and no algorithm can be faster than writing the output. Modulo n every intermediate value stays below n, and the exponent program runs in about $2 \log_2 e$ multiplications of L-bit numbers followed by reductions.

PROPOSITION 34.3 (Bit cost of modular exponentiation) CLASSICAL RESULT

GIN-PROP-041

With schoolbook multiplication and division, computing $x^e \bmod n$ for L-bit x, e, n costs $\Theta(L)$ multiplications and reductions of $\Theta(L^2)$ bit operations each: $\Theta(L^3)$ in total. With multiplication in time $M(L)$ it costs $\mathcal{O}(L \cdot M(L))$.

The measured exponent for $L \le 512$ is about 2.9 (GIN-OBS-010), approaching the predicted 3 as the lower-order terms fade. This cost — cubic but polynomial — is what makes the Fermat and Miller–Rabin tests of Chapter 35 and the RSA and Diffie–Hellman systems practical; its *converse*, the discrete logarithm (Chapter 31), has no known polynomial algorithm.

34.4 What an exponent is

In $a + b$ and $a \cdot b$ both arguments come from the same domain. In a^k they do not. Repeated multiplication defines a^k for a natural number k and an element a of any monoid; negative k makes sense exactly when a has an inverse. The exponent is an integer **acting on** the domain, not an element of it — and an evaluator that forgets this computes nonsense. The first evaluator written for this book did: it evaluated 0^{-1} in $\mathbb{Z}/6$ by reducing -1 to the residue 5 and returning $0^5 = 0$ (GIN-NEG-013).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import evaluate

for e, d in [("0^-1", "Z/6"), ("3^-1", "Z/7"), ("2^-1", "Z/6"), ("2^0.5", "Q"), ("(2/3)^-2", "Q"), ("0^0",
"Q")]:
    o = evaluate(e, d)
    print(f"{e:9s} in {d:4s}: {o.status:13s} {o.display or o.reason}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
0^-1      in Z/6 : no-solution   0^-1 needs the inverse of 0 mod 6: gcd(0, 6) = 6 does not divide 1
3^-1      in Z/7 : value         5
2^-1      in Z/6 : no-solution   2^-1 needs the inverse of 2 mod 6: gcd(2, 6) = 2 does not divide 1
2^0.5     in Q   : not-in-domain in the exponent: the literal 0.5 denotes 1/2, which is not in Z
(2/3)^-2  in Q   : value         9/4
0^0       in Q   : value         1
```

Exponents *can* sometimes be reduced — but modulo the order of the base, not modulo the modulus. For a unit a modulo n, Euler's theorem gives $a^e \equiv a^{e \bmod \varphi(n)}$. For a non-unit the reduction is false:

COUNTEREXAMPLE 34.4 (Reducing an exponent modulo φ(n) without a unit)

COUNTEREXAMPLE GIN-NEG-024

Modulo $n = 4$, $\varphi(4) = 2$, but $2^3 \equiv 0$ while $2^{3 \bmod 2} = 2^1 = 2$. The rule $a^e \equiv a^{e \bmod \varphi(n)}$ requires $\gcd(a, n) = 1$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from math import gcd
from ginsdk import numbertheory as nt

bad = [(a, e, n) for n in range(2, 40) for a in range(n) for e in range(1, 3 * n)
       if pow(a, e, n) ≠ pow(a, e % nt.phi(n), n)]
units_ok = all(pow(a, e, n) == pow(a, e % nt.phi(n), n)
               for n in range(2, 40) for a in range(n) if gcd(a, n) == 1 for e in range(1, 3 * n))
print("violations of a^e ≡ a^(e mod φ(n)) for n < 40:", len(bad), " all with gcd(a, n) > 1:", all(gcd(a, n) > 1
for a, e, n in bad))
print("the rule holds for every unit tested:", units_ok, "  first violation:", bad[0])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
violations of a^e ≡ a^(e mod φ(n)) for n < 40: 2665  all with gcd(a, n) > 1: True
the rule holds for every unit tested: True   first violation: (0, 1, 2)
```

Every violation found involves a non-unit, and the rule holds for every unit, as Euler's theorem says. The pattern is the one Chapter 9 called *typing*: the expression `a ^ e` is well formed, but the rewrite "reduce e modulo $\varphi(n)$" is admissible only under a hypothesis about a, and a system that applies it unconditionally changes the meaning of programs.

EXERCISE 34.1

Write the exponent program for $e = 45$, and compute $2^{45} \bmod 1000$ by hand with it.

▶ Solution to Exercise 34.1

EXERCISE 34.2

Show that right-to-left binary exponentiation also uses $\lfloor \log_2 e \rfloor$ squarings and $\mathrm{popcount}(e) - 1$ multiplications, if multiplications by the initial value 1 are not counted.

EXERCISE 34.3

Find an addition chain for $e = 63$ shorter than the binary method's, and explain why every chain for $e = 2^k$ of length k must consist of doublings.

▶ Solution to Exercise 34.3

SUMMARY

- Left-to-right binary exponentiation executes the exponent's binary numeral: $\lfloor \log_2 e \rfloor$ squarings, $\mathrm{popcount}(e) - 1$ multiplications, and the move word recovers the numeral.
- Exponentiation methods are addition chains; every chain has length $\geq \lceil \log_2 e \rceil$, so binary is optimal within a factor 2 and shorter chains exist (e.g. 5 instead of 6 steps for $e = 15$).
- Modular exponentiation costs $\Theta(L^3)$ bit operations with schoolbook arithmetic (measured slope ≈ 2.9 at small L); its converse, the discrete logarithm, is believed hard.
- Exponents are integers acting on a domain: negative exponents need inverses, and reduction modulo $\varphi(n)$ is valid only for units (counterexample $2^3 \bmod 4$).

Chapter 35

Sieves, Primality Tests, and Factoring Baselines

> *A test that can only say 'probably' must say what the probability is over.*

LEARNING OBJECTIVES

- Treat primality as a question about the solution set of $n = d \cdot e$ with $1 < d < n$ — a converse problem of multiplication — and classify the methods by how they answer it.
- Analyze the sieve of Eratosthenes and trial division, by counted operations.
- Prove why Fermat's test can be fooled, identify pseudoprimes and Carmichael numbers, and reproduce their published counts.
- State the Miller–Rabin test precisely, distinguish its probabilistic and deterministic uses, and say exactly what its error bound is a probability over.
- Measure two factoring baselines and state their costs.

35.1 Primality as admissibility

Chapter 30 defined a prime as a number with exactly two divisors. In the vocabulary of Chapter 9, $n > 1$ is composite exactly when the converse problem "find d, e with $d \cdot e = n$ and $1 < d, e < n$" has a solution. Methods for primality differ in what they do with that problem:

METHOD	HOW IT ANSWERS	COST FOR L-BIT N	CAN IT BE WRONG?
sieve	precomputes the answer for all $m \leq N$	$\mathcal{O}(N \log \log N)$ crossings, then $\mathcal{O}(1)$ per query	no
trial division	searches for a solution $d \leq \sqrt{n}$	up to $\approx \sqrt{n}/2 = 2^{L/2}/2$ divisions	no

METHOD	HOW IT ANSWERS	COST FOR L-BIT N	CAN IT BE WRONG?
Fermat test	checks a property every prime has	one modular exponentiation	yes: says "probably prime" for some composites
Miller–Rabin	checks a stronger property	one modular exponentiation per base	yes, with a bounded error per random base
factoring	exhibits a solution (d, e)	super-polynomial for the best known methods	no (it proves compositeness)

The two kinds of answer have different evidential status: a sieve, trial division or a found factor *prove* their answer; a Fermat or Miller–Rabin "probably prime" is only the failure to find a witness of compositeness. In CGT's terms the sieve is precomputation, trial division is search, and the tests are cheap admissibility checks that are **sound for one answer only** — a found witness proves compositeness, and its absence proves nothing.

35.2 The sieve: precomputation

Eratosthenes' sieve writes $2, \ldots, N$ and, for each prime $p \le \sqrt{N}$ not yet crossed out, crosses out $p^2, p^2 + p, \ldots$. Every composite $m \le N$ has a prime factor $\le \sqrt{m}$ and is crossed out by its smallest one. The number of crossings is $\sum_{p \le \sqrt{N}} (N - p^2)/p \approx N \ln \ln N$ by Mertens' theorem [Hardy & Wright 2008].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import math
from ginsdk import numbertheory as nt
from ginsdk.cost import Cost

for N in (10**4, 10**5, 10**6):
    c = Cost()
    ps = nt.sieve(N, c)
    print(f"N = {N:>8}: {len(ps):6d} primes, {c['cross']:8d} crossings = {c['cross'] / (N *
math.log(math.log(N))):.3f} x N ln ln N")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
N =    10000:   1229 primes,    16981 crossings = 0.765 x N ln ln N
N =   100000:   9592 primes,   193078 crossings = 0.790 x N ln ln N
N =  1000000:  78498 primes,  2122048 crossings = 0.808 x N ln ln N
```

The ratio is below 1 and slowly rising: the approximation $N \ln \ln N$ omits lower-order terms that are still visible at $N = 10^6$ (GIN-H-027). A sieve answers every query up to N in constant time after $\mathcal{O}(N \log \log N)$ preprocessing and N bits of storage — exactly CGT's precomputation mechanism, with its usual price: the table is exponential in the number of digits of N.

35.3 Fermat's test and its counterexamples

Euler's theorem (Chapter 31) says that every prime p satisfies $a^{p-1} \equiv 1 \pmod p$ for every a coprime to p. The **Fermat test** to base a checks this condition for a given n. A failure proves n composite without finding a factor — a is then a *Fermat witness*. Success proves nothing, and the converse of Euler's theorem is false (GIN-PROP-043):

THEOREM 35.1 (Korselt's criterion) CLASSICAL RESULT GIN-PROP-043

*A composite n satisfies $a^{n-1} \equiv 1 \pmod n$ for every a coprime to n if and only if n is squarefree and $p - 1 \mid n - 1$ for every prime $p \mid n$. Such n are **Carmichael numbers**; there are infinitely many [Korselt 1899; Alford, Granville & Pomerance 1994].*

Proof.

(Sufficiency.) If $n = p_1 \cdots p_k$ with distinct primes and $p_i - 1 \mid n - 1$, then for a coprime to n, $a^{n-1} = (a^{p_i - 1})^{(n-1)/(p_i-1)} \equiv 1 \pmod{p_i}$ for each i, hence modulo n by the Chinese remainder theorem. (Necessity is proved with primitive roots modulo p_i and p_i^2; see the cited sources.) The infinitude is the theorem of Alford, Granville and Pomerance. ∎

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

N = 10**5
psp = [n for n in range(3, N, 2) if not nt.is_prime(n) and nt.fermat_test(n, 2)]
carm = [n for n in range(3, N, 2) if nt.is_carmichael(n)]
print(f"base-2 Fermat pseudoprimes below {N}: {len(psp)}, the first: {psp[:6]}")
print(f"Carmichael numbers below {N}: {len(carm)}, the first: {carm[:6]}")
print("561 = 3 * 11 * 17 passes every coprime base:",
      all(pow(a, 560, 561) == 1 for a in range(2, 561) if all(a % p for p in (3, 11, 17))))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
base-2 Fermat pseudoprimes below 100000: 78, the first: [341, 561, 645, 1105, 1387, 1729]
Carmichael numbers below 100000: 16, the first: [561, 1105, 1729, 2465, 2821, 6601]
561 = 3 * 11 * 17 passes every coprime base: True
```

Below 10^6 the book's experiment finds 245 base-2 pseudoprimes and 43 Carmichael numbers, the published values (GIN-EXP-010, [Pomerance, Selfridge & Wagstaff 1980]).

35.4 Miller–Rabin: a stronger necessary condition

Write $n-1=2^s d$ with d odd. For a prime p the only square roots of 1 modulo p are ± 1, so the sequence $a^d, a^{2d}, \dots, a^{2^s d} \equiv 1$ either starts at 1 or reaches -1 before reaching 1. A base a for which this fails is a **strong witness** of compositeness; a composite n for which a is not a witness has a as a **strong liar** [Miller 1976; Rabin 1980].

THEOREM 35.2 (At most a quarter of the bases lie) CLASSICAL RESULT GIN-HIST-019

For an odd composite $n>9$, at most $\varphi(n)/4$ of the bases $1 \le a < n$ are strong liars; for $n=9$ the two liars are $2/8$ of the bases [Rabin 1980].

The theorem is about one fixed n and a **base chosen at random**: if k bases are chosen independently and uniformly, a composite n survives all k rounds with probability at most 4^{-k}. It is *not* a statement that a random number which passes is prime with probability $1-4^{-k}$ — that probability depends on how the candidates were produced, and requires a separate argument. The census below checks the bound exhaustively for small n:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numbertheory as nt

worst, worst_n, count = 0.0, None, 0
for n in range(9, 5001, 2):
    if nt.is_prime(n):
        continue
    liars = sum(1 for a in range(1, n) if nt.strong_probable_prime(n, a))
    count += 1
    if liars / (n - 1) > worst:
        worst, worst_n = liars / (n - 1), n
print(f"odd composites 9..5000 checked: {count}; largest fraction of strong liars: {worst:.4f} at n = {worst_n}")
print("strong base-2 pseudoprimes below 10^5:", [n for n in range(3, 10**5, 2) if not nt.is_prime(n) and nt.strong_probable_prime(n, 2)][:8], "...")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
odd composites 9..5000 checked: 1831; largest fraction of strong liars: 0.2500 at n = 9
strong base-2 pseudoprimes below 10^5: [2047, 3277, 4033, 4681, 8321, 15841, 29341, 42799] ...
```

The largest fraction of liars in the census is exactly $1/4$, attained at $n = 9$ — and, by Theorem 35.2, nowhere else (GIN-OBS-021).

Probabilistic and deterministic use. With random bases, Miller–Rabin is a probabilistic algorithm whose error probability is bounded for every input. With a *fixed* list of bases it is deterministic, and it is correct below a bound found by exhaustive computation: the first 13 prime bases suffice for all $n < 3\,317\,044\,064\,679\,887\,385\,961\,981$ [Sorenson & Webster 2017]. That is how the SDK's `is_prime` works, and it agrees with the sieve on every $n \le 10^6$ (GIN-EXP-010). Above the bound the fixed-base test has no proved error bound at all — a fact that should be visible in any interface that offers it.

35.5 Factoring baselines

When a number is composite, a factor is the strongest possible answer. The SDK implements two baselines:

- **trial division** — at most about $\sqrt{n}/2$ divisions by odd d, exponential in the number of digits;
- **Pollard's rho** — iterate $x \mapsto x^2 + c \bmod n$ and detect, with Floyd's cycle finding, two values congruent modulo an unknown prime factor p; heuristically about $\sqrt{p}$ steps [Pollard 1975].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import math
from ginsdk import numbertheory as nt
from ginsdk.cost import Cost

for p, q in ((1009, 1013), (104729, 1299709), (15485863, 32452843)):
    n = p * q
    c = Cost()
    d = nt.pollard_rho(n, 1, c)
    print(f"n = {n:16d}: rho found {d} after {c['step']:5d} steps (sqrt(min factor) = {math.isqrt(min(p,
q)):5d})")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n =          1022117: rho found 1013 after    17 steps (sqrt(min factor) =    31)
n =     136117223861: rho found 104729 after   278 steps (sqrt(min factor) =   323)
n =  502560280658509: rho found 32452843 after  3615 steps (sqrt(min factor) =  3935)
```

Both methods are exponential in the number of digits; rho replaces $\sqrt{n}$ by about $\sqrt{p}$ for the smallest prime factor p (GIN-OBS-012). The asymptotically fastest known methods (the quadratic and number field sieves) are sub-exponential and are not implemented here. No polynomial-time

classical factoring algorithm is known, and nothing in this book suggests one: the admissibility vocabulary describes *what* factoring asks, not how to answer it faster (Chapter 41).

EXERCISE 35.1

Show that $341 = 11 \cdot 31$ passes the Fermat test to base 2 but is not a Carmichael number, and find a base that exposes it.

▶ Solution to Exercise 35.1

EXERCISE 35.2

Show that 2 is a strong witness for 341 even though it is not a Fermat witness.

▶ Solution to Exercise 35.2

EXERCISE 35.3

A program draws random odd 512-bit integers and outputs the first one passing $k = 20$ Miller–Rabin rounds with random bases. Explain why "the output is composite with probability at most 4^{-20}" does not follow from Theorem 35.2 alone, and what additional fact is needed.

SUMMARY

- Primality is a converse problem of multiplication; sieves precompute, trial division searches, tests check necessary conditions, factoring exhibits a solution.
- The sieve performs $\approx N \ln \ln N$ crossings (measured 0.77–0.81 of that for $N = 10^4, 10^5, 10^6$).
- Fermat's test is fooled by pseudoprimes and by every Carmichael number (Korselt's criterion); published counts below 10^6 are reproduced.
- Miller–Rabin: at most $1/4$ of bases lie for odd composite n (census: maximum exactly $1/4$, at $n = 9$); the bound is a probability over random bases for a fixed n.
- With the first 13 prime bases the test is deterministic below $\approx 3.3 \cdot 10^{24}$, and unproved above.
- Trial division and Pollard's rho are exponential baselines; no faster factoring is claimed.

PART XIV

The Complexity of Arithmetic

Unit cost, word RAM and bit complexity kept apart; measured exponents; and what the shape of an evaluation does and does not change.

Chapter 36

Three Cost Models, Kept Apart

> *"Addition takes one step" is true, false, or meaningless, depending on what is allowed to grow.*

LEARNING OBJECTIVES

- Define the unit-cost model, the word RAM and the bit-complexity model, naming in each what grows and what is fixed.
- Translate a cost statement between models, and recognize when the translation fails.
- Explain why unit-cost multiplication on unbounded integers is not a reasonable model, with an explicit computation.
- Use the lower bounds proved earlier in the book (linear time and logarithmic depth for addition) to anchor the models.
- Apply the reporting rule that every cost in this book names its model and its growing parameter.

36.1 Why a cost needs a model

Chapters 10 to 19 gave many costs: addition in $\Theta(L)$ bit operations, schoolbook multiplication in $\Theta(L^2)$, Euclid's algorithm in $\mathcal{O}(L)$ steps and $\mathcal{O}(L^2)$ bit operations, modular exponentiation in $\Theta(L^3)$. Each is true in one model and false or meaningless in others. The three models below are classical [Knuth 1997; Cormen et al. 2009]; what this chapter adds is the discipline of never mixing them silently.

DEFINITION 36.1 (Three cost models for arithmetic) DEFINITION GIN-DEF-097

1. **Unit cost.** Each arithmetic operation on numbers costs 1, whatever the numbers. The growing parameter is the number of operations. Admissible only when every operand is bounded by a fixed constant — and then it is the word RAM with a fixed word.
2. **Word RAM.** Numbers are stored in w-bit words; an operation on one or two words costs 1; the model assumes $w \geq \log_2 n$ for an input of size n, so that a word can hold an index into the input. An L-bit number occupies $\lceil L/w \rceil$ words, and operations on it cost what the limb algorithms of Chapter 15 charge.
3. **Bit complexity.** Each operation on bits costs 1 (Boolean circuits, or a Turing machine's steps). The growing parameter is the bit length L of the operands. This is the model in which "$\Theta(L^2)$ for schoolbook multiplication" is a theorem.

The counted limb operations of the SDK (Chapter 37) are the word RAM with $w = 32$: they become bit costs up to the constant factor w, and unit costs only while every operand fits in one limb.

36.2 One algorithm, three costs

TASK	UNIT COST	WORD RAM (L-BIT OPERANDS, W-BIT WORDS)	BIT COMPLEXITY
add two numbers	1	$\lceil L/w \rceil$	$\Theta(L)$ (GIN-PROP-058)
multiply (schoolbook)	1	$\Theta((L/w)^2)$	$\Theta(L^2)$
multiply (Karatsuba)	1	$\Theta((L/w)^{\log_2 3})$	$\Theta(L^{\log_2 3})$ (GIN-PROP-063)
gcd by Euclid	$\mathcal{O}(L)$ divisions	$\mathcal{O}(L^2/w)$	$\mathcal{O}(L^2)$ (GIN-HIST-017)
$x^e \bmod n$	$\mathcal{O}(\log e)$	$\mathcal{O}(L(L/w)^2)$	$\Theta(L^3)$ schoolbook
x^e over $\mathbb{Z}$	$\mathcal{O}(\log e)$	$\geq eL/w$ (writing the output)	$\geq eL$
add, as a circuit	—	—	depth $\geq 1 + \log_2 L$ (GIN-PROP-059)

Two rows deserve emphasis. The last-but-one shows unit cost *undercounting by an exponential factor*: $\mathcal{O}(\log e)$ multiplications produce a number with about eL bits, which no machine can even write in fewer than eL bit steps. The gcd row shows that "the number of steps" is not a cost in any model until the cost of a step is fixed.

36.3 The pathology of unit-cost multiplication

COUNTEREXAMPLE 36.2

(Unit-cost multiplication computes doubly exponential numbers in linear time)

COUNTEREXAMPLE GIN-NEG-025

Starting from 2, k squarings compute 2^{2^k}, a number with $2^k + 1$ bits. Under unit cost this takes k steps; in any model where a step reads and writes a bounded number of bits it takes at least 2^k steps. A model in which multiplication of unbounded integers costs 1 can therefore "compute" objects whose mere description is exponentially longer than the computation, which is why the word RAM bounds the word size by $\mathcal{O}(\log n)$ [Cormen et al. 2009].

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import bigint as B
from ginsdk.cost import Cost

x = B.from_int(2)
for k in range(1, 13):
    c = Cost()
    x = B.mul_school(x, x, c)
    print(f"square #{k:2d}: unit cost 1, value 2^(2^{k}) has {B.to_int(x).bit_length():5d} bits, limb
operations for this square: {B.word_ops(c)}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
square # 1: unit cost 1, value 2^(2^1) has     3 bits, limb operations for this square: 2
square # 2: unit cost 1, value 2^(2^2) has     5 bits, limb operations for this square: 2
square # 3: unit cost 1, value 2^(2^3) has     9 bits, limb operations for this square: 2
square # 4: unit cost 1, value 2^(2^4) has    17 bits, limb operations for this square: 2
square # 5: unit cost 1, value 2^(2^5) has    33 bits, limb operations for this square: 2
square # 6: unit cost 1, value 2^(2^6) has    65 bits, limb operations for this square: 8
square # 7: unit cost 1, value 2^(2^7) has   129 bits, limb operations for this square: 18
square # 8: unit cost 1, value 2^(2^8) has   257 bits, limb operations for this square: 50
square # 9: unit cost 1, value 2^(2^9) has   513 bits, limb operations for this square: 162
square #10: unit cost 1, value 2^(2^10) has  1025 bits, limb operations for this square: 578
```

```
square #11: unit cost 1, value 2^(2^11) has  2049 bits, limb operations for this square: 2178
square #12: unit cost 1, value 2^(2^12) has  4097 bits, limb operations for this square: 8450
```

Each squaring costs one unit. The limb work is constant while the value fits in one 32-bit limb, and then grows by a factor approaching 4 per step, because the operand doubles in length and schoolbook multiplication is quadratic. After twelve unit steps the value has 4097 bits.

36.4 Floating point: unit cost of a different operation

Floating-point arithmetic restores unit cost honestly by changing the *operation*: a binary64 multiplication costs one instruction because it computes the product rounded to 53 significant bits, not the product (Chapter 25; [Goldberg 1991]). A cost statement "multiplication is $\mathcal{O}(1)$" about floating point is therefore a statement about $\mathrm{fl}(x \cdot y)$, and an algorithm analysed this way must also be analysed for rounding error. The cost model and the semantic layer cannot be separated: cheaper here means *different*.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate

for e in ["9007199254740992 + 1", "(10000000000000000 + 1) - 10000000000000000", "0.1 * 3", "1 / 3 * 3"]:
    print(f"{e:44s}  Q: {evaluate(e, 'Q').display:18s} binary64: {evaluate(e, 'binary64').display or evaluate(e, 'binary64').status}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
9007199254740992 + 1                         Q: 9007199254740993   binary64: 9007199254740992
(10000000000000000 + 1) - 10000000000000000  Q: 1                  binary64: 0
0.1 * 3                                      Q: 3/10               binary64: 0.30000000000000004
1 / 3 * 3                                    Q: 1                  binary64: 1
```

The second line loses the 1 entirely: $10^{16} + 1$ is not a binary64 number, so it rounds to 10^{16} before the subtraction. The last line shows the opposite accident — $\mathrm{fl}(\mathrm{fl}(1/3) \cdot 3)$ rounds back to exactly 1 — and is no evidence of exactness: correct answers produced by cancelling rounding errors are still produced by a different operation.

36.5 Translating between models

Two rules suffice for every translation in this book:

1. **Word RAM to bit complexity**: multiply by the cost of a word operation — $\mathcal{O}(w)$ for addition, $\mathcal{O}(w^2)$ for schoolbook word multiplication, or $\mathcal{O}(1)$ if words are treated as hardware.
2. **Unit cost to word RAM**: valid only when every intermediate value fits in $\mathcal{O}(1)$ words. The operation counts of Chapters 33–35 (division steps, squarings) are unit-cost counts; their word-RAM costs require the operand lengths, which is why each of those chapters states both.

The translation fails in exactly the situations the earlier chapters measured: operands that grow (exponentiation over $\mathbb{Z}$, Peano arithmetic in Chapter 4, the doubly exponential squarings above); and operations whose unit-cost version computes something else (floating point, wraparound integers).

LABORATORY (WEBSITE EDITION)

Text alternative: choose an operation and a bit length; the widget plots the counted 32-bit limb operations from the book's experiment GIN-EXP-008 against the bit length on logarithmic axes, with the fitted slope, and states the model ("word RAM, w = 32") under the chart.

36.6 The reporting rule

Every cost in this book is reported with (i) the model, (ii) the growing parameter and what is held fixed, (iii) whether it is a bound, an exact count or a measurement. "Addition is $\mathcal{O}(1)$" is reported as "one instruction for operands of at most 64 bits"; "addition is $\Theta(L)$" as "bit operations, L the operand length". The rule is not decoration: Chapter 4 showed a factor-2^k difference hiding behind "addition takes one step", and the pathology above an exponential one.

EXERCISE 36.1

Under unit cost, an $n \times n$ integer matrix product takes n^3 operations. Give its word-RAM cost if all entries have L bits, and explain why entries of the k-th power of the matrix may not have L bits.

▶ Solution to Exercise 36.1

EXERCISE 36.2

Restate the cost of the sieve of Eratosthenes (Chapter 35) in bit complexity, as a function of the number of digits of N.

▶ Solution to Exercise 36.2

SUMMARY

- Unit cost, word RAM and bit complexity differ in what grows; every cost names its model and its growing parameter.
- Unit cost is valid only for bounded operands; unbounded unit-cost multiplication computes 2^{2^k} in k steps, a number of $2^k + 1$ bits (executed).
- The word RAM ($w \geq \log n$) is the model of the SDK's counted limb operations; bit complexity is the model of the book's theorems.
- Floating point has unit cost because it computes a rounded result: cost and semantics change together.
- Step counts (divisions, squarings) become costs only with the operand lengths.

Chapter 37

Measured Complexity: What the Experiments Show and What They Cannot

> *A slope fitted to nine points is a description of nine points.*

LEARNING OBJECTIVES

- Read the counted-operation data of GIN-EXP-008 as the word-RAM costs of Chapter 36, and fit exponents by least squares on logarithmic axes.
- Quantify how much a fitted exponent depends on the fitting range and on the random operands.
- Explain why wall-clock time on CPython is secondary evidence, and what it does and does not confirm.
- Interpret the numeral-evaluation experiment (GIN-EXP-011) as a statement about grammar shape and cost model, without inflating it into a speedup claim.
- State what measurements cannot establish.

37.1 What was measured

Every cost experiment in this book counts operations rather than timing them (GIN-D-003). GIN-EXP-008 draws random L-bit operands for $L = 64, 128, \ldots, 16384$ and counts the 32-bit limb operations (add or subtract with carry, limb product, two-by-one quotient estimate, limb comparison) performed by the SDK's implementations of Chapter 15. The counts are exact and reproducible from the seed; the experiment's re-run during the 2026-10 audit reproduced every counted field (Appendix B).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
rows = json.load(open(os.path.join(ROOT, "experiments", "results", "exp008.json")))["data"]["rows"]
```

```
print(f"{'bits':>6} {'add':>6} {'mul school':>11} {'Karatsuba':>10} {'divmod':>9} {'gcd':>9}")
for r in rows:
    print(f"{r['bits']:6d} {r['add']:6d} {r['mul_schoolbook']:11d} {r['mul_karatsuba']:10d}
{r['divmod_2L_by_L']:9d} {str(r.get('gcd_euclid', '-')):>9}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
  bits    add  mul school  Karatsuba    divmod       gcd
    64      2           8          8        16       130
   128      4          32         32        50       476
   256      8         128        128       155      1582
   512     16         512        512       568      6085
  1024     32        2048       1796      2167     22000
  2048     64        8192       5838      8399     81034
  4096    128       32768      18409     33200    326516
  8192    256      131072      56771    131928         -
 16384    512      524288     172598    526069         -
```

37.2 Fitting exponents, and how much to trust them

If a cost is cL^{α}, then $\log(\text{cost}) = \log c + \alpha \log L$, and α is the slope on logarithmic axes. Fitting by least squares over different ranges shows how stable the estimate is:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, math, os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
rows = json.load(open(os.path.join(ROOT, "experiments", "results", "exp008.json")))["data"]["rows"]

def slope(points):
    xs, ys = [math.log2(x) for x, _ in points], [math.log2(y) for _, y in points]
    mx, my = sum(xs) / len(xs), sum(ys) / len(ys)
    return sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sum((x - mx) ** 2 for x in xs)

theory = {"add": "1", "mul_schoolbook": "2", "mul_karatsuba": "log2 3 = 1.585", "divmod_2L_by_L": "2",
"gcd_euclid": "2"}
for key, th in theory.items():
    pts = [(r["bits"], r[key]) for r in rows if r.get(key)]
    print(f"{key:15s} all points {slope(pts):.3f}   last four {slope(pts[-4:]):.3f}   first four
{slope(pts[:4]):.3f}   theory {th}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
add             all points 1.000   last four 1.000   first four 1.000   theory 1
mul_schoolbook  all points 2.000   last four 2.000   first four 2.000   theory 2
mul_karatsuba   all points 1.797   last four 1.628   first four 2.000   theory log2 3 = 1.585
```

```
divmod_2L_by_L  all points 1.891   last four 1.990   first four 1.708   theory 2
gcd_euclid      all points 1.875   last four 1.912   first four 1.838   theory 2
```

EMPIRICAL OBSERVATION 37.1 (Fitted exponents depend on the range)

EMPIRICAL OBSERVATION GIN-OBS-022

Least-squares exponents fitted to the counted limb operations of GIN-EXP-008 agree with the classical exponents for addition (1) and schoolbook multiplication (2) on every range. For Karatsuba multiplication, long division and Euclid's gcd they depend on the range: small operands fall below the asymptotic regime (Karatsuba falls back to schoolbook below 16 limbs, so its first points have slope 2; division and gcd carry linear terms that dominate at small L). Only the largest-range fits approach the classical values.

The observation is a reason for caution, not a refutation: a fitted exponent describes the measured range. "Karatsuba is $\Theta(L^{1.585})$" is a theorem (GIN-PROP-063); "the SDK's Karatsuba has slope about 1.6 between 4096 and 16384 bits" is a measurement; the second supports the first and does not prove it.

37.3 The spread over random operands

Addition and schoolbook multiplication cost the same for all operands of a given length; Euclid's algorithm does not — the number of steps depends on the quotients. Repeating the measurement over many random pairs gives the spread:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import random, statistics
from ginsdk import bigint as B
from ginsdk.cost import Cost

for L in (256, 1024):
    rng = random.Random(L)
    ops = []
    for _ in range(40):
        a, b = rng.getrandbits(L) | 1 << (L - 1), rng.getrandbits(L) | 1 << (L - 1)
        c = Cost()
        B.gcd_euclid(B.from_int(a), B.from_int(b), c)
        ops.append(B.word_ops(c))
    print(f"L = {L:5d}: 40 random pairs, limb operations min {min(ops)}, median {statistics.median(ops):.0f},
max {max(ops)}, "
          f"relative spread {(max(ops) - min(ops)) / statistics.median(ops):.1%}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
L =   256: 40 random pairs, limb operations min 1500, median 1638, max 1844, relative spread 21.0%
L =  1024: 40 random pairs, limb operations min 20769, median 21920, max 23487, relative spread 12.4%
```

The spread (at most about a fifth of the median) is small compared with the factor of about 13 between the two medians, so the quadratic growth is not an artefact of the particular operands drawn in GIN-EXP-008 — but a single pair per length, as in that experiment, is a sample, and is reported as one.

37.4 Wall-clock time is secondary

GIN-EXP-008 also timed CPython's built-in `int` multiplication. Those numbers depend on the machine, the interpreter and its internal thresholds (CPython switches from schoolbook to Karatsuba multiplication above a fixed size), and on noise:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
# wall-clock: these digits change at every re-run of GIN-EXP-008; the build checks the shape only
import json, math, os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
rows = json.load(open(os.path.join(ROOT, "experiments", "results", "exp008.json")))["data"]["rows"]
prev = None
for r in rows:
    t = r["cpython_mul_s"]
    note = f"slope from previous {math.log2(t / prev):+.2f}" if prev else ""
    print(f"{r['bits']:6d} bits: {t * 1e6:9.3f} µs   {note}")
    prev = t
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
    64 bits:     0.061 µs
   128 bits:     0.074 µs   slope from previous +0.28
   256 bits:     0.123 µs   slope from previous +0.72
   512 bits:     0.314 µs   slope from previous +1.36
  1024 bits:     1.042 µs   slope from previous +1.73
  2048 bits:     4.447 µs   slope from previous +2.09
  4096 bits:    13.425 µs   slope from previous +1.59
  8192 bits:    41.047 µs   slope from previous +1.61
 16384 bits:   124.652 µs   slope from previous +1.60
```

At small sizes the time is dominated by interpreter overhead, and the growth per doubling is far below any multiplication exponent. At large sizes it varies from one recording to the next: in the

recording printed here it lies between about 1.6 and 2.1, and an earlier recording of the same experiment on the same host gave values between about 1.1 and 2.2, with a slope below zero at 128 bits. A single noisy measurement per size cannot separate the algorithm from caches, allocation and timer resolution. These numbers are rewritten at every re-run, are not reproducible to the digit — the build checks this block's shape, not its digits — and the book never uses them to support an asymptotic claim (GIN-D-003).

37.5 Shape changes depth, the cost model decides work

GIN-EXP-011 converts decimal numerals of k digits to binary in two shapes — Horner's right-linear evaluation and a balanced tree of compositions — with schoolbook or Karatsuba multiplication in the balanced shape:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
d = json.load(open(os.path.join(ROOT, "experiments", "results", "exp011.json")))["data"]
for r in d["rows"][-4:]:
    print(f"{r['digits']:5d} digits: Horner {r['horner']:9d}   balanced+school {r['balanced_schoolbook']:8d}
"
          f"balanced+Karatsuba {r['balanced_karatsuba']:7d}   depth {r['depth_horner']} vs
{r['depth_balanced']}")
print("last slopes:", {k: v[-1] for k, v in d["loglog_slopes"].items()})
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
 1024 digits: Horner    159519   balanced+school    29177   balanced+Karatsuba   24659   depth 1024 vs 10
 2048 digits: Horner    633291   balanced+school   104333   balanced+Karatsuba   73694   depth 2048 vs 11
 4096 digits: Horner   2529644   balanced+school   390586   balanced+Karatsuba  221901   depth 4096 vs 12
 8192 digits: Horner  10105494   balanced+school  1507887   balanced+Karatsuba  669698   depth 8192 vs 13
last slopes: {'horner': 1.998, 'balanced_schoolbook': 1.949, 'balanced_karatsuba': 1.594}
```

The balanced shape reduces the *depth* from k to $\lceil \log_2 k \rceil$ in every case. Its *work* has the same asymptotic exponent as Horner's when multiplication is schoolbook (both slopes tend to 2; the balanced shape is cheaper by a constant factor), and a smaller exponent only when it is combined with Karatsuba multiplication (GIN-OBS-013). This is the precise content of the "re-association" mechanism conjectured in GIN-CONJ-001: re-association buys parallel depth unconditionally and sequential work only together with a cost model that rewards balanced operands. It is a classical radix-conversion technique [Knuth 1997; Brent & Zimmermann 2010], not a new algorithm.

37.6 What measurements cannot establish

- **Asymptotic behaviour.** Every experiment covers a finite range; fitted exponents describe that range (Empirical observation 37.1).
- **Lower bounds.** A measured cost of an implementation bounds that implementation, not the problem; the lower bounds in this book (Chapters 10 and 22) are proofs.
- **Novelty.** Agreement with classical exponents confirms the implementations; it does not suggest a new algorithm, and none is claimed.
- **Other machines.** Counted operations transfer between machines; wall-clock times do not.

EXERCISE 37.1

Using the table of §37.2, predict the number of limb operations of schoolbook multiplication at $L = 32768$ bits, and explain why the same extrapolation for Karatsuba from its first four points would be wrong.

▶ Solution to Exercise 37.1

EXERCISE 37.2

Name three reasons why a wall-clock time measured in CPython could decrease when the operand length increases, and why counted operations cannot show that behaviour.

SUMMARY

- The cost experiments count word-RAM operations; the counts are exact and reproduced by the audit.
- Fitted exponents match the classical ones for addition and schoolbook multiplication on every range; for Karatsuba, division and gcd only on the largest ranges (Empirical observation 37.1).
- The spread over random operands is small relative to the growth; wall-clock times are secondary, host-specific and non-monotone at small sizes.
- Balanced evaluation reduces depth always and work only with subquadratic multiplication — a classical technique, measured, not a speedup claim.
- Measurements describe ranges and implementations; they do not prove asymptotics, lower bounds or novelty.

PART XV

Numerical Grammar and Computational Grammar Theory

Exact correspondences, useful analogies and non-equivalences between this book and Computational Grammar Theory.

Chapter 38

Numerical Grammar and Computational Grammar Theory

> *A correspondence is a claim; an analogy is a mood.*

LEARNING OBJECTIVES

- State which definitions of Computational Grammar Theory (CGT) apply to numbers without change, and check the exact correspondences by computation.
- Identify the CGT notions that arithmetic refines: the number of evaluation layers, converse moves, and the growing parameter of cost.
- State the non-equivalences precisely, including a CGT property that digit grammars do *not* have.
- Distinguish exact correspondences from analogies, and know which proposals to CGT are only proposals.

38.1 Two publications

Computational Grammar Theory, the research program of Abed Kadan, asks when the grammar of a computational structure — its moves, admissible operation sequences, semantics and costs — becomes part of the machinery that computes over it [Kadaan (CGT) 2026]. This book was written afterwards and is, conceptually, its arithmetic prequel: numbers are the simplest structures on which CGT's questions can be asked. The two are independent. Each has its own ledger and software; no result here depends on a CGT result; CGT was read, never modified, and its identifiers below link to the separate CGT publication (GIN-D-002).

This chapter has three sections, and an item appears in only one of them: **exact correspondences** (a CGT definition instantiated on numbers, with a proof or check), **refinements** (a CGT notion that numbers force to change), and **non-equivalences** (where an apparent correspondence fails). Analogies that are none of these are not listed.

38.2 Exact correspondences

Numerals are operation expressions. CGT defines an operational grammar as moves with a relational semantics on a structure and an admissibility language (CGT-DEF-003). A digit grammar with base b and digit set D is one: the structure is $\mathbb{Z}$, the moves are $x \mapsto bx + d$, and a numeral is a word over the moves whose value is its denotation at 0 (Chapter 6). The CGT distinction between syntax and semantics *is* the distinction between numeral and number.

Canonical numerals are grammatical addresses. CGT's address of an element is the shortlex-least word reaching it from the root (CGT-DEF-006). For a standard digit grammar rooted at 0 the address of n is its numeral without leading zeros, and the address of 0 is the empty word (GIN-PROP-002).

CGT's heap numeration is bijective binary, shifted. CGT studies the heap numbering of binary trees — root 1, children $2x$ and $2x + 1$ — as its main example of arithmetic addressing (CGT-THM-003, CGT-THM-006). The map $x \mapsto x + 1$ conjugates the bijective base-2 grammar (digits 1, 2, root 0) to it (GIN-PROP-006):

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numerals as N

pairs = [(n, N.bijective(2).render(N.bijective(2).address(n)), N.heap_word(n + 1)) for n in range(1, 9)]
for n, bij, heap in pairs:
    print(f"n = {n}: bijective binary {bij:4s}  heap address of node {n + 1}: {heap}")
print("conjugacy holds for all n < 100000:", all(N.bijective2_as_heap(n) == N.heap_word(n + 1) for n in
range(100000)))
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
n = 1: bijective binary 1     heap address of node 2: L
n = 2: bijective binary 2     heap address of node 3: R
n = 3: bijective binary 11    heap address of node 4: LL
n = 4: bijective binary 12    heap address of node 5: LR
n = 5: bijective binary 21    heap address of node 6: RL
n = 6: bijective binary 22    heap address of node 7: RR
n = 7: bijective binary 111   heap address of node 8: LLL
n = 8: bijective binary 112   heap address of node 9: LLR
conjugacy holds for all n < 100000: True
```

The carry automaton is arithmetically compilable. CGT calls a grammar *arithmetically compilable* when every word of moves compiles to one element of a class of $\mathcal{O}(1)$-word operations closed under composition (CGT-DEF-011). The carry automaton of binary addition has the three-element transition monoid $\{\mathsf{K}, \mathsf{P}, \mathsf{G}\}$ (GIN-PROP-020): every word of digit pairs compiles to one of three elements, and parallel prefix adders compute those compilations (Chapter 11).

Same semantics, different shape, different cost. CGT's theorem that grammar shape trades work against parallel depth (CGT-THM-004) has two exact arithmetic instances: ripple-carry versus prefix adders (GIN-PROP-021, Chapter 22) and Horner versus balanced numeral evaluation (Chapter 37).

38.3 Refinements

Four layers, not three. CGT separates syntax, semantics and execution (CGT-PROP-002). Arithmetic needs a fourth, *typing*, between syntax and semantics: the literal `0.5` in $\mathbb{Z}$ and the literal `2147483648` in a 32-bit integer domain are syntactically well formed and fail before any operation is applied. The four layers are independent (GIN-PROP-056).

Converse moves. CGT's relational semantics allows a move to have no result or several. Arithmetic singles out a systematic source of both: every inverse operation is the converse of a forward one, and its admissibility is decided by the size of a solution set — none, one or many (Chapter 9; GIN-THM-007). CGT has no separate notion of a converse move; the book proposes one (below), and does not claim CGT needs it.

The growing parameter of cost. CGT's cost profiles use the word RAM with $w = \Theta(\log n)$ for structure size n (CGT-DEF-005). For arithmetic the growing parameter is the *bit length of the operands*, and unit cost is admissible only while operands fit in a word (Chapter 36). The CGT bound "arithmetic addressing is $\mathcal{O}(1)$ only while the address fits in a word" (CGT-THM-003) is the arithmetic statement that L-bit numbers cost $\lceil L/w \rceil$ words.

38.4 Non-equivalences

Digit grammars are not arithmetically compilable. A word of digit moves compiles to the affine map $x \mapsto b^k x + c$ (GIN-PROP-005) — one element, closed under composition, as CGT-DEF-011 requires — but the element is not $\mathcal{O}(1)$ words: its coefficients grow with the word.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import numerals as N

g = N.DECIMAL
for k in (4, 16, 64, 256):
    f = g.compile(g.parse("7" * k))
    print(f"word of {k:3d} digits compiles to x → a*x + c with a of {f.a.bit_length():4d} bits and c of
{f.c.bit_length():4d} bits")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
word of   4 digits compiles to x → a*x + c with a of   14 bits and c of   13 bits
word of  16 digits compiles to x → a*x + c with a of   54 bits and c of   53 bits
word of  64 digits compiles to x → a*x + c with a of  213 bits and c of  213 bits
word of 256 digits compiles to x → a*x + c with a of  851 bits and c of  851 bits
```

The carry monoid has three elements; the affine monoid of a digit grammar is infinite. That difference is the difference between an operation a finite automaton can perform on numerals (addition) and one it cannot (multiplication; GIN-THM-005, [Büchi 1960; Cobham 1969]). It is the sharpest single statement this book has found connecting a CGT property to a classical boundary.

No arithmetic counterpart. CGT's material on PageRank, retrieval and AI-system grammars has no counterpart here beyond fixed-point iteration, and CGT's boundary result that grammars cannot move problems across complexity classes (CGT-PROP-006) has no arithmetic refinement in this book: nothing here bears on the complexity of factoring or discrete logarithms (Chapters 31 and 35).

Analogies that are not correspondences. The book sometimes uses CGT words informally — "the exponent is a program" (Chapter 34), "long division is a transition system" — and in those places the precise statement is the proposition cited next to the phrase, not the phrase. Such phrases are explanatory; the correspondences in §38.2 are the claims.

38.5 Proposals to CGT

These are proposals for the CGT researcher, not changes made to CGT:

1. add a notion of *converse move* and the none/one/many classification of its admissibility;
2. add a typing layer to the three-layer model;
3. consider re-association as a sixth mechanism of computational advantage, alongside CGT's five (CGT-DEF-009); whether it is distinct from the grammar-shape effect is open (GIN-CONJ-001, GIN-OPEN-004);
4. record the heap conjugacy (GIN-PROP-006) next to CGT's heap numeration.

EXERCISE 38.1

For the balanced-ternary digit grammar (digits $-1, 0, 1$), what is the CGT address of -4, and why does the address of every integer exist?

▶ Solution to Exercise 38.1

EXERCISE 38.2

Show that the words over the digit moves of base b that compile to maps with $a = b^k$ for a *fixed* k form a finite set, and explain why that does not make the grammar arithmetically compilable.

SUMMARY

- Exact correspondences: numerals are CGT operation expressions; canonical numerals are CGT addresses; CGT's heap numeration is bijective binary shifted by one (checked for $n < 100000$); the carry automaton is arithmetically compilable; adders and numeral evaluation instantiate CGT's shape theorem.
- Refinements: a fourth (typing) layer; converse moves with none/one/many admissibility; bit length as the growing parameter of cost.
- Non-equivalences: digit grammars compile to single affine maps whose size grows with the word — finite versus infinite transition monoids separate addition from multiplication; CGT's PageRank and complexity-class material has no arithmetic counterpart here.
- Proposals to CGT are proposals; CGT is cited, not modified.

PART XVI

Programming and AI Interfaces

Operation contracts as an interface between a request and an execution system, measured against real toolchains; and the SDK.

Chapter 39

Operation Contracts as an Interface Between Request and Execution

> *Before asking a machine for an answer, ask it which question it will answer.*

LEARNING OBJECTIVES

- Read an operation contract: input grammar, domain, admissibility, outcome classes, signals and cost model of one operator in one domain.
- Use the machine–mathematics relation to compare an execution with the exact mathematical answer.
- Interpret GIN-EXP-013: what "the contract model predicts real execution" means, and what it does not.
- State the experiment's baselines precisely, including the one that was not run.

39.1 The interface problem

A request such as "compute $(-2147483647 - 1)/-1$" passes from whoever writes it — a programmer, a user, an AI system generating code — to whatever executes it. The two sides can disagree about every layer of Chapter 2: the writer may mean the integer 2147483648, while an x86-64 `idiv` instruction traps, Java returns -2147483648, and ISO C gives the program no meaning at all (Chapter 24). An **interface** between request and execution is a description, precise enough to compute with, of which question the executing system will actually answer.

DEFINITION 39.1 (Operation contract) DEFINITION GIN-DEF-080

An **operation contract** for an operator $\circ$ in a domain D records: the input grammar (which literals and values belong to D); the admissibility condition of $a \circ b$; the outcome classes (value, special datum, no solution, non-unique, overflow, trap, exception, undefined behaviour); the signal raised for each failure; the representation of results; and the cost model. The SDK publishes the contracts of all its domains as a JSON document (schema `gin-contracts/1`).

DEFINITION 39.2 (Machine–mathematics relation) DEFINITION GIN-DEF-081

For an expression e evaluated in a machine domain and in the exact domain it implements ($\mathbb{Z}$ for machine integers, $\mathbb{Q}$ for floating point and Python), the **relation** classifies the pair of outcomes as one of: *agrees*; *rounded*; *wrapped*; *totalized* (a value where mathematics has none, as AArch64 returning 0 for $1/0$); *selected* (a quotient with remainder where the exact quotient does not exist); *different-selection* (the machine selects a different answer from the one the reference selects); *special-datum*; *overflowed-to-special*; *signalled* (the machine refuses where mathematics has a value); *undefined*; *differs*.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from ginsdk import contracts

for e, d in [("(-2147483647 - 1) / -1", "int32 x86-64"), ("(-2147483647 - 1) / -1", "int32 Java"),
             ("1 / 0", "int32 AArch64"), ("2147483647 + 1", "int32 C"), ("0.1 + 0.2", "binary64")]:
    r = contracts.inspect(e, d)
    print(f"{e:24s} {d:14s} machine: {r['execution']['status']:10s} {str(r['execution']['value'] or ''):12s} "
          f"mathematics: {str(r['mathematics']['value'] or r['mathematics']['status']):12s} → {r['relation']
['kind']}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
(-2147483647 - 1) / -1   int32 x86-64   machine: trap                    mathematics: 2147483648   → signalled
(-2147483647 - 1) / -1   int32 Java     machine: value      -2147483648  mathematics: 2147483648   → wrapped
1 / 0                    int32 AArch64  machine: value      0            mathematics: no-solution  → totalized
2147483647 + 1           int32 C        machine: undefined               mathematics: 2147483648   → undefined
0.1 + 0.2                binary64       machine: value      0.30000000000000004 mathematics: 3/10         →
rounded
```

The same tuple — expression, domain, outcome, relation — is what the command-line tool prints (`python3 -m ginsdk inspect` , Chapter 40) and what the browser laboratory shows.

LABORATORY (WEBSITE EDITION)

Text alternative: type one integer expression; the widget evaluates it in every machine and language domain of the SDK side by side — x86-64, AArch64, RISC-V, ISO C, Java, Rust, Go, Python, JavaScript — and shows for each the outcome class, the value, and its relation to the exact integer answer.

39.2 Does the contract predict real execution?

Chapter 1 promised one measurement in which the framework demonstrably improves a prediction. GIN-EXP-013 executed 845 single operations — 13 operands including 0, ± 1, the 32-bit extremes and squares near overflow, with the operators $+, -, \times, /, \%$ — at run time in C (gcc, x86-64), Java, Go, Rust (debug build), JavaScript (BigInt and Number) and Python, and predicted each outcome three ways:

- **model** — the SDK's contract model for that toolchain's domain;
- **naive-py** — evaluate the expression in Python;
- **naive-c** — unbounded integers with C's truncating division.

A prediction scores a hit when it gets both the outcome class (value or refusal) and the value right.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import json, os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
langs = json.load(open(os.path.join(ROOT, "experiments", "results", "exp013.json")))["data"]["languages"]
print(f"{'toolchain':22s} {'model':>17s} {'naive-py':>9s} {'naive-c':>8s}")
for name, r in langs.items():
    model = [(k, v) for k, v in r.items() if k.startswith("model[")]
    m = " / ".join(f"{v['hit']}" + (f" (+{v['abstain']} abst.)" if v["abstain"] else "") + (f" (+{v['not-modelled']} n/m)" if v["not-modelled"] else "") for k, v in model)
    print(f"{name:22s} {m:>17s} {r['naive-py']['hit']:9d} {r['naive-c']['hit']:8d}   of 845")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
toolchain                          model  naive-py  naive-c
C (gcc -O0, x86-64)    731 (+114 abst.) / 845       612      757   of 845
Java                                 845       613      758   of 845
Go                                   845       613      758   of 845
Rust (debug)                         845       612      757   of 845
JavaScript BigInt                    845       700      845   of 845
JavaScript Number          676 (+169 n/m)       767      718   of 845
Python                               845       845      700   of 845
```

EMPIRICAL OBSERVATION 39.3

(Contract models predict real execution; naive models do not)

EMPIRICAL OBSERVATION GIN-OBS-017

On the 845 operations of GIN-EXP-013, the contract models predicted outcome class and value in every case they covered, for every toolchain. For ISO C the model *abstained* on the 114 operations whose behaviour is undefined, and was right on the rest; for JavaScript Number the operator `%` was not modelled. "Evaluate in Python" was right on 612–767 of the 845 on the six non-Python toolchains, and "unbounded integers with C division" on 700–758 except JavaScript BigInt. The naive predictors fail on overflow, negative division, true versus integer division, and zero divisors.

CLAIM BOUNDARY

What this measurement shows, and what it does not.

- It shows that the SDK's domain models *cover* the behaviour of these seven toolchains on these operands: the model is an accurate executable summary of their specifications.
- It does **not** show that the model was learned, discovered or predicted independently: it was built from the same language and instruction-set specifications these toolchains implement. A perfect score validates coverage, not insight.
- It is one host, one version of each toolchain, single operations, and a fixed operand set chosen to include boundaries. Compound expressions, optimizer effects (Chapter 23 showed one) and other platforms are not covered.
- **Baselines.** The two naive predictors are the baselines that were run. A *tool-only* baseline — executing the expression in the target toolchain itself — is the ground truth, not a competitor. An **AI-only baseline** — asking a language model to predict each outcome without tools — was **not run**: no model API was available to the experiment, and no number for it is reported or implied (GIN-OPEN-013).

39.3 Why an interface needs all four layers

The naive predictors fail exactly where a single layer is missing from their picture. "Evaluate in Python" has the right *semantics* for integers and the wrong *execution* model for 32-bit machines; "unbounded integers with C division" has the right *division rounding* and no *representation boundary*. The contract model is right because it records every layer separately — typing (which literals exist), semantics (the converse problem), execution (wrap, trap, undefined) — and reports

which layer decided the outcome. An interface that reports only a value, or only "error", loses exactly the information that distinguishes the toolchains.

For AI systems that write or check code, the practical consequence is modest and concrete: an arithmetic request carries an implicit domain, and the domain changes the answer. A generator or checker that consults an explicit contract can state which answer a given toolchain will produce; one that reasons "mathematically" will be wrong in the cases measured above. Whether contracts measurably improve AI-generated code is a separate experiment that has not been done (GIN-OPEN-013).

EXERCISE 39.1

Predict, without running anything, the relation of Definition 39.2 for `-7 // 2` in Python, `-7 / 2` in Java, and `-7 / 2` in exact $\mathbb{Q}$; then check with `contracts.inspect`.

▶ Solution to Exercise 39.1

EXERCISE 39.2

Design an experiment that would test whether giving an AI code generator the SDK's contract document reduces arithmetic bugs. State the outcome measure, the control condition, and what result would falsify the hypothesis.

SUMMARY

- An operation contract states input grammar, admissibility, outcome classes, signals, representation and cost of one operator in one domain; the machine–mathematics relation compares an execution with the exact answer.
- On 845 operations in seven toolchains the contract models were right on every covered case; naive predictors were right on 612–767 (Python) and 700–758 (C-style) of 845 (GIN-OBS-017).
- The model was built from the toolchains' specifications: the measurement validates coverage, not independent prediction.
- The AI-only baseline was not run; no claim about AI systems' performance is made.

Chapter 40

The SDK as Executable Semantics

> *A definition you can run is a definition you can test.*

LEARNING OBJECTIVES

- Know what each module of `ginsdk` implements and which chapter defines it.
- Evaluate, compare and inspect expressions from Python and from the command line.
- Read the `Outcome` and `gin-inspect/1` records.
- Know how the SDK is tested and cross-checked, and list its domain limitations before relying on it.

40.1 Why the book has software

Every definition in this book that can be executed has been: the digit grammars of Chapter 6, the converse problems of Chapter 9, the IEEE 754 formats of Chapter 25, the machine division models of Chapter 24, the circuits of Chapter 22, the limb arithmetic of Chapter 15, the number-theoretic algorithms of Chapters 30–35. The package `ginsdk` collects them. It has two jobs: to make the book's definitions precise enough that a computer can disagree with them, and to produce every printed output in the book — 92 executed examples in the first 32 chapters were reproduced by the audit, and each chapter since adds more (GIN-D-016).

It is written in Python using only the standard library, so that it runs wherever Python $\geq$ 3.10 does, without installing anything.

MODULE	IMPLEMENTS	CHAPTERS
`numerals`	digit grammars, canonical numerals, conversion, compilation to affine maps	5–7
`peano`	successor terms, primitive recursion with rule counts, segments	3–4
`converse`	solution sets of converse problems; fractions as pairs	9, 16–18

MODULE	IMPLEMENTS	CHAPTERS
`expr`	the expression language and evaluation in 20+ domains with layered outcomes	2, 8, 23
`contracts`	operation contracts, `inspect`, `verify` (schemas `gin-contracts/1`, `gin-inspect/1`)	8, 39
`ieee`	IEEE 754 binary formats by exact rational rounding, with all five flags	25
`machine`	integer division and overflow semantics of x86-64, AArch64, RISC-V, C, Java, Rust, Go, Python, JavaScript	24, 26
`circuits`	gate-level adders, multiplier, divider; the carry monoid	11, 22
`bigint`	limb arithmetic with counted word operations	15, 36–37
`numbertheory`	Euclid, Bézout, CRT, exponentiation, primality, factoring, arithmetic functions	30–35
`totalized`	totalized division systems and their law census	18
`cost`	explicit cost counters	36

40.2 Evaluating, comparing, inspecting

Three entry points cover most uses: `evaluate` returns an `Outcome`; `compare` evaluates one expression in many domains; `contracts.inspect` adds the reference mathematics and the machine–mathematics relation of Chapter 39.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
from ginsdk import evaluate, compare

o = evaluate("1 / 0", "Q")
print(o.status, "|", o.layer, "|", o.reason)
print(o.equation, "|", o.solutions)
for out in compare("7 / -2", ["Z", "Q", "binary64", "Python", "int32 Java"]):
    print(f"  {out.domain:10s} {out.status:12s} {out.display}")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
no-solution | semantics | 0 · c = 0 for every c, so 0 · c = 1 would require 0 = 1
0 · c = 1 | ∅
  Z          no-solution
  Q          value        -7/2
  binary64   value        -3.5
  Python     value        -3.5
  int32 Java value        -3
```

The same operations are available from a shell, which is how the SDK is meant to be used by other programs — including AI systems — that need to know which question an execution system will answer:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import subprocess, sys

out = subprocess.run([sys.executable, "-m", "ginsdk", "eval", "2147483647 + 1", "--domain", "int32 C"],
capture_output=True, text=True).stdout
print(out.strip())
out = subprocess.run([sys.executable, "-m", "ginsdk", "inspect", "0 / 0", "--domain", "binary64"],
capture_output=True, text=True).stdout
import json
r = json.loads(out)
print(r["schema"], "| machine:", r["execution"]["status"], r["execution"]["value"], r["execution"]["flags"],
      "| mathematics:", r["mathematics"]["status"], "| relation:", r["relation"]["kind"])
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
2147483647 + 1  in int32 C: undefined (execution) — addition overflows 32 bits (exact result 2147483648); C17
6.5p5: signed overflow is undefined
gin-inspect/1 | machine: special NaN ['invalid'] | mathematics: non-unique | relation: special-datum
```

LABORATORY (WEBSITE EDITION)

Text alternative: type an expression and choose a domain; the widget shows the tokens, the parse tree, the four layers with the one that decided the outcome, the converse equation and its solution set for inverse operations, the step trace, the representation of the result and the cost estimate — computed in the browser by a JavaScript port of `ginsdk.expr` that is tested against fixtures exported from the SDK.

40.3 The records

An `Outcome` (Chapter 8; GIN-DEF-062) records the expression and domain, the **status** (value, special, no-solution, non-unique, not-in-domain, unsupported, overflow, trap, exception, undefined, syntax-error), the **layer** at which evaluation stopped, a **reason** in words, the **equation** and **solutions** of a failed converse problem, IEEE **flags**, **notes**, the **steps** with the rule each applied, the parse **tree**, the **representation** of the result, and a **cost** in a named unit. No outcome is a bare "error".

A `gin-inspect/1` record (Chapter 39) contains, in addition, the reference domain, a per-layer verdict, the mathematical outcome, and the relation between execution and mathematics. Both records are stable parts of the public API (version 1.x): fields may be added, not removed or renamed.

40.4 How the SDK is checked

Software that defines the semantics of a book needs independent checks; agreeing with itself proves nothing. The checks are of four kinds:

1. **Unit tests** — 74 tests at the time of the 2026-10 audit, 75 in this edition (`python3 -m unittest discover -s sdk/tests`), including exhaustive small-case checks of every circuit and every converse-problem formula.
2. **Independent oracles** — the IEEE simulator against the host processor's own arithmetic and exception flags on 80,600 cases (GIN-IMP-001); the machine models against real executions in seven toolchains (GIN-IMP-002, GIN-OBS-017) and against QEMU's emulation of AArch64 and RISC-V (GIN-IMP-008); the limb arithmetic against Python's integers.
3. **Theorem checks** — exhaustive small-case checks of every "proved here" statement (`experiments/check_theorems*.py`).
4. **The book itself** — every executed example must reproduce its printed output, and the browser laboratory must reproduce the SDK on the exported fixtures (`npm test`).

These checks found real defects, and the book records them rather than hiding them: exponents were once evaluated inside $\mathbb{Z}/n$ (GIN-NEG-013); signaling NaNs were once ignored (GIN-REJ-002); and during the writing of Chapter 39 the 32-bit division path was found not to flag the wraparound of $\text{INT_MIN}/-1$, which made the inspection relation call it a "selection" — fixed in both the SDK and the laboratory, with a regression test. Later, an experiment of the separate research lane *The Number* found that the SDK's tokenizer accepted any Unicode decimal digits (`٣٤` evaluated to 34) where the laboratory accepted only `0`–`9` ; the SDK now follows the laboratory (correction C-006 in the results ledger).

40.5 Where the models stop

CLAIM BOUNDARY

Limitations of `ginsdk` 1.0. The SDK is a reference, not a production numerical library.

- **IEEE 754**: binary formats with round-to-nearest-even only; NaN payloads and signs not modelled; no fused multiply–add, conversions beyond the basic ones, or decimal formats (GIN-OPEN-007).
- **Machines**: 32-bit integer models from specifications; measured on one x86-64 host, emulated for AArch64 and RISC-V, never run on physical AArch64 or RISC-V hardware (GIN-OPEN-012). Compound expressions and compiler optimizations are not modelled (the post-division zero test of Chapter 23 shows why that matters).
- **Exact reals**: irrational square roots are carried as flagged 40-digit decimal approximations; there is no exact real arithmetic.
- **Symbolic domain**: rational functions of one variable over $\mathbb{Q}$ only.
- **Performance**: limb arithmetic is written for counting, not speed; Python's built-in integers are far faster.
- **Cost estimates** in `evaluate` are schoolbook models, not measurements; the measurements are in `bigint` and Chapter 37.

EXERCISE 40.1

Sketch how you would add a domain for 16-bit two's-complement integers with saturating arithmetic: which class in `ginsdk.expr` you would instantiate, which overflow policy you would pass, and which test you would write first.

▶ Solution to Exercise 40.1

EXERCISE 40.2

Use `contracts.verify` on the outcome of `evaluate("0.1 + 0.2", "binary64")` and explain what is being verified independently of the evaluator.

SUMMARY

- `ginsdk` is the book's executable reference semantics: standard-library Python, twelve modules, a stable 1.x API.
- `evaluate`, `compare` and `contracts.inspect` (also `python3 -m ginsdk eval|compare|inspect`) return layered records, never a bare error.
- It is checked by 75 unit tests, by independent oracles (hardware, seven toolchains, QEMU, Python integers), by theorem checks, and by the book's executed examples and laboratory fixtures; defects found by these checks are recorded.
- Its limits — binary IEEE formats with one rounding mode, 32-bit specification models, approximated reals — are stated, not discovered by the reader.

PART XVII

Findings, Limits, and Open Problems

What has been proved, reproduced, implemented, observed, conjectured and left open.

Chapter 41

Findings: What Was Proved, Measured, Conjectured, Refuted, and Not Shown

> *The most useful sentence in a research report is the one that says what was not found.*

LEARNING OBJECTIVES

- Know how many of the book's formal claims fall into each evidence category, and where to find each.
- State the answer to the founding question in its weak and its strong reading.
- Distinguish the book's candidate contributions from its restatements of classical mathematics.
- List what the book does not claim.

41.1 The ledger, counted

Every formal statement in this book carries a ledger identifier, and the ledger files are the canonical record (Appendix C). The count below is computed from those files when the book is built:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import os, re, collections, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
ids = set()
for f in ("RESULTS.md", "DEFINITIONS.md", "COUNTEREXAMPLES.md", "HYPOTHESES.md", "OPEN_PROBLEMS.md",
"EXPERIMENTS.md"):
    ids |= set(re.findall(r"GIN-[A-Z]+-\d+", open(os.path.join(ROOT, f), encoding="utf8").read()))
kinds = collections.Counter(i.split("-")[1] for i in ids)
names = {"DEF": "definitions", "HIST": "classical results (historical)", "THM": "theorems", "PROP":
"propositions",
         "COR": "corollaries", "REI": "reinterpretations", "OBS": "empirical observations", "IMP":
"implementation results",
```

```
        "EXP": "experiments", "H": "hypotheses", "CONJ": "conjectures", "NEG": "counterexamples", "REJ":
"rejected hypotheses",
        "OPEN": "open problems"}
for k in names:
    print(f"{names[k]:32s} {kinds.get(k, 0):4d}")
text = open(os.path.join(ROOT, "RESULTS.md"), encoding="utf8").read()
heads = re.findall(r"^### (GIN-(?:PROP|THM|COR)-\d+).*?\*\((.*?)\)\*", text, re.M)
here = sum(1 for _, s in heads if "proved here" in s)
print(f"\nof {len(heads)} propositions/theorems/corollaries: {here} with a proof written here, {len(heads) -
here} classical or packaged")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
definitions                        45
classical results (historical)     19
theorems                            7
propositions                       54
corollaries                         1
reinterpretations                  11
empirical observations             22
implementation results              8
experiments                        15
hypotheses                         42
conjectures                         1
counterexamples                    25
rejected hypotheses                 4
open problems                      13

of 62 propositions/theorems/corollaries: 15 with a proof written here, 47 classical or packaged
```

The counts are deliberately unflattering in one respect: “proved here” means that the proof was written by this project and checked on small cases by computation — not that the statement is new. Most such statements are elementary, and several are almost certainly folklore (GIN-REI-010, §41.4).

41.2 Findings by kind of evidence

KIND	REPRESENTATIVE FINDINGS	WHERE
classical, restated	Dedekind’s categoricity; the recursion theorem; Euclid, Lamé, Bézout; Fermat, Euler, Wilson, Korselt; Karatsuba’s bound; Miller–Rabin’s error bound	Ch. 3–4, 14–15, 30–35
proved here (elementary)	numerals as operation expressions and zero’s empty address (GIN-PROP-002); the converse of a homomorphism and the none/one/many	Ch. 5–9, 16–18, 22, 33

KIND	REPRESENTATIVE FINDINGS	WHERE
	classification (GIN-THM-007); fractions as pairs and the null pair (GIN-THM-006); the CGT heap as shifted bijective binary (GIN-PROP-006); a test-free divider returns all ones on zero (GIN-PROP-023); unique quotients characterize integral domains (GIN-PROP-065); the binary-gcd iteration bound (GIN-PROP-076)	
reinterpretations	inverse operations as converse relations; number constructions as admissibility repairs; carry-lookahead as CGT compilation	Ch. 9, 13, 27, 38
implementation results	an IEEE 754 simulator matching x86-64 results and all five flags on 80,600 cases (GIN-IMP-001); machine models agreeing with seven toolchains and with QEMU (GIN-IMP-002, GIN-IMP-008); gate-level circuits verified exhaustively	Ch. 22, 24–25
measurements	seven toolchains disagree on `1/0` and `INT_MIN/-1`; clang deletes a post-division zero test, gcc keeps it (GIN-OBS-008); the host detects tininess after rounding, QEMU-AArch64 before (GIN-OBS-009, GIN-OBS-018); contract models predict every covered real execution (GIN-OBS-017); classical cost exponents reproduced by counted operations (GIN-OBS-022)	Ch. 23–26, 37, 39
counterexamples	float addition is not associative; Fermat pseudoprimes; exponent reduction without a unit (GIN-NEG-024); unit-cost multiplication (GIN-NEG-025); principal roots are not multiplicative	throughout
corrections of this project's own errors	exponents evaluated inside $\mathbb{Z}/n$; signaling NaNs ignored; a division-overflow misclassification; stale experiment records	RESULTS §J, §L
conjecture	re-association as a sixth CGT mechanism (GIN-CONJ-001)	Ch. 37–38

41.3 The founding question

Chapter 1 asked whether numerical objects, operations, representations and algorithms can be understood as computational grammars whose admissible transitions, semantics, representations and costs can be formally specified and experimentally validated.

The weak reading — can arithmetic be described this way? — is answered yes. Every operation in the book has a statement of its domain, admissibility, outcome classes, representation, algorithm and cost; every failure studied is classified by the layer at which it occurs and by the condition that fails; and the description is executable and validated against independent oracles (hardware, compilers, an emulator, Python's integers).

The strong reading — does the description yield new mathematics, faster algorithms or better predictions? — is answered mostly no, with one qualified yes. No new theorem of number theory, no new algorithm and no new complexity bound was found; the measured costs reproduce textbook exponents. The qualified yes is predictive: on GIN-EXP-013, explicit contract models predicted the behaviour of seven real toolchains in every covered case where naive "mathematical" predictors failed on a quarter or more of the cases (Chapter 39). That result validates the models' coverage of published specifications; it is not an independent discovery, and the AI-only comparison was not run.

41.4 Novelty, honestly

The candidate contributions are a **framing** — arithmetic as computational grammars with four separable layers and converse admissibility — a **vocabulary of failure** (no solution versus non-unique; image versus kernel), **executable reference semantics** with tests, and **reproducible measurements of real machines**. The statements most likely to be absent from the literature reviewed are elementary: the heap/bijective-binary conjugacy and the pair-level characterization of indeterminate forms. Their novelty has not been verified against primary sources or by outside review, and no priority is claimed for them or for anything else in this book (GIN-OPEN-005).

CLAIM BOUNDARY

This book does not claim: a new theorem of number theory; a new or faster algorithm; a new complexity bound or lower bound; that any behaviour measured on one x86-64 host holds elsewhere; that QEMU results describe physical AArch64 or RISC-V processors; anything about the performance of AI systems; peer review, institutional affiliation or endorsement; that "proved here" means "new".

EXERCISE 41.1

Choose three formal items from Chapters 16–18 and classify each as classical, proved here, reinterpretation or counterexample, giving the reason from its status badge and ledger entry.

EXERCISE 41.2

Propose one experiment whose positive outcome would move the strong reading of the founding question from “mostly no” toward “yes”, and state in advance what result would count against it.

SUMMARY

- The ledger, counted at build time, separates classical results, statements proved here, reinterpretations, measurements, implementation results, counterexamples, conjectures and open problems.
- Weak reading of the founding question: established. Strong reading: no new theorems, algorithms or bounds; one qualified predictive result whose limits are stated.
- The candidate contributions are a framing, a vocabulary, executable semantics and measurements; novelty of individual statements is unverified and not claimed.

Chapter 42

Open Problems

> *An open problem is a promise about what would change our minds.*

LEARNING OBJECTIVES

- Know the open problems of the program in priority order, and which are mathematical, empirical or bibliographic.
- For each, know what result would settle it and what result would count against the hypothesis behind it.
- Know how to reproduce the evidence each problem starts from.

42.1 How the problems are stated

A problem enters this list only when it can be stated so that someone else could work on it: a precise question, the result that would answer it, a **falsifier** (an outcome that would count against the hypothesis motivating it), the literature to read *first* — most questions about arithmetic have been asked before — and a reproduction path. The priorities reflect how much a solution would change the book's conclusions (Chapter 41).

42.2 Priority 1: questions that could change the conclusions

Novelty of the candidate statements (GIN-OPEN-005). *Question:* are the heap/bijective-binary conjugacy (GIN-PROP-006) and the pair-level characterization of indeterminate forms (GIN-THM-006) stated in the literature? *Settled by:* a primary-source search of numeration systems, automatic structures, wheels and projective arithmetic, and outside review. *Falsifier for novelty:* an earlier statement of either — which would make it a classical result restated, and would be recorded as such. *Start with:* [Carlström 2004] for the null pair; [Knuth 1997] §4.1 and [Blumensath & Grädel 2000] for numerations.

An AI-only baseline for execution prediction (GIN-OPEN-013). *Question:* how accurately does a language model, without tools and then with `python3 -m ginsdk inspect`, predict the outcomes of the 845 operations × 7 toolchains of GIN-EXP-013? *Settled by:* the same corpus and scoring, with prompts, model versions and repeated samples recorded. *Falsifier for the usefulness of contracts as an AI interface:* tool-free accuracy matching the contract models', or no improvement when the tool is available. *Reproduce:* `python3 experiments/exp013_execution_prediction.py`.

Hardware confirmation of emulated results (GIN-OPEN-012). *Question:* do physical AArch64 and RISC-V processors behave as their QEMU emulations did for division by zero, INT_MIN/ − 1, and floating-point tininess? *Falsifier for the models:* any disagreement between a physical processor and its SDK model. *Reproduce:* the freestanding test program is `experiments/machine/isa_div.c`; `experiments/exp014_isa_emulation.py` currently always runs it under QEMU, so a hardware run needs the program compiled and executed natively on the target and its output compared field by field with `experiments/results/exp014.json`.

42.3 Priority 2: mathematical structure

A grammar-class hierarchy of arithmetic operations (GIN-OPEN-001). *Question:* place gcd, modular exponentiation with variable modulus, integer square root and base conversion in a precise hierarchy of automata and transducer classes, related to bit complexity. *Known:* addition is synchronous rational and multiplication is not (GIN-THM-005). *Start with:* [Büchi 1960; Cobham 1969; Khoussainov & Nerode 1995; Blumensath & Grädel 2000]. *Decision criterion:* a table in which every entry is a theorem with a citation or a proof.

The cost of admissibility (GIN-OPEN-002). *Question:* when is deciding whether a converse problem has a solution asymptotically cheaper than solving it? Divisibility costs a division; unit-ness costs a gcd; quadratic residuosity modulo a prime costs one exponentiation without finding the root; the discrete logarithm is decidable cheaply and believed hard to solve (Chapter 31). *Settled by:* a classification of the book's converse problems by decision and search complexity, with proofs or citations.

Repairs as universal constructions (GIN-OPEN-003). *Question:* which of the repairs of inadmissibility (extension, selection, restriction, quotient, totalization, absorption, signal; Chapter 18) are universal constructions — Grothendieck group, localization, completion, quotient — and which are not? *Falsifier for the trilemma's generality* (GIN-THM-004): a system giving $1/0$ a value that keeps the ring laws, the converse reading of division and $0 \neq 1$ — impossible by GIN-THM-003, so the open part is the categorical *statement*, not the impossibility.

Admissibility closures (GIN-OPEN-010). *Question:* make the correspondence between sets of converse problems and classical closures (constructible numbers, real algebraic numbers) precise, with bit complexities.

Is re-association a distinct mechanism? (GIN-OPEN-004, GIN-CONJ-001). *Falsifier:* a reduction of every re-association example to CGT's grammar-shape theorem without loss, which would make M-ASSOC a name rather than a mechanism.

42.4 Priority 3: completeness of the models

Completeness of the four-layer outcome model (GIN-OPEN-006). *Falsifier:* a failure mode of a real numerical system — decimal floating point, posits, interval arithmetic, a computer-algebra system — that fits none of syntax, typing, semantics and execution.

A complete IEEE 754 reference (GIN-OPEN-007). Extend the simulator to all rounding attributes, fused multiply–add, conversions, remainder, decimal formats and NaN payloads; validate against an established test suite and more than one ISA.

Machine semantics on more hosts (GIN-OPEN-009). Repeat GIN-EXP-004 on other operating systems, compilers and versions, and map which compilers delete post-division zero tests under which flags (GIN-OBS-008).

Mechanized proofs (GIN-OPEN-011). Formalize GIN-THM-001 to GIN-THM-007 and GIN-PROP-006 in a proof assistant. A useful complication: Lean's mathlib totalizes $0^{-1} = 0$, so the statements must carry explicit admissibility hypotheses — a test of whether the converse vocabulary survives formalization. Small-case computation, which this book uses throughout, is not a substitute for these proofs.

42.5 Closed or answered

Does the segment notation generalize? (GIN-OPEN-008). Answered: a calculus of segments exists and is the classical calculus of paths and 1-chains modulo 2 (GIN-PROP-051); it adds no new content. The notation `0{1,2,3}4` remains a checkable display of boundary and interior.

42.6 Reproducing the starting point

Every problem above starts from evidence that can be regenerated without accounts or privileged access:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```python
import os, ginsdk

ROOT = os.path.dirname(os.path.dirname(os.path.dirname(os.path.dirname(ginsdk.__file__))))
scripts = sorted(f for f in os.listdir(os.path.join(ROOT, "experiments")) if f.startswith(("exp0",
"check_theorems")) and f.endswith(".py"))
print(len(scripts), "experiment and check scripts:")
print("  " + "\n  ".join(scripts))
print("run all:  sh experiments/run_all.sh   (Appendix B lists the toolchains each one needs)")
```

OUTPUT (CHECKED BY EXECUTING THE CODE ABOVE)

```
17 experiment and check scripts:
  check_theorems.py
  check_theorems_edition1.py
  exp001_numerals.py
  exp002_increment_carries.py
  exp003_converse_census.py
  exp004_machine_division.py
  exp005_ieee_validation.py
  exp006_adder_circuits.py
  exp007_mul_div_circuits.py
  exp008_bit_complexity.py
  exp009_euclid.py
  exp010_primes.py
  exp011_numeral_shapes.py
  exp012_representation_boundaries.py
  exp013_execution_prediction.py
  exp014_isa_emulation.py
  exp015_law_census.py
run all:  sh experiments/run_all.sh   (Appendix B lists the toolchains each one needs)
```

EXERCISE 42.1

Choose one problem from Priority 2 and write its falsifier in the form “if an experiment or proof shows X, the hypothesis Y is rejected”, with X stated precisely enough to check.

EXERCISE 42.2

Formulate a new open problem raised by a chapter of your choice, in the format of this chapter: question, decision criterion, falsifier, literature to read first, reproduction path.

SUMMARY

- Priority 1: verify or refute the novelty of the two candidate statements; run the AI-only baseline of GIN-EXP-013; confirm the emulated ISA results on hardware.
- Priority 2: a grammar-class hierarchy of operations; the cost of admissibility; repairs as universal constructions; admissibility closures; whether re-association is a mechanism.
- Priority 3: completeness of the four-layer model; a complete IEEE reference; more hosts; mechanized proofs.
- The segment-notation question is closed: the calculus exists and is classical.
- Every problem starts from evidence that `sh experiments/run_all.sh` regenerates.

Appendices

Appendix A

Notation and Conventions

A.1 Sets and number systems

SYMBOL	MEANING	DEFINED IN
$\mathbb{N}$	natural numbers $0, 1, 2, \ldots$ (zero included)	Chapter 3
$\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$	integers, rationals, reals, complex numbers	Chapters 27–28
$\mathbb{Z}/n$	integers modulo n, residues $0, \ldots, n-1$	Chapter 29
$\mathbb{F}_q$, $\mathrm{GF}(2^8)$	finite fields	Chapter 29
$\mathbb{Z}_p$	p-adic integers	Chapter 29
S, P	successor and predecessor moves	Chapter 3

A.2 Numerals and representation

SYMBOL	MEANING	DEFINED IN
$\mathrm{val}_b(w)$, $\mathrm{val}(w)$	value of the numeral w in base b: the denotation of w at 0	Chapter 5
$\mathrm{addr}(n)$	canonical numeral: shortlex-least word denoting n; $\mathrm{addr}(0) = \varepsilon$	Chapter 5
ε	the empty word	Chapter 5
10_2, `0b10`	a numeral read in base 2	Chapter 5

SYMBOL	MEANING	DEFINED IN
enc, dec	encoding of values as bit strings; decoding (total on bit strings)	Chapter 7
wrap	reduction to w-bit two's complement	Chapter 21
trunc	rounding toward zero	Chapter 19
$\mathrm{fl}(x)$, rnd	the floating-point value nearest x (ties to even)	Chapter 25
$\mathrm{ulp}(x)$	unit in the last place	Chapter 25
NaN, ± 0, $\pm\infty$	IEEE 754 special data	Chapter 25
K, P, G	kill, propagate, generate: the carry monoid	Chapter 11
M, S	multiply and square moves of the exponent program	Chapter 34

A.3 Operations and solution sets

SYMBOL	MEANING	DEFINED IN
$[\![e]\!]$	the denotation of an expression or move word	Chapter 2
$\mathrm{Sol}(a, b)$	solution set $\{c : b \circ c = a\}$ of a converse problem	Chapter 9
$\mathrm{Ann}(b)$	annihilator $\{t : bt = 0\}$	Chapter 16
$\mathrm{im}\, f$, $\ker f$	image and kernel of a homomorphism	Chapter 16

SYMBOL	MEANING	DEFINED IN
$a \mid b$, $a \nmid b$	divides, does not divide	Chapter 30
gcd, lcm	greatest common divisor, least common multiple (with $\gcd(0,0) = 0$)	Chapter 30
$a \dot{-} b$	truncated subtraction $\max(a - b, 0)$	Chapter 12
$\mathrm{ord}(a)$, $\varphi(n)$	multiplicative order; Euler's totient	Chapter 31
$\mathrm{popcount}(n)$	number of 1 digits in the binary numeral of n	Chapter 34
$\bot$	the absorbing element of a wheel	Chapter 18

A.4 Cost

SYMBOL	MEANING	DEFINED IN
L	bit length of the operands	Chapter 36
w	machine word length (the SDK's limbs: $w = 32$)	Chapter 36
$M(L)$	cost of multiplying two L-bit numbers	Chapter 15
$\mathcal{O}$, Θ	asymptotic upper bound; tight bound	Chapter 36

Every cost names its model (unit cost, word RAM or bit complexity), its growing parameter, and whether it is a bound, an exact count or a measurement (GIN-DEF-097).

A.5 Status badges and environments

BADGE	MEANING
Definition	a precise object; not a discovery
Classical result	a known theorem, cited; a proof may be included for completeness
Proved here	proof written in this book and checked by computation on small cases; *not* a claim of novelty
Reinterpretation	a known fact restated in this book's vocabulary
Empirical observation	a reproducible measurement; not a proof
Implementation result	a property of the software, checked against an independent oracle
Counterexample	an instance in which a plausible claim fails
Conjecture / Open problem	not established

Ledger identifiers have the form `GIN-KIND-nnn` (Appendix C); identifiers of the separate CGT publication have the form `CGT-KIND-nnn`.

A.6 Typographic conventions

- Code, numerals as strings and command lines are set in `monospace`; a numeral as a mathematical object is set in math.
- Executed examples are followed by their output, checked at every build: the book never shows output its code does not produce.
- Machine claims say whether they were **measured** (on the host named in Appendix B), **emulated** (QEMU) or **specified** (from a standard).

Appendix B

Reproducibility, the SDK, and the Laboratory

B.1 What can be reproduced

Three kinds of evidence appear in this book, and each can be regenerated from the repository without accounts, keys or paid services:

1. **Executed examples** — every `python run` block is executed at build time by `book/tools/check_examples.py`, and the build fails if the printed output differs from the real output.
2. **Experiments** — `experiments/exp001_…` to `exp015_…` write JSON results under `experiments/results/`; counted fields are deterministic given the recorded seeds, and wall-clock fields (names ending in `_s`) are not.
3. **Theorem checks** — `experiments/check_theorems.py` and `check_theorems_edition1.py` check every "proved here" statement exhaustively on small cases.

B.2 Commands

TEXT

```
# SDK tests (Python ⩾ 3.10, standard library only)
cd sdk && PYTHONPATH=src python3 -m unittest discover -s tests -v

# every experiment, both theorem checkers, and experiments/results/SUMMARY.md
sh experiments/run_all.sh

# re-run everything into a scratch copy and compare counted fields with the committed results
python3 experiments/verify_reproduction.py

# the book: website, laboratory tests, executed examples, link check, print edition
cd book && npm ci && npm run verify && npm run pdf
```

B.3 What each experiment needs

EXPERIMENT	NEEDS BEYOND PYTHON	EVIDENCE CLASS
EXP-001 – 003, 006 – 012, 015	nothing	computation
EXP-004 machine division	gcc; optionally clang, javac/java, rustc, go, node	**measured** on the host
EXP-005 IEEE simulator vs hardware	gcc with `fenv.h`	**measured** (x86-64 SSE)
EXP-013 execution prediction	gcc, javac/java, rustc, go, node	**measured**
EXP-014 AArch64 and RISC-V	clang, lld, qemu-user (`qemu-aarch64`, `qemu-riscv64`)	**emulated**; if the tools are missing the script prints `NOT RUN` and leaves the committed results unchanged

AArch64 and RISC-V results are emulator results: they test the SDK's models against QEMU's independent implementation of the architectures, not against physical processors (GIN-OPEN-012).

B.4 The 2026-10 reproduction audit

In October 2026 the repository was re-run in a clean environment (Linux 6.18 x86-64; Python 3.13.16; Node 22.22.0; gcc 13.3.0; clang 18.1.3; OpenJDK 21; rustc 1.97.0; go 1.24.7; qemu-user 8.2.2). The full record, with logs, is `AUDIT_2026-10.md`. In summary:

- all SDK tests passed;
- the counted fields of every experiment result were identical to the committed ones, including the 80,600-case IEEE comparison and, once QEMU was installed, the emulation results of EXP-014;
- every executed example of the first 32 chapters reproduced its printed output, and every lab fixture matched the browser core;
- two defects of the experiment harness were found and fixed: a skipped experiment overwrote recorded evidence, and one theorem checker was not run by `run_all.sh`;

- while the last ten chapters were written, one defect of the SDK was found and fixed — 32-bit division on wrapping machines did not flag the wraparound of INT_MIN/ − 1 (RESULTS §L, correction C-004).

B.5 The laboratory

The interactive laboratory on the website runs `book/site/assets/js/gin-core.js`, a JavaScript port of the SDK's evaluator, IEEE rounding and machine models. It is not an independent theory: `npm test` checks that it reproduces the SDK on every fixture exported by `book/tools/export_data.py` (status, layer, displayed value, IEEE flags, and the machine–mathematics relation), and that its binary64 rounding matches the host's arithmetic on 8,000 random operations (GIN-D-015). A disagreement fails the build.

B.6 The print edition

`npm run pdf` renders the same canonical sources as the website (`dist/print.html`) to PDF with headless Chromium and refuses to publish if any element is wider than the printed text block, any chapter title is missing from the extracted text, any glyph failed to map, any font is not embedded, or the content version printed in the PDF differs from the sources'. The published file, its SHA-256 and page count are recorded in `book/site/downloads/pdf.json`.

Appendix C

A Guide to the Research Ledger

C.1 Why a ledger

A book of this kind makes several hundred formal statements of very different standing: classical theorems, new proofs of elementary facts, measurements on one machine, emulator results, conjectures. The **research ledger** — a set of plain-text files at the root of the repository — is the canonical record of every one of them. The book cites the ledger; it does not replace it. When the two disagree, the ledger is corrected, and the correction is recorded rather than overwritten.

C.2 The files

FILE	CONTENTS
`RESULTS.md`	propositions, theorems, historical results, reinterpretations, observations, implementation results, conjectures, corrections
`DEFINITIONS.md`	every definition, marked standard, classical or provisional
`HYPOTHESES.md`	hypotheses with their status (open, supported, established, rejected)
`COUNTEREXAMPLES.md`	counterexamples and rejected hypotheses
`EXPERIMENTS.md`	experiment protocols: design, falsification criterion, result, limitations
`OPEN_PROBLEMS.md`	open problems
`PROVENANCE.md`	who contributed what, and the status of each result relative to prior work

FILE	CONTENTS
DECISION_LOG.md	dated methodological decisions
LITERATURE_REVIEW.md	prior work by area
CGT_CONCEPT_MAP.md	the relation to Computational Grammar Theory
PUBLICATION_STATUS.md , EDITION_HISTORY.md	the state of this edition and its history
AUDIT_2026-10.md	the October 2026 reproduction audit

C.3 Identifiers

PREFIX	KIND
GIN-DEF	definition
GIN-THM , GIN-PROP , GIN-COR	theorem, proposition, corollary (proved here or classical; the entry says which)
GIN-HIST	historical (classical) result, cited
GIN-REI	reinterpretation of a known fact
GIN-OBS	empirical observation
GIN-IMP	implementation result
GIN-EXP	experiment
GIN-H	hypothesis
GIN-CONJ	conjecture
GIN-NEG , GIN-REJ	counterexample, rejected hypothesis
GIN-OPEN	open problem
GIN-D	decision
NUM-…	the separate research lane *The Number* (not part of this edition's claims)

PREFIX	KIND
`CGT-…`	the separate Computational Grammar Theory publication

Identifiers are never reused. A withdrawn claim keeps its identifier and gains a correction.

C.4 The ladder of claims

Each entry carries exactly one category, and no category is silently promoted to another: a visualization is not a proof, a benchmark is not a theorem, a new notation is not a new theory, a reinterpretation is not a new result, and an implementation is not an algorithmic improvement. “Proved here” means a proof is written in the ledger and its exact statement is checked on small cases by `experiments/check_theorems*.py` ; it says nothing about novelty, which is assessed separately in `PROVENANCE.md` and Chapter 41.

C.5 Following a statement to its evidence

Every formal environment in the book shows its status badge and ledger identifier. The identifier links to the ledger page on the website, where the entry gives the proof or its citation, the name of the check in `experiments/check_theorems*.py` , the experiment that measured it and the chapter that uses it. The build fails if a chapter cites an identifier that the ledger does not contain, if a citation key is missing from the bibliography, or if a cross-reference does not resolve.

Index of definitions and results

38 definitions · 26 theorems · 55 propositions · 1 lemmas · 1 corollaries · 1 conjectures · 13 empirical observations · 5 implementation results · 12 counterexamples · 1 open problems · 1 algorithms. Status badges distinguish results proved in this book (self-checked, not externally reviewed, and almost never new) from classical results, reinterpretations, reproduced results, empirical and implementation results, counterexamples, conjectures and open problems.

ITEM	TITLE	STATUS	LEDGER	CH.
Definition 1.1	Layered analysis of an arithmetic request	DEFINITION	GIN-DEF-090	1
Definition 2.1	Number (value)	DEFINITION	GIN-DEF-001	2
Theorem 2.2	Dedekind's categoricity theorem	CLASSICAL RESULT	GIN-HIST-002	2
Definition 2.3	Numeral and digit	DEFINITION	GIN-DEF-002	2
Definition 2.4	Encoding	DEFINITION	GIN-DEF-003	2
Definition 2.5	Machine state	DEFINITION	GIN-DEF-004	2
Proposition 2.6	Encoding and decoding are not inverse to each other	PROVED HERE	GIN-PROP-050	2
Counterexample 2.7	A bit pattern determines a value	COUNTEREXAMPLE	GIN-NEG-018	2
Definition 3.1	Successor grammar	DEFINITION	GIN-DEF-009	3
Definition 3.2	Successor transition system	DEFINITION	GIN-DEF-011	3
Definition 3.3		DEFINITION		3

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 3.4	Gluing segments; the boundary of a glued segment	PROVED HERE	GIN-PROP-051	3
Proposition 3.5	Fixed-width successor violates exactly one Peano axiom	PROVED HERE	GIN-PROP-052	3
Theorem 4.1	The recursion theorem	CLASSICAL RESULT	GIN-HIST-003	4
Definition 4.2	Primitive recursion on the successor grammar	DEFINITION	GIN-DEF-010	4
Proposition 4.3	Associativity and commutativity of addition	CLASSICAL RESULT	GIN-HIST-004	4
Proposition 4.4	The cost of Peano arithmetic	PROVED HERE	GIN-PROP-003	4
Implementation result 4.5	Rule counts witnessed by rewriting	IMPLEMENTATION RESULT	GIN-IMP-004	4
Definition 4.6	Church numeral	DEFINITION	GIN-DEF-013	4
Definition 5.1	Digit grammar	DEFINITION	GIN-DEF-005	5
Definition 5.2	Value of a numeral	DEFINITION	GIN-DEF-006	5
Proposition 5.3	Numeral ≠ number	CLASSICAL RESULT	GIN-PROP-001	5
Definition 5.4	Canonical numeral (address)	DEFINITION	GIN-DEF-007	5
Proposition 5.5	Zero's canonical numeral is the empty word	REINTERPRETATION	GIN-PROP-002	5
Proposition 5.6	The length of a canonical numeral	CLASSICAL RESULT	GIN-PROP-053	5
Proposition 5.7	Compilation and the concatenation law	CLASSICAL RESULT	GIN-PROP-005	5

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 6.1	Residue digit systems; bijective numeration	CLASSICAL RESULT	GIN-PROP-004	6
Counterexample 6.2	A complete residue system need not give unique numerals	COUNTEREXAMPLE	GIN-NEG-014	6
Proposition 6.3	Hyperbinary representations are counted by Stern's sequence	REPRODUCED RESULT	GIN-PROP-054	6
Proposition 6.4	CGT's heap numeration is bijective binary, shifted by one	PROVED HERE	GIN-PROP-006	6
Proposition 6.5	Positional numerals are straight-line programs for unary terms	REINTERPRETATION	GIN-PROP-007	6
Proposition 7.1	The cost of the successor in binary	PROVED HERE	GIN-PROP-010	7
Proposition 7.2	Divisibility automata	CLASSICAL RESULT	GIN-PROP-055	7
Proposition 7.3	Long division is a finite transition system	CLASSICAL RESULT	GIN-PROP-009	7
Proposition 7.4	Representations trade operations against each other	REINTERPRETATION	GIN-REI-011	7
Definition 8.1	Operation grammar	DEFINITION	GIN-DEF-014	8
Definition 8.2	Closure of a domain	DEFINITION	GIN-DEF-018	8
Definition 8.3	Arithmetic expression language	DEFINITION	GIN-DEF-060	8
Definition 8.4	Four evaluation layers	DEFINITION	GIN-DEF-061	8
Definition 8.5	Outcome	DEFINITION	GIN-DEF-062	8

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 8.6	The four layers are independent	PROVED HERE	GIN-PROP-056	8
Definition 8.7	Operation contract	DEFINITION	GIN-DEF-080	8
Definition 9.1	Converse problem	DEFINITION	GIN-DEF-015	9
Definition 9.2	Converse admissibility	DEFINITION	GIN-DEF-016	9
Theorem 9.3	Converse of a homomorphism	CLASSICAL RESULT	GIN-THM-007	9
Counterexample 9.4	Solution sets of converse problems need not be cosets	COUNTEREXAMPLE	GIN-NEG-019	9
Definition 9.5	Repairs of inadmissibility	PROPOSED	GIN-DEF-017	9
Definition 9.6	Admissibility closure	PROPOSED	GIN-DEF-019	9
Proposition 9.7	The integers and rationals keep unanswered converse problems as data	REINTERPRETATION	GIN-PROP-017	9
Open problem 9.8	The cost of admissibility	OPEN PROBLEM	GIN-OPEN-002	9
Algorithm 10.1	Column addition in base b			10
Proposition 10.1	Correctness of column addition; the carry is at most 1	CLASSICAL RESULT	GIN-PROP-057	10
Proposition 10.2	Addition cannot be done in sublinear time on numerals	CLASSICAL RESULT	GIN-PROP-058	10
Counterexample 10.3	Floating-point addition is associative	COUNTEREXAMPLE	GIN-NEG-001	10
Definition 11.1	Carry automaton and carry monoid	DEFINITION	GIN-DEF-041	11

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 11.2	The carry monoid; carries are prefix products	CLASSICAL RESULT	GIN-PROP-020	11
Empirical observation 11.3	Gate counts and depths of four adders	EMPIRICAL OBSERVATION	GIN-EXP-006	11
Proposition 11.4	Ripple-carry cost	PROVED HERE	GIN-PROP-021	11
Proposition 11.5	Addition needs logarithmic depth	CLASSICAL RESULT	GIN-PROP-059	11
Empirical observation 11.6	Mean longest carry chain	EMPIRICAL OBSERVATION	GIN-OBS-002	11
Theorem 11.7	Addition is finite-state; multiplication is not	CLASSICAL RESULT	GIN-THM-005	11
Conjecture 11.8	Re-association as a mechanism	CONJECTURE	GIN-CONJ-001	11
Proposition 12.1	Truncated subtraction is a totalization, not a converse	PROVED HERE	GIN-PROP-016	12
Proposition 12.2	The Grothendieck construction of $\mathbb{Z}$	CLASSICAL RESULT	GIN-PROP-017	12
Proposition 12.3	Adjoining negatives to saturating addition destroys it	PROVED HERE	GIN-PROP-060	12
Proposition 12.4	Subtraction as addition of the complement; the carry is the comparison	PROVED HERE	GIN-PROP-061	12
Proposition 13.1	Laws kept by wraparound and by saturation (8-bit census)	PROVED HERE	GIN-PROP-062	13
Counterexample 13.2	Saturating addition is associative	COUNTEREXAMPLE	GIN-NEG-021	13

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 14.1	Why 2 × 3 = 3 × 2	CLASSICAL RESULT	GIN-PROP-013	14
Proposition 14.2	Why a · 0 = 0, derived four ways	CLASSICAL RESULT	GIN-PROP-012	14
Implementation result 15.1	The array multiplier: size and depth	IMPLEMENTATION RESULT	GIN-IMP-006	15
Proposition 15.2	Karatsuba's bound	CLASSICAL RESULT	GIN-PROP-063	15
Empirical observation 15.3	Measured exponents of schoolbook and Karatsuba multiplication	EMPIRICAL OBSERVATION	GIN-OBS-010	15
Definition 16.1	Division as a converse problem	DEFINITION	GIN-DEF-015	16
Definition 16.2	Annihilator	DEFINITION	GIN-DEF-091	16
Theorem 16.3	The structure of a converse in a commutative ring	CLASSICAL RESULT	GIN-THM-001	16
Proposition 16.4	0 / 1 = 0 as a complete case	CLASSICAL RESULT	GIN-PROP-011	16
Theorem 16.5	Division in $\mathbb{Z}/n$	CLASSICAL RESULT	GIN-THM-002	16
Proposition 16.6	Counting the solvable divisions in $\mathbb{Z}/n$	PROVED HERE	GIN-PROP-064	16
Proposition 16.7	Unique quotients characterize integral domains	CLASSICAL RESULT	GIN-PROP-065	16
Corollary 17.1	Why 1/0 and 0/0 fail differently, and why 0/1 = 0	CLASSICAL RESULT	GIN-COR-001	17
Theorem 17.2	No non-trivial ring can invert zero	CLASSICAL RESULT	GIN-THM-003	17

ITEM	TITLE	STATUS	LEDGER	CH.
Proposition 17.3	Symbolic simplification is not evaluation	CLASSICAL RESULT	GIN-PROP-014	17
Counterexample 17.4	Overflow and division by zero are the same event	COUNTEREXAMPLE	GIN-NEG-017	17
Theorem 18.1	The extension trilemma for 1/0	REINTERPRETATION	GIN-THM-004	18
Implementation result 18.2	The wheel of fractions of $\mathbb{Z}$ satisfies the wheel axioms	IMPLEMENTATION RESULT	GIN-IMP-007	18
Proposition 18.3	IEEE division by zero is not the converse of IEEE multiplication	CLASSICAL RESULT	GIN-PROP-015	18
Theorem 18.4	Fractions as pairs: ∞ can be adjoined, 0/0 cannot	PROVED HERE	GIN-THM-006	18
Empirical observation 18.5	Which laws each way of dividing by zero keeps	EMPIRICAL OBSERVATION	GIN-EXP-015	18
Theorem 19.1	The division theorem	CLASSICAL RESULT	GIN-HIST-005	19
Proposition 19.2	A divider that never tests its divisor returns all ones	PROVED HERE	GIN-PROP-023	19
Proposition 19.3	Division by an invariant integer using multiplication	CLASSICAL RESULT	GIN-PROP-066	19
Counterexample 20.1	0^0 has a determined value	COUNTEREXAMPLE	GIN-NEG-009	20
Proposition 21.1	Two's complement is the truncated 2-adic expansion	CLASSICAL RESULT	GIN-PROP-008	21
Proposition 21.2	Signed overflow is a disagreement of carries	CLASSICAL RESULT	GIN-PROP-067	21

ITEM	TITLE	STATUS	LEDGER	CH.
Definition 22.1	Circuit	DEFINITION	GIN-DEF-040	22
Theorem 22.2	The circuit complexity of the four operations	CLASSICAL RESULT	GIN-HIST-006	22
Empirical observation 23.1	One symbol, six instructions per architecture family	EMPIRICAL OBSERVATION	GIN-OBS-007	23
Empirical observation 23.2	Compilers differ on deleting a zero test after a division	EMPIRICAL OBSERVATION	GIN-OBS-008	23
Counterexample 23.3	Compilers always exploit undefined division to delete later tests	COUNTEREXAMPLE	GIN-NEG-004	23
Empirical observation 24.1	Asymmetric treatment of the two boundaries	EMPIRICAL OBSERVATION	GIN-OBS-005	24
Implementation result 24.2	ISA models agree with an emulator	IMPLEMENTATION RESULT	GIN-IMP-008	24
Definition 25.1	IEEE binary format	DEFINITION	GIN-DEF-030	25
Definition 25.2	Correct rounding	DEFINITION	GIN-DEF-031	25
Implementation result 25.3	The IEEE simulator agrees with this CPU, results and flags	IMPLEMENTATION RESULT	GIN-IMP-001	25
Definition 25.4	Exception flags	DEFINITION	GIN-DEF-032	25
Proposition 25.5	The least unrepresentable positive integer	CLASSICAL RESULT	GIN-PROP-030	25
Proposition 25.6	Which decimals are binary floating-point numbers	PROVED HERE	GIN-PROP-031	25
Empirical observation 25.7	Summation order changes a sum	EMPIRICAL OBSERVATION	GIN-OBS-015	25

ITEM	TITLE	STATUS	LEDGER	CH.
Empirical observation 25.8	Tininess detection on three architectures	EMPIRICAL OBSERVATION	GIN-OBS-009	25
Empirical observation 26.1	A cost boundary enforced as an admissibility boundary	EMPIRICAL OBSERVATION	GIN-OBS-014	26
Proposition 27.1	Every positive rational, once: the Calkin–Wilf sequence	CLASSICAL RESULT	GIN-PROP-068	27
Proposition 28.1	√2 is irrational	CLASSICAL RESULT	GIN-HIST-007	28
Definition 28.2	Complete ordered field	CLASSICAL RESULT	GIN-DEF-092	28
Theorem 28.3	The reals are uncountable	CLASSICAL RESULT	GIN-HIST-008	28
Proposition 28.4	Equality of computable reals is not decidable	CLASSICAL RESULT	GIN-PROP-069	28
Proposition 28.5	$\mathbb{C}$ has no order compatible with its field operations	CLASSICAL RESULT	GIN-PROP-070	28
Counterexample 28.6	The principal square root is multiplicative	COUNTEREXAMPLE	GIN-NEG-022	28
Empirical observation 28.7	Naive complex division fails on representable inputs whose quotient is 1	EMPIRICAL OBSERVATION	GIN-OBS-019	28
Proposition 29.1	In a finite commutative ring, every non-zero element is a unit or a zero divisor	CLASSICAL RESULT	GIN-PROP-071	29
Definition 29.2	The field GF(2^n)	CLASSICAL RESULT	GIN-DEF-093	29
Proposition 29.3	Newton iteration for inverses modulo a power of two	CLASSICAL RESULT	GIN-PROP-072	29
Definition 29.4	p-adic integers	CLASSICAL RESULT	GIN-DEF-094	29

ITEM	TITLE	STATUS	LEDGER	CH.
Definition 30.1	Divisibility	CLASSICAL RESULT	GIN-DEF-095	30
Proposition 30.2	$\mathbb{N}$ under divisibility is a lattice with 1 at the bottom and 0 at the top	CLASSICAL RESULT	GIN-PROP-073	30
Lemma 30.3	Euclid's lemma	CLASSICAL RESULT	GIN-HIST-009	30
Theorem 30.4	The fundamental theorem of arithmetic	CLASSICAL RESULT	GIN-HIST-010	30
Counterexample 30.5	Factorization into irreducibles is unique in every ring of integers	COUNTEREXAMPLE	GIN-NEG-023	30
Theorem 30.6	There are infinitely many primes (Euclid, Elements IX.20)	CLASSICAL RESULT	GIN-HIST-011	30
Theorem 31.1	Euler's theorem (Fermat's little theorem for n = p)	CLASSICAL RESULT	GIN-HIST-012	31
Theorem 31.2	Wilson's theorem	CLASSICAL RESULT	GIN-HIST-013	31
Theorem 31.3	Which moduli have primitive roots	CLASSICAL RESULT	GIN-HIST-014	31
Proposition 31.4	The structure of a discrete logarithm	PROVED HERE	GIN-PROP-074	31
Theorem 31.5	Euler's criterion	CLASSICAL RESULT	GIN-HIST-015	31
Definition 32.1	Dirichlet convolution and the Dirichlet ring	CLASSICAL RESULT	GIN-DEF-096	32
Proposition 32.2	Units and division in the Dirichlet ring	PROVED HERE	GIN-PROP-075	32
Theorem 33.1	Correctness of Euclid's algorithm	CLASSICAL RESULT	GIN-PROP-040	33
Theorem 33.2	Lamé's theorem	CLASSICAL RESULT	GIN-PROP-040	33

ITEM	TITLE	STATUS	LEDGER	CH.
Theorem 33.3	Bézout's identity, constructively	CLASSICAL RESULT	GIN-HIST-016	33
Theorem 33.4	The bit cost of Euclid's algorithm	CLASSICAL RESULT	GIN-HIST-017	33
Theorem 33.5	The average number of steps	CLASSICAL RESULT	GIN-PROP-040	33
Proposition 33.6	An iteration bound for the binary algorithm	PROVED HERE	GIN-PROP-076	33
Proposition 34.1	The cost of the exponent program	CLASSICAL RESULT	GIN-PROP-041	34
Theorem 34.2	The doubling lower bound for addition chains	CLASSICAL RESULT	GIN-HIST-018	34
Proposition 34.3	Bit cost of modular exponentiation	CLASSICAL RESULT	GIN-PROP-041	34
Counterexample 34.4	Reducing an exponent modulo φ(n) without a unit	COUNTEREXAMPLE	GIN-NEG-024	34
Theorem 35.1	Korselt's criterion	CLASSICAL RESULT	GIN-PROP-043	35
Theorem 35.2	At most a quarter of the bases lie	CLASSICAL RESULT	GIN-HIST-019	35
Definition 36.1	Three cost models for arithmetic	DEFINITION	GIN-DEF-097	36
Counterexample 36.2	Unit-cost multiplication computes doubly exponential numbers in linear time	COUNTEREXAMPLE	GIN-NEG-025	36
Empirical observation 37.1	Fitted exponents depend on the range	EMPIRICAL OBSERVATION	GIN-OBS-022	37
Definition 39.1	Operation contract	DEFINITION	GIN-DEF-080	39

ITEM	TITLE	STATUS	LEDGER	CH.
Definition 39.2	Machine–mathematics relation	DEFINITION	GIN-DEF-081	39
Empirical observation 39.3	Contract models predict real execution; naive models do not	EMPIRICAL OBSERVATION	GIN-OBS-017	39

Glossary

Persistent definitions of the vocabulary used throughout the book. Terms link to the place where they are defined formally.

admissible

An expression is **admissible** in a domain when every operation in it has exactly one result there. For an inverse operation this means the converse problem has exactly one solution.

totalization (totalized)

A **totalization** supplies a value, by convention, where the defining equation has no solution or no unique solution — for example AArch64 returning 0 for integer division by zero, or meadows defining $0^{-1} = 0$. The value is consistent but is not a solution.

Bibliography

73 entries: 45 metadata verified, 14 secondary sources only, 14 unverified. **Metadata verified** means authors, title, venue and year were checked against publisher, DBLP or institutional records; it does *not* mean the full text was read. **Unverified** entries are standard works cited from the collaborator's knowledge; their metadata should be checked before citation.

Alford, Granville & Pomerance 1994

W. R. Alford, A. Granville, C. Pomerance (1994). *There are infinitely many Carmichael numbers*. Annals of Mathematics 139:703–722. METADATA VERIFIED

Apostol 1976

T. M. Apostol (1976). *Introduction to Analytic Number Theory*. Springer, Undergraduate Texts in Mathematics, ISBN 978-0-387-90163-3. doi:10.1007/978-1-4757-5579-4. METADATA VERIFIED

Arm ARM

Arm Ltd. (2024). *Arm Architecture Reference Manual for A-profile architecture (DDI 0487), SDIV/UDIV*. Arm Ltd.. SECONDARY SOURCES ONLY

No-trap, zero-result behaviour of AArch64 integer division confirmed via secondary sources; the pseudocode was not consulted.

Avizienis 1961

A. Avizienis (1961). *Signed-digit number representations for fast parallel arithmetic*. IRE Transactions on Electronic Computers EC-10, 389–400. METADATA VERIFIED

Bergstra & Ponse 2015

J. A. Bergstra, A. Ponse (2015). *Division by zero in common meadows*. Software, Services, and Systems, LNCS 8950, Springer, 46–61. doi:10.1007/978-3-319-15545-6_6. METADATA VERIFIED

Bergstra & Tucker 2007

J. A. Bergstra, J. V. Tucker (2007). *The rational numbers as an abstract data type*. Journal of the ACM 54(2), Article 7. doi:10.1145/1219092.1219095. METADATA VERIFIED

Blelloch 1990

G. E. Blelloch (1990). *Prefix sums and their applications*. Technical report CMU-CS-90-190, Carnegie Mellon University. link. METADATA VERIFIED

Blumensath & Grädel 2000

A. Blumensath, E. Grädel (2000). *Automatic structures*. Proceedings of the 15th IEEE Symposium on Logic in Computer Science (LICS), 51–62. METADATA VERIFIED

Brent & Kung 1982

R. P. Brent, H. T. Kung (1982). *A regular layout for parallel adders*. IEEE Transactions on Computers C-31(3):260–264. METADATA VERIFIED

Brent & Zimmermann 2010

R. P. Brent, P. Zimmermann (2010). *Modern Computer Arithmetic*. Cambridge University Press. UNVERIFIED

Büchi 1960

J. R. Büchi (1960). *Weak second-order arithmetic and finite automata*. Zeitschrift für mathematische Logik und Grundlagen der Mathematik 6:66–92. METADATA VERIFIED

Burks, Goldstine & von Neumann 1946

A. W. Burks, H. H. Goldstine, J. von Neumann (1946). *Preliminary discussion of the logical design of an electronic computing instrument*. Report to the U.S. Army Ordnance Department, Institute for Advanced Study. SECONDARY SOURCES ONLY

The $\log_2 n$ average carry-chain argument is cited via secondary sources.

Calkin & Wilf 2000

N. Calkin, H. S. Wilf (2000). *Recounting the rationals*. The American Mathematical Monthly 107(4), 360–363. doi:10.1080/00029890.2000.12005205. METADATA VERIFIED

Cantor 1891

G. Cantor (1891). *Über eine elementare Frage der Mannigfaltigkeitslehre*. Jahresbericht der Deutschen Mathematiker-Vereinigung 1 (1890–91):75–78. METADATA VERIFIED

Carlström 2004

J. Carlström (2004). *Wheels — on division by zero*. Mathematical Structures in Computer Science 14(1):143–184. doi:10.1017/S0960129503004110. METADATA VERIFIED

Charikar et al. 2005

M. Charikar, E. Lehman, D. Liu, R. Panigrahy, M. Prabhakaran, A. Sahai, A. Shelat (2005). *The smallest grammar problem*. IEEE Transactions on Information Theory 51(7):2554–2576. doi:10.1109/TIT.2005.850116. METADATA VERIFIED

Church 1936

A. Church (1936). *An unsolvable problem of elementary number theory*. American Journal of Mathematics 58(2):345–363. UNVERIFIED

Church 1941

A. Church (1941). *The Calculi of Lambda-Conversion*. Annals of Mathematics Studies 6, Princeton University Press. UNVERIFIED

Cobham 1969

A. Cobham (1969). *On the base-dependence of sets of numbers recognizable by finite automata*. Mathematical Systems Theory 3:186–192. doi:10.1007/BF01746527. METADATA VERIFIED

Cormen et al. 2009

T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press. UNVERIFIED

§17.1: amortized analysis of the binary counter.

CVE-2020-10735

Python Software Foundation (2022). *Integer string conversion length limitation (CVE-2020-10735)*. Python documentation, stdtypes: int_max_str_digits. SECONDARY SOURCES ONLY

Dedekind 1888

R. Dedekind (1888). *Was sind und was sollen die Zahlen?*. Vieweg, Braunschweig. UNVERIFIED

Dixon 1970

J. D. Dixon (1970). *The number of steps in the Euclidean algorithm*. Journal of Number Theory 2:414–422. METADATA VERIFIED

ECMA-262

Ecma International (2024). *ECMAScript Language Specification (Number::divide, BigInt::divide)*. Ecma International. SECONDARY SOURCES ONLY

Euclid, Elements

Euclid (c. 300 BCE). *Elements, Book VII, Propositions 1–2*. c. 300 BCE; e.g. T. L. Heath (trans.), Cambridge University Press 1908. UNVERIFIED

FIPS 197

National Institute of Standards and Technology (2001 (updated 2023)). *Advanced Encryption Standard (AES), FIPS PUB 197 (197-upd1)*. NIST, Gaithersburg, MD. doi:10.6028/NIST.FIPS.197-upd1. METADATA VERIFIED

Forslund 1995

R. R. Forslund (1995). *A logical alternative to the existing positional number system*. Southwest Journal of Pure and Applied Mathematics 1:27–29. SECONDARY SOURCES ONLY

Cited via OEIS A214679; not consulted.

Furst, Saxe & Sipser 1984

M. Furst, J. B. Saxe, M. Sipser (1984). *Parity, circuits, and the polynomial-time hierarchy*. Mathematical Systems Theory 17(1), 13–27. METADATA VERIFIED

Gauss 1801

C. F. Gauss (1801). *Disquisitiones Arithmeticae*. G. Fleischer, Leipzig. METADATA VERIFIED

Go Spec
The Go Authors (2024). *The Go Programming Language Specification: arithmetic operators, integer operators*. go.dev. SECONDARY SOURCES ONLY

Gödel 1931
K. Gödel (1931). *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*. Monatshefte für Mathematik und Physik 38:173–198. UNVERIFIED

Goldberg 1991
D. Goldberg (1991). *What every computer scientist should know about floating-point arithmetic*. ACM Computing Surveys 23(1):5–48. doi:10.1145/103162.103163. METADATA VERIFIED

Granlund & Montgomery 1994
T. Granlund, P. L. Montgomery (1994). *Division by invariant integers using multiplication*. Proceedings of PLDI 1994, ACM SIGPLAN Notices 29(6), 61–72. link. METADATA VERIFIED

Hardy & Wright 2008
G. H. Hardy, E. M. Wright; revised by D. R. Heath-Brown, J. H. Silverman (2008). *An Introduction to the Theory of Numbers (6th edition)*. Oxford University Press, ISBN 978-0-19-921985-8. METADATA VERIFIED

Harvey & van der Hoeven 2021
D. Harvey, J. van der Hoeven (2021). *Integer multiplication in time O(n log n)*. Annals of Mathematics 193(2):563–617. doi:10.4007/annals.2021.193.2.4. METADATA VERIFIED

Heilbronn 1969
H. Heilbronn (1969). *On the average length of a class of finite continued fractions*. In Number Theory and Analysis (Papers in honor of Edmund Landau), Plenum, 87–96. UNVERIFIED

Hesse, Allender & Barrington 2002
W. Hesse, E. Allender, D. A. M. Barrington (2002). *Uniform constant-depth threshold circuits for division and iterated multiplication*. Journal of Computer and System Sciences 65(4), 695–716; corrigendum 80(2), 496–497 (2014). METADATA VERIFIED

Hodgson 1983
B. R. Hodgson (1983). *Décidabilité par automate fini*. Annales des Sciences Mathématiques du Québec 7(1):39–57. METADATA VERIFIED

IEEE 754-2019
IEEE Computer Society (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE. doi:10.1109/IEEESTD.2019.8766229. SECONDARY SOURCES ONLY

Intel SDM
Intel Corporation (2024). *Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 2 (IDIV, DIV)*. Intel. SECONDARY SOURCES ONLY

Isabelle/HOL Fields

The Isabelle developers (2024). *HOL/Fields.thy: the division_ring class (axiom inverse_zero: inverse 0 = 0; lemma divide_zero)*. Isabelle/HOL theory library. link. SECONDARY SOURCES ONLY

ISO C17

ISO/IEC JTC1/SC22/WG14 (2018). *ISO/IEC 9899:2018, Programming languages — C (§6.5.5 multiplicative operators)*. ISO. SECONDARY SOURCES ONLY

JLS SE 21

J. Gosling et al. (2023). *The Java Language Specification, Java SE 21 Edition (§15.17.2)*. Oracle. SECONDARY SOURCES ONLY

Kadaan (CGT) 2026

A. Kadaan (2026). *Computational Grammar Theory: Structure, Operation, and the Cost of Resolution — research ledger and working edition 0.1*. github.com/akadaan310/computationalgrammartheory (read-only reference, commit b3a29f3). link. METADATA VERIFIED

Cited under the byline it was published with. Abed Kadaan and Abed Kadan (this book's author) are the same researcher (decision GIN-D-017). The CGT ledger records that it was developed with an AI research assistant.

Kahan 1987

W. Kahan (1987). *Branch cuts for complex elementary functions, or much ado about nothing's sign bit*. In A. Iserles, M. J. D. Powell (eds.), The State of the Art in Numerical Analysis, Clarendon Press, 165–211. METADATA VERIFIED

Kaplan 1999

R. Kaplan (1999). *The Nothing That Is: A Natural History of Zero*. Oxford University Press. METADATA VERIFIED

Karatsuba & Ofman 1962

A. Karatsuba, Yu. Ofman (1962). *Multiplication of many-digital numbers by automatic computers*. Doklady Akademii Nauk SSSR 145:293–294. UNVERIFIED

Khoussainov & Nerode 1995

B. Khoussainov, A. Nerode (1995). *Automatic presentations of structures*. Logic and Computational Complexity, LNCS 960, Springer, 367–392. doi:10.1007/3-540-60178-3_93. METADATA VERIFIED

Knuth 1978

D. E. Knuth (1978). *The average time for carry propagation*. Indagationes Mathematicae (Proc. KNAW) 40:238–242. METADATA VERIFIED

Knuth 1997

D. E. Knuth (1997). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms (3rd ed.)*. Addison-Wesley. UNVERIFIED

§4.1 positional number systems; §4.3 multiple-precision arithmetic (Algorithm D); §4.5.3 analysis of Euclid's algorithm; §4.6.3 evaluation of powers, addition chains.

Kogge & Stone 1973

P. M. Kogge, H. S. Stone (1973). *A parallel algorithm for the efficient solution of a general class of recurrence equations*. IEEE Transactions on Computers C-22(8):786–793. METADATA VERIFIED

Korselt 1899

A. Korselt (1899). *Problème chinois*. L'Intermédiaire des Mathématiciens 6:142–143. METADATA VERIFIED

Ladner & Fischer 1980

R. E. Ladner, M. J. Fischer (1980). *Parallel prefix computation*. Journal of the ACM 27(4):831–838. doi:10.1145/322217.322232. METADATA VERIFIED

Lamé 1844

G. Lamé (1844). *Note sur la limite du nombre des divisions dans la recherche du plus grand commun diviseur entre deux nombres entiers*. Comptes rendus de l'Académie des sciences 19:867–870. UNVERIFIED

mathlib Community 2020

The mathlib Community (2020). *The Lean mathematical library*. Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), 367–381. doi:10.1145/3372885.3373824. SECONDARY SOURCES ONLY

The convention $0^{-1} = 0$ was confirmed in mathlib's online documentation (Mathlib.Algebra.GroupWithZero), 2026-10-08.

Miller 1976

G. L. Miller (1976). *Riemann's hypothesis and tests for primality*. Journal of Computer and System Sciences 13(3):300–317. METADATA VERIFIED

Moore 1966

R. E. Moore (1966). *Interval Analysis*. Prentice-Hall, Englewood Cliffs, NJ (Series in Automatic Computation). METADATA VERIFIED

O'Connor & Robertson (MacTutor)

J. J. O'Connor, E. F. Robertson (2000). *Brahmagupta (biography)*. MacTutor History of Mathematics Archive, University of St Andrews. link. METADATA VERIFIED

Peano 1889

G. Peano (1889). *Arithmetices principia, nova methodo exposita*. Bocca, Turin. UNVERIFIED

Plofker 2009

K. Plofker (2009). *Mathematics in India*. Princeton University Press. UNVERIFIED

Pollard 1975

J. M. Pollard (1975). *A Monte Carlo method for factorization*. BIT 15:331–334. METADATA VERIFIED

Pomerance, Selfridge & Wagstaff 1980

C. Pomerance, J. L. Selfridge, S. S. Wagstaff Jr. (1980). *The pseudoprimes to 25·10⁹*. Mathematics of Computation 35:1003–1026. METADATA VERIFIED

Porter 1975

J. W. Porter (1975). *On a theorem of Heilbronn*. Mathematika 22:20–28. UNVERIFIED

Existence and year confirmed via secondary sources; title and pages not confirmed.

Python Reference

Python Software Foundation (2024). *The Python Language Reference, §6.7 Binary arithmetic operations*. python.org. SECONDARY SOURCES ONLY

Rabin 1980

M. O. Rabin (1980). *Probabilistic algorithm for testing primality*. Journal of Number Theory 12:128–138. METADATA VERIFIED

RISC-V ISA Manual

RISC-V International (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA — M extension*. RISC-V International (ratified 20191213; later versions keep the division semantics). link. METADATA VERIFIED

Division-by-zero and overflow results and their rationale confirmed in the current online text.

Rust Reference

The Rust Project (2024). *The Rust Reference: arithmetic and logical binary operators; integer overflow*. rust-lang.org. SECONDARY SOURCES ONLY

Sklansky 1960

J. Sklansky (1960). *Conditional-sum addition logic*. IRE Transactions on Electronic Computers EC-9:226–231. METADATA VERIFIED

Smith 1962

R. L. Smith (1962). *Algorithm 116: Complex division*. Communications of the ACM 5(8):435. METADATA VERIFIED

Smullyan 1961

R. M. Smullyan (1961). *Theory of Formal Systems*. Annals of Mathematics Studies 47, Princeton University Press. METADATA VERIFIED

Uses the term k-adic for bijective base-k notation (per secondary sources).

Sorenson & Webster 2017

J. Sorenson, J. Webster (2017). *Strong pseudoprimes to twelve prime bases*. Mathematics of Computation 86:985–1003. doi:10.1090/mcom/3134. METADATA VERIFIED

The numerical value of ψ_{13} is taken from secondary sources.

Stein 1967

J. Stein (1967). *Computational problems associated with Racah algebra*. Journal of Computational Physics 1(3):397–405. METADATA VERIFIED

Turing 1936

A. M. Turing (1936). *On computable numbers, with an application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society (2) 42(1):230–265 (issued 1936, volume dated 1937); correction ibid. 43:544–546. doi:10.1112/plms/s2-42.1.230. METADATA VERIFIED